

TECHNISCHE UNIVERSITÄT DRESDEN

FACULTY OF COMPUTER SCIENCE
INSTITUTE OF SYSTEMS ARCHITECTURE
CHAIR OF DATABASES
PROF. DR.-ING. WOLFGANG LEHNER
PROF. APL. DR.-ING. DIRK HABICH

Diploma Thesis

submitted for the academic degree
Diplom-Informatiker

Active Cache-Aware Hardware Adaptation: Cache Engine for Trie-Based Index Structures

Benjamin-Elias Probst
(Born 11 April 1996 in Potsdam)

Professor: apl. Prof. Dr.-Ing. Dirk Habich
Tutor: apl. Prof. Dr.-Ing. Dirk Habich, Dr.-Ing. Alexander Krause

Dresden, September 23, 2026

Thesis Assignment

*Active Cache-Aware Hardware Adaptation:
A Cache Engine for Trie-Based Index Structures*

Context and motivation

Search algorithms and index structures are a core building block of modern in-memory database systems. The Chair of Databases at TU Dresden addresses the hardware-conscious optimisation of such systems. Since the introduction of the Adaptive Radix Tree, numerous cache-aware techniques and related algorithm families have been proposed — from node layout through prefetching and succinct representations to allocation —, yet almost every publication uses its own data structures, different load profiles, and inconsistent measurement methodology, which makes results hard to compare across publications. Finally, the weightings across improvements of individual algorithm components are currently not separable.

Problem statement

Search methods — from balanced search trees (B-/B⁺-trees) through tries and radix trees to specialised, custom-built constructs — span a large yet conceptually uniform *design space*: even a two- or three-component data structure admits astronomically many designs, and a concrete algorithm is a single point in this space, described by many individual design decisions — how nodes are represented, traversed, laid out in memory, and allocated [IZA⁺18, IZH⁺18]. Cache-line-aware behaviour arises not at a single one of these places but at *many different structural places* of an algorithm or of its memory management, and from the interplay of multiple conceptual layers of an algorithm. The state of the art comprises numerous mature methods, yet each optimises only individual ones of these places and demonstrates its benefit in its own measurement environment, which makes progress hard to compare across methods. A common, generic concept and **principle** is missing that can measure and evaluate cache-line-aware behaviour, alongside other measurement focal points, at the different places of a search algorithm — interchangeably, systematically, and fairly.

Objective

The aim of the thesis is to find, design, and prototype — as an experimentation framework — such a principle (a framework) which, with its focal point on a dynamic composition of the distributed algorithm concepts: (i) decomposes cache-aware search algorithms into interchangeable design components, (ii) reconstructs state-of-the-art algorithms as explicit configurations (points in the design space), (iii) measures and evaluates cache-line-aware behaviour both at the individual directly and indirectly affected places and for the whole algorithm on a common, CPU-only basis, and (iv) on the basis of the fine-grained measurement results, delivers the best overall algorithm as a standalone binary behind a unified search-algorithm interface. As a probe for the trial integration of a new technology into the state of the art, the thesis contributes PRT-ART (a Probst Redirect Tree / ART hybrid), which is studied as a conceptually novel search algorithm, yet contributes only a few new aspects within the classification of search algorithms and not a complete algorithm.

The separability of the individual algorithm components demanded here is to be established constructively and made measurable in this thesis.

Sub-tasks

1. Systematically capture the general design components of search algorithms — in particular the cache-aware ones — and identify the places at which cache-line-aware and search-algorithm-specific decisions take effect. This includes the systematic subdivision of the design components into design levels of a generalised algorithm categorisation.
2. Reconstruct state-of-the-art designs — where possible from the original code of the respective publication and original compiler binary linking — (ART, HOT, Masstree, CoCo-trie, START, B²-tree, Wormhole, SuRF, and further methods) as explicit, comparable configurations.

-
3. Design and implement PRT-ART as a probe with several design components of its own, competing against the captured state of the art.
 4. Extend the overall process into a consistently declaratively driven chain: this chain starts from an XML description of the experiment, from which an automated, platform-specific system build is derived for the system in question, which in turn produces every conceivable binary permutation of the design components. Their measurement results are evaluated towards the best measured system binary and system-binary recombination; a final measurement of that one binary or of hybrid binary recombinations produces an evaluation via CSV/xlsx files, which is finally translated into LaTeX and diagrams. That declarative configuration descriptions drive the automated construction of modular system variants is an established construct [CE00, ABKS13].
 5. Build a reproducible measurement pipeline (binary $\rightarrow$ CSV $\rightarrow$ LaTeX $\rightarrow$ diagram) with profile-aware workload routing of all workload frameworks found in the state of the art over *all* available workloads and hardware-counter collection.
 6. Measure systematically and as completely as possible across the configurations in three mandatory measurement series — (A) probe vs. the state of the art, (B) systematic variation of the design decisions, (C) merge/regression of old vs. new variants —, with each series collected at three granularities: *micro-benchmarking* (each individual design component is measured — within the search-algorithm shell as well as the algorithm-category shell — in each common interface function of its category, in isolation against load profiles; the algorithms of the category are evaluated comparatively against one another per interface function, in compliance with latency-aware best practices in database research), *macro-benchmarking* (the individual interface operations of the search algorithm as a whole — e.g. insert, point lookup, delete — are measured through the interface functions of the algorithm category against sample loads or test subranges of the algorithm components), and *whole-system benchmarking* (established YCSB-like load profiles and comparable frameworks over every recombination of the design components).
 7. Evaluate at which places and in which combinations cache-line-aware decisions are beneficial or detrimental for which data-type-specific load patterns — derived from the ranking across the three benchmarking granularities and presented as analysable diagrams and tables.
 8. Automation of system detection and binary selection of the most optimal cache-line-aware algorithm, based on empirically substantiated measurements, together with a measurement-driven, dynamically controlled cache-line-aware adaptation of the algorithm behaviour as a static heuristic system policy for the algorithm behaviour under varying loads and operations. This so-called extraction is to be stored as an XML load-profile result for the testing environment of an overall algorithm, under reference to the various applied load profiles, and is reusable for each architecture focus in order to generate heuristics automatically.

Method and evaluation

The evaluation covers, besides the CPU-only core, SIMD and concurrent multi-threading paths, and compares the configurations against established baselines, including a non-tree-like hash table (`flat_hash_map`) as a counterprobe within the same genus. The data basis consists of real string datasets of differing characteristics — e.g. `url`, `dna`, `protein`, `xml`, `tpcds-id`, and `trec-terms` — under varied data volumes, key lengths, prefix overlap, local density, skew, hit/miss ratios, and value sizes. Measured quantities include exact/prefix lookup, prefix enumeration, insert/update/delete under all variants of the C++ standard interfaces of the well-known `std::map` and `std::vector` container shells, p50/p95/p99 latency, throughput, bytes per key, and hardware counters (cycles, instructions, L1/L2/LLC and dTLB misses, branch mispredictions). Single-thread behaviour is studied first, followed by read-parallel multi-thread scenarios.

Fair comparability is ensured by fixed rules: a common minimal mode (common-denominator) is separated from the PRT-ART native mode, all methods receive identical key spaces and seeds, missing capabilities are reported as N/A rather than covertly emulated, and the compiler, flags, ISA path, allocator, and commit hash of each baseline are logged. For reproducibility, every overall-algorithm configuration is measured in several separate runs whose spread remains part of the statement (percentiles via HDR histograms rather than averaging); each dataset carries a record of source, checksum, and seed rule in order to attest the original code beyond doubt.

Scope

The core contribution is CPU-focused with SIMD extensions and multi-threading, but restricted to read-parallel methods with serialised writing (multi-read, single-write). The GPU is for now out of scope (extendable in the future); full write parallelism and an engine/optimiser integration are optional extensions. All architecture extensions are themselves dynamic extensions against the CPU baseline run and lead to deeper measurement permutations with multiple runtimes. The system-optimized binary can be built optionally with or without the measurement observer — as a debug, measurement, or release variant.

Statement of Authorship

I hereby declare that I, Benjamin-Elias Probst (Mat.-No. 4510512, born on 11 April 1996 in Potsdam), have written the diploma thesis submitted on this day to the examination board of the Faculty of Computer Science on the topic:

*Active Cache-Aware Hardware Adaptation:
Cache Engine for Trie-Based Index Structures*

entirely on my own, using no sources or aids other than those indicated, and that I have marked all passages taken from other works, verbatim or in substance, as such.

Dresden, September 23, 2026

Benjamin-Elias Probst
Mat.-No. 4510512

Abstract

Search algorithms and index structures are central to modern in-memory databases. Since the Adaptive Radix Tree, numerous cache-aware optimisations—spanning node layout, prefetching, succinct representations, and allocation—have been proposed, yet they have never been combined and compared within a single, fair measurement framework: each publication uses its own data structures, workloads, and methodology, which makes results hard to compare across publications. This thesis presents a cache-engine framework whose modules are separated at the binary level by a single, ABI-stable C interface (exactly one living being exported per module); behind it, a three-layer abstraction within the generated tier binary (CacheEngine $\rightarrow$ ExecutionEngine $\rightarrow$ SearchEngine) provides the search API and is kept distinct from the anatomy model (genus, living being, organs). The framework decomposes cache-aware search algorithms, by means of a planner and a CacheEngineBuilder, into interchangeable, distributed (scattered across the code) design decisions of a large design space [IZA⁺18, IZH⁺18], and reassembles them through dynamic composition by compile-time permutation (not runtime adaptation). Structurally different methods—from tree-like indexes (tries, B⁺-trees) to non-tree-like hash tables—are expressed as points of the same design space behind a single interface; non-tree-like methods occupy a restricted subset of the design decisions. State-of-the-art algorithms are reconstructed as explicit configurations (points in the design space) and measured at three granularities—individual design decisions, individual interface operations, and whole algorithms—under known workload-framework load profiles (YCSB and further frameworks integrated by this single framework), each module being buildable optionally with or without a measurement observer for debug, measurement, or release. As a probe, this thesis contributes PRT-ART, a Probst Redirect Tree / ART hybrid with eight building-block layers of its own; a measurement driver with profile-aware workload routing evaluates it, under three mandatory measurement series, against thirty reconstructed state-of-the-art methods (eight established reference designs with building blocks, and further ones); the long-term goal is to deliver the best recombination as a standalone binary.

Contents

1	Introduction	5
1.1	Motivation	5
1.2	Problem Statement	6
1.3	Research Questions	6
1.4	Objective and Contributions	8
1.5	Structure of the Thesis	10
2	Search Trees and Fundamentals	11
2.1	Concept and Overview of the State of the Art	11
2.2	Hardware: Historical Adaptation to Search Performance	13
2.2.1	Cache Organization: Associativity, Miss Classes, and Parallelism	14
2.2.2	Cache-Line-Oriented Layout Research	14
2.2.3	Prefetching	14
2.2.4	Main Memory and Address Translation: DRAM, NUMA, and TLB	15
2.2.5	Write Path, Replacement, and Coherence	16
2.2.6	Cache-Aware and Cache-Oblivious Data Structures	16
2.3	Workloads and Benchmark Conventions	17
2.4	Software Means and Synthesis into the Abstraction	18
2.5	Heuristics as Decision Rules	19
3	Concepts of a Cache-Aware Measurement System	21
3.1	Problem Statement and Requirements for a Measurement System	21
3.2	State of the Art as Axis Instances	22
3.3	Solution Concept: the Anatomy Model	31
3.3.1	Axis Composition and the Living-Being Metaphor	31
3.3.2	The Three-Level Model and the Three Genera	33
3.3.3	Design Patterns and Compile-Time Patterns	34
3.3.4	Five Interface Layers: E4 down to E0	35
3.4	The Measurement System	37
3.4.1	The Triad: Organ, System, and Measurement Axes	38
3.4.2	Complex Axes: the Target ISA as a Fixed Recombination	40
3.4.3	Hardware Detection as a Two-Way-Split Measurement Axis	41
3.4.4	Identity Stamps: One Line per Realm	42
3.4.5	Subsystems and the Chain from the XML to the Tier Binary	42
3.5	PRT-ART as Demonstrator	45
3.6	Sharpening through Heuristics	46
3.6.1	Heuristic Loop and Learned Load Profiles	47

3.6.2	The Four Operating Modes of the Builder	47
3.6.3	Measurement-Curve Type System, Filter Chain, and Measurement-Error Detection	48
4	Implementation and Concrete Algorithm Components	51
4.1	Three-Repository Architecture	51
4.2	PRT-ART in Detail	52
4.3	Axis Implementation and Measurement-System Levels	53
4.3.1	The One Architecture in Code	55
4.3.2	From Profile to Tier Binary: Planner, Builder, and Measurement Pipeline	56
4.4	Quality Assurance: Contract Tests per Layer	57
4.5	Realm Roots in Code: System and Measurement Axes	60
5	Evaluation	65
5.1	Methodology	65
5.2	Results and Analysis	68
6	Conclusion and Outlook	71
6.1	Answering the Research Questions	71
6.2	Limitations	73
6.3	Outlook	74
	Bibliography	77
	List of Figures	93
	List of Tables	99
A	Complete Measurement Series	101
A.1	Cartesian smoke series (43 permutations, real DLL measurement)	101
A.2	Bias-break matrix	101
A.3	Latency surfaces per interface function	104
A.4	Three-dimensional raw-data view	105
A.5	Working-set sweep	107
A.6	Per-axis latency attribution	107
A.6.1	Per-axis observer detail	108
A.7	Latency distribution	110
A.8	Reference-relative evaluation	110
A.8.1	Ratio matrices	111
A.8.2	Normalised bars	111
A.9	Axis exchangeability	112
A.10	Limitations	115
B	Code Structure	117

C	Glossary	123
D	Building-Block Matrix	127
D.1	T0 Search Algorithm Axis T0 (Search Structures and Methods)	127
D.2	T1 Cache Traversal (Fanout Lookup)	130
D.3	T2 Slot-to-Offset Mapping (Traversal Sublayer)	131
D.4	T3 Path Compression	131
D.5	T4 Node Types (Node-Type Axis)	132
D.6	T5 Memory Layout	134
D.7	T6 Allocator Axis (T6)	135
D.8	T7 Prefetch Strategy	137
D.9	T8 Concurrency Synchronization Axis (Concurrency Axis T8)	138
D.10	T9 Serialization (Encoding and Compression)	140
D.11	T10 Value Handle Storage Strategy	140
D.12	T11 Index Organization	141
D.13	T12 I/O Dispatch Axis	142
D.14	T13 Migration Strategy Axis	144
D.15	T14 Filter Axis	145
D.16	T15 Buffering Q1 (Operations Buffer)	146
D.17	T16 Buffering Q2 (Flush Policy)	148
D.18	T17 Persistence Target	149
D.19	System sphere: target ISA (<code>target_isa</code>)	150
D.20	System sphere: operating system (<code>operating_system</code>)	152
D.21	System sphere: extension hardware (<code>external_utils</code>)	153
D.22	Build-SIMD: Vector/Accelerator Extension Axis	155
D.23	System sphere: outer complex bracket (<code>build_target_complex</code>) and build toolchain (<code>build_toolchain</code>)	156
D.24	Measurement sphere: measurement categories (<code>measurement_category</code>)	158
D.25	Measurement sphere: measurement tooling/collector (<code>collector</code>)	159
D.26	Measurement sphere: telemetry building blocks (conserved catalogue)	159
D.27	Measurement sphere: load framework (<code>load_framework</code>) — meta-meta axis	161
D.28	Build-PG Page Type Axis (PAGE-TYPE)	162
D.29	Build-HW — Platform Hardware Strategy	163
D.30	A-Korpus Allocator Corpus (A01-A23)	164
D.31	Axis inventory from the generated registries	166
E	Architecture Decisions	167
F	Comparison Interfaces: <code>std::map</code> and <code>std::vector</code>	171
F.1	<code>std::map</code> (ordered associative)	171
F.2	<code>std::vector</code> (sequence)	174
F.3	Composite Functions and the Conformance Subset	176

1 Introduction

1.1 Motivation

Search trees and index structures underpin modern data-intensive systems — from in-memory databases through file systems to language processing. At the same time, today’s processors are dominated by the memory hierarchy: not the number of operations but their cache behaviour determines run time. From this tension arises the guiding question of this thesis. At its centre stand trie-based indexes and cache-line-optimized processing. Since the Adaptive Radix Tree (ART) [LKN13] of 2013, research has diversified: trie-height optimization (HOT [BZP⁺18]), hybrid B⁺ structures (Masstree [MKM12], B²-tree [SSL⁺21]), succinct representations (CoCo-Trie [BFTV24], SuRF [ZLL⁺18]), and self-tuning approaches (START [FJKN20]).

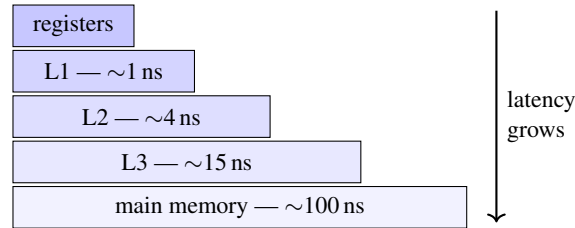


Figure 1.1: The “cache wall”: access latency grows across the memory hierarchy by about two orders of magnitude.

In parallel, a dedicated body of research on cache awareness, prefetching, and allocation strategies has emerged. Heuristics such as Cache-Sensitive Search Trees, cache-oblivious layouts, and hierarchical prefetching each address a separately considered aspect of the search-algorithm architecture, with the focus usually resting on empirically discovered improvements and measurements. Because so much rests on cache-line-optimized behaviour, segmenting the search-algorithm constituents into *axes* suggests itself — a backward abstraction of known building blocks into reusable modules across the most important works of the state of the art.

This thesis carries out this segmentation with a framework whose building blocks can be treated as interchangeable design dimensions of a large design space (*design space*) [IZA⁺18, IZH⁺18]. Structurally different search methods — from tree-like indexes to non-tree-like hash tables, including special properties such as cache-line awareness — thereby become comparable as points of the same design space; non-tree-like methods occupy a restricted subset of the dimensions over the same search-algorithm interface, as if they were the sole fan of a tree root. As a probe for state-of-the-art membership, PRT-ART, a Probst Redirect Tree / ART hybrid and an abstract, incomplete assembly of novel search-algorithm components, serves this role.

1.2 Problem Statement

In today’s search structures cache-awareness is either hard-wired or ignored entirely; above all, however, one cannot *separate* which individual design decision benefits which load pattern — the *separability problem*, the central thread of this thesis.

Existing cache-aware index designs optimize one or a few aspects in isolation and report results that are hard to compare across publications, because each work uses its own data structures, workloads, and measurement methodology. The weights of improvements from individual components remain inseparable: a reported performance gain cannot be unambiguously attributed to a single component (page type, layout, value placement, prefetching), because the components stay intertwined in the original designs. There is no common framework that (i) decomposes such designs into interchangeable axes, (ii) reconstructs published algorithms as explicit configurations, and (iii) measures both individual axes and whole algorithms on a single, fair basis.

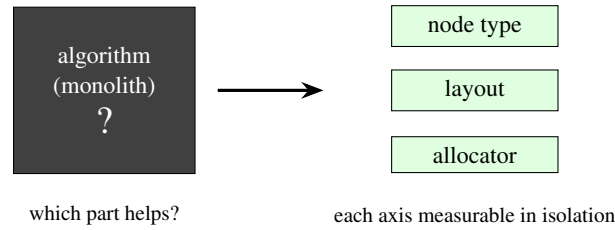


Figure 1.2: The separability problem: as a monolith it remains unclear which design decision matters; only the decomposition into orthogonal axes makes each contribution measurable in isolation.

1.3 Research Questions

From the task description and the goal of cache-line optimization, and given the ubiquity of the influence of cache-line settings, the following research questions arise: one main question (RQ0) and four measurable research questions (RQ1–RQ4); complementary aspects are attached to them as deepening *sub-questions*. Across all five questions lies a categorization question that this thesis carries as a coinage of terms: the task description already distinguishes what is measured (the design constituents of the algorithm), the platform on which building and measuring take place, and the measuring itself. Whether the axes of the design space can be cleanly separated along these three concerns into *organ axes* (the interchangeable design constituents of the algorithm), *system axes* (the concerns of the build and target platform), and *measurement axes* (the concerns of measuring itself) — and how such a separation is to be accomplished if a design does not yet provide it — deliberately remains open at this point; the sub-questions are therefore named along these three *axis categories*, and the answers are developed in Chapters 3 and 4.

RQ0 (Main question) By how much do *active, measurement-driven* cache-aware decisions — page type, memory layout, value placement, and prefetching — improve the performance of trie-based search structures over *passive, statically dimensioned* variants on different CPU platforms (hybrid CPUs, Sapphire Rapids)? *Extension (axis categories)*: Can a measured gain be attributed unambiguously at all — to the design decision itself (organ), to the build and target platform (system), or to the measurement setup (measurement) — and which separation of the axis categories does this

require?

RQ1 (Composition) How can a search-algorithm design be canonically decomposed into orthogonal axes and systematized in a permutation matrix, such that the state-of-the-art methods can be reconstructed bias-free as profile configurations — ideally from their original code — and permuted arbitrarily? *Sub-question (category cut)*: Does the canonical decomposition itself fall apart along the three axis categories — must platform and measurement concerns be kept out of the permutation matrix so that the decomposition of the design constituents remains orthogonal —, and how is such a cut to be accomplished if the design does not yet provide for it? *Sub-question (organ couplings and system variation)*: Which axis couplings (node type $\times$ SIMD $\times$ path compression) limit the isolation of individual axes, and how does the same axis-constant structure behave with respect to cache-update and cache-line fill behaviour when only the ISA/SIMD axis (SSE2/AVX2/AVX-512, NEON/SVE2, RVV) is varied?

RQ2 (Measurement methodology) How must a reproducible measurement pipeline with hardware-counter collection be designed so that every axis permutation is measured fairly (all else held equal, i.e. identical compiler and flag basis) across three measurement series (A–C) over three granularities (micro-, macro-, and overall benchmarking), and under which platform-calibrated criteria does a platform count as successfully verified? *Sub-question (measurement axes)*: Must the measurement instruments themselves — timing, counter collection, load generation — be carried as axes of their own that are always present without ever changing what is built and measured, and how is this separation to be accomplished if it does not yet exist? *Sub-question (organ axes)*: How can contradictory findings in the literature on the optimal node/cache-line size (one cache line for CSS/CSB⁺ versus up to sixteen for Hankins/Patel) be re-measured bias-free through axis-isolated permutation with all else held equal rather than adopted from incompatible original setups? *Sub-question (extensibility)*: How extensible is such a measurement pipeline — how do new probes and their design constituents integrate, how are they abstracted, and do they join existing algorithm categories, found new categories, or a hybrid form; at which conceptual level should this cut run? Central here: what constitutes an extension of the system in the first place, and how does adding or cutting away individual extension building blocks — such as the design constituents originating from a single publication — affect the attainable degree of cache-line awareness of the overall system? Does this require an operating mode that offers the design constituents of a selectable reference design from the literature in isolation as a closed axis set, in order to measure exactly this one publication faithfully — with measurement and system axes remaining free-standing?

RQ3 (Probe comparison) Does the probe PRT-ART win against the eight Rank-1 state-of-the-art (SOTA) designs (ART, HOT, Masstree, CoCo-Trie, START, B²-tree, Wormhole, SuRF)¹ and the Rank-2/3 SOTA *not* through asymptotic novelty but through better microarchitectural properties (higher cache-line utilization, fewer LLC and dTLB misses, lower branch cost, smaller hot search paths) for exact lookup, prefix lookup, and prefix enumeration — measured under the three mandatory measurement series A (probe vs. state of the art), B (systematic variation

¹The *rank*s order the state-of-the-art publications by centrality for this thesis: Rank-1 comprises the ten most important core works — among them the eight standalone search designs as well as ARTSync (P08) [LABN16] and LOUDS (P09) [Jac89], which enter as organs —, Rank-2 the cache-aware layout and B⁺ precursors, and Rank-3 the prefetching, synchronization, and TU-Dresden hardware works.

of the design decisions), and C (merge/regression old vs. new)? *Sub-question (organ axes)*: Which local page representation (8-/16-bit point page versus compact page) is cheapest at a given branch point, and by which platform-calibrated switching rules — including the order $\text{Array}_{256} \rightarrow \text{Array}_{65535} \rightarrow \text{Vector}\langle u8, u8 \rangle \rightarrow \text{Vector}\langle u16, u16 \rangle$ by fill level and search type — should switching occur adaptively? *Sub-question (compile-time force)*: Do latency-critical measurements force decisions that are in principle selectable at run time — such as those adaptive switching rules — to be fixed as requirements already at compile time, so that every axis splits into a compile-time-fixed main decision and run-time-variable sub-decisions — a sharpening that RQ4 takes up?

RQ4 (Compile time vs. run time) Which axis properties must be fixed at compile time to produce platform-optimized, provenance-verifiable binaries per permutation, and which can or must be selected at run time (bit-encoded permutation identifier, adaptive layout selection) without violating scientific comparability (identical compiler and flag basis)? Does the measurement make it possible to generate ISA+OS-optimized binaries for a search-algorithm recombination whose heuristic optimization is perfectly tailored to its runtime environment? *Extension (system axes)*: Does the boundary between compile time and run time follow the axis categories — do the platform properties in particular (ISA, operating system, hardware extensions) belong, as system concerns, to the compile-time identity of the binary, while measurement choices may remain run-time-variable?

1.4 Objective and Contributions

The objective follows the task description in four steps: decompose cache-aware search algorithms into interchangeable design constituents, reconstruct state-of-the-art designs as explicit configurations, measure and evaluate cache-line-aware behaviour both per constituent and for the whole algorithm on a common CPU basis, and deliver the best overall algorithm — following an autonomous heuristic-profile evaluation and heuristic-function modelling — as a standalone binary. The code of the diploma thesis is here the *formal operator* of the cache engine: it loads the probe PRT-ART, compiles it via metaprogramming against the core constituents of the cache engine, delivers ABI-stable *living beings* (search-algorithm instances), and drives them to measurement via the probe dock — the measurement interface of the genus (Chapter 3). The underlying architecture model — the three layers (genus interface, genus-interface subclass, axis-organs), the four subsystems, and the ABI-stable delivery of the search-algorithm *living beings* — is developed in Chapter 3 and Chapter 4; the axis-architecture concept itself is adopted from prior work by the author (on its origin see Section 3.3.3), and the contribution of this thesis is its transfer to cache-aware search structures and its measurement-oriented realization. The central contributions are:

- An ABI-stable module interface: a single C module boundary (`comdare_create_anatomy()` $\rightarrow$ `IAnatomyBase`) exports exactly one living being per permutation; behind it lies a three-layer abstraction chain (`CacheEngine` $\rightarrow$ `ExecutionEngine` $\rightarrow$ `SearchEngine`) with variadically templated hybrid-API specializations (1 parameter $\Rightarrow$ `std::vector` API; 2 parameters $\Rightarrow$ `std::map` API; $N > 2 \Rightarrow$ `std::map<K, std::tuple<V1...VN>>`).
- An axis permutation matrix in a three-part order: eighteen organ axes (composition slots T0–T17) carry the binary permutation of a partial-design algorithm used in research or commonly known,

three system main axes — target ISA, operating system, and extension hardware — describe the build and target platform, and a separate measurement-axis realm keeps the measurement concerns apart from both; an optional nineteenth axis exists only when disk IO is activated and included in the algorithm design. The matrix is realized with Boost.MP11 compile-time type-list building blocks (no runtime tag, no `std::variant`; the `std::variant` catalogue persists only as a historical eleven-axis precursor) and compile-time fallback between the probe stack and the state-of-the-art stack.

- Thirty SOTA algorithm profiles as persisted XML configurations (8 standalone Rank-1 + 22 Rank-2/3) that the `ExperimentDriver` automatically transforms into pre-compiled C++23 modules; prospectively, the profile set is to cover every analysed publication that contributes a standalone search design. By means of the cache engine, the XML design genuinely maps the individual algorithm constituents at compile time and is thereby a compiler-compiler build recipe within a model-driven optimization stack.
- PRT-ART as a probe with eight building-block layers of its own (4+1+2 pool allocator, optimistic lock coupling with reserved blocks, multi-level layout, path-oriented prefetch, density tracker, hypothesis metrics H1/H2/H3, inline/external/chain-ref value handles, signaling-bit serialization); in the measurement apparatus the probe overrides six slots — the build axis page type plus path compression (redirect), path-oriented prefetch, OLC concurrency, telemetry, and the chain-ref value-handle slot —, while the multi-level layout, the pool allocator, and serialization currently run via the cache-engine default in the measurement path (Section 3.5). The deployment of a probe proves the full extensibility of the system towards the efficient advancement of novel approaches to cache-aware algorithms and accelerated search.
- A measurement driver with defined-/full-mode distinction, profile-aware workload routing, and three granularities — micro-, macro-, and overall benchmarking (individual constituents, interface operations, whole algorithms).
- A build-and-measurement chain behind this driver that is declaratively controlled throughout: an experiment *planner* resolves the XML description of the experiment against the registered axis offerings, and the `CacheEngineBuilder` builds and measures from it the binary permutations per platform — up to the evaluation via xlsx or CSV tables and LaTeX tables and diagrams.
- A bias-free comparison matrix that runs *all* load profiles known in research against *all* reconstructed living beings (design-pattern-driven original paper binaries), thereby countering the methodological home-field bias of the individual publications.
- A generic platform probe that determines cache properties (line size, hierarchy depth, topology) per target platform and feeds them as fixed quantities into a system-optimized binary; the two production machines are to be measured for this purpose under two operating systems (immutable Talos and root Linux with full hardware-counter access) — at submission the root-Linux regime is in place, the Talos mirror is outstanding.
- *As objective and outlook*: delivery and substantiated evaluation of the best overall algorithm under the measurement aspect of the best cache-aware behaviour under comparison of all measurable

parameters against binary competitors and acceleration under cache-line adaptation — automatically selected from the fine-grained measurement results — as a standalone, ABI-stable binary behind the unified search-algorithm interface; the measurement pipeline already provides the required ranking, and a first increment of the automatic selection and binary shipping is implemented as a standalone tool (`best_binary_selector`); the heuristics-driven full automation remains outlook.

1.5 Structure of the Thesis

The structure follows the path from the general to the specific: Chapter 2 lays — conceptually and visually — the foundations from established knowledge: the landscape of search structures, the hardware properties of the memory hierarchy, the workload conventions of the comparison literature, the software means of abstraction, and the notion of a heuristic. Chapter 3 develops from them the concepts of a cache-aware measurement system: the problem statement and its requirements, the state of the art as axis instances, the anatomy model with its layers, the measurement system itself, the test candidate PRT-ART as demonstrator, and the heuristic evaluation with its operating modes. Chapter 4 realizes this concept as a software architecture — repositories, the test candidate in code, the axis architecture and compilation pipeline, layer contracts with contract tests, and the system axes in code. Chapter 5 evaluates along the research questions, including the heuristic counter-evidence; Chapter 6 closes with conclusion and outlook. The appendices deepen alongside: the measurement-series documentation (Appendix A), the code structure (Appendix B), the glossary (Appendix C), the complete building-block matrix (Appendix D), the architecture decisions (Appendix E), and the comparison interfaces (Appendix F).

2 Search Trees and Fundamentals

This chapter leads from the familiar object — search trees — via the hardware that shapes their performance and the workloads under which they are compared, to the software means and the notion of heuristics by which their design decisions can be abstracted and selected. Everything stays with *known* terms, concepts, and publications; the systematic catalogues and the thesis’s own conceptual framework follow in Chapter 3.

2.1 Concept and Overview of the State of the Art

We first map the landscape of search structures — comparison-based trees, digital and prefix-based methods, hashing, spatial and flat structures — and classify the known representatives with their core ideas and supporting references. The literature refers to the entirety of the best published methods at any given time as the *state of the art* (SOTA); in this thesis the term always means the published search-structure designs against which a new design must measure itself. Here it already becomes apparent that every algorithm can be located in a shared design space; how this design space is systematically spanned is developed in Chapter 3. Search structures can be classified by the mechanism with which they map

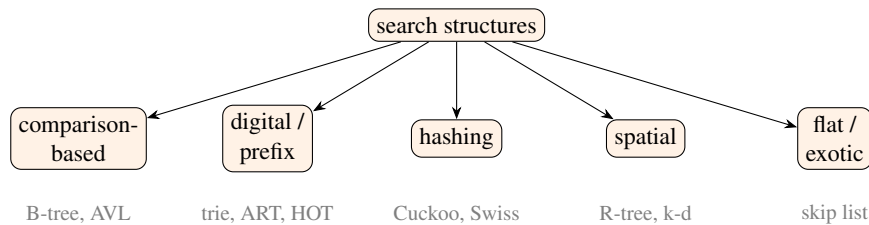


Figure 2.1: The landscape of search structures: five classes with typical representatives.

a key to a record [Knu98]. *Comparison-based* structures locate the key via order comparisons and presuppose a total order — from the self-balancing AVL trees [AVL62], whose sibling subtrees differ in height by at most one, and the red-black trees [Bay72, GS78], which secure the same logarithmic height via a node colouring with looser invariants that are cheaper to maintain, through the block- and external-memory-oriented B/B⁺ trees [BM72, Com79], whose nodes typically correspond to a cache or memory block size [GL01, HP03] and which in the B⁺ variant keep all payload data in the leaves and reduce inner nodes to signposts, to the main-memory-oriented T-tree [LC86], which combines node-local value ranges with AVL balancing and thereby reduces the number of pointer indirections per search step. *Digital* structures, by contrast, branch on the bytes of the key rather than by comparison, so that the search-path length depends on the key length and not on the number of keys: the *trie* [dlB59, Fre60] as the basic form, in which every root path spells out a key and common prefixes are shared, the path-compressed radix/PATRICIA trees [Mor68] that collapse sparse one-child chains, and the *Adaptive Radix Tree* (ART) [LKN13], which adapts the node representation to the actual fanout and serves throughout

this thesis as the reference design.

Beyond the tree-like methods, *hashing* maps keys directly via a hash function and has no inherent order, which makes range queries expensive [Knu98]; cuckoo hashing [PR04], which unlike open addressing and chaining guarantees constant worst-case lookups, and tables such as SwissTable [Goo18], which achieve high practical density thanks to SIMD group probing, stand alongside this work as a non-tree-like baseline. *Spatial* structures (k-d tree [Ben75], quadtree [FB74], R-tree [Gut84]) partition a multidimensional key space and lie outside the scope, as do the theoretical “exotics” — the van Emde Boas tree [vEB75] and the y-fast trie [Wil83] with $O(\log \log u)$ over a universe of size u , or the Fusion Tree [FW93] that undercuts the comparison bound — as well as the randomized skip list [Pug90], the flat, sorted array with binary or interpolation search [Knu98] — strictly speaking the single leaf of a tree — and the suffix array, which generalizes this layout on string keys into a space-saving full-text index and whose lightweight, memory-budgeted construction counts as a textbook example of algorithm engineering [MF02]. This thesis concentrates on the tree-like structures of the comparison- and digital-based classes (Figure 2.1); as outwardly different as the classes appear, they share recurring, mutually separable partial decisions.

Orthogonal to this search-algorithm view stands the established taxonomy of the *containers* of the C++ standard library [ISO24]: sequence containers (`std::vector`) order by insertion position, ordered-associative ones (`std::map`, `std::set`) by key order, unordered ones (`std::unordered_map`) map via a hash function. These standardized interface contracts form the common basis against which structurally different methods can be measured uniformly — the ordered map as the common denominator of a contract shell for comparison- and digital-based search structures, the `std::vector` as the sequence view, and the hash table as the unordered counter-check; their placement in the conceptual framework of this thesis is developed in Chapter 3.

All methods named so far are *statically designed* structures: their inner organization is fixed before the first query arrives. An established line of research, by contrast, moves the design into run time. *Database cracking* [IKM07] builds the index adaptively as a by-product of query processing — each query partitions the data a little further —, and *stochastic cracking* [HIKY12] makes this approach robust against unfavourable query sequences; *learned indexes* [KBC⁺18] replace the search structure with a model of the key distribution that approximately predicts the position of a key. More recent work continues this line in an actively tuning fashion: VEGA readjusts a learned index at run time with group-wise learning granularity [LCL⁺25], and AirIndex designs the hierarchy of an index automatically from the data and storage-I/O profile of the target system [CLP23]. These approaches show a common trend: away from the hard-wired single structure, towards configurable, self-adapting designs. For the present thesis they are the closest related known approaches, since they demonstrate that the design decisions of a search structure can be kept variable; how such decisions can be decomposed systematically and measured individually is developed in Chapter 3.

For the cache-related properties of these designs, the literature has developed a graded terminological taxonomy that this thesis uses throughout. Its starting point is the *I/O model* (external-memory model) of Aggarwal and Vitter: a two-level memory whose levels exchange data only in blocks of fixed size; the cost measure of an algorithm is solely the number of block transfers [AV88]. Transferred to the cache hierarchy, a design is called *cache-aware* when it knows and explicitly uses the concrete parameters of the target platform — block size, capacity; *cache-conscious* has, since Chilimbi, Hill, and Larus, denoted the same attitude from the data structure’s point of view: its layout is deliberately adapted to cache blocks

through clustering, coloring, and compression [CHL99]. *Cache-oblivious* is the counter-position of Frigo, Leiserson, Prokop, and Ramachandran: the algorithm knows no parameters at all; analysis takes place in the *ideal-cache model* — two levels, capacity Z , line length L , full associativity, optimal replacement —, and an optimal cache-oblivious design is then nearly optimal on *every* level of a multi-level hierarchy at once [FLPR99]. *Cache-sensitive*, finally, is the wording of Rao and Ross for their search trees tailored to the cache line (Section 2.2) and is used largely synonymously with cache-aware [RR99, RR00]; as the umbrella term for the parameter-knowing variants, this thesis uses *cache-aware*.

Orthogonal to this parameter question lies the question of timing: *when* is the cache-related design decision made? A *static* design fixes it ahead of the first query — be it cache-aware on determined parameters or cache-oblivious without parameters —, whereas the *active or adaptive* approaches of the previous paragraph move the decision into run time and readjust it from observed behaviour. Both distinctions — parameter knowledge and decision timing — are independent of each other; the notion of heuristics that later connects them follows in Section 2.5.

2.2 Hardware: Historical Adaptation to Search Performance

The performance of search structures has always been driven by hardware. We trace the historical advent of cache-aware hardware — cache hierarchy, cache line, address translation, coherence — focusing on the *static and compile-time-relevant* properties that later carry the abstraction in Section 2.4. Modern

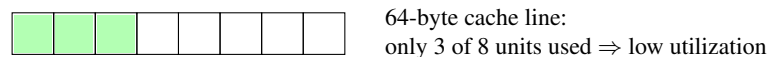


Figure 2.2: Cache-line utilization: a cache line is always loaded in full; if only a few of its bytes are used, effective bandwidth drops — cache-aware layout raises utilization.

processors interpose several cache levels (L1, L2, and — not on every platform — L3) between the registers and main memory [HP19]; an access that finds its datum nearby is cheap, whereas a miss down to main memory costs several hundred cycles [Dre07], on multi-socket and chiplet systems additionally depending on the owning NUMA node. The transfer unit between the levels is the *cache line* (Figure 2.2); its size is no universal constant but hard-wired per microarchitecture — 64 bytes on x86-64 and common ARM cores (such as Cortex-A76 or Neoverse V2), 128 bytes on Apple Silicon L2 and system-level cache [App24], narrower 32 bytes on older or embedded cores [HP19, Int24a, Adv23]. Programs can *read* it at runtime — via `CPUID` (x86), `CTR_EL0` (ARM), or `riscv_hwprobe` (RISC-V; restricted, among other things to CBO block sizes) [Arm24, Int24b] — but not change it; the depth of the hierarchy also varies (simple RISC-V cores such as the SiFive U74 end at L2, some AMD processors bring a stacked 3D V-Cache L3). For a cache-line-aware layout, the line size determined per target platform is therefore an *input to compilation*, not a fixed assumption. Moreover, the transfer of a line does not proceed strictly block-at-once: *critical-word-first* delivers the requested word first, and DDR5 splits the memory channel into two independent 32-bit sub-channels [KAS21]; non-temporal or write-combining stores additionally bypass the normal write-allocate path instead of pulling the affected cache line into the cache hierarchy [Int24b].

2.2.1 Cache Organization: Associativity, Miss Classes, and Parallelism

Where a line may reside in the cache is determined by *associativity*: the cache is organized into *sets*, each holding n *ways* (line slots); the memory address decomposes into tag, set index, and line offset, and each address may reside only in the n slots of *its* set — direct-mapped ($n = 1$) and fully associative (a single set) are the limiting cases, and common data caches lie in between at 4- to 16-way [HP19, Int24a, Adv23]. By their cause, Hill and Smith classify misses into three classes (the *3-C model*): *compulsory* (first access — the line was never in the cache), *capacity* (the working set exceeds the capacity), and *conflict* (more hot lines fall onto one set than it has ways) [HS89]; the textbook presentation adds *coherence* misses as a fourth class in multi-core systems [HP19]. The classification is more than bookkeeping, since it separates the countermeasures: compulsory misses are addressed by prefetching (Section 2.2.3), capacity misses by more compact layouts, and conflict misses arise precisely with the regular address patterns of large node arrays — if hot elements lie apart at power-of-two distances, they compete for the same sets [HP19, Dre07].

Moreover, a miss does not block a modern cache: since Kroft’s *lockup-free* organization, the cache keeps one status register per outstanding miss (*miss status holding register*, MSHR) and serves further accesses in the meantime [Kro81]. The small, fixed number of these registers — realized at Intel as *line fill buffers* between L1 and L2 — limits how many memory accesses can be in flight at once: the *memory-level parallelism* (MLP) [HP19, Int24a]. Whether a search structure can offer this parallelism at all is a design question — the search path of a tree is a chain of mutually dependent accesses and does not exploit it as a matter of course (Section 2.2.3).

Two further properties act at the line boundaries: *alignment* — an object spanning a line boundary costs two transfers instead of one, which is why cache-aware layouts align nodes to line boundaries — and *false sharing*: if two cores write to *disjoint* fields of the same line, the line migrates back and forth between the cores despite logical independence, because coherence (Section 2.2.5) always manages whole lines [Dre07, Int24a].

2.2.2 Cache-Line-Oriented Layout Research

This very cache-line-oriented layout is the historical guiding idea of the relevant research: the CSS tree [RR99] introduces the cache-line node, the CSB⁺ tree [RR00] the contiguous sibling cluster, Hankins and Patel [HP03] measure the effect of node size, and Samuel et al. [SPB05] make the node processor-aware; the Adaptive Radix Tree [LKN13] grades node size cache-line-orientedly with Node4/16/48/256, and Schmidt et al. [SKK⁺25] measure stride versus sequential accesses on high-bandwidth memory. As concrete layout options, the literature knows the dense and the cache-line-aligned array-of-structs, the column-oriented struct-of-arrays, the bit-packed LOUDS [Jac89], and the SIMD-tiled AoSoA. The multi-level hierarchy itself is moreover the subject of software/hardware-cooperative prefetching methods such as the hierarchical instruction prefetcher for server applications of Zhang et al. [ZGH⁺25], which enters this thesis as a prefetching instance and not as a data-structure-specific tree data prefetch.

2.2.3 Prefetching

Prefetching brings data into the cache *before* their use and is the primary countermeasure against compulsory misses and unhidden memory latency (Section 2.2.1). The hardware prefetchers of modern

cores form a few documented classes: next-/adjacent-line prefetchers fetch the neighbouring line of every missed line, stream prefetchers detect ascending or descending access sequences and run ahead of them, stride prefetchers learn constant distances per load instruction (recognized by instruction pointer), and more complex spatially or temporally correlating methods are the subject of research; the survey is given by Mittal [Mit16], and the concrete prefetchers of the L1/L2 stages are documented in the vendor manuals [Int24a, Adv23]. A prefetcher is judged by three quantities: *accuracy* (how many fetched lines were used), *coverage* (how many misses were avoided), and *timeliness* — prefetches that come too late hide the latency only partially, those that come too early evict lines still needed [Mit16].

In addition, the architectures provide *software* prefetch instructions (x86: the `PREFETCHH` family including a non-temporal variant; ARM: `PRFM`) with which the program itself announces an address together with a target level [Int24b, Arm24]. Their central tuning knob is the *prefetch distance* — how far ahead the announcement is made: large enough to hide the memory latency, small enough not to lose the fetched line again before use; it depends on latency and processing speed and is thus a quantity to be calibrated per platform [Mit16]. In index research, Chen et al. justify wider B^+ nodes precisely for this reason: if all lines of a multi-line node are announced in parallel, the effective node load time drops, and the optimal node width grows beyond the single cache line [CGM01]; fractal prefetching transfers the same principle across levels to disk pages [CGM02].

Prefetching finds its limit at *pointer chasing*, the core of every tree search: the address of the next node is known only once the line of the current node has arrived. Such a dependent load chain offers no pattern to either a stride or a stream prefetcher, and it serializes the memory accesses — the available memory-level parallelism (Section 2.2.1) remains unused [Mit16]. The known countermeasures therefore start not at prediction but at design: more useful bytes per dependent step (wider, line-aligned nodes [CGM01]) or several *independent* searches whose accesses are interleaved so that the load chain of one fills the waiting time of the others — design answers that recur as parameters in Section 2.2.6.

2.2.4 Main Memory and Address Translation: DRAM, NUMA, and TLB

Behind the last cache lies the DRAM main memory, whose characteristics the JEDEC standard fixes for the current generation DDR5 (JESD79-5) [JED20]. Two quantities are frequently conflated: the *data rate* in megatransfers per second (MT/s; two transfers per clock) measures the bandwidth of the interface, while the *CAS latency* (CL) counts clock cycles until the first response of an opened row — since the CL cycle count grows with the nominal rate, rising data rates hardly lower the absolute access latency [HP19, KAS21]. Parallelism is supplied by the organization: several memory channels per system, the already-mentioned DDR5 split of each channel into independent 32-bit sub-channels, and bank groups within each chip whose interleaved operation covers row activation times [JED20, KAS21]. Stored on the module are the standard’s *JEDEC nominal profiles*; alongside them exist vendor-specific overclocking profiles (Intel XMP, AMD EXPO) beyond the standard rates — which rate a machine actually runs is therefore a property of the target platform that must be documented, not a constant readable from the module.

In multi-socket and chiplet systems, topology is added: with *NUMA* (non-uniform memory access), each node owns local memory, and accesses to remote nodes cost noticeably more than local ones. By default the operating system places pages according to the *first-touch* policy — a page ends up on the node of the thread that first writes it [Lam13]. For measurements on search structures this is a silent precondition:

whoever builds the structure on a different node than the one measuring it measures the interconnect latency as well.

Besides the data-cache hierarchy there exists a second locality boundary: the *translation lookaside buffer* (TLB) caches virtual-to-physical mappings, and a TLB miss triggers an expensive page walk [HP19]. Its *reach* — entry count times page size — limits the memory region addressable without a page walk. For locality reasons processors separate the instruction TLB from the data TLB (dTLB) — since instruction accesses are considerably more local than data accesses — and lay it out in multiple stages (L1 dTLB plus a shared L2 STLB) [Int24b, Dre07]; for search structures the dTLB is the second, independent boundary alongside the cache line, since large or widely scattered nodes raise dTLB pressure independently of cache utilization. In the state of the art the dTLB appears mainly as a *measurement target*; algorithmically it is addressed by large pages (huge pages, 2 MiB/1 GiB), which enlarge the TLB reach, are supported by several allocators (mimalloc, jemalloc, TCMalloc, snmalloc), and which Schmidt et al. [SKK⁺25] use empirically with 2-MiB pages — a standalone dTLB optimization nevertheless remains underrepresented in the search-algorithm papers, an observation to which the research gap in Chapter 3 (Section 3.1) returns.

2.2.5 Write Path, Replacement, and Coherence

The write and update behaviour of the caches, too, is largely microarchitecturally fixed. Writes follow *write-through*, which writes immediately through to the next level, or *write-back*, which at first merely marks the change in the cache line (*dirty*) and writes it back only upon its eviction — which saves bus traffic and lowers write latency; orthogonally to this the allocation policy (write-allocate, typical with write-back, or no-write-allocate, typical with write-through) determines the handling of a write miss; ARMv8 makes this choice explicit via page attributes — write-through, write-back, or non-cacheable together with read- or read-write-allocate hints — [Arm24], while x86 normally uses write-back with write-allocate [Int24b]. Which line is evicted is decided by a heuristic replacement strategy from LRU/pseudo-LRU to the two-bit-frugal, scan- and thrash-resistant *re-reference interval prediction* (dynamic variant DRRIP) [JTJE10], which builds on the adaptive insertion policy [QJP⁺07]; how many levels hold a line at once is governed by the inclusion policy — an *inclusive* cache holds every line of the higher levels redundantly, an *exclusive* one (victim cache) excludes them, a *non-inclusive* one allows both —, and in multi-core systems an invalidation-based coherence protocol secures consistency: the widespread MESI keeps every line in one of the states *Modified*, *Exclusive*, *Shared*, or *Invalid*, extended by vendors with variants such as MOESI (AMD [Adv23]) or MESIF (Intel [Int24a]) [HP19]. As a concrete instance of these fixed properties, the L3 inclusion policy varies between generations — inclusive on earlier Intel client generations (such as Nehalem), non-inclusive on Intel servers from Skylake-SP/Xeon Scalable, and predominantly victim-like on AMD Zen [Int24a, Adv23]. Since for the most part this is not freely selectable at runtime, these properties are given per target platform and accessible only through measurement; their effect — in particular the *cache-line ping-pong* under competing writes to the same line — must be made visible explicitly by a fair measurement procedure (Chapter 3).

2.2.6 Cache-Aware and Cache-Oblivious Data Structures

With the terminological taxonomy from Section 2.1 and the parameters now introduced, the state of the art of cache-near data structures can be ordered. The empirical impulse of the cache-aware line was given

by Ailamaki et al., whose time breakdown of commercial database systems showed that around half of the execution time lies in memory stall cycles — above all in L2 data and L1 instruction misses [ADHW99]. The answers of the following years each bind part of the parameters from the previous sections explicitly into the design: the cache-line size (CSS/CSB⁺ tree [RR99, RR00]), the prefetch distance together with multi-line nodes (pB⁺ tree [CGM01], across levels fractal prefetching [CGM02]), page size and SIMD width at once (FAST, an architecture-sensitive tree layout with interlocked page, line, and SIMD blocking [KCS⁺10]), or the fanout per cache line (ART [LKN13]); the layout techniques of clustering and compression go back to Chilimbi et al. [CHL99]. The cache-oblivious counter-position is instantiated as well: Bender et al. ground the cache-oblivious B-tree on the van Emde Boas layout [BDFC05], building on the order-maintenance/packed-memory primitive [BCD⁺02], the cache-oblivious traversal of ordered sets with a locality-preserving $O(k/B)$ bound [BCDFC02], and the layout of a tree of fixed topology that minimizes the expected number of block transfers in a multi-level memory model nearly optimally even under unknown block size [BDFC02, BDFC00]; Saikkonen and Soisalon-Soininen maintain a cache-sensitive layout invariant even under updates [SSS08, SSS16], and Graefe/Larson [GL01] compare both approaches across the CPU cache levels, while all practical trie and B⁺ indexes are designed cache-aware. Table 2.1 orders the designs named here by which hardware parameters they explicitly bind into the design.

Table 2.1: Cache-near structure designs and the hardware parameters they explicitly bind into the design.

Design	Cache line	Page/TLB	SIMD width	Prefetch distance	parameter-free
CSS tree [RR99]	•	—	—	—	—
CSB ⁺ tree [RR00]	•	—	—	—	—
pB ⁺ tree [CGM01]	•	—	—	•	—
Fractal prefetching [CGM02]	•	•	—	•	—
FAST [KCS ⁺ 10]	•	•	•	—	—
ART [LKN13]	◦	—	•	—	—
Cache-oblivious B-tree [BDFC05]	—	—	—	—	•
Node-size study Hankins/Patel [HP03]	•	—	—	—	—

Legend: • = explicit design parameter of the original work; ◦ = considered indirectly (ART grades its node sizes cache-line-orientedly without parameterizing the line size); — = not addressed. Classification by this thesis based on the cited original works.

Two observations on the table carry further. First, every design binds a *handful* of parameters firmly — no known design treats the parameter set of the previous sections as an open input space. Second, both philosophies leave the same question open: how can *measured* locality knowledge of the concrete target platform — whose cache properties are predominantly hard-wired and only to a lesser extent configurable (Section 2.2.5) — be combined systematically with the cache-near settings that are actually selectable? This question is taken up again in Chapter 3 (Section 3.1).

2.3 Workloads and Benchmark Conventions

Whether a search structure makes good use of the hardware properties from Section 2.2 only shows under load; for this, fixed terms and benchmark conventions have become established. A *load* (operation) is a single call of the search interface — a point query, an insertion, a deletion, or a range scan. A *workload* (in this thesis synonymously: *load profile*) is a structured profile of such operations: an operation mix

(e.g. 50 % read / 50 % update) over a key access distribution (uniform, Zipf-distributed, latest values) on a dataset of defined size and key characteristic. Realistic workloads are thereby sequences of many consecutive interface calls — not isolated, unrealistic single accesses.

The most widespread workload convention is the Yahoo! Cloud Serving Benchmark (YCSB) [CST⁺10]. It defines a family of key-value workloads: YCSB-A (50/50 read/update, Zipf-distributed), YCSB-B (95/5 read/update), YCSB-C (read only), YCSB-D (latest values), YCSB-E (short range scans), and YCSB-F (read-modify-write). Alongside this, the literature relies on further established families: the index benchmark *SOSD* [KMvR⁺19], the benchmarks of the *TPC* family (transactional TPC-C, analytical TPC-DS) [Tra24], the integration into real systems (such as SuRF [ZLL⁺18] as a Bloom-filter replacement in RocksDB), real string corpora (URLs, DNA, dictionary terms), and — for the hardware-near works — *SPEC CPU* [Sta17] and *CloudSuite* [FAK⁺12]. These frameworks produce different loads that are not directly comparable with one another.

The *goal of workload variation* is to break a methodological bias: each publication typically chooses the load profile under which its own algorithm performs best. Only when all load profiles run against all comparison candidates can bias-reduced comparability be gained. For this thesis, the conventions named above provide the raw material of a common set of load profiles; how this set is constructed and driven over all candidates is developed in Chapter 3, and the corresponding measurement matrix belongs to the evaluation methodology (Chapter 5).

Observation under load uses an established set of metrics. At the cache level one counts the misses per level, expressed as a *miss rate* (misses per access) or — more robust when comparing programs of different access intensity — as *MPKI* (misses per thousand instructions); at the core level, *IPC* and *CPI* (instructions per cycle and its reciprocal) measure the achieved execution density, and stall-cycle breakdowns show which share of the clock cycles is spent on waiting memory accesses [HP19]; that this share can dominate under database loads was precisely the finding of Ailamaki et al. [ADHW99] (Section 2.2.6). For cache-aware layouts, the *cache-line utilization* is added — the share of actually used bytes per loaded line (Figure 2.2) — which directly measures what a layout gains at the transfer unit.

These quantities are collected via the processors’ *performance monitoring counters* (PMC): a few programmable count registers per core to which a hardware event (such as an L2 miss or a dTLB miss) is assigned via event selection; they are made accessible by tools such as `perf` [dM10] or VTune [Rei05]. Their limits are just as well documented: counter values are not strictly deterministic across repetitions and, depending on the event, systematically overcount [WM08], and if more events are to be observed than registers exist, the tool multiplexes the counters — the result is then an extrapolation, not a count. A reliable comparison therefore needs repetitions and an honest statement of dispersion [HB15, WM08]. Which of these metrics this thesis collects and how they are ordered into measurement categories is developed in Chapter 3.

2.4 Software Means and Synthesis into the Abstraction

With which software-system means — compile-time metaprogramming, static use of the hardware properties from Section 2.2 — can the abstraction left open there be *realized*? Available are the established means of software-engineering design: the named design patterns [GHJV94], the static polymorphism of the *Curiously Recurring Template Pattern* together with *policy-based design* [Ale01], and the com-

position from interchangeable building blocks as described by invasive software composition from gray-box components [Aßm03], generative programming [CE00], and the feature-oriented software product lines [ABKS13, BO92]; C++23 modules structure the translation units of such building-block compositions but by themselves do not yet guarantee portable ABI stability [Xu24] — relevant as soon as pre-compiled building blocks are to be exchanged.

For interchangeable building blocks to become a concrete system, what is needed besides the composition techniques is a *form of description* for the configuration chosen in each case. The established answer are *declarative configuration models*: feature models have described configurable variant spaces as feature hierarchies with selection constraints since the FODA study [KCH⁺90], generative programming produces the system variant in the solution space from such configuration knowledge in the problem space [CE00], and the product-line literature systematizes the associated binding times from compile time to run time [ABKS13]. Industrial tooling follows the same principle: Maven’s *Project Object Model* is the declarative — held as an XML file — manifestation of *what* makes up a modular Java system, while the build lifecycle takes over the *when* and *how* [Apa26]; CMake produces a native buildsysteem per platform from one declarative description via interchangeable *generators* [Kit26]; and Mokhov, Mitchell, and Peyton Jones show that build systems as a whole can be scientifically captured and recombined as a combination of a few scheduler and rebuilder building blocks over declarative dependency descriptions [MMP18]. That declarative models can accompany a system beyond build time into operation is developed by the models@run.time line [BBF09]; Götz shows in the TU Dresden MQuAT dissertation how declarative (configuration) models steer the self-optimization of component-based systems through contract negotiation [Göt13].

For this thesis, both strands together form the toolbox: the composition techniques with which the hardware properties measured per target platform can be turned into a concrete layout at compile time, and the declarative configuration models with which the chosen configuration can be described and reproduced; how both are combined for this purpose is developed in Chapter 3.

2.5 Heuristics as Decision Rules

Both the adaptive and learned indexes (Section 2.1) and the replacement strategies of the caches (Section 2.2) make their decisions *heuristically*. A *heuristic* is a decision rule without a guarantee of optimality: it maps observable features — such as the cache properties determined per target platform (Section 2.2) or the measured load profile (Section 2.3) — to a concrete configuration without proving that configuration best. It takes the place of an exact solution wherever the latter is unknown or too expensive to determine; the cache’s own heuristic replacement strategies (LRU, RRIP; Section 2.2) are the hardware-side archetype of this way of thinking [JTJE10].

Heuristics can be distinguished by *when* their decision is made. A *static heuristic* is fixed at compile time: the chosen configuration is compiled firmly into the translated program and no longer changes at run time. A *dynamic heuristic*, by contrast, selects the configuration at run time, depending on the actually observed behaviour, and may adjust it during operation. Both variants presuppose the same *heuristic function* but shift the moment of its evaluation.

Formally, this heuristic function is a *mapping* from a measurement result or load profile to a configuration: $h : \mathcal{M} \rightarrow \mathcal{K}$, where $\mathcal{M}$ denotes the space of collected features (measurements, load-profile metrics) and

$\mathcal{K}$ the space of admissible configurations. The present thesis first measures this relationship systematically (Chapter 3); the *automated* determination of h itself — the mechanism that derives the respective best configuration from the measurement results — is an objective and outlook of this thesis (Section 6.3).

Finally, the terms used partly synonymously in the literature must be delimited. A heuristic is called *adaptive* if it adjusts its decision to changing conditions; *dynamic* refers more narrowly to the moment — evaluation at run time rather than at compile time; and *measurement-driven* emphasizes the data source — the decision rests on collected measurements rather than on fixed model assumptions. An adaptive heuristic is thus always dynamic, a dynamic one not necessarily adaptive, and both may, but need not, be measurement-driven.

With this, the foundations are assembled: the landscape of search structures, the hardware properties that shape their performance, the workload conventions under which they are compared, the software means of abstraction, and the notion of heuristics that connects measured knowledge with configuration choice. How exactly these means allow a search algorithm to be decomposed into orthogonal, individually measurable design decisions is developed in Chapter 3; the concrete realization of the building blocks introduced there is shown in Chapter 4.

3 Concepts of a Cache-Aware Measurement System

Building on the foundations from Chapter 2, this chapter first develops the problem statement and the requirements for a measurement system, and as their *solution* the anatomy model and the measurement system built upon it — demonstrated on the probe PRT-ART.

3.1 Problem Statement and Requirements for a Measurement System

The starting point is the finding of the foundations (Chapter 2): the performance of cache-aware search structures is decided not by asymptotic complexity but by cache-line behaviour under real load. This behaviour, however, is the result of many interlocking design decisions — how nodes branch, how pages are arranged in memory, how allocation, prefetching, and synchronization are done —, and in the published methods these decisions have grown together monolithically: whoever measures a method always measures the whole bundle, never the individual decision. Precisely this is the problem statement of this thesis, the *separability problem*: the contribution of each individual design decision to cache-line behaviour shall become visible in isolation — per decision as well as for the whole algorithm.

The state of the art sharpens this problem in two directions. In substance, each of the studied works optimizes only a few design decisions in isolation, and in the studied corpus *not a single method occupies all design dimensions together*: some dimensions are filled exclusively with baseline or textbook variants (the mapping between algorithm and cache traversal), others appear only as a *measurement target* (the dTLB), the spatial search class is missing entirely, and decisions such as I/O, migration, filtering, and buffering are each occupied individually but never together with the search decisions in one design. Methodologically a second gap is added: results are reported in incomparable settings, and the frequent choice of scenario-specific, often favourably tailored load profiles hampers a fair cross-comparison; what is missing is a bias-reduced measurement of *all* load profiles against *all* methods on a common basis.

From this double gap follow four requirements for a solution. It must, first, *decompose* cache-aware search algorithms into interchangeable design constituents, so that each decision becomes individually graspable; second, *reconstruct* the state-of-the-art designs as explicit configurations of these constituents, so that the known and the new become comparable in the same space; third, *measure* the cache-line-aware behaviour per constituent as well as for the whole algorithm on a common basis — all load profiles against all methods, bias-reduced; and fourth, derive the best overall algorithm from the measurements and *deliver* it system-optimized. The cache-aware focus is no side condition but the guiding quantity: precisely the cache-line behaviour that decides performance shall become measurable in isolation per design decision. The measurement requirement sharpens, at the juxtaposition of cache-aware and cache-oblivious designs (Section 2.2), into a third, intermediate path: the actual cache properties (line size, hierarchy depth, topology) are *determined* per target platform and enter a system-optimized binary as fixed quantities, while the *configurable* cache-near settings (prefetch distance, batching and inline thresholds, pool budgets, and,

where architecture and operating system permit, the hardware prefetcher) are permuted as axis parameters against an already-loaded binary — graded by what hardware and operating system allow, from fully runtime-configurable (mutable Linux with MSR access) through boot- or compile-static (immutable operating systems) to fixed (read-only cache registers on ARM and RISC-V). Thus cache-oblivious behaviour is augmented with *measured* locality knowledge.

The four requirements condense into a target picture, the *design space*: each partial decision — say node type, memory layout, or allocator — spans an axis of its own, and every state-of-the-art algorithm is a point in the Cartesian product of these axes (Figure 3.1).

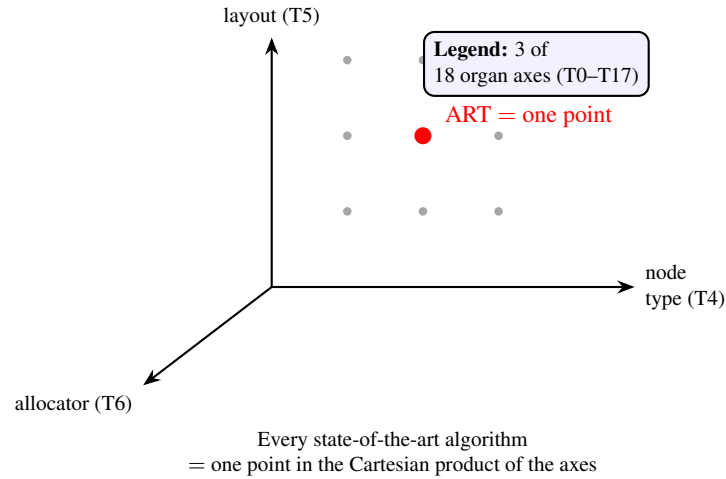


Figure 3.1: The design space: the axes span a multidimensional Cartesian product; a concrete algorithm is a point within it. For illustration, three of the eighteen organ axes are chosen — the real storage organ of node type (T4), memory layout (T5) and allocator (T6); the full design space has eighteen organ dimensions.

Once the design space is spanned this way, the separability problem becomes solvable: varying a design decision means moving along exactly one axis while all others stay fixed — its contribution becomes measurable in isolation. What this thesis contributes is consequently the systematic, orthogonal axis decomposition of the known designs together with a *permuted measurement architecture* that reconstructs them as explicit configurations and measures them on a common basis; for cache-aware algorithms cannot be differentiated without a separation of the properties that decisively govern caching. The four requirements at the same time describe a processing chain — from the definition of an axis assignment through its generation and measurement to the evaluation —, and a system of this size remains manageable only if these stations are developed as separate layers with fixed contracts; the layer model for this is developed in Section 3.3.4. Which of the catalogued instances the framework adopts and which it replaces as obsolete is developed dialectically by the following Section 3.2; the solution concept (Section 3.3) and the measurement system built upon it (Section 3.4) are the solution that closes the double gap.

3.2 State of the Art as Axis Instances

The methods known from Chapter 2 now appear systematically as axis compositions; the complete catalogues of this dissection are gathered in the present section. This adoption proceeds *dialectically* (thesis–antithesis–synthesis): per axis a foreign organ is *adopted* (thesis) or an obsolete assumption is

replaced (antithesis), from which the own contribution emerges as synthesis. The guiding idea is the “third way” from Section 3.1 — measured locality knowledge instead of hard-wired or ignored cache awareness —, which makes each axis explicit as an orthogonal, permutable, and individually measurable organ. For many axes the antithesis is not an obsolete foreign instance but the implicit assumption of not regarding the decision as selectable at all; the synthesis then consists in making it an explicit axis in the first place (marked “—” in the replaced column). Two cluster assignments are deliberate approximations: the cache-traversal axis (T1) draws its inspiration from the layout theory as well as from prefetching, and the reclamation sub-axis of the allocator (T6) shares its sources with the synchronization (T8) — an inter-organ wiring, not a cluster violation. Table 3.1 summarizes this adoption per composition axis.

Table 3.1: Dialectical adoption of the state-of-the-art instances per composition axis (in canonical slot order T0–T17; the earlier numbering T0–T18 still counted telemetry and ISA as composition axes — both have moved into the system sphere, Section 3.4.1. Cluster codes: A trie/node, B hybrid/layout, C allocation, D prefetching, E synchronization, F hardware/measurement/engineering)

Axis		adopted (thesis)	replaced (antithesis)	synthesis core
T0	Search-Algorithm (A)	ART, SuRF, HOT, Wormhole; START, k-ary/interpolation search	monolith “structure is method”	search paradigm as a permutable axis
T1	Cache-Traversal (B)	cache-line stride (Chen), cache-oblivious (Bender), HBM (Schmidt)	implicit textbook linear search	algorithm and cache view separated
T2	Mapping (B)	linear 1:1 mapping; hash redirect, permutation index	—	direct vs. pool-relative mapping made explicit
T3	Path-Compression (A)	byte-wise (ART), Patricia (HOT), macro node (CoCo), multi-byte (START)	uncompressed raw path	compression granularity selectable
T4	Node-Type (A)	ART Node4/16/48/256	fixed node size	adaptive node class + storage delegation
T5	Memory-Layout (B)	AoS-aligned, SoA, LOUDS bit-packed, CSS/CSB ⁺ , AoSoA	—	four layout dimensions permutable
T6	Allocator (C)	A01–A23 (Hoard, mimalloc, jemalloc, TCMalloc, ...) + RCU/Hazard	algorithm-hardwired allocator	allocation as shared interface organ (seven sub-axes)
T7	Prefetch (D)	distance estimator (ART), hardware (Wormhole), stride (Chen), adaptive	prefetch not isolated as an axis	path-oriented bundle prefetch (own)
T8	Concurrency (E)	OLC (ART), RCU, hazard pointers; lock-/wait-free	coarse locking as the only assumption	pattern $\perp$ reclamation as two sub-axes
T9	Serialization (F)	raw binary, varint (ART), succinct (LOUDS/FST), LZ77	—	encoding/compression selectable
T10	Value-Handle (F)	inline, external pool, immutable shared (RCU), versioned	implicitly fixed embedded value	placement, ownership and versioning (+ chain ref)
T11	Index-Organization (B)	clustered, IOT, non-clustered, heap	—	storage order as its own axis
T12	I/O-Dispatch (F)	buffered, mmap, direct I/O	mmap as anti-pattern	persistence decoupled from the search structure

Axis	adopted	replaced	synthesis core
T13 Migration (F)	hot/cold, multi-tier NVM, LeCaR-ML	tiering never combined with search axes	migration logic as a measurable axis
T14 Filter (F)	Bloom, Cuckoo, SuRF-range, Xor	filter never integrated with search axes	filter as an orthogonal organ
T15 Queuing-q1 (F)	Disruptor ring, delta chain, CoW, epoch-RCU, SPSC/MPMC	operations buffer never an axis	buffer with access-pattern classes
T16 Queuing-q2 (F)	eager/lazy, watermark (LSM), timed	flush policy implicit	adaptive EWMA flush policy (own)
T17 Persistence-Target (F)	in-memory storage as the standard target	implicit in-memory assumption	persistence target as its own axis (pinned to in-memory in the measurement catalog)
<i>Moved into the system sphere (Section 3.4.1):</i>			
target_isa (system)	vendor-spec x86-64/AArch64/Power/RISC-V (+ SIMD)	—	target ISA as a system complex axis
Telemetry (system/meas.)	density tracker, insert counter, HdrHistogram, path counter	all-node counter (cache-line ping-pong)	coherence-friendly leaf-only; split in two: measurement sub-axis + build quantity

The systematic core is the axis-wise *dissection*: every overall algorithm is decomposed into its organs, and per axis the contributing organ algorithms are listed, with a brief reference to the originating living being. The framework carries *eighteen organ composition axes* in fixed slot order (T0–T17); alongside them stand the quantities of the system sphere — target ISA, SIMD and extension hardware as well as the two-way-split telemetry (the triad of axis categories is ordered by Section 3.4.1) — and the probe build axis page type. The earlier nineteen-axis numbering (T0–T18, interim state of July 2026) has been superseded by this order. Many axes are refined into sub-axes (e.g. SA1–SA4 or AA1–AA7) and provide other axes with distributed interfaces for their detail operations — exactly these *axis interfaces* later carry the micro benchmarking (Section 3.4.5). Table 3.2 gives the overview of which publications contribute an organ per axis; the thematic origin clusters of the prior works thereby appear *dissolved* into their organ contributions.

The *search axis* (T0) carries the most organs — as original bindings ART [LKN13], HOT [BZP⁺18], START [FJKN20], SuRF [ZLL⁺18], and Wormhole [WNJ19], plus generic search methods such as k-ary search [SGL09], interpolation search [PIA78], Eytzinger-layout search [KM17a], skip list [Pug90], B-tree [BM72], and open-addressing hashing [Knu98]. The traversal axes separate the algorithm and cache views: T1 ranges from cache-line walking through stride prefetch [CGM01] and HBM proximity [SKK⁺25] to cache-oblivious recursion [FLPR99], while T2 (mapping) remains a sparsely occupied axis via linear 1:1 mapping, hash redirect [WNJ19], and permutation index [MKM12]. The *path compression* (T3) instantiates the byte-wise variant [LKN13], the Patricia split [Mor68, BZP⁺18], the macro node [BFTV24], and the multi-byte rewiring [FJKN20], and the *node-type axis* (T4) bundles the adaptive layouts from Node4/16/48/256 [LKN13] through HOT compound [BZP⁺18], Internal/Border [MKM12], Decision/Span [SSL⁺21], LOUDS-Dense/Sparse [ZLL⁺18], and the CSS/CSB⁺/Hankins nodes to the PRT-ART family.

Table 3.2: Axis dissection of the search methods: eighteen organ axes in canonical slot order (T0–T17), the quantities moved into the system sphere, and the probe build axis; per axis the sources contributing an organ.

Axis	Sub-axes	Contributing sources
T0 search_algo	SA1–SA4	P01,P02,P05,P07,P10; k-ary (TUD 2009), interpol., Eytzinger, skip list, B-tree, hash
T1 cache_traversal	CT1–CT2	cache-line default; P17,P21,P32
T2 mapping	MP1–MP2	P03,P07; CE baseline
T3 path_compression	PC1–PC3	P01,P02,P04,P05,P09
T4 node_type	NT1–NT3	P01,P02,P03,P04,P05,P06,P11,P12,P13
T5 memory_layout	HM1–HM4	P06,P09,P11,P12,P32; default
T6 allocator	AA1–AA7	A01–A23; P29,P30
T7 prefetch	PF1–PF3	P07,P21,P23,P25,P26,P27
T8 concurrency	CC1–CC2	P08,P29,P30; Dijkstra,Courtois,Herlihy,Michael/Scott
T9 serialization	SR1–SR3	P01,P09,P10; LZ77
T10 value_handle	VH1–VH3	P01,P03,P07,P29
T11 index_organization	io1–io3	Bayer/McCreight,Comer,Oracle
T12 io_dispatch	IO1–IO3	Stonebraker,Crotty; baselines
T13 migration_policy	MG1–MG3	anti-caching,NVM-tiering,LeCaR; none
T14 filter	FT1–FT3	Bloom,Cuckoo,P10,Xor
T15 queuing_q1	QS1–QS6	Disruptor, Bw-tree, Driscoll, P29, Pugh, LSM, Lamport, Michael/Scott
T16 queuing_q2	FS1–FS4	eager/lazy/watermark/Kafka/RocksDB
T17 persistence_target	—	CE baseline (in-memory); disk writeback as a disabled option
Sys. target_isa	IS1–IS3	x86-64/AArch64/Power/RISC-V (vendor-spec)
Sys. external_utils: SIMD	SE1–SE3	SSE2/AVX2/AVX-512/NEON/SVE2/RVV; GPU: CUDA
Sys. external_utils: ext. HW	HW1–HW4	HW strategy per paper (Tab. 3.4)
Sys./meas. telemetry	TM1–TM3	P01,P02,P16,P17,P26,P28; HdrHistogram (Gil Tene)
Build (probe) page_type	PG1–PG3	P01,P02,P04,P05,P06,P10,P11,P12,P13,P20; PRT-ART (2026)

The *memory-layout axis* (T5) ranges from the cache-line-aligned AoS through pointer-free arrangement [RR99], sibling cluster [RR00], and embedded tree [SSL⁺21] to the bit-packed LOUDS [Jac89]. The *allocator axis* (T6) is the most densely occupied with the corpus A01–A23 and is refined into seven sub-axes — from free-list topology through thread locality/NUMA affinity [UHL⁺17] and the allocation strategy to reclamation (epoch, RCU [McK01], hazard pointers [Mic04a], QSBR); the code provides 25 allocator implementations for the 23-work corpus (individual works contribute several variants), of which Table 3.3 lists the ten runnable profiles with license and workload affinity. The *value axis* (T10) distinguishes inline value [LKN13], external pool [WNJ19], versioned pointer [MKM12], and immutable shared ref [McK01], the *serialization axis* (T9) raw binary, succinct LOUDS/FST, and varint [LKN13]. On the hardware-near side, the *prefetch axis* (T7) ranges from no prefetch through software-fixed [CGM01], adaptively distanced [Kha11], and fill-buffer-aware [MWR25] to path-oriented [ZSZ⁺24] and hierarchical bundle prefetch [ZGH⁺25]; the explicit hardware prefetch instruction stems from Wormhole [WNJ19]. The system axis *target ISA* and the SIMD extension of the extension-hardware hub (both system sphere, Section 3.4.1) carry the vendor specifications (x86-64, AArch64, Power, RISC-V with SSE2/AVX2/AVX-512, NEON/SVE2, RVV). Two cross-cutting strategy axes record what an algorithm actively *uses* and how it *distributes* work — the *hardware strategy* (SIMD family, cache-level targeting, NUMA, prefetch hardware, atomic family) and the *scheduling strategy* (worker-pool

Table 3.3: Allocator profiles with license and workload affinity

Profile	Family	License	expected_workload
hoard	A01	Apache-2.0	YCSB_A
michael_lockfree	A03	LGPL-2.1-or-later	YCSB_B
mimalloc	A04	MIT	YCSB_A
jemalloc	A05	BSD-2	YCSB_B
tcmalloc	A06	Apache-2.0	YCSB_C
snmalloc	A07	MIT	YCSB_A
scallocc	A08	BSD-3	YCSB_A
rpmalloc	A10	Public Domain	YCSB_C
lrmalloc	A11	MIT	YCSB_B
dlmalloc	A20	CC0	YCSB_C

expected_workload = YCSB workload class (A/B/C).

layout, SIMD worker limit — hardware-bound typically two of N cores —, heterogeneous P-/E-core dispatch, co-routines, and batch granularity) —, which Table 3.4 assigns per paper. The *telemetry* — in this thesis’s triad no longer an organ axis but split in two, into a measurement-tooling sub-axis and a build quantity of the system sphere (Section 3.4.1) — instantiates density tracker [LKN13], insert counter [BZP⁺18], HdrHistogram, path-read counter [ZSZ⁺24], and the probability hints (P16/P17 Bender [BDFC00, BDFC05]); the coherence-friendly leaf-only variants go back to Kuehn (P28 [Kue23]; personal communication 2023, beyond the published DaMoN-2023 state on the cache- and distribution-aware B⁺ layout), while the counter in all (including inner) nodes counts as an *anti-pattern* (cache-line ping-pong).

Table 3.4: Target hardware and scheduling profiles assigned in this thesis per reconstructed SOTA method (selection; the profiles are reconstruction targets of this thesis, not necessarily original-paper properties; the full catalogue follows in the appendix).

Paper	Hardware strategy	Scheduling strategy
P01 ART	AVX2, L1-aware, none, CAS	thread-per-core
P02 HOT	AVX2 (bit vector), L1-aware, CAS	thread-per-core
P03 Masstree	scalar, L1-aware, CAS	thread-per-core + work-stealing
P05 START	AVX2 (multi-byte span), L1-aware, CAS	thread-per-core
P20 B-Trees Are Back	AVX2, HBM-aware, NUMA-aware, CAS	work-stealing + hybrid
P27 hp-soft	PREFETCH (instr. bundle), all-cache, CAS	thread-per-core + co-routine
P28 Kuehn	PREFETCH (scalar counter), L1-aware, CAS	thread-per-core + sampling
P31 Ungethuem	AVX-512 (HBM), HBM-aware, NUMA-aware, CAS	hybrid (P/E cores)
P32 Schmidt	AVX-512 (SIMD stride), HBM-aware, NUMA-aware, CAS	hybrid
PRT-ART	AVX2, L1-aware (TLB-aligned), local NUMA, PREFETCH, CAS (OLC)	thread-per-core + 2-SIMD-worker limit + hybrid + co-routine + micro-batch

The *concurrency axis* (T8) splits into the two sub-axes pattern and reclamation (pattern $\perp$ reclamation,

Table 3.1): the patterns range from the serial zero point through coarse locking [Dij65], reader/writer locks [CHP71], and optimistic lock coupling [LABN16] to lock-free [MS96], wait-free [Her91], as well as RCU [McK01] and hazard-pointer [Mic04a] methods; the locking mode within a pattern distinguishes read-only, read-write, upgradeable, and optimistic validation. The *I/O axis* (T12) instantiates buffered [Sto81], direct, and memory-mapped I/O [CLP22]. The axis thereby builds on the block-transfer thinking of the external-memory tradition (Section 2.2): for I/O-efficient data-structure engineering, TPIE provides a reference library of the external-memory line (ESA 2002) [APV02]; the textbook foundation is supplied by Mehlhorn and Sanders [MS08]. The *migration axis* (T13) instantiates hot/cold separation and multi-tier RAM→NVM→SSD migration, and the *buffering axes* (T15/T16) range from the Disruptor ring buffer [TFB⁺11] through the Bw-tree delta chain [LLS13], the copy-on-write buffer [DSST89], and the tombstone buffer [OCGO96] to lock-free SPSC [Lam77] and MPMC queues [MS96]. The *index-organization axis* (T11) distinguishes clustered arrangement [BM72], heap storage, and non-clustered secondary index [Com79], the *filter axis* (T14) the Bloom [Blo70], Cuckoo [FAKM14], SuRF [ZLL⁺18], and Xor filter [GL20]. The *persistence-target axis* (T17), finally, makes the storage target explicit — in-memory as the standard, a disk-writeback target as a disabled option —; in the canonical measurement catalog it is pinned to its in-memory representative so that the permutation space stays at 2^{17} identities (Section 3.4.5).

The *loads*, too, appear as instances. The workloads used in the literature stem from *several*, mutually incomparable benchmark frameworks; Table 3.5 maps them to the using publications. Most widespread is the Yahoo! Cloud Serving Benchmark (YCSB) [CST⁺10]; besides it appear the index benchmark SOSD [KMvR⁺19], the TPC family [Tra24], the CPU suite SPEC CPU [Sta17], the scale-out suite CloudSuite [FAK⁺12], and — for the allocator corpus — mimalloc-bench [Lc21] and the classics Larson, threadtest, and shbench. Since these frameworks produce incomparable loads, the thesis abstracts them into a common set of load profiles; the frameworks not yet realized in the present measurement apparatus (TPC, SOSD, SPEC CPU, CloudSuite, and the allocator suites) are attached via a *dataset loader*.

Table 3.5: Benchmark frameworks and using publications

Framework	Type	Using publications
YCSB [CST ⁺ 10]	KV workload	P02, P03, P07, P20, P26
SOSD [KMvR ⁺ 19]	index benchmark	P05
TPC-C / TPC-DS [Tra24]	OLTP / analytical	P19
real string corpora	string datasets	P04, P06, P10
RocksDB integration	real system	P10 (SuRF)
SPEC CPU [Sta17]	CPU suite	P23, P24, P27, A18
CloudSuite [FAK ⁺ 12]	scale-out server	P24, P27
IBM DB2	production DBMS	P22
mimalloc-bench [Lc21]	allocator suite	A04, A07, A09, A10, A18
Larson / threadtest / shbench	allocator classics	A01, A03, P08, P30
LADDIS	kernel allocator	A02
AggSum / Memory-Stream	bandwidth	P32
custom microbenchmarks	per paper (Zipf/uniform)	P01, P04, P06, P11–P14, P21, P28

From this analysis the thesis distils *fourteen* distinct load profiles (LP01–LP14, Table 3.6): they give the heterogeneous loads a common, uint64-based operation model — the op-mix names the shares of insert,

lookup, delete, scan, and read-modify-write over defined key distributions — and are each derived from concrete publications.

Table 3.6: Load-profile catalogue (14 profiles, derived from the 33-paper analysis)

LP	Key	Op-mix	Distribution	neg%	Derived from
LP01	bulk-insert-dense	100/0/0/0/0	uniform-seq	0	P01, P02, P03, P05, P12
LP02	bulk-insert-sparse	100/0/0/0/0	uniform-rand	0	P01,P02,P08
LP03	insert-value-length-sweep	100/0/0/0/0	uniform-seq	0	P01,P08
LP04	static-read-positive	0/100/0/0/0	uniform	0	P02, P05, P11, P12, P16, P25
LP05	static-read-zipfian	0/100/0/0/0	zipfian(.99)	0	P10,P20,P25,P28
LP06	static-read-negsweep	0/100/0/0/0	uniform	0–100	P04,P06,P22
LP07	static-read-string-corpus	0/100/0/0/0	real-corpus	50	P04,P06,P10
LP08	range-scan-heavy	0/0/0/100/0	uniform-seq	0	P01,P10,P12,P32
LP09	insert-lookup-5050	50/50/0/0/0	uniform	0	P03,P29
LP10	delete-heavy-post-build	50/0/50/0/0	uniform-rand	0	P01,P08
LP11	mixed-ycsb-oltp	25/50/0/0/25	zipfian(.99)	0	P10,P19,P20
LP12	concurrent-rmw-stress	25/9/36/9/9	uniform	0	P07,P29
LP13	concurrent-read-scaling	0/100/0/0/0	uniform	0	P07,P08,P25,P28
LP14	dynamic-realistic-trace	9/91/0/0/0	tpcc-nonuniform	0	P19,P20

Op-mix = shares of insert/lookup/delete/scan/RMW; neg% = share of negative lookups.

The state of the art dissected here rests on a fixed corpus: the analysis draws on 33 deeply analysed search-algorithm publications (P01–P33) and a parallel allocator corpus of 23 works (A01–A23), whose axis assignments are persisted as XML profiles in the `comdare-cache-engine` (three works enter only dissected, see below); the complete building-block and allocator matrices are given in the appendix. The axis-wise dissection of this corpus follows a web-verified provenance map over roughly 110 organ algorithms; every state-of-the-art algorithm thereby appears as a point in the Cartesian product of the axes (Figure 3.1). That the axis framework also carries non-tree-like methods is shown by a pure hash table as a counter-check within the Map genus — as two selectable T0 building blocks, of which exactly one is chosen per permutation and both of which are permuted: an open-addressing hash after Knuth (S14, implemented) and the SwissTable [Goo18] (envisioned as a building block of its own). The counter-check satisfies the point-query subset of the `std::map` interface (`find/insert/erase`); the order-based operations drop out by principle, since a hash table is unordered. Across this subset it occupies a limited axis subset: allocation, prefetch, concurrency, and memory layout remain shared — and on the system side the SIMD extension concretely carries the SwissTable group lookup [Goo18] —, while the tree- and trie-specific axes (path compression, node type, range index) do not drop out but are assigned pass-through variants (`None`); the counter-check thus requires no genus of its own.

The publications stem from six thematic *origin clusters* that here serve only the holistic placement of each living being before it is decomposed into its organs. The *trie family* comprises ART (P01) [LKN13], HOT (P02) [BZP⁺18], Masstree (P03) [MKM12], CoCo-Trie (P04) [BFTV24], START (P05) [FJKN20], and the succinct tries LOUDS (P09) [Jac89] and SuRF (P10) [ZLL⁺18]. The *hybrid and B⁺ family* unites the B²-tree (P06) [SSL⁺21], Wormhole (P07) [WNJ19], and the cache-sensitive CSS, CSB⁺, and Hankins trees (P11–P13) [RR99, RR00, HP03]. The *layout theory* ranges from Samuel (P14) [SPB05] and Graefe/Larson (P15) [GL01] through the cache-oblivious designs of Bender et al. (P16/P17) [BDFC00, BDFC05, BDFC02, BCDFC02, BCD⁺02] to the layout-invariant trees of Saikkonen/Soisalon-Soininen (P18/P19) [SSS08, SSS16]. The *prefetching* comprises *B-Trees*

Are Back (P20) [MBL25], the prefetching B⁺ trees of Chen et al. (P21/P22) [CGM01, CGM02], Khan (P23) [Kha11], Naderan-Tahan (P24) [NTSA16], Mahling (P25) [MWR25], as well as Zhang (P26/P27) [ZSZ⁺24, ZGH⁺25] and Kuehn (P28) [Kue23]. The *synchronization* carries the optimistic lock coupling of ART (P08) [LABN16], RCU (P29) [McK01], and hazard pointers (P30) [Mic04a]; the hardware-near *TU-Dresden line*, finally, Ungethüm (P31) [UHL⁺17], Schmidt (P32) [SKK⁺25], and VAMPIR (P33) [BH23]. The parallel *allocator corpus* (A01–A23) spans from the classics dlmalloc [Lea96], Hoard [BMBW00], Slab together with its magazines/Vmem extension [Bon94, BA01], and Michael’s lock-free allocation [Mic04b] through the multi-core-scalable mimalloc [LZdM19], jemalloc [Eva06], TCMalloc [GM05], snmalloc [LDPB19], scallocc [AKLS15], rpmalloc [Jan17], and lrmalloc [LR18] to the cache-aware/predictable CAMA [HBHR11], NUMAlloc [YZZ⁺23], StarMalloc [RFP24a], and PIM-malloc [VIA26]. Figure 3.2 condenses the most important reconstructed search structures into a gallery — per method its origin class, the characteristic structural idea and the axes at which it makes its main contribution; all thus appear as points in the common design space.

ART (P01) Trie adaptive nodes 4/16/48/256, byte-wise branching, path compression axes <i>T4, T3</i>	HOT (P02) Trie height-optimized dynamic span, compact SIMD nodes axes <i>T4, T0</i>	Masstree (P03) Trie + B ⁺ trie of B ⁺ -trees, 8-byte slices, B ⁺ -leaf nodes axes <i>T4, T10</i>	CoCo-Trie (P04) Succinct compact-collapsed trie, succinct-encoded paths axes <i>T3, T4</i>
START (P05) Trie self-tuning adaptive radix span axes <i>T0, T4</i>	B²-Tree (P06) Hybrid B ⁺ with binary inner nodes, cache-aware axes <i>T4, T5</i>	Wormhole (P07) Hybrid trie + hash + B ⁺ -leaf, $O(\log L)$ point query axes <i>T0, T2</i>	LOUDS (P09) Succinct unary degree sequence, bitmap-encoded trie axes <i>T9, T5</i>
SuRF (P10) Succinct succinct range filter over LOUDS axes <i>T14, T9</i>	CSS-Tree (P11) cache-sensitive pointerless implicit tree, arithmetic navigation axes <i>T4, T5</i>	CSB⁺-Tree (P12) cache-sensitive B ⁺ with <i>one</i> child-group pointer axes <i>T4, T5</i>	SwissTable Hash (control) SIMD control bytes; non-tree, in the same axis system axis <i>T0 + SIMD (system)</i>

Figure 3.2: Gallery of the most important reconstructed search structures: per method (with paper ID) its origin class, the characteristic structural idea and the axes at which it makes its main contribution. Trie, hybrid, succinct and cache-sensitive methods stand next to a non-tree hash control (SwissTable) — all as points in the same axis design space. The complete axis dissection is given in Table 3.2.

For each SOTA algorithm (*state of the art* — the published designs of Section 2.1) and each allocator a profile XML records the axis assignment (page, node, traversal, value_handle, concurrency, allocator, prefetch, telemetry, isa, layout, reclamation), a key-value signature, and an optional <expected_workload> tag that overrides the measurement heuristic (Table 3.7); all 30 SOTA profiles and all 10 allocator profiles are tagged. Three of the 33 corpus works deliberately have no standalone profile but enter only dissected: the optimistic lock coupling of ART [LABN16] as a concurrency organ, the succinct LOUDS representation [Jac89] as a memory-layout organ (e.g. in SuRF [ZLL⁺18]), and the profiling tool VAMPIR [BH23] as a measurement technology; the remaining 30 each carry their own profile XML.

Across the profiles these affinities distribute into 13× YCSB_C (read-heavy), 9× YCSB_A (zipf-distributed read and update), 6× YCSB_E (range scans), and 2× YCSB_B (95/5 read/update); PRT-ART

Table 3.7: SOTA algorithm profiles with workload affinity

Profile	Paper ref.	expected_workload
art	P01 (Leis 2013)	YCSB_C
hot	P02 (Binna 2018)	YCSB_C
masstree	P03 (Mao 2012)	YCSB_A
coco_trie	P04 (Boffa 2024)	YCSB_C
start	P05 (Fent 2020)	YCSB_C
b2tree	P06 (Schmeisser 2022)	YCSB_E
wormhole	P07 (Wu 2019)	YCSB_A
surf	P10 (Zhang 2018)	YCSB_A
css_tree	P11 (Rao/Ross 1999)	YCSB_C
csb_tree	P12 (Rao/Ross 2000)	YCSB_E
hankins	P13 (Hankins 2003)	YCSB_C
samuel	P14 (Samuel 2005)	YCSB_C
graefe	P15 (Graefe 2001)	YCSB_C
bender_treelayout	P16 (Bender 2002)	YCSB_C
bender_cacheoblivious	P17 (Bender 2005)	YCSB_C
saikkonen_multilevel	P18 (Saikkonen 2008)	YCSB_E
saikkonen_layoutinvariant	P19 (Saikkonen 2016)	YCSB_E
btreesareback	P20 (Mueller 2025)	YCSB_E
chen_prefetch	P21 (Chen 2001)	YCSB_A
chen_fractal	P22 (Chen 2002)	YCSB_A
khan_adaptive	P23 (Khan 2010)	YCSB_A
naderan_tahan	P24 (Naderan-Tahan 2016)	YCSB_A
mahling	P25 (Mahling 2025)	YCSB_E
zhang_fgcs	P26 (Zhang 2024)	YCSB_A
zhang_asplos	P27 (Zhang 2025)	YCSB_A
kuehn	P28 (Kuehn 2023)	YCSB_C
rcu	P29 (McKenney 2001)	YCSB_B
hazard_pointers	P30 (Michael 2004)	YCSB_B
ungethuem	P31 (Ungethuem 2017)	YCSB_C
to_stride	P32 (Schmidt 2025)	YCSB_C

expected_workload = YCSB workload class (A/B/C/E/F).

itself is tagged with YCSB_F (read-modify-write, for the density-tracker updates).

The core finding of this load-profile analysis is a systematic *bias*: each publication chooses the load profile under which its own design performs best. Static, read-only-oriented tries and filters (HOT [BZP⁺18], CoCo [BFTV24], B²-tree [SSL⁺21], SuRF [ZLL⁺18], CSS [RR99]) frame “build-once, 100 % positive lookups” and avoid updates; compressed tries and filters (CoCo, B²-tree, SuRF) make the negative-query share the primary axis, because they abort early on misses; zipfian-centred works (B-Trees Are Back [MBL25], Mahling [MWR25], Kuehn [Kue23]) favour cache-aware reordering layouts; write- and concurrency-heavy works (ART [LKN13], Masstree [MKM12], Wormhole [WNJ19], OLC-ART [LABN16], RCU [McK01]) conversely compete only where static structures fail; and prefetch- and hardware-near works (CSB⁺ [RR00], Chen [CGM01], Schmidt [SKK⁺25]) measure cache and TLB rather than algorithm logic, via node size and prefetch. Fourteen of the publications contribute no distinct load profile to the catalogue — partly survey, theory, and hardware works without an index load profile of their own, partly works whose custom microbenchmarks are subsumed by profiles already catalogued. Only when *all* load profiles run against *all* living-being binaries (and the container hulls, Section 3.3.2) — the workload as a dynamic axis (Chapter 5) — can the “winner only in its self-chosen home profile” be

made visible and bias-reduced comparability be gained.

Finally, the quality criteria of scientific measurement (cf. the definitional classes in Section 3.4), too, appear as concrete technologies. *Hardware performance counters* (PMC, via `perf`) [dM10] are nearly universal and deliver cache-miss rates, dTLB misses, branch misses, and IPC/CPI — the majority of the works P01–P28 report such counters —; *profilers* such as OTF2/VAMPIR [BH23] and VTune [Rei05] collect trace-based runtime profiles, *simulators* such as `gem5` [BBB⁺11] provide hardware-accurate estimates where real counters are absent (so P25 Mahling [MWR25], P27 Zhang [ZGH⁺25]), and the *HdrHistogram* [Ten12] collects the latency percentiles (p50/p95/p99). On the collection the evaluation builds: instead of arithmetic means, median and percentiles are reported, differences tested for significance with the distribution-free *Mann-Whitney U test* [MW47] (with Holm correction of the family-wise error rate [Hol79]) and quantified via the effect measure *Cliff's δ* [Cli93], following the recommendations for scientific benchmarking [HB15]. A methodological line of its own is the hardware-near measurement methodology of TU Dresden (Habich group): Ungethüm et al. [UHL⁺17] systematize the hardware optimizations for database machines, Schmidt et al. [SKK⁺25] measure stride versus sequential accesses on HBM with dTLB and L2/L3 counters, and VAMPIR [BH23] profiles non-functional properties of heterogeneous memory. From all these technologies follows the set of measurement categories — cache-line utilization, cache misses (L1/L2/L3), dTLB misses, memory footprint, branch misses, IPC/CPI, latency (mean and percentiles p50–p999), throughput, energy, and fill-buffer occupancy —, whose time- and observer-based subset the present apparatus already resolves (wall clock, HdrHistogram in the downstream evaluation, per-axis observers, the statistics triad of median/percentiles, significance test, and effect measure, the two-phase operation loop, and the conformity gate against `std::map`), while the counter-based PMC categories remain bound to hardware-counter access; the complete measurement architecture is developed by the measurement system (Section 3.4) and Chapter 5.

3.3 Solution Concept: the Anatomy Model

From the dialectical adoption of the state of the art emerges the solution concept of this thesis; this section develops it as the *anatomy model* and in doing so defines the terminology and conceptual special constructs that carry the following sections.

3.3.1 Axis Composition and the Living-Being Metaphor

The central idea is to treat an algorithm not as a monolith but as a *composition of orthogonal axes* — each axis a partial decision of a cache-aware index (how pages are arranged, how nodes branch, how memory is allocated). A concrete algorithm is then a point in the Cartesian product of all axes, a state-of-the-art design an explicit configuration. The decisive property is *modular exchangeability*: a new axis variant can be studied in isolation — its effect measured per axis *and* for the whole algorithm — without reimplementing the others; this is precisely what solves the separability problem, since the contribution of each constituent becomes measurable in isolation while the other axes are held fixed. This decomposition rests on one observation and is carried by a metaphor. First, search algorithms share, in the large, the same *tree anatomy* in different expressions — from balanced search trees through tries and radix trees to custom constructs; even a sorted key–value list is, strictly speaking, a single leaf of a tree whose values point onward to data leaves. The tree structure is therefore universal across search-algorithm architectures

and forms the canvas on which the axes are placed; non-tree-like search methods — such as the hash counter-check of Section 3.2 — remain the same genus with a restricted profile, for a separate genus would arise only if the base axis set differed fundamentally. Second, a vivid metaphor treats a concrete algorithm as a *living being* whose *organs* are the axes — and the measurement system as its medical diagnostics (Figure 3.3). This metaphor is owed to the author’s background in biomedical engineering and is meant to ease the analysis through vividness.

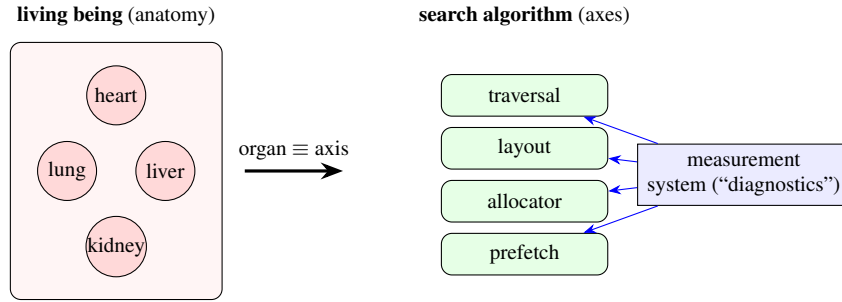


Figure 3.3: The anatomy bridge: just as a living being consists of interacting organs, a search algorithm is a composition of axis-organs; the measurement system takes the role of medical diagnostics and measures each organ in isolation.

The permutation of axes then resembles a genetic experiment: exchanging organs between living beings produces new composite algorithms with hitherto unknown cache-line behaviour. The organs themselves are won from the state of the art — the design building blocks of the known methods (Section 2.1) are decomposed into orthogonal axes and collected in a shared axis matrix, so that each axis harvests the contributions of the entire corpus (Figure 3.4).

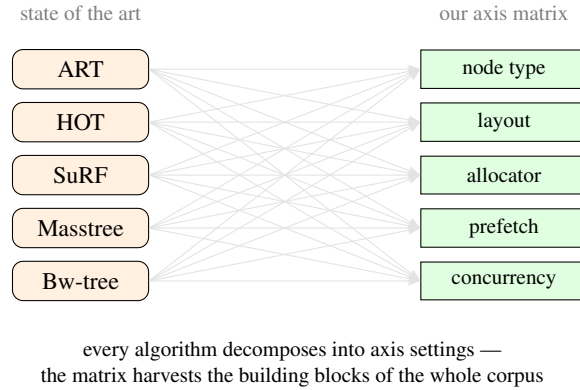


Figure 3.4: Synthesis state of the art → axes: the design building blocks of the known methods are decomposed into orthogonal axes; each axis collects the contributions of the entire corpus — the inverse of Figure 3.1 (there an algorithm is a point, here an axis is a collecting basin).

The anatomy thus won is more than the enumeration of the organs: it is their *wiring*. Each organ exports a generalized interface through which the others use its services without knowing its concrete realization — thus the allocation organ provides a shared allocation interface that node layout, value placement, and serialization use for their memory needs (Figure 3.5). In software-engineering terms this is the *feature interaction* of feature-oriented software product lines [ABKS13, BO92]: an axis corresponds to a feature, a living being to a feature composition, a living-being type (the genus subclass, Section 3.3.2) to a static

feature-set definition, a genus to a (search-)algorithm interface category, and the anatomy to the wiring between the features. Which axis uses which is shown by the inter-organ usage graph (Figure 3.6).

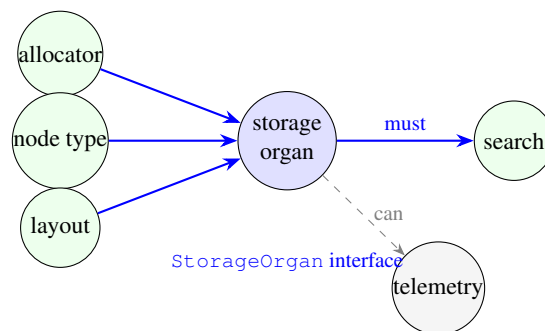


Figure 3.5: Axis = organ, anatomy = wiring: the storage organ (composed of node type, layout and allocator) exports a generalized `StorageOrgan` interface over which the search runs (solid, *mandatory*); further organs such as telemetry *can* use it via optional observer hooks (dashed, *optional*), but need not.

Composing these pairwise couplings across all organs yields a directed inter-organ usage graph.

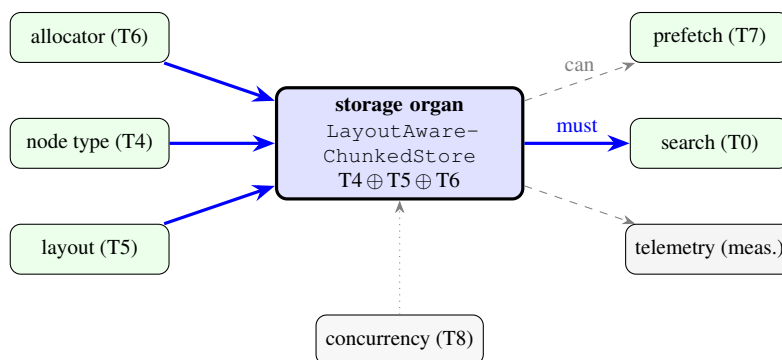


Figure 3.6: Inter-organ usage graph (“which axis uses which”): there is exactly *one* canonical hub — the storage organ `LayoutAwareChunkedStore`, which composes node type (T4), layout (T5) and allocator (T6). *Mandatorily* (solid) the store consumes these three axes, and the search (T0) runs over it. All remaining couplings are *optional* (dashed): prefetch (T7) and the telemetry (measurement instrumentation, Section 3.4.1) read the store only via `requires`-gated observer hooks — an axis *can* use foreign interfaces, but *need* not. The coupling of concurrency (T8) to the store is not (yet) wired in the code (dotted).

3.3.2 The Three-Level Model and the Three Genera

These organs are embedded in *one* architecture — a single, self-contained hierarchy without parallel trees. At its root stands the `ExecutionEngine` as the common, measurable abstraction over everything measurable; beneath it three levels can be distinguished: the *genus* as the outward-visible algorithm interface (the *probe dock*), beneath it the *genus subclass* (also: *tier subclass*) with a fixed axis set, beneath that the *organs* themselves (Figure 3.7). “Living being” and `SearchAlgorithm` denote the same entity — metaphor and technical term —; the anatomy is not a second model but the *body* of this living being: the fixed composition of its eighteen axis-organs.

At the topmost level there stand exactly three genera. The `Map` genus — the key-value-oriented probe dock

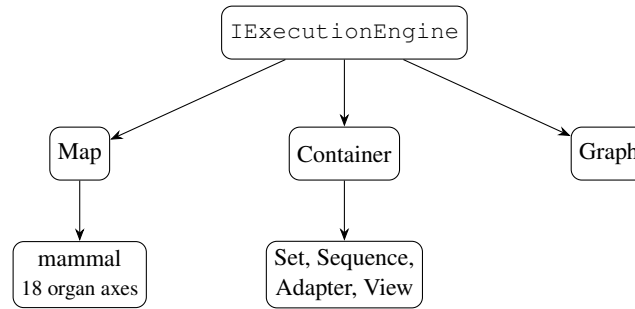


Figure 3.7: The three-level model: genus (level 1, outer interface) → tier subclass (level 2, fixed axis set) → organs/axes (level 3). The `Map` genus carries the “mammal” with the eighteen organ axes through its sole tier subclass `SearchAlgorithm`; the `Container` genus the subclasses `Set/Sequence/Adapter/View`.

and the focus of this thesis — carries as its sole subclass (`SearchAlgorithm`) the full eighteen-axis “mammal”; the `Container` genus (keyless storage) fans out into the subclasses `Set`, `Sequence`, `Adapter`, and `View`; the `Graph` genus carries its own, different organ set (Figure 3.8). The kingdom “Animalia” ($\equiv$ `IAnatomyBase`) shown there as the root is the anatomy-bearing view immediately beneath the measurable root `IExecutionEngine` of Figure 3.7; the code separation of the two interfaces is deepened in Chapter 4. Non-tree-like methods thus require no genus of their own but occupy a restricted axis subset of the same genus.

3.3.3 Design Patterns and Compile-Time Patterns

The actual contribution thus lies less in a single structure than in a *tool*: instead of solving the comparison problem directly, this thesis first builds the “shovel” — the permutation and measurement apparatus — with which arbitrary axis combinations can be generated and fairly measured. Scientifically this apparatus situates itself in the software engineering of *composition*: invasive software composition from gray-box components [Aßm03], generative programming [CE00], and feature-oriented software product lines [ABKS13, BO92]. The axis concept itself is adopted from prior work by the author (the Comdare / BEP Venture UG brand, originally for the deduplication optimization of the UltiHash database); the contribution of this thesis is its transfer to cache-aware search structures and its design-pattern-driven, measurement-oriented realization. In software-engineering terms the apparatus is a composition of named design patterns and two new compile-time patterns (Figure 3.9).

The classical patterns are not ornament but the wiring of the apparatus: *Strategy* encapsulates each of the eighteen organ axes as an interchangeable algorithm, *Abstract Factory*, *Composite*, and *Iterator* build and traverse the permutation tree, *Adapter* binds foreign code behind the stable engine interface, and *Observer* as well as *Decorator* (non-intrusive measurement hulls) collect the measurements, while a *Memento* (copy-on-write) secures the state between warm-up and measurement [GHJV94]; not used, by contrast, are *Singleton*, *Flyweight*, and *Bridge*, while the *Command* pattern appears solely as the command channel of the hybrid mode (Section 3.6.2). Static polymorphism is provided by the *Curiously Recurring Template Pattern* (CRTP) together with *Template-Method* concept checks; concept-checked `if constexpr` (`resolve_baustein`) selects per axis between the probe and the cache-engine standard, and *policy-based design* captures the composition as a tuple of axis policies [Ale01]. C++23 modules structure the translation units but by themselves do not yet guarantee portable ABI stability [Xu24] — only the C-stable

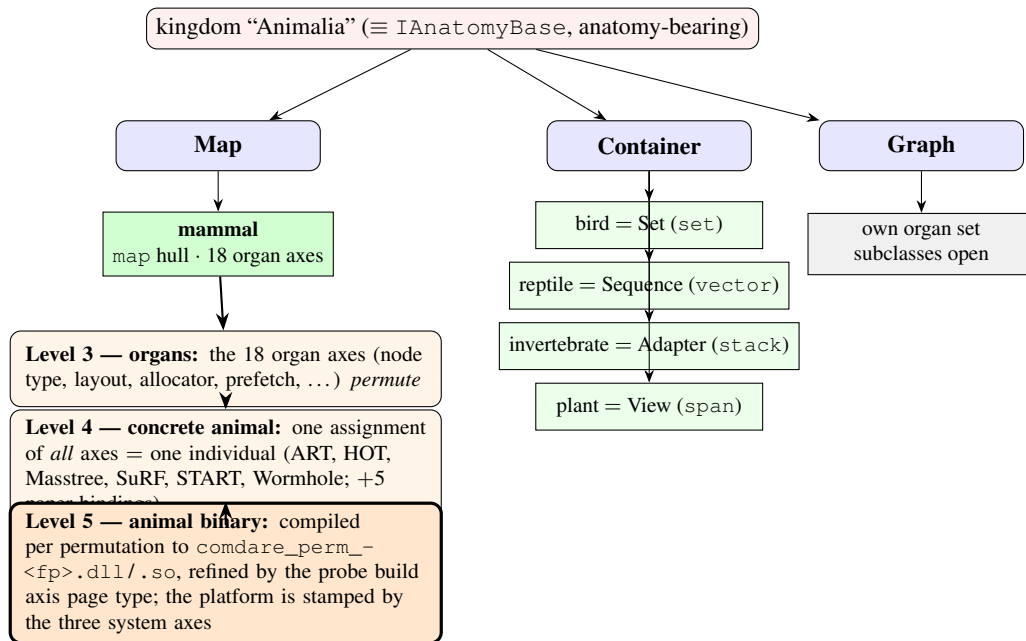


Figure 3.8: The genus/tier hierarchy across five levels. Map, Container and Graph are the three *genera* (level 1, outer interface). Each genus hosts its *own* tier subclasses (level 2): Map exclusively the “mammal” (tier subclass `SearchAlgorithm`, full eighteen-axis anatomy), Container every *non-mammal* subclass (bird/reptile/invertebrate/plant = `Set/Sequence/Adapter/View`), Graph still-open ones. The mammal branch is drawn out here down to the individual: the eighteen axis organs (level 3) permute into a concrete axis composition = one concrete animal (level 4, e.g. ART, HOT, Masstree), which is compiled per permutation into a standalone animal binary (level 5, additionally refined by the probe build axis page type and platform-stamped by the three system axes, Section 3.4.1). *Every animal binary is thus an animal of its own*; the hierarchy reaches from the genus down to the single compiled permutation. Cf. the condensed model in Figure 3.7.

module boundary establishes that. Beyond the established patterns, the thesis names two constructions for which a web search found no established term: the *lazy-sparse cartesian materialization*, which counts the combinatorially exploding variant product space of all axis catalogs only arithmetically and materializes each path individually on demand in mixed radix, rather than blowing up the compiler with the full type tree; and the *type-driven source emission*, which extracts fully qualified type names from a compile-time axis composition, renders them to C++ source, and loads each permutation as a standalone module across the ABI boundary — thus the probe PRT-ART is compiled, by metaprogramming, in matrix layers against the cache engine.

How this *one* architecture becomes visible in the code through an ABI-stable hull is deepened in Chapter 4 (Section 4.3.1); before that, the following subsection orders the apparatus as a whole: in which layers it is organized from experiment definition to evaluation, and how it is completed.

3.3.4 Five Interface Layers: E4 down to E0

The anatomy model orders the *measured*: what a living being is and which organs it consists of. Orthogonal to that, the apparatus that generates and measures such living beings needs a second ordering principle — a view of the *experiment pipeline* that transforms a declared axis assignment into measured results.

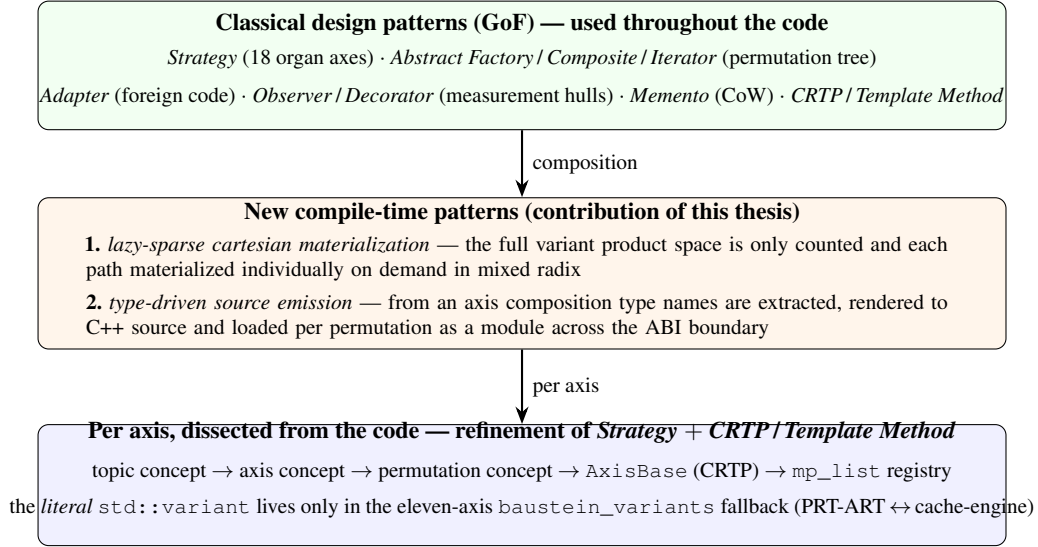


Figure 3.9: Design patterns as building blocks of the architecture: above the classical GoF patterns used throughout the code with their respective role, in the middle the two new compile-time patterns named in this thesis, below the code-dissected *three-stage concept chain* per axis (topic concept → axis concept → cache-engine permutation concept → CRTP base → *mp_list* registry) as a refinement of *Strategy*/*CRTP* — where the *literal* `std::variant` appears not here but only in the eleven-axis `baustein_variants` fallback model. Their composition forms the design-pattern-driven, measurement-oriented architecture (Chapter 4).

This pipeline is organized into five layers *E4* down to *E0*, complemented by the cross-cutting concern *M* (Figure 3.10). The numbering runs from the most abstract to the most concrete level. *E4* is the definition and evaluation layer — the level a user of the framework typically works with: experiments are described declaratively as XML profiles, and at the end of the pipeline the same layer receives the measurement results as a return channel and prepares them into tables and diagrams. *E3* is the permutation B^+ -tree: it unfolds the declared assignment into the experiment tree, where the compile-static axes determine the identity of the binary to be built and the dynamic axes span the runtime dimension. *E2* are the tier binaries themselves: per permutation one standalone binary loaded across the ABI-stable interface (level 5 of the genus hierarchy, Figure 3.8). *E1* is the runtime configuration — the most precise detail layer, in which the dynamic sub-axis parameters are varied on the already-loaded binary. *E0*, finally, is the infrastructure (build, tooling, and platform substrate); it is deliberately completed last. The measurement system permeates all layers as the cross-cutting concern *M* without being one itself — this special role is deepened in Section 3.4.

Carrying this layering is its completion principle: each layer is completed *individually*, with an explicitly *frozen interface contract* to its respective neighbouring level — the *E4*→*E3* boundary, say, freezes how an XML definition is translated into axis levels, the *E3*→*E2* boundary how a tree path becomes a build order, and the *E2*→*E1* boundary which runtime knobs a binary accepts across the ABI. The test question per boundary is always: does the layer pass its definition on to the next *losslessly*? A system whose product space rules out any complete materialization (Section 3.4.5) is too large for simultaneous changes across several levels; only the frozen contracts make each level individually acceptable before the next one begins. This principle corresponds to the matrix programming style of the apparatus (Section 3.3.3): just as the probe is compiled in matrix layers against the cache engine, the apparatus itself is developed

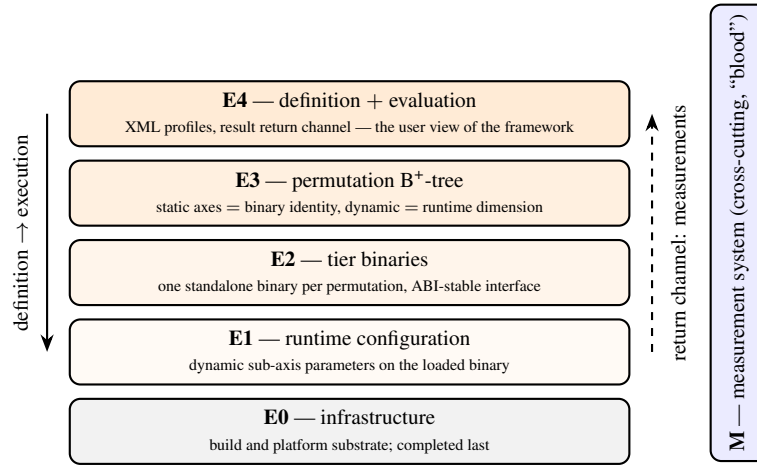


Figure 3.10: The five interface layers of the experiment pipeline: from the declarative definition and evaluation layer E4 (user view) through the permutation B⁺-tree (E3) and the tier binaries (E2) to the runtime configuration (E1); the infrastructure (E0) carries everything and is completed last. The return channel leads the measurements back to the evaluation in E4; the measurement system permeates all layers as the cross-cutting concern M (“blood”, Section 3.4). Each layer is completed individually, with a frozen interface contract to its neighbouring level.

in interface layers against frozen contracts; how each layer is thereby tested individually — against a substitute of its neighbouring level — is shown in Section 4.4.

The E-layers are expressly to be distinguished from the three-level model (Section 3.3.2), however similar both may look at first glance: the three-level model is the *anatomy view* — it orders *one* living being from the genus through the subclass to the organs and thus describes the structure of the measured. The E-layers are the *pipeline view* — they order the path of an experiment from the definition through tree, binary, and runtime back to the evaluation and thus describe the process of measuring. Both views are orthogonal to each other: the same living being that anatomically decomposes into genus, subclass, and organs passes through the layers E3 to E1 as a tier binary. Now that it is settled what is composed and in which layers the apparatus processes it, the following section turns to the question of how the living beings thus composed can be measured fairly.

3.4 The Measurement System

To compare structurally different search algorithms *fairly*, the thesis maps them all onto a single, well-understood interface: the C++ contract of the ordered map `std::map<Key, Value>` with point query (`find`), insertion/update (`insert`, `emplace`), deletion (`erase`), and order-based navigation (`lower_bound/upper_bound`, range iteration) [ISO24]. The axes describe the *inner* behaviour behind this fixed interface, so that the comparison of two conformant `std::map` implementations becomes the central measurement act; in parallel the sequence hull `std::vector` (indexed access, resizing, contiguous storage) serves as a container comparison case, to check whether an axis composite also carries the container and not only the search semantics. Behind the fixed outer interface the system distinguishes several *levels of dynamics* that determine *when* an axis decision is made: the topmost split of compile time versus run time; the living-being type, which occupies only a subset of the axes; the axis algorithms compiled into a binary at compile time (static recombination); the dynamic sub-axes,

whose runtime parameters (prefetch distance, batch size, inline threshold, pool budget) are varied without recompilation; and the static sub-axes as a compile-time choice among partial algorithms of differing complexity. The idea of regarding data structures not individually but as a design space shapes Idreos’ *Periodic Table of Data Structures* [IZA⁺18] and the *Data Calculator* [IZH⁺18]: structures arise from the combination of few design primitives whose costs can be synthesized — which coincides directly with the axis dissection of this thesis (Section 3.2), in which the classical families are mere reading groupings of orthogonal axis assignments. Against the static design-space enumeration of the Data Calculator and classical *policy-based design* [Ale01], however, this taxonomy expressly carries compile- and runtime-variable axis levels together; a web search found no established name for this grading (cf. Section 3.3.3).

These building blocks are only as meaningful as their measurement. Scientific measurement is subject to established quality criteria [HB15]: *reproducibility* (fixed seeds, deterministic order, documented configuration, so that runs are methodically repeatable and statistically comparable), *validity* (internal: causally isolated effects; construct: the metric actually measures the intended concept; external: transferability to other platforms and loads), *fairness* (identical keys, workloads, and warm-up; missing capabilities reported as *N/A* rather than covertly emulated), and *dispersion* as part of the statement (percentiles instead of arithmetic means, with significance testing; the distributional view is provided by HDR histograms in the downstream evaluation). Reported percentiles follow the nearest-rank method: the rank $R(q, n) = \lceil q \cdot n \rceil$ is one-based, the corresponding array index $k = R - 1$ zero-based; the median is the special case $q = 0.5$ of the same method and, for even n , yields the lower middle value, not the mean of the two middle values; p95, p99, and p999 are the same ranks with $q = 0.95$, $q = 0.99$, and $q = 0.999$. On these criteria the measurement patterns used here build: a *two-phase operation loop* separates a discarded warm-up from the actual measurement; collection is *hybrid* — isolated axis algorithms directly against one another, a composed living being centrally over an ABI-stable per-axis observer interface —; and a *conformance gate* checks the `std::map` contract before each measurement, so that only comparable things are compared. These concepts form the *definitional classes* of measurement; their technical *instances* — hardware performance counters, OTF2/VAMPIR profiling, the TU-Dresden hardware measurement methodology — have already been introduced as concrete technologies in Section 3.2.

3.4.1 The Triad: Organ, System, and Measurement Axes

Within these definitional classes the measurement system answers the category question deliberately left open in the introduction (Section 1.3): the axes of the design space separate along the three concerns named there — the measured, the build and target platform, the measuring itself — into *three* axis categories, developed in the following as standalone *realms* with one class root each, one offer catalog each, and one stamp line each (Section 3.4.4). This triad is the second shape of the separability problem: not only the design decisions among one another, but also the measured, the platform, and the measurement instrument must stay separated so that a measured effect can be attributed unambiguously.

The *organ axes* are the *measured*: organs of the living being, part of the genera, permutable — every assignment changes the identity of the tier binary. Canonically there are *eighteen* organ main axes in fixed slot order T0–T17 (Table 3.2); the T numbers denote the slots of the composition, not an implementation numbering. The earlier nineteen-axis numbering (interim state of July 2026) is a historical precursor of this order: from it, telemetry and the ISA have moved into the system sphere, and the persistence target

has been added as the eighteenth organ axis.

The *system axes* are the concerns of the build and target platform. Canonically there are *exactly three* main axes in fixed order: the target ISA (`target_isa`), the operating system (`operating_system`), and the extension hardware (`external_utils`); the latter is exclusively the collecting hub of the hardware meta axes (SIMD/vector extensions, GPU/FPGA/NPU, external accelerators). In the anatomy metaphor the system axes are not another organ but the *blood*: they permeate the whole body, are always present as soon as measurement is on, and belong to no genus. From this follow three invariants: system axes are not permutable (they are non-interchangeable, central system constituents); they do not stand in the permutation tree; and they never touch the binary identity — no measurement instrument may change *what* is built. Conversely, the effects of the organ axes must never be blended with the effects of the platform and the measurement instruments. Build control quantities such as optimization level and SIMD extension therefore multiply only the *build* matrix — they appear in the build version tag of the binary (Section 3.4.4) —, never the organ permutation space.

The *measurement axes*, finally, are the concerns of measuring itself — the measurement instruments. They form the third realm, separate from both others: the load framework (`load_framework`), which carries the workload conventions of Section 2.3 into the apparatus, is its first main axis; the *measurement tooling* spans the three granularities of benchmarking (Section 3.4.5); and the measurement categories (Section 3.2) — differentiated in the apparatus into sixteen categories — are a *sub-axis* of this tooling: they manifest as columns of the result tables, not as main axes of their own. The run methodology of a run (debug, measurement, release, or compare execution) likewise remains a sub-axis and never becomes part of a binary identity; like the system axes, the measurement axes are strictly binary-id-neutral. Within the sixteen categories there is a *regime bipartition*: the time- and observer-based regime (runtime clocks and observer hulls; nine categories, among them latency, throughput and memory footprint) and the counter-based PMC regime (hardware performance counters; seven categories, among them cache, dTLB and branch misses as well as IPC/CPI). The regime is a property of the measurement axis, not of the measured organ: the same organ axis can be illuminated by both regimes. The observer-based subset is already resolved by the apparatus, the PMC subset remains bound to hardware counter access (Section 3.2).

Conceptually, a double root cut with *two* realm roots follows — instead of the *one* common root of all non-organ axes that earlier versions of this thesis still envisioned: the system realm is rooted at the `CacheEngineBuilder`, which knows the platform and builds the tier binaries; the measurement realm is rooted at the experiment *planner*, which resolves the experiments from the XML description (Section 3.4.5). Each of the three categories thus carries its own abstract class root and its own offer catalog; the code shape of these roots is developed in Section 4.5. Also binding is the canonical overall order of all main axes: first the measurement tooling, then the three system axes in their fixed order, then the organ slots T0–T17.

This order is the result of datable *relocations*, not a mere renaming. An earlier five-item list of the system axes carried, besides the three main axes of today, also the compiler, the scheduling, and the load framework: the compiler has moved as the sub-axis group of the build toolchain into the outer complex bracket (Section 3.4.2), the scheduling as a sub-axis onto the wrapper of the target ISA, and the load framework has left the system world *without replacement* towards the measurement realm — the system axes are thereby slimmed down to their thematically fitting functions. From the organ numbering,

conversely, the ISA — today the complex axis `target_isa` (Section 3.4.2) — and telemetry have moved into the system sphere; telemetry *split in two*, since counter instrumentation must be built in for measuring: as a genuine sweep sub-axis of the measurement tooling in the planner it generates a genuine compile-time main quantity in the builder.

This last pattern is generic and carries a name of its own: the *two-way-split main axis* — an axis that runs across the runtime of the preceding stage *and* the compile time of the following stage, along the chain planner → builder → tier binary; the sub-axis release of the preceding stage becomes the main-axis assumption of the following stage. The load framework is the first instance (the planner chooses it at runtime, the builder carries it compiled in), telemetry the second, and the hardware detection (Section 3.4.3) the third.

All three axis realms are bound by a methodological principle, the *honest-0 doctrine* (honest zero or honest emptiness): where a value cannot truly be collected — an organ axis without a real runtime parameter, a counter without hardware access, a measurement source that does not exist on the platform —, the system reports *honestly invalid or empty* instead of a plausible phantom value. Better a measurement marked invalid than an invented effect: a phantom value would systematically poison the heuristic derivation (Section 3.6), whereas an honest gap stays visible and can be closed deliberately. The principle holds throughout — from the axis wiring (only axes with a genuinely measurable runtime knob report runtime effects; the rest count honestly as compile-static) to the availability reporting of the counter sources.

What these measurement patterns concretely collect are three opposing components — *latency, throughput and memory footprint* —, against which every axis permutation spans as a trade-off profile: the same permutation that is a point in the design space in Figure 3.1 appears here as a point inside the trade-off triangle (Figure 3.11). A fourth component, the one actually central for cache-aware search structures — the cache-aware behaviour at the node, data and index level via hardware performance counters — is not yet collected in the current implementation state (PMC capture is off by default; even with the switch enabled, L2 misses and coherence invalidations remain structurally invalid for lack of a portable counter, and the last-level counter can additionally fail to open on individual AMD platforms) and therefore remains marked as a pending fourth measurement component.

3.4.2 Complex Axes: the Target ISA as a Fixed Recombination

To the outside, the three system main axes behave like *one*: a *complex axis* is a bracket over a fixed recombination of several main axes — all members remain main axes but appear together as one (bundled as a command bracket in the sense of the *Command* pattern). The *outer* complex bracket spans `target_isa × operating_system × external_utils` and carries as its sub-axis group the build toolchain (compiler, optimization level, atomics capability) — the relocation target of the former compiler system axis; as a bracket it is itself *no* member of the canonical order. The target ISA is *in itself* a second, inner complex axis: a fixed recombination of the identity members RAM frequency, CAS latency, and CPU fabrication, recursively wrapped by the outer bracket; on its wrapper hang the agreed sub-system axes (scheduling, NUMA node, memory page size). The members are *declared* per machine — in the same user XML that also displays the experiment (Section 3.4.5) — and determined per operating system and ISA; on an exact property match the same axis is reusable across formally different machine types, because the host name serves only the instance lookup — the class identity is the property tuple. In the identity stamp (Section 3.4.4) a complex axis occupies exactly *one* field of its type line, no bracket per

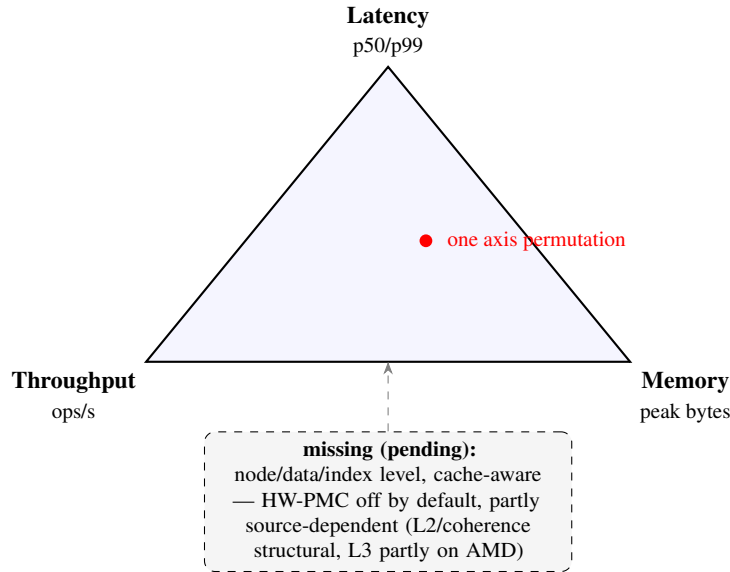


Figure 3.11: The measurement trade-off triangle: every axis permutation spans against the three opposing measurement components latency, throughput and memory footprint, and is a point inside it (cf. the design space in Figure 3.1, where the same permutation is a point). The component actually central for cache-aware search structures — the behaviour at the node, data and index level via hardware performance counters — is not yet collected in the current implementation state (dashed).

member; the operating-system distribution is a stamp variable there, not a fourth expression. The CAS latency is, in the current state, a purely *declared* member (provenance level “declared, not measured”, Section 3.4.3); the thesis reports this instead of feigning a measured value.

3.4.3 Hardware Detection as a Two-Way-Split Measurement Axis

So that the declared platform members do not become silently aging constants, the concept introduces hardware detection as a further two-way-split main axis — and specifically as a *measurement* axis: it measures properties of the hardware but changes neither system nor organ axes. *Level 1* lies with the planner: at *its* runtime it coarsely detects the cell of ISA vendor family and operating system and selects by metaprogramming which fine-grained detection software, including operating-system accesses, is compiled into the `CacheEngineBuilder` — a compile-time cell selection, not a runtime switch across operating systems. The selection matrix is *total* over all declared ISA complexes and all documented operating systems: unimplemented cells exist as an honest declaration-only specialization, and a new catalog entry without a cell breaks the compilation instead of leaving a silent gap. *Level 2* lies with the builder: with the compiled-in detection it collects, at its runtime, the actual properties of the machine and thereby sets the compile-time stamps of the tier binaries correctly. The stamp of this measurement axis itself carries the vendor detection family and the operating system — as the device segment of the measurement stamp line (Section 3.4.4) —, never a measured value.

Every collection result carries a *provenance* level together with its collection time, in descending order of trust: *configured-measured* (privileged system query), *SPD JEDEC base value* (read unprivileged from the module memory, deliberately only the JEDEC nominal rate; overclocking profiles appear as a separate offer field, never as the actual value), and *declared-not-measured* (user XML). A lower level may displace

a higher one only with a marker that travels along — otherwise a declared nominal rate would stand in the result table while the machine possibly runs faster: worse than honestly declared. What is not collected receives an explicit n/a token, never a zero in the value column (honest-0 doctrine, Section 3.4.1); four error classes separate missing, unreadable, corrupt, and unknown-format sources, and a corrupt source never degrades silently to the next level. The gain of this axis is therefore not a better single value but the *automated resubmission*: a tool result collected once and then frozen — this is how the previously recorded RAM nominal rate of 4800 MT/s arose from a manual `dmidecode` query — is replaced by a collection that takes place anew on every run and carries its trust level along. In the implementation state of this thesis, the provenance vocabulary including the SPD parser and the error domain as well as the collection chain and the platform selection matrix exist for this axis as code (Chapter 4).

3.4.4 Identity Stamps: One Line per Realm

The triad becomes readable on the tier binary itself. Its *permutation* identity is the `binary_id`: the path of eighteen `axis=value` segments in canonical slot order — one segment per organ axis, and *only* organ axes. In addition, every tier binary carries compiled-in *stamp lines*, exactly one per axis type. The *organ line* records, for each of the eighteen main axes, the chosen algorithm together with its algorithm version. The *system line* holds the three system main axes; as their version they carry their *build* version — the ordered suffix tag of toolchain, optimization level, extension, and further build control quantities, in which an empty member simply produces no segment —, because a platform axis has no algorithm version. The *measurement line* leads with the load framework as its first segment and then the set of compiled-in measurement toolings; an *empty* measurement line is thereby an honest statement — no measurement tooling compiled in. A complex axis occupies exactly one field in its line (Section 3.4.2). A fourth *realm* line for the mixing strategy of a probe, by contrast, cannot exist: a probe mixes against the organ axes and is therefore already described by the organ line. The mixing becomes readable there in the algorithm name, which differing mixing constellations extend hierarchically (pattern `pvt-art.memory.abc@1.0.0.c`); the version tag itself no longer marks such a mixing — its suffix `e` has denoted an efficiency-core algorithm since flag grammar v2, not a specimen origin. It is on exactly this name level that the hybrid and full-join states remain distinguishable at the compile; byte-different mixed builds therefore never carry organ segments of the same name. What closes the stamp is consequently not a mixing line but the *fingerprint line*: a SHA512 whose preimage extends the three realm lines by a format identifier and injected build and measurement members — the toolchain segment, the enabled sets of the build axes, the sub-axis value set, the overlay source hash, and the actually effective measurement-gate state of the translation unit (Chapter 4) — and which condenses the entire stamp identity into *one* comparable value on which the recognition of an already built permutation hangs. Measured *values* — say, a collected RAM rate — never appear in stamp or `binary_id`: what is stamped is the compile-time platform *class*, never a measured value (Section 3.4.3).

3.4.5 Subsystems and the Chain from the XML to the Tier Binary

The apparatus is organized as *one* main channel: *one* user XML is the only source of the entire process. It *displays* which experiments, axes, settings, and procedures are to be carried out, while per realm a *registry* catalog holds the *offer* of what exists — the declarative configuration model of Section 2.4, modeled

on the Java POM: a declarative manifest resolved against a repository; an unregistered reference is a hard validation error, not a silent omission. The experiment *planner* resolves the XML against the three offer catalogs and builds, per target platform and measurement task, *one* `CacheEngineBuilder` into which the chosen measurement axes are compiled; this builder orchestrates with them the build of the tier binaries and stamps them with its platform (Section 3.4.4); the tier binary, finally, carries the organ permutation. Each stage of the chain thus feeds from its kind-specific offer — measurement catalog → planner, system catalog → builder, organ catalog → tier binary —, and the tier binary stays readable in two strictly separated layers: the hardware and environment layer of the system axes and the application logic layer of the organ axes; as little as one equates an operating system with the application front end, as little do these two layers merge in the binary.

Building on these measurement patterns and this chain, the measurement system separates four subsystems and tests in three stages that feed the three mandatory measurement series; it reduces the permutation space deliberately. Four formally separated subsystems with distinct responsibilities form the model: the `messung_driver` (the evaluation orchestrator that runs the pipeline per mandatory measurement series A/B/C), the `CacheEngineBuilder` (an autonomous platform-measurement system that executes the seven-phase pipeline of Section 4.3.2 via its `ExperimentDriver`), the `CacheEngine` (the axis building-block library with the default implementations per axis), and the probe PRT-ART (which supplies some axis-organs itself and consumes cache-engine services for the rest); the planner role of the chain and the builder role today share one binary in two roles (Chapter 4). The `ExecutionEngine` is not a fifth subsystem but the shared measurable root (the `IExecutionEngine` interface) through which these four are connected: beneath it, the probe and every engine register as `IExecutionEngine`, and the builder enumerates the permutation space per registered engine. Defining is the *bidirectional* cache-engine ↔ probe relationship: the cache engine instantiates the probe per permutation configuration, and the probe calls cache-engine services (telemetry, prefetch, heuristics) at runtime during its own lookups. This symmetry carries the *multi-probe* capability, with which measurement series A compares the probe against many SOTA adapters in a single pipeline (Figure 3.12).

The builder orchestrates a *three-stage evaluation*: **Stage 1** (cache-engine only) uses the default variant per axis, **Stage 2** (probe only) replaces the overridden axes by the probe variants and otherwise keeps the defaults, and **Stage 3** (full join, non-redundant) combines default and all probe variants. Each stage can produce up to thousands of non-redundant permutation binaries that are loaded at runtime across the C-stable module boundary as C++23 modules; the present measurement run materializes a sparse, capped subset of these (Figure 3.13).

These stages do *not* map one-to-one onto the three mandatory measurement series but feed them: the default binaries of Stage 1 (including the reconstructed SOTA profiles) and the probe binaries of Stage 2 *jointly* form the comparison series A; the non-redundant full join of Stage 3 is the basis of the variation series B; and the merge/regression series C (old vs. new) is bound to no single stage but runs build-spanning as an old-versus-new comparison — over successive states of the same configuration as well as over concrete merge points (Table 3.8).

The three measurement series — all executed by the `messung_driver` via the `ExperimentDriver` library, configured either as three fixed series or via an external `messreihen.xml` — are thus: *series A* (PRT-ART vs. state of the art) with the modes `A_defined` (fast verification against the 8 Rank-1 profiles) and `A_full` (all 30 SOTA profiles; the standard, since only the full run delivers the bias-reduced measurement

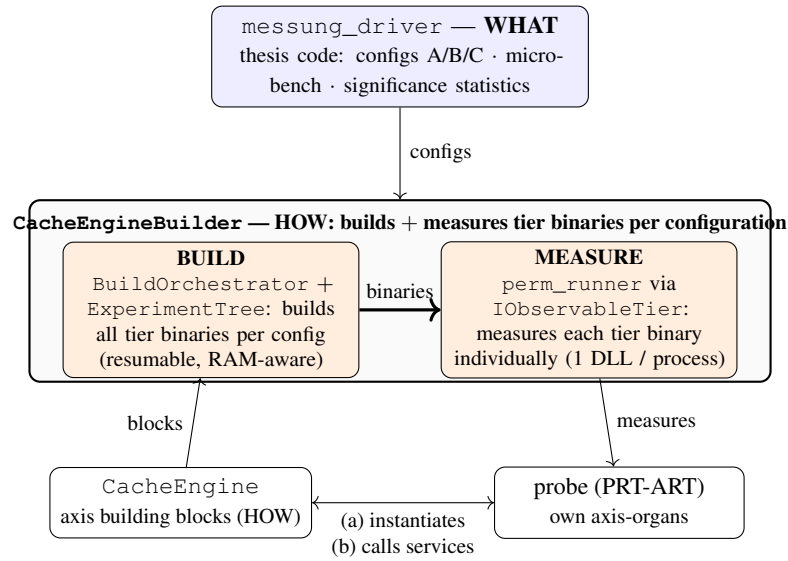


Figure 3.12: The M-model focused on the build and measurement phases. The outer `messung_driver` loop (**WHAT**, thesis code) feeds the configurations of the three measurement series A/B/C; the `CacheEngineBuilder` (**HOW**, library) splits into the two genuinely interesting phases: **BUILD** (`BuildOrchestrator/ExperimentTree` builds a standalone tier binary per configuration, resumable and RAM-aware) and **MEASURE** (`perm_runner` drives and measures each tier binary individually through the observer interface `IObservableTier`). The `CacheEngine` supplies the standard axis building blocks, the probe PRT-ART its own organs; their bidirectional relationship ((a) instantiates, (b) calls services) carries the multi-probe capability. The `ExecutionEngine` remains the shared measurable root.

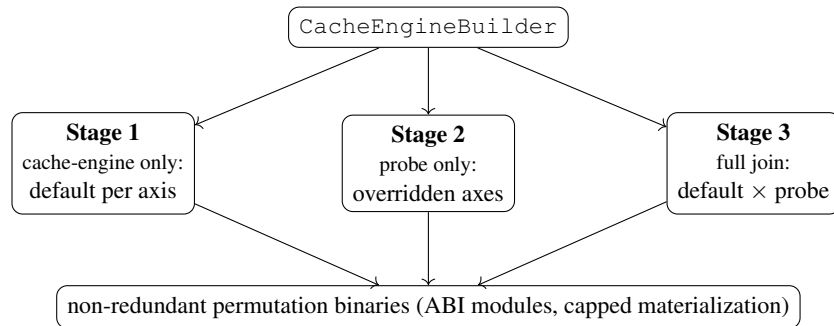


Figure 3.13: The builder's three-stage evaluation: each stage produces non-redundant permutation binaries; their mapping onto the measurement series A/B/C is shown by Table 3.8.

Table 3.8: Mapping of the three-stage evaluation onto the mandatory measurement series

Evaluation stage	Content	feeds series
Stage 1 (cache-engine only)	default building blocks + SOTA profiles	A (baseline also for C)
Stage 2 (probe only)	overridden axes = probe variants	A
Stage 3 (full join)	default × probe, non-redundant	B
build-spanning	old vs. new (regression and merge points)	C

The three evaluation stages map not one-to-one but in this assignment onto the three mandatory measurement series A/B/C; series C is build-/version-spanning (not bound to a single evaluation stage).

of all load profiles against all methods, Section 3.1); *series B* (systematic variation of the axis decisions all else held equal, to isolate the contribution of each axis); and *series C* (merge/regression old vs. new: concrete merge points between PRT-ART building blocks and cache-engine permutations — e.g. the PRT-ART OLC with `tcmalloc` and the ART page, the 4+1+2 pool with the HOT page, the path prefetch with the B²-tree page, or the multi-level layout with the Wormhole page —, run build-spanning as an old-versus-new comparison). Each series is collected in three granularities. *Micro-benchmarking* measures each axis over its *axis interface* — the set of functions that *all* algorithms of this axis must supply (every allocator provides memory, whichever one it is): the axis-specific main properties, measured internally. The axis is thereby *not* an interface function of the search algorithm; its genus interface functions *use* the axis interfaces — just as axis algorithms may use foreign axis interfaces, since containers, say, are needed everywhere. *Macro-benchmarking* measures the individual `std::map` interface operations (insertion, point and range lookup, update, deletion) against example loads, and *overall benchmarking* runs the YCSB load profiles over every recombination. As a counter-check to the internal micro measurement, the apparatus additionally measures three *wallclock levels*: the wallclock of the axis interfaces under each algorithm, the wallclock of the genus interface functions, and the wallclock of a test load from the load frameworks. The architectural obligation of this cut is that all axis algorithms use exclusively axis interfaces instead of generic operating-system calls — compiled in inline by metaprogramming, without `std::variant` —, so that the micro measurement of an axis actually measures the axis. Precisely this methodology is the answer to the separability problem: by measuring each constituent first in isolation over its axis interface before it appears in the operation and overall context, the contributions become separately measurable instead of remaining blended in the overall throughput.

The system reduces the space thus spanned deliberately. Already the canonical measurement catalog — two enabled representatives per organ axis, the persistence axis pinned to its in-memory representative — counts $2^{17} = 131072$ binary identities per system permutation (per combination of the build control quantities, Section 3.4.1), each with its full eighteen-segment path (Section 3.4.4); the raw variant product space over all axis catalogs including their sub-axes lies orders of magnitude above that (already the historical eleven-axis precursor counted billions of combinations). Three mechanisms dampen the explosion: the `<expected_workload>` profile filter routes only compatible workloads, the `Defined` mode (`MessreihenMode::Defined`) runs only the declared tuples (around 100–1000 per series, which enter the manuscript), and the `Full` or `Full-Sampled` mode (e.g. every thousandth permutation) runs only on the ZIH cluster, observing the axis compatibility reported by the platform probe.

3.5 PRT-ART as Demonstrator

PRT-ART is the *probe* that shows the system holds: an abstract living being that supplies some axis-organs itself and falls back via a compile-time fallback (`resolve_baustein.hpp`) on the cache-engine default building blocks where it does not override an axis; the hybrid `PrtArtSearchEngine<Ts...>` API remains unaffected, and the runtime binding occurs via the execution-engine adapter, not via a separate search-engine layer. The probe repo is loaded via the plugin controller of the cache engine (`COMDARE_CE_PRUEFLINGE`): its entry snippet must explicitly *declare* itself via `comdare_pruefling_deklarieren` — today with the single registered capability `pruefling_slots_v1`, the axis slots of this fallback — because presence may only refute, never permit: the deposited evidence

headers can only break the declaration, never open a gate, and a requested but non-declaring probe breaks the configuration loudly instead of being skipped silently; the historical probe-own adapter path, by contrast, is formally decommissioned and not part of this declaration. Concretely, PRT-ART overrides in the measurement apparatus the build axis page type, the four organ axes path compression (redirect), value handle (ChainRef slot), prefetch, and concurrency pattern, as well as the runtime telemetry (in the triad a measurement-tooling sub-axis, Section 3.4.1) — path-oriented (bundle) prefetch, optimistic lock coupling with reserved value blocks, and the hypothesis metrics H1/H2/H3 —; its further own building blocks, the 4+1+2 pool allocator (free-list sub-axis) and the multi-level layout, live in the probe repo and currently run via the cache-engine default in the measurement path (binding as follow-up step E6 of the internal implementation planning; the identifier is independent of the interface layers E4–E0); for the remaining axes it reuses existing SOTA building blocks via the permutation configuration. Thus the probe shows that the measurement system holds: a novel living being arises from a few of its own organs plus the default toolkit (Figure 3.14) and is measurable over the same probe dock as every reconstructed SOTA method.

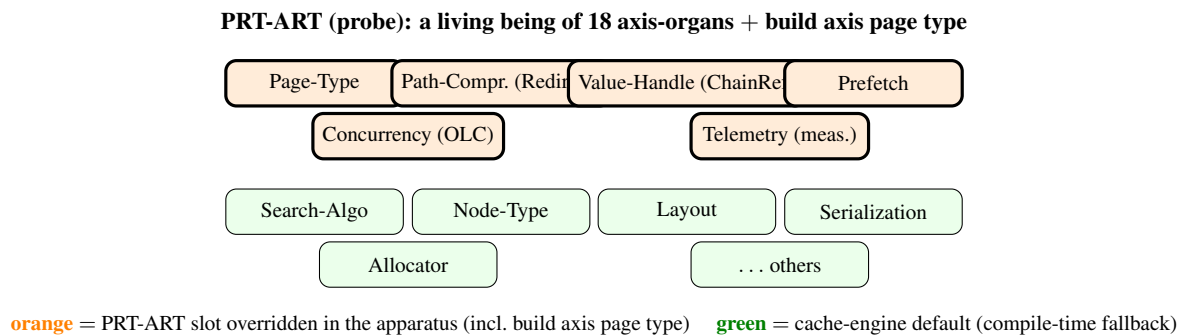


Figure 3.14: PRT-ART as a demonstrator in the M-model: the probe overrides six slots in the measurement apparatus — the build axis page type, the four organ axes path compression (redirect), value handle (ChainRef), prefetch, and OLC concurrency, as well as the runtime telemetry (measurement-tooling sub-axis, Section 3.4.1) — and obtains the remaining fourteen organ axes via the compile-time fallback from the cache engine; its further own building blocks (4+1+2 pool allocator, multi-level layout) currently run via the cache-engine default in the measurement path (binding: follow-up step E6). Thus a complete, measurable living being arises from a few own plus many default organs.

3.6 Sharpening through Heuristics

The measurement series deliver more than rankings: from them heuristics can be won that sharpen the measurement system step by step from a test bench into a self-optimizing delivery system. This section develops this sharpening as a concept in three stages — the heuristic loop over learned load profiles, the four operating modes of the builder up to the hybrid mode as the target picture of this thesis, and the measurement-curve type system together with the filter chain and the measurement-error detection that feed these modes with data. Throughout, the accomplished and the planned are expressly kept apart.

3.6.1 Heuristic Loop and Learned Load Profiles

From the measurement series one can derive the best axis composition per load and data-type profile and store it as a reusable profile. *The full loop is an outlook mechanism; what is implemented within this thesis*

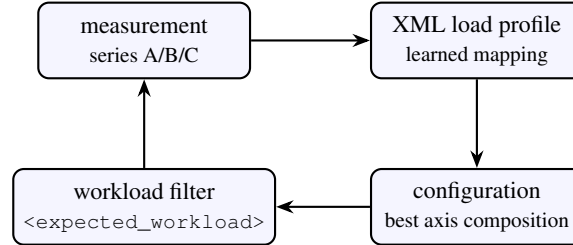


Figure 3.15: The heuristic loop: from the measurement series the best axis composition per load profile is learned, stored as an XML load profile, and fed back via the workload filter into the next measurement.

is only a first increment of the automatic selection and binary delivery (*best_binary_selector*, Section 1.4). Beyond mere ranking, a *heuristic extraction* is envisioned as a follow-up phase after **Persist** — the last phase of the seven-phase pipeline (Section 4.3.2) — that automatically derives from the collected measurements which axis configuration is best per workload and dataset. The intended mechanism is an ML classification in three steps: first, features are extracted from the measurement CSV per permutation — the load-profile and dataset features (workload type, key distribution, write/read share, alphabet and prefix characteristics, as well as key and value data type) and the axis assignment —, with the best performance measured per load pattern (say, throughput or tail latency) as the target; over these an *ML classifier* learns the mapping from load-profile/axis features to the best configuration; and it delivers per workload/dataset class the recommended axis composition without re-running the entire permutation space. The result is persisted as a reusable *XML load profile* — symmetric to the measurement-series XML configurations — and can be fed back into the `<expected_workload>` filter without a full re-measurement; this closes the loop between measurement, learned heuristic, and configuration-guiding selection (Figure 3.15). This *measure* → *profile* → *config* → *filter* loop is the classical, established form of *empirical, measurement-driven autotuning*: it stands in the line of autotuning frameworks such as OpenTuner [AKV⁺14] and the measure-in-the-loop autotuners FFTW [FJ98] and ATLAS [WPD01]; the load-profile-driven configuration choice with workload filter finds its database counterpart in OtterTune [APGZ17] and the Database Tuning Advisor with workload compression [ACK⁺04], the online runtime-near variant in Active Harmony [TCH02]; the term “software-in-the-loop” itself originates from the X-in-the-loop validation taxonomy [TMS⁺16]. The concrete realization of this ML-based selection mechanism remains reserved for future work (Chapter 6); the loop itself, however, is only the first stage of a larger operating target picture, which the following subsection develops.

3.6.2 The Four Operating Modes of the Builder

The heuristic loop is the core of an operating regime of *four* operating modes of the builder interface that build on one another. Of these four modes, the first is the apparatus’s operation of today; modes two through four are the declared goal of this thesis and are recorded here as a concept.

In the *measurement mode* — the operation that Section 3.4 describes — the builder runs the measurement

series over the admitted permutations. After the measurement completes, the framework knows the properties of all admitted axis algorithms against the recombination of all admitted load-profile frameworks and workloads. The *evaluation mode* condenses these measurements: from them emerge minimal strands of the permutations of all properties against one another for certain *workload clusters* — for groups of similar loads, only the small set of axis compositions that dominates them remains. How workloads are aggregated into clusters (which clustering methods carry) is deliberately left open and subject to further investigation.

The *working mode* is the production goal: the builder interface keeps all tier binaries relevant for the currently stochastically frequent workload load and interface function hot in main memory and switches, under the hood, the tier binary with its specifically optimal axis permutations at the ABI-stable interface — the same binary-switching mechanism the measurement mode uses, only load-driven instead of measurement-driven. Level-correctly, the compile-time doctrine remains preserved: there is no runtime switch *inside* the tier — the binaries themselves remain compile-time-permuted; switching happens *between* preloaded binaries at the deliberate ABI boundary. The probe dock of the genus (Section 3.3.2) is called the *working dock* in this mode: the same interface through which the measurement mode tests now delivers the productive answers.

The *hybrid mode*, finally — the goal of this thesis — combines working mode and measurement. Its centrepiece is that the heuristic optimization is itself a *system axis* (Section 3.4): it adopts, via metaprogramming, the genus of its tier binaries, is itself compiled into a tier binary, and docks at the builder’s probe dock like every real tier; it passes commands via the *Command* pattern [GHJV94] on to the real tier binaries statically assigned to it. The recombination of the heuristic tier binaries to be used is itself permutable against the existing real tier binaries; the heuristic axis can therefore use the measured function curves for estimates and compile an optimal multidimensional recombination that is unambiguously usable as a search-algorithm hull through its ABI-stable genus interface — towards the outside a *virtual whole tier binary*, indistinguishable from a single binary. At the same time, the hybrid mode measures along: over the optimized recombination, the combined wall-clock time as well as macro- and micro-benchmarks are logged per measurement command across all tier binaries and operations.

The hybrid mode thereby carries the *heuristic counter-proof*: after the working mode is entered, the heuristic tier binary is measured anew — its wall-clock behaviour compared against the best single binaries and against the reconstructed paper algorithms decides whether the heuristic recombination delivers what the curves promise. The goal of this thesis is thus stated twice over: to determine the best recombination or indeed the best single binary against all axis properties — especially cache-line awareness — and in the end to provide both in production-ready form, the best single binary *and* the heuristic tier binary. All four modes shall thereby be documented automatically via the evaluation chain of the definition layer (Section 3.3.4); the measurement tables and diagrams of this thesis itself are the first product of this chain, and the assessment of the counter-proof is defined by Chapter 5.

3.6.3 Measurement-Curve Type System, Filter Chain, and Measurement-Error Detection

For the evaluation and hybrid modes to carry, the evaluation needs more than flat result tables. As a target picture this thesis formulates a *measurement-curve type system* for this; in the current state only its germ exists — a curve fit over one axis and one x-dimension —, the full structure is stated as a

goal. Every tier-binary total permutation receives, per measurement axis, a wall-clock measurement curve over the working set — say, the lookup latency as a function of the fill size —, separated by load-profile framework, workload type, working-set size, operation type, and observer type; each axis is swept separately in the experiment tree, compile-statically as well as runtime-dynamically, so that each axis receives a complete overview of its properties and individual measurements. Taken together over all permutations, a multidimensional measurement database arises per tier binary, from which the heuristic curves of the loop (Section 3.6.1) are read. The cardinality of each dimension of this type system is to be strategically organized into classes and hierarchies: design-fixed constants (say, the eighteen organ axes and the sixteen measurement categories), profile-bound products (what an experiment profile admits as assignments), and runtime-free iteration spaces (the dynamic sampling points per static binary) — so the system axes remain able to carry the evaluation across all levels.

From the evaluations there emerges — likewise as a target picture — automatically a *filter chain*: each chain link checks, based on the curves so far, whether a permutation family is pursued further, and otherwise passes the decision on to the next link — the design pattern *Chain of Responsibility* [GHJV94], located strictly in the builder, for the chain controls the *generation* of the tier binaries: evaluation, filter, then the decision which permutations are built next and which are discarded. This chain is not implemented in the current code; it is recorded as a concept and follows after the first complete measurement runs.

Finally, the hybrid mode needs a control of the measurement error that the measurement instances themselves introduce — observers cost time. Three build variants of the same experiment are envisioned: first, observers in all tier binaries *and* in the heuristic hybrid tier; second, observers only in the heuristic tier binary; third, observers in none — only the wall clock. The differences between the three variants quantify the measurement error caused by the measurement instances. Complementarily follows a *fourth step* of the final heuristic assembly: the tier binaries subordinate to the heuristic tier (not the finally optimized main binary) are rebuilt without micro- and macro-benchmarks and without observers and compared purely on the wall clock against all known tier-binary permutations and against the known paper algorithms — with the paper comparison taking precedence. This too is a target picture: neither the build-variant matrix nor the complementary fourth step exists in the current code; what is laid is the foundation — the per-compilation measurement switches and the honest validity marking per measured value in the sense of the honest-0 doctrine (Section 3.4).

This keeps clearly apart what is accomplished and what is a goal: accomplished are the measurement apparatus of the measurement mode including the three-stage evaluation, the evaluation chain up to tables and diagrams, and the germ of the curve fit; still pending are the complete measurement runs; goals are the evaluation, working, and hybrid modes, the full measurement-curve type system, the filter chain, and the measurement-error control. The assessment of the heuristic counter-proof is taken up by Chapter 5; the placement of the outlook beyond this thesis is given by Chapter 6. First, however, Chapter 4 deepens the concrete implementation of the building blocks introduced here.

4 Implementation and Concrete Algorithm Components

This chapter descends into detail: first the repository architecture, then each key point of PRT-ART individually and visually, next the axis implementation together with the *one* architecture in code and the code shape of the build chain, and finally the quality assurance via contract tests per layer including the two-gate protocol and the code shape of the realm roots — system and measurement axes including the hardware probe.

4.1 Three-Repository Architecture

The implementation is distributed across three Git repositories with clearly separated responsibilities:

- `comdare-cache-engine` — **how** measurement is done: the building-block library and the builder (XML config parser, `generate_module_from_profile` code generation, permutation loop, module loader, the seven-phase `ExperimentDriver`, and the experiment planner `ExperimentPlanDirector`, Section 4.3.2), the libraries `search_engine` and `execution_engine`, the ABI headers under `abi/` (`baustein_variants`, `resolve_baustein`, `module_abi_v1`), the 30 SOTA profiles under `algorithm_profiles/sota`, and the test suites; the source tree follows a Pitchfork layout (`apps/` for executables, `libs/` for libraries, `libs/common` for cross-cutting code).
- `comdare-prt-art` — the **probe**: the hybrid `PrtArtSearchEngine` (vector/map/tuple API), its ABI adapters, and its building blocks (4+1+2 allocator pools — seven pools: four size-class pools A–D, one rest pool R, and two reserved value pools —, OLC with reserved blocks, multi-level layout, path-oriented prefetch, density tracker, hypothesis metrics H1/H2/H3) as well as the probe-side comparison re-implementations of the Rank-2/3 papers without public source code (`legacy_reimpl`, P11–P27), which are compared against the real cache-engine adapters in the SOTA admission test; the pool, prefetch, and layout components are currently descriptor skeletons without a real memory backend — the prefetch building block nonetheless already overrides the prefetch slot of the measurement apparatus, while pool and layout still run there via the cache-engine default (Section 4.2).
- `probst-Diplomarbeit-cache-engine` — **what** is tested: the `messung_driver` (binary `comdare-messung-driver`), the experiment configurations, the evaluation pipeline, and the manuscript.

The four subsystems (measurement driver, `CacheEngineBuilder`, `CacheEngine`, probe) are orthogonal to this three-repository split: builder and `CacheEngine` share `comdare-cache-engine` — the builder as an executable (`apps/`) on top of its builder library (`libs/cache_engine/builder`), the `CacheEngine` as a library (`libs/cache_engine`). Across this, the terminology separates two engine

terms that must not be conflated. The `CacheEngine` (`libs/cache_engine`) is the *library of axis building blocks*: it provides per axis the interchangeable design constituents as concept/CRTP components and contains *no* measurement logic. The `ExecutionEngine`, by contrast, is the execution and measurement infrastructure lying across all axes: its abstract measurable root is `IEExecutionEngine` (Section 3.4); the concrete runtime adapter `SearchAlgorithmAbiAdapter<A>` provides, via the `SearchAlgorithmAnatomy`, the OS primitives (cache-line-aligned allocation, NUMA-conformant read pinning, coherence-aware writing); the adapter’s measurement and observer couplings are gated by `COMDARE_MEASUREMENT_ON` — this yields four delivery variants of the same permutation (debug with observer, measurement with aggregator, release without measurement couplings, and the compare label kept build-identical to release) —, and the measurement aggregator is kept host-side by the `ExperimentDriver`; the CMake option `COMDARE_EXPERIMENT_MODE` marks experiment compiles and encodes at compile time the invariant that an experiment build is always a measurement build — it does not itself enable any hooks. In short: the `CacheEngine` provides the *building blocks*, the `ExecutionEngine` the *tools*; this separation keeps the production path measurement-free.

4.2 PRT-ART in Detail

Building on the known state of the art, we describe each key point of PRT-ART individually and visually. The probe contributes six key points of its own, two of them on the telemetry axis, which since the axis reordering resides in the system sphere (Section 3.4.1). On the *allocator axis*, a *4+1+2 pool allocator* (four size-class pools, one rest pool, plus two reserved value pools) replaces the algorithm-hardwired allocation with a cache-line-aligned, low-fragmentation free list. On the *concurrency axis* — itself split into pattern (8.1) and locking mode (8.2) — PRT-ART contributes *optimistic lock coupling (OLC) with reserved value blocks*, coordinated by a concurrency manager that handles the disciplines OLC, HTM, STM, lock-free, RCU, and hazard pointers; reclamation is here a sub-axis of the allocator (epoch, RCU, hazard-pointer, and QSBR strategies selectable per permutation). On the *prefetch axis* a *path-oriented bundle prefetch* fetches the hot search paths ahead of time, on the *layout axis* a *multi-level layout*. Particularly cache-critical is the *telemetry axis*: per-node counters cause cache-line ping-pong on multi-core systems and falsify precisely the measurements they collect. Following the Kuehn research line [Kue23], PRT-ART therefore offers cache-coherence-aware strategies — *leaf-only counters* (the main variant), *sampling* (every N th leaf access), and *offline recompute* (bottom-up aggregation before reordering) —, while the classical per-node counter remains only as a labelled anti-pattern. The density tracker and the hypothesis metrics $H1/H2/H3$ the probe adds as its own telemetry organs. These key points are *building blocks of the probe repo*; of these, currently prefetch, concurrency, and telemetry are overridden in the measurement apparatus (together with the build axis page type, the path-compression redirect, and the `ChainRef` value handle), while the 4+1+2 pool allocator and the multi-level layout run via the cache-engine default in the measurement path until their binding (follow-up step E6) lands (Section 3.5). Condensed into a gallery, Figure 4.1 shows the six own key points of PRT-ART, each located at its axis. The remaining two of the eight building-block layers from the contribution list (Section 1.4) — the inline/external/chain-ref value handles and the signaling-bit serialization — do not appear as gallery points of their own: the former provide the `ChainRef` slot overridden in the apparatus (T11), the latter is catalogued as the `VarLen Serialization` building block with signaling bits in Appendix D.

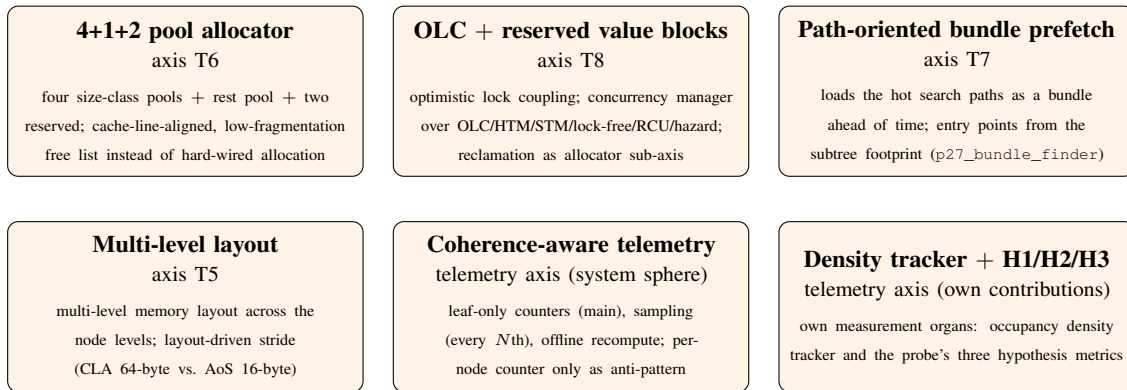


Figure 4.1: Gallery of the six own key points of PRT-ART, each at its axis: a 4+1+2 pool allocator (T6), optimistic lock coupling with reserved value blocks (T8), path-oriented bundle prefetch (T7), a multi-level layout (T5), and cache-coherence-aware telemetry together with the density tracker and the hypothesis metrics H1/H2/H3 — the telemetry axis has resided in the system sphere since the axis reordering (Section 3.4.1). The tiles show the *building blocks of the probe repo*; currently T7/T8 and the telemetry are overridden in the apparatus, while the building blocks for T5/T6 still run there via the cache-engine default (E6). In addition, the probe overrides the path-compression redirect (T3), the ChainRef value handle (T10), and the build axis page type; the remaining fourteen organ axes (currently including T5/T6) the probe draws via the compile-time fallback from the cache-engine.

4.3 Axis Implementation and Measurement-System Levels

Each axis is implemented as a generalized interface; alongside lie the levels of the measurement system from code generation to the seven-phase pipeline. Each axis is a topic directory containing a two-layer *concept* (a broad topic concept and a narrow axis concept) and a CRTP base class secured by a `requires` clause. The probe-versus-default choice is resolved for the fourteen PRT-ART fallback axes at compile time via `resolve_baustein.hpp` (concept-checked `if constexpr` over tag specializations; the literal tag catalogue of the historical eleven-axis precursor persists there only as the `baustein_variants` fallback), so that the hot path contains neither `virtual` calls nor runtime switches — the zero-cost abstraction from Section 3.3. Every SOTA design and every allocator is integrated via a thin *adapter* that provides the original implementation behind the engine ABI: allocator vendors are switched on or off at compile time via per-vendor include shims (`#if COMDARE_AXIS_06_USE_<X>` with forward stubs), SOTA building blocks via `constexpr` enable flags plus `mp_filter` (the `COMDARE_AXIS_*_USE_/ENABLE_` flag family), each with a safe fallback chosen per family — allocator adapters substitute the missing vendor with `std::malloc/std::free`, data-structure adapters with a stub that throws a controlled `std::runtime_error` exception on use —, so that the build remains independently runnable. The former detection `define COMDARE_HAVE_<X>` persists only historically. The integration of hierarchical prefetching [ZGH⁺25] additionally provides a compile-time call-graph analyzer (`p27_bundle_finder`, a C++23 port of the original tool `hp_soft` that identifies functions whose subtree footprint exceeds a threshold as bundle entry points).

The eighteen organ axes (T0–T17, Section 3.4.1) are here not abstract but each a concrete *strategy concept* with several interchangeable building blocks, from which a concrete composition picks exactly one per slot. Figure 4.2 gathers all eighteen as a gallery — per axis the short name, the sub-axis family and a

representative selection of the building blocks; the complete building-block matrix with all origin sources is given in Appendix D, the SOTA contributions per axis in Table 3.2. The nineteen-axis numbering of earlier versions of this chapter (T0–T18, interim state of July 2026) has been superseded by this order (Section 3.4.1). That no composition omits an organ is enforced by the `IsComposition` concept: it demands per algorithm the assignment of exactly these eighteen organs — the fifteen search axes, the two queuing axes, and the persistence target —, so that no composition compiles that leaves an axis open; the complete organ set is thus not a convention but a compile-time-checked condition.

T0 Search algorithm SA1–SA4 Array256, OriginalART/HOT, KArY, Interpol., Hash ... (21: 17 base + 4 per-K) <i>Def. Array256</i>	T1 Cache traversal CT1–CT2 Linear-fanout, Hash- lookup, Binary-fanout <i>Def. Linear</i>	T2 Slot mapping MP1–MP2 Direct-placement, Pool-relative <i>Def. Direct</i>	T3 Path compression PC1–PC3 None, Byte-wise (ART), Patricia <i>Def. None</i>
T4 Node type NT1–NT3 Node4 / 16 / 48 / 256 <i>Def. Node256</i>	T5 Memory layout HM1–HM4 AoS, Cache-line-aligned, SoA, Bitmap (LOUDS), AoSoA <i>Def. Cache-line</i>	T6 Allocator AA1–AA7 Hoard, Slab, Mimalloc, Jemalloc, TCMalloc, Buddy ... (25) <i>Def. Std-malloc</i>	T7 Prefetch PF1–PF3 None, Distance-estimator (ART), Hardware, Path-orient. <i>Def. None</i>
T8 Concurrency CC1–CC2 None, Blocking, R/W, OLC, Lock-free, RCU, Hazard <i>Def. None</i>	T9 Serialization SR1–SR3 Raw-binary, VarLen (LEB128), Succinct, LZ4/Delta <i>Def. Raw-binary</i>	T10 Value handle VH1–VH3 Inline, External-pool, RCU- shared, MVCC, Chain-ref <i>Def. Inline</i>	T11 Index org. io1–io3 Clustered, Heap, IOT (em- bedded), Non-clustered <i>Def. Heap</i>
T12 I/O dispatch IO1–IO3 In-memory, Buffered, Di- rect (O_DIRECT), Mmap <i>Def. In-memory</i>	T13 Migration MG1–MG3 None, Hot/Cold, Tier- based, Adaptive (ML) <i>Def. None</i>	T14 Filter FT1–FT3 Bloom, Cuckoo, Range- SuRF (LOUDS), Xor <i>(no None)</i>	T15 Op. buffer (Q1) QS1–QS6 None, Append, FIFO/LIFO, Ring, Priority ... (15) <i>Def. No-buffer</i>
T16 Flush policy (Q2) FS1–FS4 Eager, Lazy, Watermark, Timed, Adaptive-LSM <i>Def. Lazy</i>	T17 Persistence target 18th slot (ORG-18) In-memory; disk write-back (declared, disabled via option) <i>Def. In-memory</i>	Relocations (July 2026) telemetry (TM1–TM3) → system sphere, split in two; ISA (IS1–IS3) → system main axis target_isa (Section 3.4.1)	Reading Each axis = a <code>Concept</code> with interchangeable building blocks (<i>Strategy</i>); a composition picks exactly one per slot.

Figure 4.2: Gallery of the eighteen organ axes (T0–T17, canonical slot order): per axis the implementation concept — short name, sub-axis family and a representative selection of the interchangeable building blocks. Each axis is a `Concept` interface with several strategy building blocks (*Strategy* pattern); a concrete composition occupies exactly one per slot. The *default* (“Def.”) is a derived comparison baseline, not a code flag; for the large axes T0/T6/T15 the total building-block count is given in parentheses. The T numbers are slots of the composition, not directory numbers of the implementation. Compared with the nineteen-axis gallery of earlier versions (T0–T18, interim state of July 2026), telemetry and ISA have moved to the system sphere, and the persistence target has been added as the eighteenth slot — pinned to its in-memory representative in the canonical measurement catalogue (Section 3.4.1). The complete matrix with origin sources is given in Appendix D.

4.3.1 The One Architecture in Code

The *one* architecture from Section 3.3 is made visible to the outside at code level through an ABI-stable *hull*. The living being `SearchAlgorithm` and its ABI-runtime view — the “`SearchEngine`” — are merely two views of the same entity across the module boundary, not a parallel construct: the layering `CacheEngine` → `ExecutionEngine` → `SearchEngine` separates the roles (cf. Section 4.1) — the `CacheEngine` as the axis building-block library, the `ExecutionEngine` as the root of the execution and measurement tooling (OS primitives; in experiment mode the measurement aggregator over exactly one observer interface), and the `SearchEngine` as the search-API view measured over precisely that interface (Figure 4.3). Axis-less *viruses* stand as siblings beneath the same root but carry no anatomy.

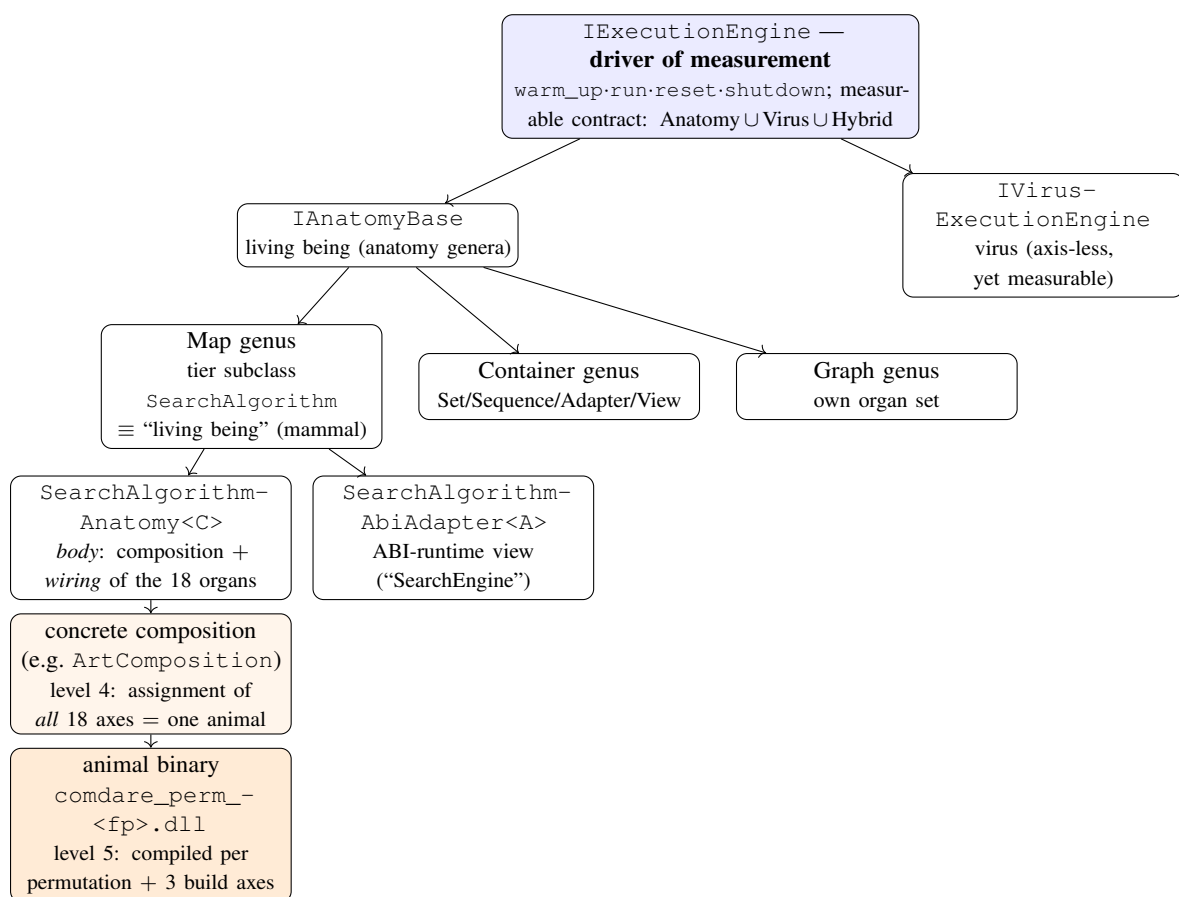


Figure 4.3: The *one* architecture as a single hierarchy without parallel trees. At the root stands `IExecutionEngine` as the *driver of measurement* — the universal measurable contract (`warm_up/run/reset/shutdown`) across all three engine kinds `Anatomy`, `Virus` and `Hybrid`; beneath it branch the anatomy-bearing *genera* (`Map`, `Container`, `Graph`) and the axis-less *viruses*. `SearchAlgorithm` is the focal living being (mammal); its anatomy is its *body* — the fixed composition *and wiring* of the 18 organs —, and “`SearchEngine`” only its ABI-runtime view. The mammal branch is drawn out down to the individual: a concrete axis composition (level 4) is compiled per permutation into a standalone animal binary (level 5). `Container` (keyless storage) and `Graph` carry their own organ sets.

Concretely, the module boundary is a C-stable interface that exports exactly one living being per permutation; its variadic hybrid API specializes by parameter count, and complex keys are reduced by a

fingerprint component to 16-byte binary strings of fixed length (Figure 4.4). How these and the remaining code interfaces relate in detail is summarized by the UML map in Figure 4.5.

```
// C-stable module boundary: exports one living being per permutation
extern "C" IAnatomyBase* comdare_create_anatomy();

// Variadic hybrid API by parameter count (key K, values V):
// 1 parameter -> std::vector<V>
// 2 parameters -> std::map<K, V>
// N > 2         -> std::map<K, std::tuple<V1, ..., VN>>
template<class... Ts> class SearchEngine; // ABI-runtime view of the being
```

Figure 4.4: The ABI-stable C module boundary and the variadic hybrid API. Complex keys are reduced by a fingerprint component to 16-byte binary strings of fixed length.

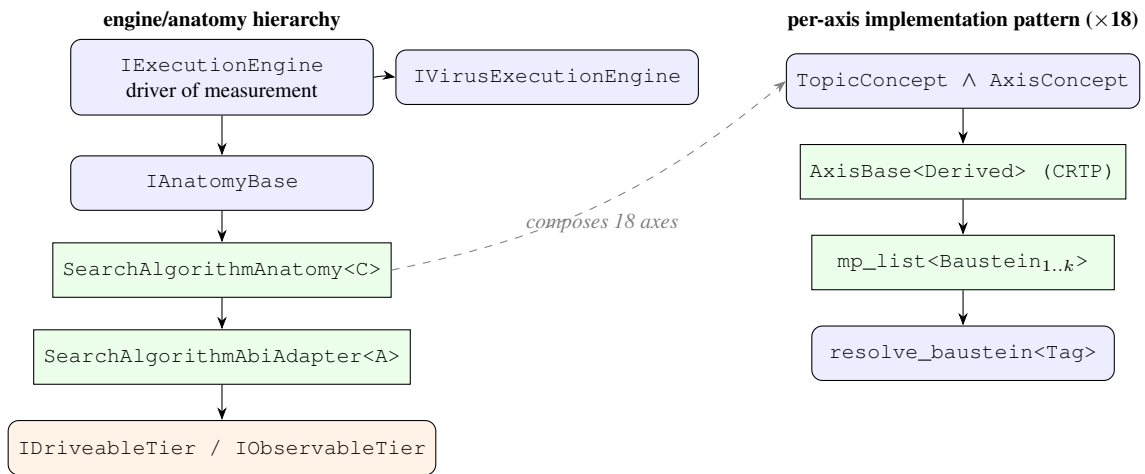


Figure 4.5: UML map of the code interfaces: on the left the engine/anatomy hierarchy — the measurable root `IExecutionEngine` as the *driver of measurement* ($\text{Anatomy} \cup \text{Virus} \cup \text{Hybrid}$) over `IAnatomyBase` and the axis-less `IVirusExecutionEngine`, beneath it the *body* `SearchAlgorithmAnatomy<C>` (composition and wiring of the eighteen axes) and its ABI-runtime view `SearchAlgorithmAbiAdapter<A>` (“SearchEngine”), measured and driven through the `IDriveableTier/IObservableTier` contracts; on the right the per-axis implementation pattern repeated for each axis — a two-layer Concept, a requires-guarded CRTP base, the `mp_list` building-block family, and the compile-time resolution `resolve_baustein`, which chooses per axis between the probe slot and the cache-engine default.

4.3.2 From Profile to Tier Binary: Planner, Builder, and Measurement Pipeline

The chain from the user XML to the tier binary (Section 3.4.5) has a named shape in code. Its entry point is the measurement driver `comdare-messung-driver`, which carries the roles of the chain as subcommands: `validate` checks, purely read-only, the profile resolved by the XML parser against the compiled-in registry catalogues (an unregistered reference is a hard validation error, Section 3.4.5), builds no binary, and measures nothing. The planner role `plan dump|ci|cmake` drives the deterministic walk of the `ExperimentPlanDirector` — separated along the *Director* and *Builder* design patterns: the director owns the *one* enumeration walk over measurement combinations, system permutations, and

organ steps, and per output syntax (text plan, CI pipeline description, CMake fragment) a concrete builder emits via hook methods; two runs are byte-identical, and the plan header annotates the registry trio of the three offer catalogues organ/system/measurement. The builder role `tier ci|cmake`, finally, rests with the *built* CEB: it emits itself the view of the second chain stage — system permutations and tier build orders of its released build space —, for the planner steers the CEB orders and the CEB steers the tier orders. Implementation-wise, planner role and CEB role currently share one binary; this is documented honestly, no sham split. The emitted build and check attestations keep the layer separation of the chain visible: they always carry the system and organ brackets separately, and only measurement attestations carry all three brackets.

Two properties secure the *one* main channel. First, since its version 1.1 the text plan reports per build step the actually built file stem — produced by the *same* function that the build orchestrator calls during the real build: the plan claims no name that the build would not produce. Second, the former second emission channel of the permutation codegen — a CMake-side tool trio that generated permutation sources outside this chain — was shut down in two steps in July 2026: first disabled in the consuming repositories, then removed byte-neutrally from the engine repository. What remains is *one* XML source and *one* channel that builds it; variant formation lies as far as possible in C++23 metaprogramming, and CMake stays confined to the role of building.

Inside the CEB, the builder transforms each profile into a pre-compiled C++23 module; each permutation is identified by a bit-packed *permutation flag* structure that grows, with the N-phase refinement, from nine banks (around 50 bits) to a wider *permutation identifier* with fourteen banks (82 bits) and sub-bank bit fields; the eighteen organ axes are identified in the active measurement path via the string-serialized eighteen-segment path (the `binary_id`, Section 3.4.4) plus fingerprint, and a constraint filter excludes invalid combinations before each module generation. Over these building blocks runs the *seven-phase measurement pipeline* (Figure 4.6) of the `ExperimentDriver` library per series: (1) **Enumerate** (parse XML configs, build the permutation list), (2) **Codegen** (one `comdare_perm_<fp>.cpp` per permutation, plus `generate_module_from_profile` for the SOTA profiles), (3) **Compile**, (4) **Load** (LoadLibrary/dlopen of all modules), (5) **Execute** (`create_instance` + profile-aware `run_workload`), (6) **Measure** (aggregate the binary measurement records), and (7) **Persist** (`measurements.csv/.json`); two opt-in phases hot-compile missing modules and verify the ABI contract per module. The downstream evaluation toolchain (Figure 4.7) is a sequence of small programs — `sample_data_generator` (deterministic test data without real hardware) → `messung_driver` (drives the series) → `binary_to_csv` (deserializes the binary measurement records) → `csv_to_latex` (booktabs tables with per-permutation-ID algorithm fact sheets) → `diagram_generator` (A4-aware TikZ plots) → `latex_to_pdf` —; the measurement couplings are separated from the production path by the `COMDARE_MEASUREMENT_ON` gating (Section 4.1), and continuous integration (GitLab CI plus a mirrored GitHub workflow) runs across all three repositories.

4.4 Quality Assurance: Contract Tests per Layer

The layer model E4–E0 with its frozen interface contracts was developed as a concept in Section 3.3.4; this section describes its realization as test infrastructure. The occasion is a concrete project experience: two implementation increments that changed several layers at once had to be rolled back completely because

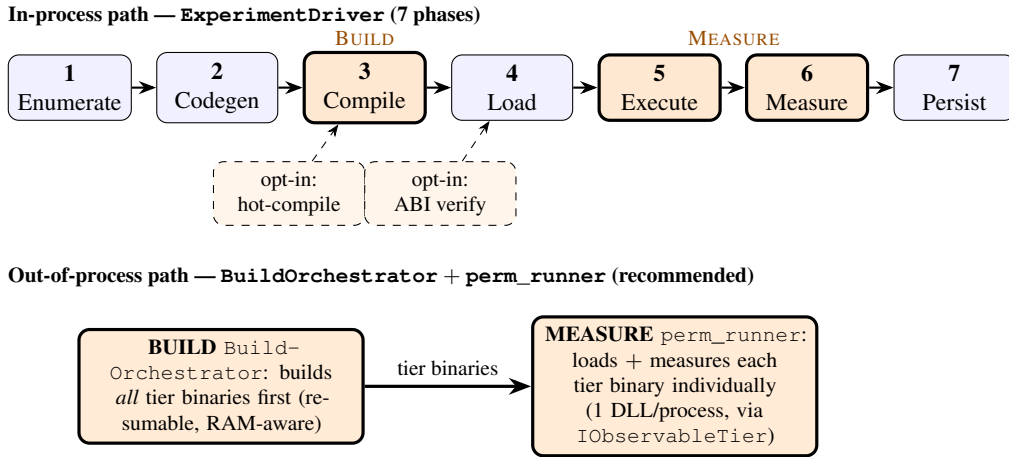


Figure 4.6: The measurement pipeline in two forms. The **in-process path** (top) is the seven-phase pipeline of the `ExperimentDriver` per measurement series (Enumerate → Codegen → Compile → Load → Execute → Measure → Persist; two opt-in phases hot-compile missing modules and verify the ABI contract). Highlighted are the genuinely substantial phases: **COMPILE** (the *build* of the tier binaries) and **EXECUTE/MEASURE** (the *measurement*); Codegen/Load/Persist are pure plumbing. The **out-of-process path** (bottom, the recommended route) separates the two tasks out-of-process: the `BuildOrchestrator` builds *all tier binaries first* (resumable, RAM-aware), after which `perm_runner` loads and measures each binary individually (one DLL per process) through the observer interface `IObservableTier`.

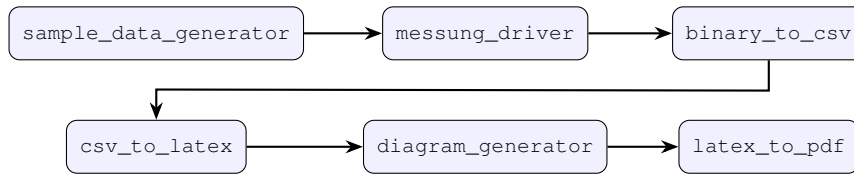


Figure 4.7: The downstream evaluation toolchain as a six-stage chain: `sample_data_generator` produces deterministic test data, `messung_driver` drives the measurement series, `binary_to_csv` deserializes the binary measurement records, `csv_to_latex` renders the booktabs fact sheets per permutation ID, `diagram_generator` the A4-aware TikZ diagrams, and `latex_to_pdf` typesets the manuscript.

their defects arose from interactions between the levels that neither local tests nor line-by-line review made visible. For a system whose product space rules out any complete materialisation (Section 3.4.5), multi-level changes are thus practically not acceptable for sign-off; the consequence is to complete each layer individually — with an explicitly frozen interface contract to the neighbouring level, its own test harness, and its own definition of done.

The carrying test principle reads: *each layer is tested against a compile-time substitute of its neighbouring level*. The contract tests are ordinary test targets of the build pipeline; the neighbouring level is replaced by a *fake* — a template- and concept-based stand-in in the test scope, no runtime switch in production code. A layer can therefore be finished and signed off before the deeper level even consumes its values, and the test question of every boundary — does the layer pass its definition on to the next one *without loss*? (Section 3.3.4) — turns from a design principle into an executable acceptance test.

Per layer this concretizes as follows: for the definition and evaluation layer E4 the harness checks the translation of the declarative XML profiles into the axis-level structure — golden fixtures compare the

axis levels emitted from a profile structurally against an expected reference, with a fake E3 replacing the permutation tree; neither tree construction nor build orders are needed to sign off the E4→E3 boundary. For the permutation B⁺ tree E3 the harness checks the bijection of the binary identity: from axis-level fixtures the permutation enumeration is run and, per path, the mixed-radix-encoded binary identifier is translated back and forth (counting plus signature round trip) — against a fake E2 that builds not a single binary; added to this is a conformity gate in the full-run path that checks every imported binary against the frozen contract before measurement. For the tier binaries E2 the contract is the ABI itself: an emitted reference DLL is loaded and must satisfy the frozen interface sequence — `IAatomyBase`, `IObservableTier`, and `IResourceControllableTier` including a correct capability report —, and a control request above the reported capabilities must be answered deterministically with the frozen error value of the apply response (`apply == 5`) instead of being silently accepted; the accompanying fake E1 creates and applies the runtime controls but does not demand any consumption effect — that belongs to the acceptance of E1. For the runtime configuration E1, finally, edge tests check the corner cases of the dynamic regime: that a reused tier is fully reset between two runs, that a pool budget below the live data set is handled honestly instead of silently violated, and that every applied control becomes visible as a named column in the persisted result file — checked against the CSV output, not the in-process state —, plus the preservation of the two-phase operation loop (discarded warm-up, measured run) as an invariant.

Above the layer harnesses, a *two-gate protocol* secures every implementation package. *Gate 1* is the strict gate of the engine repository: the full test suite — 320 registered tests in the state of this work — runs uncut *twice*, and only “0 failed” is accepted, with no exception list. Part of this double run is the golden anchoring of the permutation space: the 2¹⁷ binary identities of the canonical measurement catalogue (Section 3.4.5) are regenerated on demand and confirmed against a frozen CRC-64 checksum (0x56F1B721C72DC10E, CRC-64/ECMA-182 over the 131072 identities in canonical order) — the anchor is not computed but literal tool output, and changing it is permissible only within a coordinated interface window —, while a small, materializable catalogue of 320 identities serves as a byte-exact guard of the serialization form. *Gate 2* is the integration gate of the umbrella repository: the CI twin of the package builds green, subject to a *nominally frozen* exception set of five known red tests, each with a frozen failure class — a sixth red test, or a class change within the five, stops the package instead of softening the set.

This protocol is methodology, not a system component: it describes *how* changes to the apparatus are accepted, not what the binaries carry. The directory and suite structure it runs on is documented in Appendix B.

Completion follows the layering top-down — E4, E3, E2, then the measurement cross-cutting concern M immediately before E1 —, and the infrastructure E0 is deliberately consolidated last (Section 3.3.4); the E3→E2 boundary is already golden-tested in the current state, the remaining harnesses are fixed as the acceptance definition of the ongoing layer completion. The cross-cutting concern M has its own harness that gets by without any tier binary: a synthetic measurement source is driven through the system-axis root and its values traced into the result columns; its code shape is described in the following section.

4.5 Realm Roots in Code: System and Measurement Axes

The tripartition of the axis realms (Section 3.4.1) is executed in code as a type separation. All axes share a common family roof — an empty, non-polymorphic CRTP base whose concept guard rejects any smuggled-in vtable already at compile time — and declare their family via a discriminator with five values: the organ axes, the measurement system axes of the blood cross-cutting concern, the build and steering axes of the CEB configuration, the system meta axes under the extension hub, and the measurement-realm axes of the planner. Per family there is a dedicated CRTP root; an axis *is* its family through its base class, not through a re-attachable label — the CEB configuration root, for instance, returns its kind as a hard return value, which is why the measurement realm carries its own root instead of a reused one. Earlier versions of this chapter described at this point *one* common root of all non-organ axes (interim state of July 2026); the double root cut — the system realm roots at the builder, the measurement realm at the planner (Section 3.4.1) — has been completed in code since the axis reordering.

On the system side, the canonical order itself is code: a constant three-element array holds the system main axes `target_isa`, `operating_system`, and `external_utils` in their binding sequence, and every consumer that needs an order — the stamp line, the generator block sequence, the XML header, the suffix segments — must read it there instead of re-deriving it (order single source). Three *departure guards* fix the relocations of the former five-element list (Section 3.4.1) at compile time: `static_assert` checks reject compiler, scheduling, and load framework as system main axes — whoever re-enters one of the three breaks the build and must revoke the relocation explicitly instead of revising it silently. The build version identifier of the system line (Section 3.4.4) likewise draws from a single declarative source: an ordered eight-segment sequence of the build steering quantities in which an empty member simply produces no segment — it replaced three previously diverging concatenation chains.

On the measurement side, the planner carries its own meta-meta root; its first and currently only main axis is the load framework, which has left the system world without replacement (Section 3.4.1), and like all measurement-realm axes it is strictly binary-id-neutral. Its offer is held by four measurement registries as `constexpr` catalogues: the run methodology of a run (debug, measurement, release, and an already selectable compare method whose build row is deliberately identical to release and whose replay-comparison execution is still outstanding), the load framework (currently exactly one entry, YCSB in version 1.0.0), the write-back method (CSV, LaTeX table, comparison metrics), and the measurement tooling with exactly three versioned entries — wall clock, macro, micro —, whose guard turns a fourth, unagreed tooling into a compilation error. A dedicated PMC tooling does not exist: the counter instrumentation resides in the micro tooling. On the compile side, two separate gates mirror this tooling ladder: the *statistics gate* (`COMDARE_CE_ENABLE_STATISTICS`) switches the observer statistics of the macro regime, and the value-based *segment-timing gate* (`COMDARE_CE_ENABLE_SEGMENT_TIMING`) switches the eighteen per-axis segment timers of the fine grain. An unset segment-timing gate *inherits* the state of the statistics gate — existing builds do not change a single byte through the separation —, and a set segment-timing gate without the statistics gate breaks the compilation loudly, because the segment timers write into structures of the observer block. The actually effective gate state of every translation unit travels as a dedicated measurement-gate member (seven ; -separated fields) into the fingerprint preimage of the stamp (Section 3.4.4) — the stamp thus never claims an outfit the build does not carry. This registry cut is implementation *state*, not target: the target cut is the axis-interface model

from Section 3.4.5 — micro-benchmarking measures an axis through its axis interface, and the three wall-clock levels provide the counter-measurement —, and the registry converges towards this model.

The host-side measurement cross-cutting concern of the system realm — the *blood* of the apparatus, permeating all layers without ever touching the binary identity (Section 3.4.1) — roots in the CRTP base `SystemAxis` (file `system_axis.hpp`). It is built as a zero-cost abstraction — an empty, non-polymorphic base without virtual calls in the hot path, whose conformance is enforced by a dedicated concept — and translates the concept invariants into compile-time completeness gates: a `consteval` gate checks that each of the sixteen measurement categories is assigned to exactly one of the two regimes (nine time- and observer-based, seven counter-based PMC categories, Section 3.4.1), and a second one rejects any of these axes that mixes categories of both regimes — the regime-mixing ban thus breaks compilation rather than, later, the evaluation.

The sixteen measurement categories are carried by a single `constexpr` registry (`measurement_axis_registry.hpp`): per category the name and the regime derived from the one source. A static gate enforces that the table index matches the category value, that no name is empty, and that no regime deviates from the root assignment — a seventeenth category or a silent index drift thereby becomes a compilation error. Conceptually the categories are a *sub-axis* of the measurement tooling and manifest as columns of the result tables (Section 3.4.1); the registry names are at the same time the reporting vocabulary of the evaluation layer (the column names of the E4 reports), and a compile-time iteration over all entries makes the registry available to metaprogramming.

The acquisition is abstracted by a vendor-neutral measurement-source interface (`i_measurement_source.hpp`). `IMeasurementSource` describes a source by its *structural* capabilities — which of the ten event semantics (cycles, instructions, L1/L2/L3 cache, dTLB and branch misses, memory stalls, coherence invalidations, energy) it can express at all — and delivers per measurement interval a `MeasuredDelta` with *per-event validity*: the value zero with the validity flag set is a genuine zero measurement; an unset flag means *not measured*. The capability report never promises a measurement guarantee — the truth per measurement is carried by the validity flags alone. As a guaranteed, explicitly *degraded* fallback there is the wall-clock source (`WallClockSource`): it delivers only a nanosecond proxy for the cycles and leaves all hardware events honestly invalid; the existing PMC family is attached by an adapter that advertises only the seven structurally expressible channels.

The honest-0 doctrine (Section 3.4.1) is programmed into these building blocks rather than merely documented. The wall-clock system axis refuses to label its mean as a percentile: requests for p50 through p999 are answered as invalid, since the total time divided by the operation count is a mean, and a mean reported as a percentile would be a phantom value — genuine percentiles arise only downstream in the evaluation: nearest-rank over the collected raw latencies, complemented by the HDR histogram, which at three significant digits resolves with a relative bucket tolerance of $1/1024 \approx 0.0977\%$; the measurement path itself deliberately carries no histogram bucket, because it would irreversibly quantize the raw value already at recording time. The cache-line utilization is collected as a genuine utilization ratio — used payload bytes against occupied cache lines times line width —, not as a raw line counter, which would be precisely the *inverse* metric. Memory footprint and fill-buffer occupancy remain invalid until a genuine peak column or a hardware source exists (the software operation buffer of the queuing axis is no proxy for the hardware line-fill buffer), and IPC/CPI remains invalid as long as the counter POD carries no instruction and cycle columns.

The hardware probe — conceptually the third split main axis and a *measurement* axis (Section 3.4.3) — is present in code in both expansion stages in the state of this work. The first stage comprises the provenance vocabulary (the three trust levels as a dedicated result type; “not collected” is deliberately not a fourth level but a state of its own), a DDR5 SPD parser as pure, `constexpr`-capable byte functions over a passed-in memory span — under two hundred lines, without I/O and without vendored third-party code; overclocking profiles it recognizes as a separate offer, never as an actual value — as well as a dedicated error domain with four classes (source missing, unreadable, corrupt, unknown format), so that a corrupt source never silently degrades to the next level. The second stage wires the acquisition: the runtime chain boot cache → SPD → declaration is a chain of responsibility without virtual calls — its members are heterogeneous types, their wiring is a type list, so that the order (the essence of such a chain) resides in the type rather than in a runtime list; the chain stops at the highest reachable trust level, receives its source paths by injection (each stage is testable without the target platform), and runs exclusively, memoized once, at the release seam of the builder. Which chain applies is decided by a compile-time factory over the cell of ISA complex and operating system: its primary template is declared but not defined — a cell without a conscious decision is an incomplete type and breaks the build —, a totality guard checks the full matrix of all declared ISA complexes and documented operating systems, and unimplemented cells (Windows, macOS) are honest declaration-only cells with expressly occupied upper levels, so that the degradation path reports “asked, does not exist here” instead of “skipped”.

The comparison between declaration and acquisition is decided by a named runtime verdict with exactly seven outcomes — match, deviation, declared only, not collected, no declaration, incomparable levels, and label contradiction —, where only provenance-compatible quantities are compared at all; no verdict and no measured value ever reaches the stamp or identity APIs (Section 3.4.4). With this, the automated resubmission demanded in Section 3.4.3 is realized in code: the previously stored RAM nominal rate of 4800 MT/s — a once-collected, then frozen `dmidecode` result — is replaced by an acquisition that takes place anew on every run and carries its trust level along.

At the evaluation end of the cross-cutting concern lies the germ of the curve fitting from Section 3.6.3 as a skeleton (`curve_fit.hpp`): a measurement-axis curve over the working-set size is fitted by least squares to the model $y = a \log_2 x + b$ — the form expected for cache-hierarchy curves. Here too, honest emptiness replaces a phantom fit: the fit status distinguishes *no data*, *underdetermined* (fewer than two points or no spread of the x values), and *invalid data* (undefined logarithm, non-finite values) explicitly from the success case — never an *Ok* with NaN coefficients —, and the result reader accepts only strictly numeric cells and counts skipped rows as a diagnostic instead of silently turning them into phantom points.

Completed with this are the realm roots including order and guards, the registries, the source abstraction with a degraded fallback, the two-stage hardware probe, and the fit germ; the wiring of these cross-cutting building blocks into runner and probe dock as well as the mapping of the registry names onto the existing columns of the result files are the next layer increments (Section 4.4). The three concept invariants — system axes are not permutable, do not stand in the permutation tree, and never touch the binary identity — are preserved by the construction itself: the root docks onto the host-side result structures in a read-only fashion and remains invisible to the ABI boundary.

With this, the apparatus is fully described — from the repositories through the probe and the eighteen organ axes to the chain from plan to tier binary, their layer-wise quality assurance, and the realm roots in

code. Chapter 5 lays out the methodology by which this apparatus is assessed.

5 Evaluation

The evaluation first discloses the methodology and, building on it, analyses the measurement series.

5.1 Methodology

Three *hypotheses* guide the analysis:

H1 (page-type cost) The mean access cost per page-type variant differs systematically by workload: dense page types win, as expected, under Zipf distribution (YCSB-A), range-optimized pages under range scans (YCSB-E).

H2 (code quality per source) The code-quality score inherited from the original source repository — recorded per source repository as a documented quality audit over seven assessment axes (style, tests, documentation, maintainability, maintenance status, license, build system; scale 1–5, docs/quality_audit) and complemented by the documented per-function `is_original` marking (Appendix D) — correlates measurably with the achieved throughput.

H3 (inline vs. external vs. chain distribution) The observed distribution of the three ValueHandle variants correlates with page density: dense pages prefer inline, sparse pages external, and PRT-ART heuristics ChainRef.

These *evaluation hypotheses* are not to be confused with the identically named PRT-ART telemetry metrics H1–H3 (Section 3.5): the former are testable statements about the design space, the latter measurement organs of the probe that, among other things, supply the data for this test.

Phase 5 of the measurement pipeline (Execute, Section 4.3.2) derives the YCSB workload per permutation as a fallback heuristic from the profile’s traversal tag (Table 5.1), unless an `<expected_workload>` tag of the profile overrides the heuristic (Section 3.2); modules without a profile use the global default workload. The data basis is real string datasets of differing characteristics (Table 5.2), each with a reproducibility record of source, checksum, line count, preprocessing, and seed rule for the synthetic miss, update, and delete keys.

Table 5.1: Profile-aware workload routing (fallback heuristic)

Traversal tag	YCSB workload	Characteristic
RANGE_SCAN / RANGE_HOT_PATH	YCSB-E	range queries
HOT_PATH / PRTART_HOT_PATH	YCSB-A	50/50 read/update, Zipf
STANDARD (default)	YCSB-C	read-only

The measurements target a local workstation for development verification and the TU-Dresden ZIH cluster (Barnard CPU nodes with Intel Sapphire Rapids, Capella GPU) for the full sweeps, plus this work’s

Table 5.2: Real datasets and their characteristics

Dataset	Characteristic
url	very long keys, long common prefixes, medium alphabet
dna	small alphabet, high repetition, deep prefix entropy
protein	biological sequences, small to medium alphabet
xml	structured strings with recurring prefixes
tpcds-id	database-like IDs, sorted and unsorted
trec-terms	dictionary/search-index terms with defined miss keys

own production target machines (Ryzen 9 9950X3D, Intel Core i9-12900K) and an HBM platform. An `IPlatformProbe` (Phase 1) reports per platform the runnable subset of axes (e.g. AVX-512 only where available, HBM-aware only on HBM hardware), whereby the exclusion of an unavailable extension is always decided *machine-side* — with a warning and a named error class, never as a silent profile filter, for the experiment profile stays identical for all machines; per platform a scalar, an AVX2, and — on Barnard — an AVX-512 build are prepared, and on hybrid CPUs P and E cores are measured separately (via the dedicated counter domains `cpu_core` and `cpu_atom`) rather than aggregated together. The productive measurement operation itself is a *two-machine operation* — one AMD machine (the aforementioned Ryzen 9 9950X3D) alongside one Intel machine (Core i9-12900K, Alder Lake) —, and the counter-based collection runs for this in two separate vendor lanes (AMD and Intel) that are gated hard: only once both lanes have proven their counter prerequisites does a machine lane process its measurement strand — there is no silent partial measurement of only one vendor. On the production target machines, moreover, each measurement is to be collected under *two operating-system regimes* to separate production fidelity from measurement depth: an *immutable* operating system (Talos) mirrors production use and enforces boot/compile-static cache settings (Section 2.2); a *root Linux with full hardware-counter access* (`perf/MSR`), by contrast, is what makes the counter-based measurement categories (Section 3.2) and the runtime-configurable cache-near axis parameters (prefetch distance, hardware prefetcher) accessible in the first place. As of this version the second regime is *designed but not carried out*: all reported measurements originate from the root-Linux regime of the two production machines; the Talos mirror remains a requirement and will be collected after submission.

Which counters that access actually populates is a separate question. In the present state the measurement chain opens four generic counters via `perf_event_open`: L1D misses, last-level misses, dTLB misses, and, since M-3a, branch misses (`PERF_COUNT_HW_BRANCH_MISSES`, generic on both Intel and AMD). No portable generic counter exists for L2 misses or coherence invalidations, and a vendor-specific RAW counter is deliberately *not* guessed. IPC/CPI, by contrast, is not wired up; energy draw is wired up via a best-effort snapshot of the RAPL sysfs zone, but stays empty without root or zone read access — the normal case since Linux 5.10. The columns of the counters that are not wired up therefore do not carry a *measured* zero: they are to be read as *not implemented* and will follow with the next expansion stage. This distinction is part of the honesty doctrine of this work — a zero only where zero was in fact measured.

The time- and observer-based categories run identically under both regimes and secure comparability; the PMC categories are collected only under the privileged regime and logged per platform with operating system, kernel, and counter domain.

The scope of the full build is fixed as a *target*: per system permutation — per combination of the build control quantities of optimization level and SIMD extension declared in the profile (Section 3.4.1) — the $2^{17} = 131072$ organ permutations of the canonical measurement catalog will be built (Section 3.4.5). How many such build variants are due is decided by the profile alone; as of this thesis it declares four (two optimization levels times two SIMD levels), the most aggressive optimization level being deliberately absent because it would break the apparatus’s IEEE-754 floating-point determinism. Results of these full-build runs are pending (Section 5.2).

Fair comparability is secured by fixed rules: each comparison separates a common minimal mode (common denominator: external value handles, no PRT-ART special paths) from the PRT-ART native mode (inline switching and adaptive page-type selection per branch point); all methods receive identical, normalized byte keys as well as identical miss, update, and delete sets; warm-up, randomization, and thread pinning are logged per platform and workload. Missing capabilities of a baseline — e.g. range scans of a hash table — are reported as N/A rather than covertly emulated; for every foreign library the compiler, flags, ISA path, allocator, and commit hash are logged, and each configuration is measured in several separate runs; the latency percentiles are determined by the downstream evaluation nearest-rank from the raw values, complemented by the HDR distributional view (Section 4.5).

The measurement *operation* itself is part of these fairness rules. Measurement remains strictly separated from building: measurement runs never start automatically with a green build but are triggered explicitly — a green build proves buildability, not a measurement result. While the debug mode may run verification in parallel across all threads of a machine, the measurement mode measures with exactly *one* measurement thread; the genus’s own threading is left to the tier binary rather than overridden by the harness. A binary found in the store is skipped only if its green test attestation is inventoried at the binary, and every measurement is preceded by the conformity check at the probe dock (Section 4.4); build and measurement runs are processed per machine in milestone batches of 4096 binaries each at a stretch, with an evenly distributed allocation of the jointly buildable binaries across the machines and inventoried resumption after interruption.

Beyond the three hypotheses, the analysis receives a fourth, self-contained *evaluation dimension*: the heuristic counter-proof. Its concept is developed in Chapter 3 — the hybrid mode compiles the heuristically estimated recombination of the tier binaries itself into a tier binary and docks it at the probe dock (Section 3.6.2); the measurement-curve type system supplies the estimation basis, and its measurement-error detection is formulated by Section 3.6.3. The assessment question of this dimension reads: “Is the heuristically estimated recombination of the tier binaries faster than the existing paper algorithms?” Unlike H1–H3, which test statements about the design space, the counter-proof tests the thesis’s apparatus itself: whether the recombination estimated from the measurement curves delivers, in real wall-clock behaviour, what the curves promise.

The test plan for this is fixed in two stages; since the corresponding measurement runs are still pending, it is — like the target picture in Section 3.6.3 — formulated in the future tense. The first stage will quantify the measurement error that the measurement instances themselves introduce, for the heuristic’s estimation basis is *observed* curves, while its claim concerns unobserved production behaviour. To this end, the same experiment is set up in three build variants:

1. observers in *all* tier binaries *and* in the heuristic hybrid tier,

2. observers *only* in the heuristic tier binary,
3. observers in *none* of the compiled binaries — only the wall clock is measured.

The differences between the three variants quantify the observer overhead separated by its origin — measurement organs of the subordinate tiers (difference between variants 1 and 2) versus measurement commands of the hybrid tier (difference between variants 2 and 3) — and yield, per workload and platform, an *uncertainty band* below which two configurations count as indistinguishable.

The second stage will be the complementary *fourth step* of the final heuristic assembly: the tier binaries subordinate to the heuristic tier — not the finally optimized main binary — are rebuilt without micro- and macro-benchmarks and without observers, so that no measurement artefact remains in the compiled binary. The comparison is then made purely on the wall clock, in two comparison sets with a fixed precedence: primarily against the reconstructed paper algorithms — the paper comparison takes precedence, for it answers the assessment question literally —, secondarily against all known tier-binary permutations, in particular the best single binary; this second comparison classifies whether a potential lead is attributable to the heuristic as a whole or merely to a single superior permutation.

From this follows the decision rule: the counter-proof counts as *delivered* per workload class if the observer-free rebuilt recombination beats the fastest paper reconstruction on the wall clock and the margin exceeds the uncertainty band of the build variants; it counts as *missed* if a paper algorithm or the best single binary comes out ahead — this outcome, too, would be a result, for it would quantify for the first time how much estimation error lies between curve prognosis and real behaviour. The measurement data of this dimension are pending; they will be fed in after the first complete measurement runs together with series A–C in Section 5.2.

5.2 Results and Analysis

At the time of writing, methodology and evaluation pipeline are in place and verified on sample data; a first real smoke measurement series (43 permutations, real DLL measurement on the development machine) is documented in Appendix A together with its limitations (Section A.10), the complete runs of the mandatory measurement series on the target platforms are pending (Section 6.2). Per series run, the `messung_driver` writes under `Code/_runs/<date>/<spec_id>/` the files `measurements.csv`, `measurements.json`, and optional binary records. `measurements.csv` carries one row per permutation with the columns of the canonical measurement record: `op_count`, `total_cycles`, L1/L2/L3 cache misses, dTLB misses, coherence invalidations, energy, plus the memory fields (`bytes_allocated`, `bytes_in_use_peak`, `fragmentation`); branch misses have been captured for real since M-3a and are present as an additive schema extension of the record (`pmc_branch_misses`); throughput is still to be added as a schema extension and so far exists only in the command path. The evaluation enriches these files via `binary_to_csv` with profile ID, paper reference, and axis assignment, renders `csv_to_latex` fact sheets per permutation ID (in series A_defined this corresponds to one per SOTA profile) and `diagram_generator` diagrams (A4-aware TikZ bar charts, scatter plots, heatmaps), which the manuscript includes via `\input{tikz/<spec_id>/...}` and `\input{tabellen/<spec_id>...}` from the language-neutral directories. The CSV encoding is the implemented state; the xlsx encoding has been

decided as the future default of the evaluation: a dedicated writer will — as a factory over the output encodings — write *one* file per run with one sheet per selected sub-axis permutation and an info sheet (system information of the measuring machine, main axes used); the CSV output remains available as a configurable fallback of the same factory (then one file per sheet), and columns whose value never changes within a run move into the metadata instead of being repeated in every row. The three measurement series are then evaluated: series A (PRT-ART vs. the state of the art, with the evaluation hypotheses H1–H3 from Section 5.1), series B (systematic variation of the cache-engine permutations, fed from the full join of stage 3) and series C (merge/regression of old against new, across builds; Table 3.8). Orthogonal to this assignment lies the *probe’s measurement schema* declared in the user XML with its three modes: in *replace* mode the probe’s axes replace the cache engine’s standard assignment entirely (the default of stage 2), in *hybrid* mode the joint cache-engine–probe hybrid is measured per probe, and the *full-join* mode of stage 3 combines all axis variants without discarding anything. Series A operates in the first two modes, series B in the third; the hybrid and full-join states remain distinguishable at the binary itself, for the mixing strategy shows in the stamp as the *e* suffix of the version tag and in hierarchically extended algorithm names of the organ line (Section 3.4.4); it carries no stamp line of its own, because the probe mixes against the organ axes and thus already stands in their line. For pure replace as for pure cache-engine builds, suffix and name extension consequently stay absent.

The core of the evaluation is the *axis sensitivity analysis*, which directly answers the separability problem of the task: in the state of the art the weights of the improvements of *individual* algorithm constituents are so far not separable, because each publication demonstrates its benefit in its own, incompatible measurement environment. By isolating each constituent as an interchangeable axis over an otherwise constant structure, the axis framework makes the contribution of each design decision separable — for the first time under all-else-equal conditions — and measurably attributable. From the records per permutation, the analysis quantifies at the micro granularity — the measurement of each axis via its *axis interface*, the set of functions that all algorithms of that axis must provide (Section 3.4.5) — which axis contributes most to the performance variance; macro and whole-system benchmarking add the level of the genus-interface operations and the YCSB load-profile level, the wall-clock counter-measurement of the axis interfaces checks the internal micro measurement from outside, and from the ranking across the three granularities follows the recommendation of standard configurations per workload class. The results are finally discussed against the thesis’s leitmotif — the transition from heuristic to informed cache management, i.e. the question whether telemetry-driven, automatically adapted configurations outperform statically chosen ones. Chapter 6 draws the conclusion from this.

6 Conclusion and Outlook

This chapter takes stock: it answers the research questions RQ0–RQ4 from Section 1.3 together with their sub-questions, names the limits of the state reached, and sketches the next steps.

6.1 Answering the Research Questions

Orthogonal to all five questions, the introduction had deliberately left a categorization question open: whether the axes of the design space separate cleanly into organ, system, and measurement axes — and how such a separation is to be accomplished where it does not yet exist. Both are answered (Section 3.4.1): the axes separate along the three concerns into three *realms* with one class root each, one offer catalog each, and one stamp line each at the binary (Section 3.4.4). The separation was accomplished not by renaming but by datable *relocations*: telemetry and the ISA left the organ count for the system sphere, compiler and scheduling moved from the former five-item system list into the complex bracket and onto the ISA wrapper respectively, and the load framework left the system world without replacement towards the measurement realm; in the code, compile-time departure guards fix these relocations (Section 4.5). This triad is the second shape of the separability problem and carries the answers to the following sub-questions.

RQ0 (Main question). By how much active, measurement-driven cache-aware decisions outperform their passive, statically dimensioned counterparts can be quantified conclusively only after the end-to-end runs per platform (Section 6.3). What is prepared is the apparatus that isolates exactly this difference: the same axis-constant structure is permuted once actively (measurement-driven page-type, layout, prefetch, and value-placement decision) and once passively (statically dimensioned) and compared under an identical compiler and flag basis on hybrid CPUs and Sapphire Rapids. The extension sub-question — whether a measured gain can be attributed unambiguously at all — is answered structurally, and the separation it requires is precisely the triad: an organ effect permutes the `binary_id`, a platform concern stands at the binary as system stamp and build version identifier without permuting it, and a measurement concern remains strictly binary-id-neutral — no measurement instrument may change *what* is built.

RQ1 (Composition). The canonical decomposition carries: *eighteen* organ main axes in fixed slot order T0–T17 span the permutation matrix (Section 3.4.1); the former nineteen-axis count (interim state of July 2026) is a historical precursor of this order. The canonical measurement catalog — two enabled representatives per organ axis, the persistence axis pinned to its in-memory representative — counts $2^{17} = 131072$ binary identities per system permutation, each with a full eighteen-segment path; the raw variant product space across all axis catalogs including their sub-axes lies orders of magnitude above that (Section 3.4.5). State-of-the-art designs are reconstructed unambiguously through algorithm-profile XMLs: 30 SOTA profiles (8 Rank-1 + 22 Rank-2/3) demonstrate that the axis decomposition carries P01–P33, and the variadic API specialization ($1/2/N$ parameters onto `std::vector/std::map/tuple`)

makes both algorithm- and container-oriented patterns expressible on one engine. The sub-question on the category cut is answered in the affirmative: the decomposition remains orthogonal only because platform and measurement concerns are kept out of the permutation matrix — the cut was accomplished by the dated relocations and is held by the departure guards in the code (Section 4.5). The non-fully-orthogonal organ couplings (node type $\times$ SIMD $\times$ path compression) are marked as such and explicitly restricted during permutation. The sub-question on cache-update and cache-line-fill behaviour under a solely varied ISA/SIMD axis is answered methodically — precisely *because* the SIMD extension is not an organ axis: as a build control quantity it multiplies only the build matrix, so that one and the same `binary_id` can be built under several system permutations and compared with an otherwise unchanged structure (Section 3.4.1); the empirical answer follows with the full-build runs (Section 6.3).

RQ2 (Measurement methodology). Measurement is split into a production mode (`COMDARE_MEASUREMENT_ON` disabled, no overhead) and an experiment mode: the `ResultAggregator` is kept host-side in the `ExperimentDriver` (phases 5–7), and the experiment mode activates the measurement hooks and the observer coupling in the ABI adapter. Profile-aware workload routing selects the workload per permutation from the traversal tag, ensuring fair comparison between range-query- and point-lookup-oriented designs; this routing concerns only the default assignment of series A, while the answer to the home-field bias remains the bias-free full matrix, which in whole-system benchmarking runs all load profiles as a dynamic axis against all living-being binaries (Chapter 3). Each series is collected at three granularities: micro-benchmarking measures each axis over its *axis interface*, macro- and whole-system benchmarking add the genus interface operations and the load-profile level, and three wall-clock levels check the internal micro-measurement from the outside (Section 3.4.5). The sub-question on the measurement axes is answered in the affirmative: the measurement instruments are a realm of their own in the planner — the load framework as its first main axis, the measurement tooling with the sixteen categories as a sub-axis —, always present as soon as measurement is on, and strictly binary-id-neutral; this separation was accomplished by the double root cut — the system realm is rooted at the builder, the measurement realm at the planner (Section 3.4.1). Contradictory findings in the literature on the optimal node/cache-line size (one versus up to sixteen cache lines) are thus not adopted from incompatible original setups but re-measured axis-isolated, with all else held equal. The extensibility sub-question is answered in concept and honestly unfinished in build-out: a new probe joins as a living being of its genus — it docks at the probe dock, passes the three-stage check, and its measurement schema (replace, hybrid, or full-join mode) remains distinguishable at the binary via the `e` suffix of the version tag and the hierarchically extended algorithm names of the organ line — no stamp line of its own is needed for that, because the probe mixes against the organ axes (Section 5.2); whether it matches an existing category, founds a new one, or constitutes a hybrid form is decided by the genus cut — a genus of its own would arise only with a fundamentally different base axis set (Section 3.3.2). The operating mode asked for there, which offers the design components of exactly *one* publication as a closed axis set in isolation, is by contrast not built and remains outlook (Section 6.3). A platform counts as successfully verified only once the conformance gate confirms the `std::map` contract per module and the platform probe covers the reported axis subset; the platform-calibrated thresholds themselves are fixed with the real runs.

RQ3 (Probe comparison). The concrete comparison of PRT-ART against the 30 SOTA profiles is pending the first measurement runs (Section 6.3). The architecture is fully prepared: the PRT-ART profile, the three mandatory series templates, the `ExperimentDriver` with auto-pickup of the SOTA profiles, and the ABI-conformant adapter with the `std::map` core contract (conformance subset, Appendix F); the full expansion to all seventeen contract operations including the notify hooks is designated as a follow-up step. The cheapest local page representation per branch point (point versus compact page) and its platform-calibrated switching rules are likewise part of the PRT-ART profile. The compile-time sub-question — whether latency-critical measurement forces decisions selectable at run time as such into compile time — is answered in the affirmative and has become the construction principle: every axis divides into a compile-time-fixed *main* decision and runtime-variable *sub*-decisions; the main axes are compiled in statically — for organ axes as a `binary_id` segment, for system axes as stamp plus build version identifier —, while dynamic sub-axes vary their runtime parameters without recompilation (Section 3.4). Exactly the division the sub-question conjectured is thus the carrying doctrine of the apparatus; RQ4 continues it.

RQ4 (Compile time vs. run time). The boundary between compile time and run time runs along the axis categories, and the system extension sub-question is thereby answered in the affirmative: the platform concerns — the three system main axes together with their complex bracket (Section 3.4.2) — belong to the compile-time identity of the binary; they appear as system stamp line and build version identifier, never as a `binary_id` segment, and produce provenance-verifiable binaries per permutation (SHA256-validated original bodies — whose full implementation Section 6.2 lists as open —, `get_compiler()` identifier). Measurement choices remain runtime-variable, and layout and page selection remain selectable at run time via a bit-encoded permutation identifier, without violating the common compiler and flag basis. Where an axis needs both sides of the boundary, it is *two-way-split*: load framework, telemetry, and hardware detection run across the runtime of the planner *and* the compile time of the builder — the sub-axis release of the earlier stage becomes the main-axis assumption of the later one (Section 3.4.1) —, and the two-way-split hardware detection uses its level 2 to set the compile-time stamps of the tier binaries correctly from the actual properties acquired at builder runtime (Section 3.4.3). What is stamped is always the compile-time platform *class*, never a measured value; as a *design*, the concept of a *fit stamp* follows on from this: it would check the stamped platform class on every measurement run against the named runtime verdict of the hardware detection (Section 4.5) — the fit between machine and binary would thereby itself become a checked quantity; in the state of this work this concept is design, not built (Section 6.3). The forward-looking sub-question of whether the measurement itself can tailor ISA- and operating-system-optimized binaries for a recombination is taken up by the outlook as heuristics automation (Section 6.3); the operating-system dimension is already anchored in the measurement plan via the two measurement regimes from Chapter 5.

6.2 Limitations

- **Module bodies:** the ad-hoc codegen of the active measurement path emits real axis-library bodies per permutation (in the state of this work the eighteen fully qualified axis types plus the umbrella include); only the legacy profile codegen still produces ABI-conformant skeleton modules, and the

remaining gaps are the PRT-ART probe stub and the honestly zero-reported storage/prefetch axes of the pool families.

- **Full build and delivery measurement pending:** the declared build target (Section 5.1) — per system permutation the 2^{17} organ permutations of the canonical catalog — has not yet been run, nor has the delivery measurement over the small, materializable 320-identity catalog (Section 4.4); the end-to-end `messung_driver` runs across the target platforms (AVX2/AVX-512, ARM NEON/SVE) are outstanding, while the apparatus-side acceptance per package already runs through the two-gate protocol.
- **Results chapter:** currently methodological; concrete measurements follow. A first real smoke measurement series (43 permutations) is available in Appendix A, but does not replace the mandatory measurement series.
- **Hardware counters off by default and partly source-dependent:** PMC capture (L1, L3, dTLB, and, since M-3a, branch misses via `perf_event_open`) is disabled by default; L2 misses and coherence invalidations stay structurally at zero regardless of the switch for lack of a portable counter, the last-level counter can additionally fail to open on individual AMD platforms, and energy capture via RAPL is wired up best-effort but stays empty without root or zone read access; IPC is planned as an extension of the counter set and is currently not captured. In the smoke series so far the hardware columns are zero — the cache-aware node/data/index measurement component is thus still pending (Figure 3.11).
- **Hardware detection in partial build-out:** the first two build-out stages — provenance vocabulary, SPD parser, and error domain as well as acquisition chain, compile-time factory, and runtime verdict — are available as code (Section 4.5); outstanding are the wiring into runner and probe dock, the additive result write-back including its consumers, the provenance round trip of the machine declarations, and the fit-stamp check (Sections 3.4.3 and 6.1).
- **Evaluation encoding:** CSV is the implemented state; the `xlsx` factory writer decided as the future default (one sheet per selected sub-axis permutation plus an info sheet, Section 5.2) is not yet built.
- **Smoke series with apparatus caveats:** the real measurements so far vary seven axes of the then nineteen-axis count (interim state of July 2026) in two sub-blocks (27 SIMD×layout×allocator combinations plus 16 combinations over node width, path compression, internal search, and telemetry), and partly approximate axis effects via wall-clock proxies; single effects are apparatus rather than organ artefacts (“shadowing”). The complete limitation table is given in Section A.10.

6.3 Outlook

In the short term: the full build following the declared target (Section 5.1), the delivery measurement over the 320-identity catalog, the end-to-end run of the three mandatory series on real hardware including the generation of the measurement diagrams, the build-out of the hardware detection by wiring, result write-back, and fit stamp, the decided `xlsx` factory writer, and the closing of the remaining module-body gaps (PRT-ART stub, honest-0 storage/prefetch axes of the pool families). To this is added the paper-isolation

mode designated as outlook in RQ2: an operating mode that offers the design components of exactly one publication as a closed axis set in isolation, in order to measure them faithfully to the original — with measurement and system axes remaining free-standing.

Heuristic automation and binary delivery. The mechanism demanded by the thesis assignment (sub-task 7), which automatically derives the best configuration from the measurement results, is deliberately not yet implemented in its heuristics-driven *full* automation and is reserved for a follow-up phase; a first increment of the automatic selection and binary shipping, however, is already available as a standalone tool (`best_binary_selector`) that ranks and delivers the best permutation per metric from the measurement CSVs. The full expansion presupposes three components; the second is already implemented, and toward the first and the third this tool provides a first, purely measurement-driven increment: (i) a *heuristic/ML classifier* that maps the load-profile and axis features (operation mix, key characteristics, hardware identifier) to the recommended axis configuration; (ii) the already implemented *XML parser* (import and export including the extractor) for the heuristic policies stored as an XML load-profile result and reusable per architecture focus; and (iii) a *binary packager* that delivers the thus-selected overall algorithm as a standalone binary behind the unified search-algorithm interface. A prerequisite for all three is that the end-to-end measurement series be fully available beforehand — only the robust data basis across all permutations and load profiles makes a measurement-driven, automatically controlled selection meaningful; until then, the configuration recommendations of the evaluation scheme from Section 5.2 will initially be derived manually once the data are available.

In the medium to long term: a GPU adapter (Grace Hopper cluster, ZIH), PIM-allocator integration (A16), and formal verification of selected permutation classes (StarMalloc-style, A13). The three-repository architecture lets future students add further probe algorithms analogously to PRT-ART without modifying the cache-engine building-block library — a key design principle for the scalability of the research infrastructure. Beyond the tree-like search structures treated here, the anatomy principle extends through higher-level abstractions to the further structure classes of Section 2.1 — hash-based, spatial, and flat structures —, so that the same measurement and permutation apparatus can be applied to entirely new classes without rebuilding it; herein lies a key lever for future experimental progress.

Bibliography

- [ABKS13] Sven Apel, Don Batory, Christian Kästner, and Gunter Saake. *Feature-Oriented Software Product Lines: Concepts and Implementation*. Springer, 1 edition, 2013.
- [ACK⁺04] Sanjay Agrawal, Surajit Chaudhuri, Lubor Kollár, Arunprasad P. Marathe, Vivek R. Narasayya, and Manoj Syamala. Database tuning advisor for Microsoft SQL server 2005. In *30th International Conference on Very Large Data Bases (VLDB)*, pages 1110–1121, 2004.
- [ADHW99] Anastassia Ailamaki, David J. DeWitt, Mark D. Hill, and David A. Wood. DBMSs on a modern processor: Where does time go? In *Proceedings of the 25th International Conference on Very Large Data Bases (VLDB)*, pages 266–277, 1999.
- [Adv23] Advanced Micro Devices. Software optimization guide for the AMD Zen microarchitecture (family 19h). Advanced Micro Devices, Santa Clara, CA, USA, 2023.
- [AKLS15] Christoph Aigner, Christoph Kirsch, Michael Lippautz, and Ana Sokolova. Fast, multicore-scalable, low-fragmentation memory allocation. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 513–531. ACM, 2015.
- [AKV⁺14] Jason Ansel, Shoaib Kamil, Kalyan Veeramachaneni, Jonathan Ragan-Kelley, Jeffrey Bosboom, Una-May O’Reilly, and Saman Amarasinghe. Opentuner: An extensible framework for program autotuning. In *23rd International Conference on Parallel Architectures and Compilation Techniques (PACT)*, pages 303–316, 2014.
- [Ale01] Andrei Alexandrescu. *Modern C++ Design: Generic Programming and Design Patterns Applied*. Addison-Wesley, 1 edition, 2001.
- [AMH08] Daniel J. Abadi, Samuel R. Madden, and Nabil Hachem. Column-stores vs. row-stores: How different are they really? In *ACM SIGMOD International Conference on Management of Data*, pages 967–980, 2008.
- [Apa26] Apache Software Foundation. Maven — POM reference. <https://maven.apache.org/pom.html>, abgerufen am 01.08.2026, 2026.
- [APGZ17] Dana Van Aken, Andrew Pavlo, Geoffrey J. Gordon, and Bohan Zhang. Automatic database management system tuning through large-scale machine learning. In *ACM SIGMOD International Conference on Management of Data*, pages 1009–1024, 2017.
- [App24] Apple Inc. Apple silicon CPU optimization guide. Apple Inc., Cupertino, CA, USA, Version 4, 2024.

- [APV02] Lars Arge, Octavian Procopiuc, and Jeffrey Scott Vitter. Implementing I/O-efficient data structures using TPIE. In *Algorithms — ESA 2002, 10th Annual European Symposium*, volume 2461 of *Lecture Notes in Computer Science*, pages 88–100. Springer, 2002.
- [Arm24] Arm Limited. Arm architecture reference manual for A-profile architecture (DDI 0487). Arm Limited, Cambridge, UK, <https://developer.arm.com/documentation/ddi0487/latest>, abgerufen am 18.09.2026, 2024.
- [Aßm03] Uwe Aßmann. *Invasive Software Composition*. Springer, 1 edition, 2003.
- [AV88] Alok Aggarwal and Jeffrey Scott Vitter. The input/output complexity of sorting and related problems. *Communications of the ACM*, 31(9):1116–1127, 1988.
- [AVL62] Georgy Adelson-Velsky and Evgenii Landis. An algorithm for the organization of information. *Doklady Akademii Nauk SSSR*, 146:263–266, 1962.
- [BA01] Jeff Bonwick and Jonathan Adams. Magazines and vmem: Extending the slab allocator to many cpus and arbitrary resources. In *Proceedings of the USENIX Annual Technical Conference (ATC)*. USENIX, 2001.
- [Bay72] Rudolf Bayer. Symmetric binary B-trees: Data structure and maintenance algorithms. *Acta Informatica*, 1(4):290–306, 1972.
- [BBB⁺11] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K. Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R. Hower, Tushar Krishna, Somayeh Sardashti, Rathijit Sen, Korey Sewell, Muhammad Shoaib, Nilay Vaish, Mark D. Hill, and David A. Wood. The gem5 simulator. *ACM SIGARCH Computer Architecture News*, 39(2):1–7, 2011.
- [BBF09] Gordon Blair, Nelly Bencomo, and Robert B. France. Models@run.time. *Computer*, 42(10):22–27, 2009.
- [BCD⁺02] Michael A. Bender, Richard Cole, Erik D. Demaine, Martin Farach-Colton, and Jack Zito. Two simplified algorithms for maintaining order in a list. In *Algorithms — ESA 2002, 10th Annual European Symposium*, volume 2461 of *Lecture Notes in Computer Science*, pages 152–164. Springer, 2002.
- [BCDFC02] Michael A. Bender, Richard Cole, Erik D. Demaine, and Martin Farach-Colton. Scanning and traversing: Maintaining data for traversals in a memory hierarchy. In *Algorithms — ESA 2002, 10th Annual European Symposium*, volume 2461 of *Lecture Notes in Computer Science*, pages 139–151. Springer, 2002.
- [BDFC00] Michael A. Bender, Erik D. Demaine, and Martin Farach-Colton. Cache-oblivious b-trees. In *Proceedings of the 41st Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 399–409. IEEE, 2000.
- [BDFC02] Michael A. Bender, Erik D. Demaine, and Martin Farach-Colton. Efficient tree layout in a multilevel memory hierarchy. In *Algorithms — ESA 2002, 10th Annual European*

- Symposium*, volume 2461 of *Lecture Notes in Computer Science*, pages 165–173. Springer, 2002.
- [BDFC05] Michael A. Bender, Erik D. Demaine, and Martin Farach-Colton. Cache-oblivious b-trees. *SIAM Journal on Computing*, 35(2):341–358, 2005.
- [Ben75] Jon Louis Bentley. Multidimensional binary search trees used for associative searching. *Communications of the ACM*, 18(9):509–517, 1975.
- [BFTV24] Antonio Boffa, Paolo Ferragina, Francesco Tosoni, and Giorgio Vinciguerra. CoCo-trie: Data-aware compression and indexing of strings. *Information Systems*, 120:102316, 2024.
- [BH23] Johannes Berthold and Dirk Habich. VAMPIR/OTF2 profiling and performance analysis for database workloads. In *Proceedings of the 19th International Workshop on Data Management on New Hardware (DaMoN)*, 2023. Poster presentation.
- [Blo70] Burton H. Bloom. Space/Time Trade-offs in Hash Coding with Allowable Errors. *Communications of the ACM*, 13(7):422–426, 1970.
- [BM72] Rudolf Bayer and Edward M. McCreight. Organization and maintenance of large ordered indexes. *Acta Informatica*, 1(3):173–189, 1972.
- [BMBW00] Emery D. Berger, Kathryn S. McKinley, Robert D. Blumofe, and Paul R. Wilson. Hoard: A scalable memory allocator for multithreaded applications. In *Proceedings of the Ninth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS IX)*, pages 117–128. ACM, 2000.
- [BO92] Don Batory and Sean O’Malley. The design and implementation of hierarchical software systems with reusable components. *ACM Transactions on Software Engineering and Methodology*, 1(4):355–398, 1992.
- [Bon94] Jeff Bonwick. The slab allocator: An object-caching kernel memory allocator. In *Proceedings of the USENIX Summer 1994 Technical Conference*, pages 87–98. USENIX, 1994.
- [BZP⁺18] Robert Binna, Eva Zangerle, Martin Pichl, Günther Specht, and Viktor Leis. HOT: A height optimized trie index for main-memory database systems. In *ACM SIGMOD International Conference on Management of Data*, pages 521–534, 2018.
- [CE00] Krzysztof Czarnecki and Ulrich W. Eisenecker. *Generative Programming: Methods, Tools, and Applications*. Addison-Wesley, 1 edition, 2000.
- [CGM01] Shimin Chen, Phillip B. Gibbons, and Todd C. Mowry. Improving index performance through prefetching. In *Proceedings of the 2001 ACM SIGMOD International Conference on Management of Data*, pages 235–246. ACM, 2001.
- [CGM02] Shimin Chen, Phillip B. Gibbons, and Todd C. Mowry. Fractal prefetching b+-trees. In *Proceedings of the 2002 ACM SIGMOD International Conference on Management of Data*, pages 157–168. ACM, 2002.

- [CHL99] Trishul M. Chilimbi, Mark D. Hill, and James R. Larus. Cache-conscious structure layout. In *Proceedings of the ACM SIGPLAN 1999 Conference on Programming Language Design and Implementation (PLDI)*, pages 1–12, 1999.
- [CHP71] Pierre-Jacques Courtois, Frans Heymans, and David Lorge Parnas. Concurrent Control with “Readers” and “Writers”. *Communications of the ACM*, 14(10):667–668, 1971.
- [Cli93] Norman Cliff. Dominance statistics: Ordinal analyses to answer ordinal questions. *Psychological Bulletin*, 114(3):494–509, 1993.
- [CLP22] Andrew Crotty, Viktor Leis, and Andrew Pavlo. Are You Sure You Want to Use MMAP in Your Database Management System? In *12th Conference on Innovative Data Systems Research (CIDR 2022)*, Chaminade, CA, USA, January 9-12, 2022. www.cidrdb.org, 2022.
- [CLP23] Supawit Chockchowwat, Wenjie Liu, and Yongjoo Park. AirIndex: Versatile index tuning through data and storage. *Proceedings of the ACM on Management of Data*, 1(3):204:1–204:26, 2023. Artikel-Nr. 204; präsentiert auf der ACM-SIGMOD-Konferenz 2024.
- [Com79] Douglas Comer. The ubiquitous B-tree. *ACM Computing Surveys*, 11(2):121–137, 1979.
- [CST⁺10] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. Benchmarking cloud serving systems with YCSB. In *1st ACM Symposium on Cloud Computing (SoCC)*, pages 143–154, 2010.
- [Des14] Cameron Desrochers. moodycamel::ConcurrentQueue: A fast multi-producer, multi-consumer lock-free queue for C++. <https://github.com/cameron314/concurrentqueue>, 2014.
- [Dij65] Edsger W. Dijkstra. Solution of a Problem in Concurrent Programming Control. *Communications of the ACM*, 8(9):569, 1965.
- [dlB59] René de la Briandais. File searching using variable length keys. In *Western Joint Computer Conference*, pages 295–298, 1959.
- [dM10] Arnaldo Carvalho de Melo. The new linux perf tools. In *Linux Kongress*, pages 1–42, 2010. <http://oldvger.kernel.org/~acme/perf/lk2010-perf-paper.pdf>.
- [DPT⁺13] Justin DeBrabant, Andrew Pavlo, Stephen Tu, Michael Stonebraker, and Stan Zdonik. Anti-caching: A new approach to database management system architecture. *Proceedings of the VLDB Endowment*, 6(14):1942–1953, 2013.
- [Dre07] Ulrich Drepper. What every programmer should know about memory. Technical report, Red Hat, Inc., 2007. White paper, <https://www.akkadia.org/drepper/cpumemory.pdf>.
- [DSST89] James R. Driscoll, Neil Sarnak, Daniel Dominic Sleator, and Robert Endre Tarjan. Making Data Structures Persistent. *Journal of Computer and System Sciences*, 38(1):86–124, 1989.

- [Eva06] Jason Evans. A scalable concurrent malloc(3) implementation for freebsd. In *Proceedings of BSDCan 2006*. BSDCan, 2006. <https://people.freebsd.org/~jasone/jemalloc/bsdcan2006/jemalloc.pdf>.
- [FAK⁺12] Michael Ferdman, Almutaz Adileh, Onur Kocberber, Stavros Volos, Mohammad Alisafae, Djordje Jevdjic, Cansu Kaynak, Adrian Daniel Popescu, Anastasia Ailamaki, and Babak Falsafi. Clearing the clouds: A study of emerging scale-out workloads on modern hardware. In *17th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 37–48, 2012.
- [FAKM14] Bin Fan, David G. Andersen, Michael Kaminsky, and Michael Mitzenmacher. Cuckoo Filter: Practically Better Than Bloom. In *Proceedings of the 10th ACM International Conference on Emerging Networking Experiments and Technologies (CoNEXT '14)*, pages 75–88. ACM, 2014.
- [FB74] Raphael A. Finkel and Jon Louis Bentley. Quad trees: A data structure for retrieval on composite keys. *Acta Informatica*, 4(1):1–9, 1974.
- [FJ98] Matteo Frigo and Steven G. Johnson. FFTW: An adaptive software architecture for the FFT. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, volume 3, pages 1381–1384, 1998.
- [FJKN20] Philipp Fent, Michael Jungmair, Andreas Kipf, and Thomas Neumann. START: Self-tuning adaptive radix tree. In *36th IEEE International Conference on Data Engineering Workshops (ICDEW)*, pages 147–153, 2020.
- [FLPR99] Matteo Frigo, Charles E. Leiserson, Harald Prokop, and Sridhar Ramachandran. Cache-oblivious algorithms. In *40th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 285–297, 1999.
- [Fre60] Edward Fredkin. Trie memory. *Communications of the ACM*, 3(9):490–499, 1960.
- [FW93] Michael L. Fredman and Dan E. Willard. Surpassing the information theoretic bound with fusion trees. *Journal of Computer and System Sciences*, 47(3):424–436, 1993.
- [GHJV94] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1 edition, 1994.
- [GL01] Goetz Graefe and Per-Åke Larson. B-tree indexes and cpu caches. In *Proceedings of the 17th IEEE International Conference on Data Engineering (ICDE)*, pages 349–358. IEEE, 2001.
- [GL20] Thomas Mueller Graf and Daniel Lemire. Xor Filters: Faster and Smaller Than Bloom and Cuckoo Filters. *ACM Journal of Experimental Algorithmics*, 25(1):1.5:1–1.5:16, 2020.
- [GM05] Sanjay Ghemawat and Paul Menage. TCMalloc: Thread-caching malloc. Technical report, Google, 2005. Design documentation.

- [GMUW09] Hector Garcia-Molina, Jeffrey D. Ullman, and Jennifer Widom. *Database Systems: The Complete Book*. Pearson Prentice Hall, Upper Saddle River, NJ, USA, 2 edition, 2009.
- [Goo18] Google. Abseil swiss tables (flat_hash_map). <https://abseil.io/about/design/swisstables>, 2018.
- [Göt13] Sebastian Götz. *Multi-Quality Auto-Tuning by Contract Negotiation*. PhD thesis, Technische Universität Dresden, 2013. URN: urn:nbn:de:bsz:14-qucosa-119938.
- [GS78] Leo J. Guibas and Robert Sedgewick. A dichromatic framework for balanced trees. In *19th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 8–21, 1978.
- [Gut84] Antonin Guttman. R-trees: A dynamic index structure for spatial searching. In *ACM SIGMOD International Conference on Management of Data*, pages 47–57, 1984.
- [HB15] Torsten Hoefler and Roberto Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2015.
- [HBHR11] Jörg Herter, Peter Backes, Florian Hauptenthal, and Jan Reineke. CAMA: A predictable cache-aware memory allocator. In *23rd Euromicro Conference on Real-Time Systems (ECRTS)*, pages 23–32, 2011.
- [Her91] Maurice Herlihy. Wait-Free Synchronization. *ACM Transactions on Programming Languages and Systems*, 13(1):124–149, 1991.
- [HIKY12] Felix Halim, Stratos Idreos, Panagiotis Karras, and Roland H. C. Yap. Stochastic database cracking: Towards robust adaptive indexing in main-memory column-stores. *Proceedings of the VLDB Endowment*, 5(6):502–513, 2012.
- [HKT⁺21] A. H. Hunter, Chris Kennelly, Paul Turner, Darryl Gove, Tipp Moseley, and Parthasarathy Ranganathan. Beyond malloc efficiency to fleet efficiency: A hugepage-aware memory allocator. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 257–273, 2021. <https://www.usenix.org/conference/osdi21/presentation/hunter>.
- [Hol79] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- [HP03] Richard A. Hankins and Jignesh M. Patel. Effect of node size on the performance of cache-conscious b+-trees. In *Proceedings of the 2003 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, pages 120–131. ACM, 2003.
- [HP19] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 6th edition, 2019.

- [HS89] Mark D. Hill and Alan Jay Smith. Evaluating associativity in CPU caches. *IEEE Transactions on Computers*, 38(12):1612–1630, 1989.
- [IKM07] Stratos Idreos, Martin L. Kersten, and Stefan Manegold. Database cracking. In *3rd Biennial Conference on Innovative Data Systems Research (CIDR)*, pages 68–78, 2007.
- [Int24a] Intel Corporation. Intel 64 and IA-32 architectures optimization reference manual. Intel Corporation, Santa Clara, CA, USA, 2024.
- [Int24b] Intel Corporation. Intel 64 and IA-32 architectures software developer’s manual, volume 3a: System programming guide. Intel Corporation, Santa Clara, CA, USA, <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>, abgerufen am 18.09.2026, 2024.
- [ISO24] ISO/IEC. ISO/IEC 14882:2024 — programming languages — c++. International Organization for Standardization, Geneva, Switzerland, <https://www.iso.org/standard/83626.html>, abgerufen am 18.09.2026, 2024.
- [IZA⁺18] Stratos Idreos, Kostas Zoumpatianos, Manos Athanassoulis, Niv Dayan, Brian Hentschel, Michael S. Kester, Demi Guo, Lukas M. Maas, Wilson Qin, Abdul Wasay, and Yiyu Sun. The periodic table of data structures. *IEEE Data Engineering Bulletin*, 41(3):64–75, 2018.
- [IZH⁺18] Stratos Idreos, Kostas Zoumpatianos, Brian Hentschel, Michael S. Kester, and Demi Guo. The data calculator: Data structure design and cost synthesis from first principles, and learned cost models. In *ACM SIGMOD International Conference on Management of Data*, pages 535–550, 2018.
- [Jac89] Guy J. Jacobson. Space-efficient static trees and graphs. In *Proceedings of the 30th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 549–554. IEEE, 1989.
- [Jan17] Mattias Jansson. rpmalloc: A public domain license per-thread memory allocator. GitHub Repository, 2017.
- [JED20] JEDEC Solid State Technology Association. DDR5 SDRAM standard (JESD79-5). Technical report, JEDEC Solid State Technology Association, Arlington, VA, USA, July 2020. Norm.
- [JTJE10] Aamer Jaleel, Kevin B. Theobald, Simon C. Steely Jr., and Joel Emer. High performance cache replacement using re-reference interval prediction (RRIP). In *37th Annual International Symposium on Computer Architecture (ISCA)*, pages 60–71, 2010.
- [KAS21] Neal Koyle, Sujeet Ayyapureddi, and Mike Seibert. Micron DDR5: Client Module Features. White Paper CCM004676576390-11566, Rev. A, Micron Technology, Inc., October 2021. Online-Ressource.
- [KBC⁺18] Tim Kraska, Alex Beutel, Ed H. Chi, Jeffrey Dean, and Neoklis Polyzotis. The case for learned index structures. In *ACM SIGMOD International Conference on Management of Data*, pages 489–504, 2018.

- [KCH⁺90] Kyo C. Kang, Sholom G. Cohen, James A. Hess, William E. Novak, and A. Spencer Peterson. Feature-oriented domain analysis (FODA) feasibility study. Technical Report CMU/SEI-90-TR-021, Software Engineering Institute, Carnegie Mellon University, Pittsburgh, PA, USA, 1990. Technical Report.
- [KCS⁺10] Changkyu Kim, Jatin Chhugani, Nadathur Satish, Eric Sedlar, Anthony D. Nguyen, Tim Kaldewey, Victor W. Lee, Scott A. Brandt, and Pradeep Dubey. FAST: Fast architecture sensitive tree search on modern CPUs and GPUs. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pages 339–350, 2010.
- [Kha11] Minhaj Ahmad Khan. Data cache prefetching with dynamic adaptation. *The Computer Journal*, 54(5):815–823, 2011.
- [Kit26] Kitware, Inc. CMake documentation: cmake(1). <https://cmake.org/cmake/help/latest/manual/cmake.1.html>, abgerufen am 01.08.2026, 2026.
- [KM17a] Paul-Virak Khuong and Pat Morin. Array Layouts for Comparison-Based Searching. *ACM Journal of Experimental Algorithmics*, 22:1.3:1–1.3:39, 2017.
- [KM17b] Paul-Virak Khuong and Pat Morin. Array layouts for comparison-based searching. *ACM Journal of Experimental Algorithmics*, 22, 2017.
- [KMvR⁺19] Andreas Kipf, Ryan Marcus, Alexander van Renen, Mihail Stoian, Alfons Kemper, Tim Kraska, and Thomas Neumann. SOSD: A benchmark for learned indexes. In *NeurIPS Workshop on Machine Learning for Systems*, 2019.
- [Kno65] Kenneth C. Knowlton. A fast storage allocator. *Communications of the ACM*, 8(10):623–625, 1965.
- [KNR11] Jay Kreps, Neha Narkhede, and Jun Rao. Kafka: A Distributed Messaging System for Log Processing. In *6th International Workshop on Networking Meets Databases (NetDB)*, pages 1–7, 2011.
- [Knu98] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 2nd edition, 1998.
- [Kro81] David Kroft. Lockup-free instruction fetch/prefetch cache organization. In *Proceedings of the 8th Annual Symposium on Computer Architecture (ISCA)*, pages 81–87, 1981.
- [Kue23] Lena Kuehn. Towards data-based cache optimization of b+-trees. In *Proceedings of the 19th International Workshop on Data Management on New Hardware (DaMoN)*, pages 8:1–8:8. ACM, 2023.
- [LABN16] Viktor Leis, Michael Andersen, Erik Birkedal, and Thomas Neumann. The art of practical synchronization. In *Proceedings of the International Workshop on Data Management on New Hardware (DaMoN)*, pages 8:1–8:8. ACM, 2016.
- [Lam77] Leslie Lamport. Concurrent Reading and Writing. *Communications of the ACM*, 20(11):806–811, 1977.

- [Lam13] Christoph Lameter. NUMA (non-uniform memory access): An overview. *ACM Queue*, 11(7):40–51, 2013.
- [LC86] Tobin J. Lehman and Michael J. Carey. A study of index structures for main memory database management systems. In *12th International Conference on Very Large Data Bases (VLDB)*, pages 294–303, 1986.
- [Lc21] Daan Leijen and contributors. mimalloc-bench: Suite for benchmarking malloc implementations. <https://github.com/daanx/mimalloc-bench>, 2021.
- [LCL⁺25] Meng Li, Huayi Chai, Siqiang Luo, Haipeng Dai, Rong Gu, Jiaqi Zheng, and Guihai Chen. VEGA: An active-tuning learned index with group-wise learning granularity. *Proceedings of the ACM on Management of Data*, 3(1):86:1–86:26, 2025. SIGMOD 2025; Artikel-Nr. 86.
- [LDPB19] Paul Lietar, Derek Dreyer, Adam Parkinson, and Richard Bornat. snmalloc: A message passing allocator. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on Memory Management (ISMM)*, pages 122–135. ACM, 2019.
- [Lea96] Doug Lea. A memory allocator. Design article and C implementation, 1996.
- [LJY25] Ruihao Li, Lizy K. John, and Neeraja J. Yadwadkar. Old is gold: Optimizing single-threaded applications with Exgen-Malloc. *IEEE Computer Architecture Letters*, 24(2), 2025.
- [LKN13] Viktor Leis, Alfons Kemper, and Thomas Neumann. The adaptive radix tree: ARTful indexing for main-memory databases. In *29th IEEE International Conference on Data Engineering (ICDE)*, pages 38–49, 2013.
- [LLS13] Justin J. Levandoski, David B. Lomet, and Sudipta Sengupta. The Bw-Tree: A B-tree for New Hardware Platforms. In *29th IEEE International Conference on Data Engineering (ICDE 2013), Brisbane, Australia, April 8-12, 2013*, pages 302–313. IEEE Computer Society, 2013.
- [LNOM08] Erik Lindholm, John Nickolls, Stuart Oberman, and John Montrym. NVIDIA Tesla: A unified graphics and computing architecture. *IEEE Micro*, 28(2):39–55, 2008.
- [LR18] Ricardo Leite and Rui Rocha. Lrmalloc: A modern and competitive lock-free dynamic memory allocator. In *Proceedings of the 13th International Conference on Parallel Processing and Applied Mathematics (VECPAR)*, pages 188–200. Springer, 2018.
- [LYTL19] Tianlin Li, Yiping Yao, Wenjie Tang, and Zhongwei Lin. HMalloc: A Hybrid, Scalable, and Lock-Free Memory Allocator for Multi-Threaded Applications. In *2019 IEEE 25th International Conference on Parallel and Distributed Systems (ICPADS)*, pages 406–409, 2019.
- [LZdM19] Daan Leijen, Benjamin Zorn, and Leonardo de Moura. Mimalloc: Free list sharding in action. In *Proceedings of the 17th Asian Symposium on Programming Languages and Systems (APLAS)*, pages 172–192. Springer, 2019.

- [LZS⁺23] Xuchuan Luo, Pengfei Zuo, Jiacheng Shen, Jiazhen Gu, Xin Wang, Michael R. Lyu, and Yangfan Zhou. SMART: A high-performance adaptive radix tree for disaggregated memory. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2023.
- [MBL25] Marcus Müller, Lawrence Benson, and Viktor Leis. B-trees are back: Engineering fast and pageable node layouts. *Proceedings of the ACM on Management of Data*, 3(1), 2025.
- [McK01] Paul E. McKenney. Read-copy update. In *Proceedings of the 2001 Ottawa Linux Symposium (OLS)*, pages 338–367, 2001.
- [MF02] Giovanni Manzini and Paolo Ferragina. Engineering a lightweight suffix array construction algorithm. In *Algorithms — ESA 2002, 10th Annual European Symposium*, volume 2461 of *Lecture Notes in Computer Science*, pages 698–710. Springer, 2002.
- [Mic04a] Maged M. Michael. Hazard pointers: Safe memory reclamation for lock-free objects. *IEEE Transactions on Parallel and Distributed Systems*, 15(6):491–504, 2004.
- [Mic04b] Maged M. Michael. Scalable lock-free dynamic memory allocation. In *Proceedings of the ACM SIGPLAN 2004 Conference on Programming Language Design and Implementation (PLDI)*, pages 35–46. ACM, 2004.
- [Mit16] Sparsh Mittal. A survey of recent prefetching techniques for processor caches. *ACM Computing Surveys*, 49(2):35:1–35:35, 2016.
- [MKM12] Yandong Mao, Eddie Kohler, and Robert Tappan Morris. Cache craftiness for fast multicore key-value storage. In *7th ACM European Conference on Computer Systems (EuroSys)*, pages 183–196, 2012.
- [MMP18] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build systems à la carte. *Proceedings of the ACM on Programming Languages*, 2(ICFP):79:1–79:29, 2018.
- [Mor68] Donald R. Morrison. PATRICIA — practical algorithm to retrieve information coded in alphanumeric. *Journal of the ACM*, 15(4):514–534, 1968.
- [MPU⁺23] Francesco Minervini, Oscar Palomar, Osman Unsal, Enrico Reggiani, Josue Quiroga, Joan Marimon, et al. Vitruvius+: An area-efficient RISC-V decoupled vector coprocessor for high performance computing applications. *ACM Transactions on Architecture and Code Optimization*, 20(2), 2023.
- [MS96] Maged M. Michael and Michael L. Scott. Simple, Fast, and Practical Non-Blocking and Blocking Concurrent Queue Algorithms. In *Proceedings of the Fifteenth Annual ACM Symposium on Principles of Distributed Computing (PODC)*, pages 267–275. ACM, 1996.
- [MS08] Kurt Mehlhorn and Peter Sanders. *Algorithms and Data Structures: The Basic Toolbox*. Springer, 1 edition, 2008.

- [MW47] Henry B. Mann and Donald R. Whitney. On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics*, 18(1):50–60, 1947.
- [MWR25] Fabian Mahling, Marcel Weisgut, and Tilmann Rabl. Fetch me if you can: Evaluating CPU cache prefetching and its reliability on high latency memory. In *Proceedings of the 21st International Workshop on Data Management on New Hardware (DaMoN)*. ACM, 2025.
- [NR21] Ruslan Nikolaev and Binoy Ravindran. Brief announcement: Crystalline: Fast and memory efficient wait-free reclamation. In *35th International Symposium on Distributed Computing (DISC)*, volume 209 of *LIPIcs*, pages 60:1–60:4, 2021.
- [NR24] Ruslan Nikolaev and Binoy Ravindran. A family of fast and memory efficient lock- and wait-free reclamation. *Proceedings of the ACM on Programming Languages*, 8(PLDI), 2024.
- [NTSA16] Mahmood Naderan-Tahan and Hamid Sarbazi-Azad. Why does data prefetching not work for modern workloads? *The Computer Journal*, 59(2):244–259, 2016.
- [OCGO96] Patrick E. O’Neil, Edward Cheng, Dieter Gawlick, and Elizabeth J. O’Neil. The Log-Structured Merge-Tree (LSM-Tree). *Acta Informatica*, 33(4):351–385, 1996.
- [PIA78] Yehoshua Perl, Alon Itai, and Haim Avni. Interpolation Search - A Log Log N Search. *Communications of the ACM*, 21(7):550–553, 1978.
- [PR04] Rasmus Pagh and Flemming Friche Rodler. Cuckoo hashing. *Journal of Algorithms*, 51(2):122–144, 2004.
- [Pug90] William W. Pugh. Skip Lists: A Probabilistic Alternative to Balanced Trees. *Communications of the ACM*, 33(6):668–676, 1990.
- [QJP⁺07] Moinuddin K. Qureshi, Aamer Jaleel, Yale N. Patt, Simon C. Steely Jr., and Joel Emer. Adaptive insertion policies for high performance caching. In *34th Annual International Symposium on Computer Architecture (ISCA)*, pages 381–391, 2007.
- [Rei05] James Reinders. *VTune Performance Analyzer Essentials: Measurement and Tuning Techniques for Software Developers*. Intel Press, Hillsboro, OR, USA, 1 edition, 2005.
- [RFP24a] Antonin Reitz, Aymeric Fromherz, and Jonathan Protzenko. Starmalloc: Verifying a modern, hardened memory allocator. In *Proceedings of the 2024 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*. ACM, 2024. Also available as arXiv:2403.09435.
- [RFP24b] Antonin Reitz, Aymeric Fromherz, and Jonathan Protzenko. StarMalloc: Verifying a modern, hardened memory allocator. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA2), 2024.

- [RIS21] RISC-V International. RISC-V V vector extension, version 1.0. https://docs.riscv.org/reference/isa/extensions/vector/_attachments/riscv-v-spec.pdf, 2021.
- [RR99] Jun Rao and Kenneth A. Ross. Cache conscious indexing for decision-support in main memory. In *Proceedings of the 25th International Conference on Very Large Data Bases (VLDB)*, pages 78–89, 1999.
- [RR00] Jun Rao and Kenneth A. Ross. Making b+-trees cache conscious in main memory. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, pages 475–486. ACM, 2000.
- [SBB⁺17] Nigel Stephens, Stuart Biles, Matthias Boettcher, Mbou Eyole, Giacomo Gabrielli, Matt Horsnell, Grigorios Magklis, Alejandro Martinez, Nathanael Premillieu, Alastair Reid, Alejandro Rico, and Paul Walker. The ARM scalable vector extension. *IEEE Micro*, 37(2):26–39, 2017.
- [SDF⁺00] Jagannathan Srinivasan, Souripriya Das, Chuck Freiwald, Eugene Inseok Chong, Mahesh Jagannath, Aravind Yalamanchi, Ramkumar Krishnan, Anh-Tuan Tran, Samuel DeFazio, and Jayanta Banerjee. Oracle8i index-organized table and its application to new domains. In *26th International Conference on Very Large Data Bases (VLDB)*, pages 285–296, 2000.
- [SGL09] Benjamin Schlegel, Rainer Gemulla, and Wolfgang Lehner. k-Ary Search on Modern Processors. In *Proceedings of the Fifth International Workshop on Data Management on New Hardware (DaMoN 2009)*, pages 52–60. ACM, 2009.
- [SKK⁺25] Lennart Schmidt, Roland Kühn, Matti Krause, Jens Teubner, Wolfgang Lehner, and Dirk Habich. To stride or not to stride the memory access? In *Proceedings of the 3rd Workshop on Disruptive Memory Systems (DIMES)*, pages 19–26. ACM, 2025.
- [SPB05] Curt Samuel, Torben Bach Pedersen, and Philippe Bonnet. Processor-conscious indexing. In *Proceedings of the 2005 ACM SIGMOD International Conference on Management of Data*, pages 235–246. ACM, 2005.
- [SSL⁺21] Josef Schmeißer, Maximilian E. Schüle, Viktor Leis, Thomas Neumann, and Alfons Kemper. The B²-tree: Cache-friendly string indexing within B-trees. In *Datenbanksysteme für Business, Technologie und Web (BTW)*, pages 39–58, 2021.
- [SSS08] Riku Saikkonen and Eljas Soisalon-Soininen. Cache-sensitive memory layout for binary trees. In *Fifth IFIP International Conference on Theoretical Computer Science (TCS 2008)*, volume 273 of *IFIP Advances in Information and Communication Technology*. Springer, 2008.
- [SSS16] Riku Saikkonen and Eljas Soisalon-Soininen. Cache-sensitive memory layout for dynamic binary trees. *The Computer Journal*, 59(5):630–649, 2016.
- [Sta17] Standard Performance Evaluation Corporation. SPEC CPU 2017 benchmark suite. <https://www.spec.org/cpu2017/>, 2017.

- [Sto81] Michael Stonebraker. Operating System Support for Database Management. *Communications of the ACM*, 24(7):412–418, 1981.
- [ȚCH02] Cristian Țăpuș, I-Hsin Chung, and Jeffrey K. Hollingsworth. Active harmony: Towards automated performance tuning. In *ACM/IEEE Conference on Supercomputing (SC)*, pages 1–11, 2002.
- [Ten12] Gil Tene. HdrHistogram: A high dynamic range (hdr) histogram. <https://github.com/HdrHistogram/HdrHistogram>, 2012.
- [TFB⁺11] Martin Thompson, Dave Farley, Michael Barker, Patricia Gee, and Andrew Stewart. Disruptor: High Performance Alternative to Bounded Queues for Exchanging Data Between Concurrent Threads. Technical paper, LMAX, May 2011. Version 1.0.
- [TMS⁺16] Ghizlane Tibba, Christoph Malz, Christoph Stoermer, Natarajan Nagarajan, Licong Zhang, and Samarjit Chakraborty. Testing automotive embedded systems under X-in-the-loop setups. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 1–8, 2016.
- [Tra24] Transaction Processing Performance Council. TPC benchmark C and TPC-DS specifications. Transaction Processing Performance Council, San Francisco, CA, USA, 2024.
- [UHL⁺17] Sigmund Ungethuen, Dirk Habich, Wolfgang Lehner, et al. Hardware acceleration for database systems: A comprehensive survey. *ACM Computing Surveys*, 50(1):3:1–3:35, 2017.
- [vEB75] Peter van Emde Boas. Preserving order in a forest in less than logarithmic time. In *16th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 75–84, 1975.
- [VIA26] VIA Research. Pim-malloc: A fast and scalable dynamic memory allocator for processing-in-memory systems. In *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2026. Accepted to HPCA 2026; also available as arXiv:2505.13002 (2025).
- [vRLK⁺18] Alexander van Renen, Viktor Leis, Alfons Kemper, Thomas Neumann, Takushi Hashida, Kazuichi Oe, Yoshiyasu Doi, Lilian Harada, and Mitsuru Sato. Managing non-volatile memory in database systems. In *ACM SIGMOD International Conference on Management of Data*, pages 1541–1555, 2018.
- [VRM⁺18] Giuseppe Vietri, Liana V. Rodriguez, Wendy A. Martinez, Steven Lyons, Jason Liu, Raju Rangaswami, Ming Zhao, and Giri Narasimhan. Driving cache replacement with ML-based LeCaR. In *10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage)*, 2018.
- [Vyu11] Dmitry Vyukov. Bounded MPMC queue. <https://www.1024cores.net/home/lock-free-algorithms/queues/bounded-mpmc-queue>, 2011.

- [Wil64] J. W. J. Williams. Algorithm 232: Heapsort. *Communications of the ACM*, 7(6):347–348, 1964.
- [Wil83] Dan E. Willard. Log-logarithmic worst-case range queries are possible in space $\Theta(N)$. *Information Processing Letters*, 17(2):81–84, 1983.
- [WM08] Vincent M. Weaver and Sally A. McKee. Can hardware performance counters be trusted? In *Proceedings of the IEEE International Symposium on Workload Characterization (IISWC)*, pages 141–150, 2008.
- [WNJ19] Xingbo Wu, Fan Ni, and Song Jiang. Wormhole: A fast ordered index for in-memory data management. In *14th ACM European Conference on Computer Systems (EuroSys)*, 2019.
- [WPD01] R. Clint Whaley, Antoine Petitet, and Jack J. Dongarra. Automated empirical optimization of software and the ATLAS project. *Parallel Computing*, 27(1–2):3–35, 2001.
- [Xu24] Chuanqi Xu. Modules ABI Requirement. Technical Report P3092R0, ISO/IEC JTC1/SC22/WG21, January 2024. Online-Ressource.
- [YZZ⁺23] Hanmei Yang, Xin Zhao, Jin Zhou, Wei Wang, Sandip Kundu, Bo Wu, Hui Guan, and Tongping Liu. NUMAlloc: A faster NUMA memory allocator. In *2023 ACM SIGPLAN International Symposium on Memory Management (ISMM)*, 2023.
- [ZDL⁺13] Matei Zaharia, Tathagata Das, Haoyuan Li, Timothy Hunter, Scott Shenker, and Ion Stoica. Discretized Streams: Fault-Tolerant Streaming Computation at Scale. In *24th ACM Symposium on Operating Systems Principles (SOSP)*, pages 423–438, 2013.
- [ZGH⁺25] Tingji Zhang, Boris Grot, Wenjian He, Yashuai Lv, Peng Qu, Fang Su, Wenxin Wang, Guowei Zhang, Xuefeng Zhang, and Youhui Zhang. Hierarchical prefetching: A software-hardware instruction prefetcher for server applications. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Volume 2, pages 529–544. ACM, 2025.
- [ZGV⁺24] Zhuangzhuang Zhou, Vaibhav Gogte, Nilay Vaish, Chris Kennelly, Patrick Xia, Svilen Kanev, Tipp Moseley, Christina Delimitrou, and Parthasarathy Ranganathan. Characterizing a memory allocator at warehouse scale. In *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 192–206, 2024.
- [ZL77] Jacob Ziv and Abraham Lempel. A universal algorithm for sequential data compression. *IEEE Transactions on Information Theory*, 23(3):337–343, 1977.
- [ZLL⁺18] Huanchen Zhang, Hyeontaek Lim, Viktor Leis, David G. Andersen, Michael Kaminsky, Kimberly Keeton, and Andrew Pavlo. SuRF: Practical range query filtering with fast succinct tries. In *ACM SIGMOD International Conference on Management of Data*, pages 323–336, 2018.

-
- [ZSZ⁺24] Qian Zhang, Haoyun Song, Kaiyan Zhou, Jianhao Wei, and Chuqiao Xiao. A prefetching indexing scheme for in-memory database systems. *Future Generation Computer Systems*, 156:179–190, 2024.

List of Figures

1.1	The “cache wall”: access latency grows across the memory hierarchy by about two orders of magnitude.	5
1.2	The separability problem: as a monolith it remains unclear which design decision matters; only the decomposition into orthogonal axes makes each contribution measurable in isolation.	6
2.1	The landscape of search structures: five classes with typical representatives.	11
2.2	Cache-line utilization: a cache line is always loaded in full; if only a few of its bytes are used, effective bandwidth drops — cache-aware layout raises utilization.	13
3.1	The design space: the axes span a multidimensional Cartesian product; a concrete algorithm is a point within it. For illustration, three of the eighteen organ axes are chosen — the real storage organ of node type (T4), memory layout (T5) and allocator (T6); the full design space has eighteen organ dimensions.	22
3.2	Gallery of the most important reconstructed search structures: per method (with paper ID) its origin class, the characteristic structural idea and the axes at which it makes its main contribution. Trie, hybrid, succinct and cache-sensitive methods stand next to a non-tree hash control (SwissTable) — all as points in the same axis design space. The complete axis dissection is given in Table 3.2.	29
3.3	The anatomy bridge: just as a living being consists of interacting organs, a search algorithm is a composition of axis-organs; the measurement system takes the role of medical diagnostics and measures each organ in isolation.	32
3.4	Synthesis state of the art → axes: the design building blocks of the known methods are decomposed into orthogonal axes; each axis collects the contributions of the entire corpus — the inverse of Figure 3.1 (there an algorithm is a point, here an axis is a collecting basin).	32
3.5	Axis = organ, anatomy = wiring: the storage organ (composed of node type, layout and allocator) exports a generalized <code>StorageOrgan</code> interface over which the search runs (solid, <i>mandatory</i>); further organs such as telemetry <i>can</i> use it via optional observer hooks (dashed, <i>optional</i>), but need not.	33

- 3.6 Inter-organ usage graph (“which axis uses which”): there is exactly *one* canonical hub — the storage organ `LayoutAwareChunkedStore`, which composes node type (T4), layout (T5) and allocator (T6). *Mandatorily* (solid) the store consumes these three axes, and the search (T0) runs over it. All remaining couplings are *optional* (dashed): prefetch (T7) and the telemetry (measurement instrumentation, Section 3.4.1) read the store only via `requires`-gated observer hooks — an axis *can* use foreign interfaces, but *need* not. The coupling of concurrency (T8) to the store is not (yet) wired in the code (dotted). 33
- 3.7 The three-level model: genus (level 1, outer interface) → tier subclass (level 2, fixed axis set) → organs/axes (level 3). The Map genus carries the “mammal” with the eighteen organ axes through its sole tier subclass `SearchAlgorithm`; the Container genus the subclasses `Set/Sequence/Adapter/View`. 34
- 3.8 The three genera and the tier hierarchy down to the compiled binary 35
- 3.9 Design patterns as building blocks of the architecture: above the classical GoF patterns used throughout the code with their respective role, in the middle the two new compile-time patterns named in this thesis, below the code-dissected *three-stage concept chain* per axis (topic concept → axis concept → cache-engine permutation concept → CRTP base → `mp_list` registry) as a refinement of *Strategy/C RTP* — where the *literal* `std::variant` appears not here but only in the eleven-axis `baustein_variants` fallback model. Their composition forms the design-pattern-driven, measurement-oriented architecture (Chapter 4). 36
- 3.10 The five interface layers of the experiment pipeline: from the declarative definition and evaluation layer E4 (user view) through the permutation B⁺-tree (E3) and the tier binaries (E2) to the runtime configuration (E1); the infrastructure (E0) carries everything and is completed last. The return channel leads the measurements back to the evaluation in E4; the measurement system permeates all layers as the cross-cutting concern M (“blood”, Section 3.4). Each layer is completed individually, with a frozen interface contract to its neighbouring level. 37
- 3.11 The measurement trade-off triangle: every axis permutation spans against the three opposing measurement components latency, throughput and memory footprint, and is a point inside it (cf. the design space in Figure 3.1, where the same permutation is a point). The component actually central for cache-aware search structures — the behaviour at the node, data and index level via hardware performance counters — is not yet collected in the current implementation state (dashed). 41

- 3.12 The M-model focused on the build and measurement phases. The outer `messung_`-driver loop (*WHAT*, thesis code) feeds the configurations of the three measurement series A/B/C; the `CacheEngineBuilder` (*HOW*, library) splits into the two genuinely interesting phases: **BUILD** (`BuildOrchestrator/ExperimentTree` builds a standalone tier binary per configuration, resumable and RAM-aware) and **MEASURE** (`perm_runner` drives and measures each tier binary individually through the observer interface `IObservableTier`). The `CacheEngine` supplies the standard axis building blocks, the probe PRT-ART its own organs; their bidirectional relationship ((a) instantiates, (b) calls services) carries the multi-probe capability. The `ExecutionEngine` remains the shared measurable root. 44
- 3.13 The builder's three-stage evaluation: each stage produces non-redundant permutation binaries; their mapping onto the measurement series A/B/C is shown by Table 3.8. . . . 44
- 3.14 PRT-ART as a demonstrator in the M-model: the probe overrides six slots in the measurement apparatus — the build axis page type, the four organ axes path compression (redirect), value handle (`ChainRef`), prefetch, and OLC concurrency, as well as the runtime telemetry (measurement-tooling sub-axis, Section 3.4.1) — and obtains the remaining fourteen organ axes via the compile-time fallback from the cache engine; its further own building blocks (4+1+2 pool allocator, multi-level layout) currently run via the cache-engine default in the measurement path (binding: follow-up step E6). Thus a complete, measurable living being arises from a few own plus many default organs. 46
- 3.15 The heuristic loop: from the measurement series the best axis composition per load profile is learned, stored as an XML load profile, and fed back via the workload filter into the next measurement. 47
- 4.1 Gallery of the six own key points of PRT-ART, each at its axis: a 4+1+2 pool allocator (T6), optimistic lock coupling with reserved value blocks (T8), path-oriented bundle prefetch (T7), a multi-level layout (T5), and cache-coherence-aware telemetry together with the density tracker and the hypothesis metrics H1/H2/H3 — the telemetry axis has resided in the system sphere since the axis reordering (Section 3.4.1). The tiles show the *building blocks of the probe repo*; currently T7/T8 and the telemetry are overridden in the apparatus, while the building blocks for T5/T6 still run there via the cache-engine default (E6). In addition, the probe overrides the path-compression redirect (T3), the `ChainRef` value handle (T10), and the build axis page type; the remaining fourteen organ axes (currently including T5/T6) the probe draws via the compile-time fallback from the cache-engine. 53

- 4.2 Gallery of the eighteen organ axes (T0–T17, canonical slot order): per axis the implementation concept — short name, sub-axis family and a representative selection of the interchangeable building blocks. Each axis is a `Concept` interface with several strategy building blocks (*Strategy* pattern); a concrete composition occupies exactly one per slot. The *default* (“Def.”) is a derived comparison baseline, not a code flag; for the large axes T0/T6/T15 the total building-block count is given in parentheses. The T numbers are slots of the composition, not directory numbers of the implementation. Compared with the nineteen-axis gallery of earlier versions (T0–T18, interim state of July 2026), telemetry and ISA have moved to the system sphere, and the persistence target has been added as the eighteenth slot — pinned to its in-memory representative in the canonical measurement catalogue (Section 3.4.1). The complete matrix with origin sources is given in Appendix D. 54
- 4.3 The *one* architecture as a single hierarchy without parallel trees. At the root stands `IExecutionEngine` as the *driver of measurement* — the universal measurable contract (`warm_up/run/reset/shutdown`) across all three engine kinds Anatomy, Virus and Hybrid; beneath it branch the anatomy-bearing *genera* (Map, Container, Graph) and the axis-less *viruses*. `SearchAlgorithm` is the focal living being (mammal); its anatomy is its *body* — the fixed composition *and wiring* of the 18 organs —, and “SearchEngine” only its ABI-runtime view. The mammal branch is drawn out down to the individual: a concrete axis composition (level 4) is compiled per permutation into a standalone animal binary (level 5). Container (keyless storage) and Graph carry their own organ sets. 55
- 4.4 The ABI-stable C module boundary and the variadic hybrid API. Complex keys are reduced by a fingerprint component to 16-byte binary strings of fixed length. 56
- 4.5 UML map of the code interfaces: on the left the engine/anatomy hierarchy — the measurable root `IExecutionEngine` as the *driver of measurement* (`Anatomy ∪ Virus ∪ Hybrid`) over `IAnatomyBase` and the axis-less `IVirus-ExecutionEngine`, beneath it the *body* `SearchAlgorithmAnatomy<C>` (composition *and wiring* of the eighteen axes) and its ABI-runtime view `SearchAlgorithmAbiAdapter<A>` (“SearchEngine”), measured and driven through the `IDriveableTier/IObservableTier` contracts; on the right the per-axis implementation pattern repeated for each axis — a two-layer `Concept`, a requires-guarded CRTP base, the `mp_list` building-block family, and the compile-time resolution `resolve_baustein`, which chooses per axis between the probe slot and the cache-engine default. 56

4.6	The measurement pipeline in two forms. The in-process path (top) is the seven-phase pipeline of the <code>ExperimentDriver</code> per measurement series (Enumerate → Codegen → Compile → Load → Execute → Measure → Persist; two opt-in phases hot-compile missing modules and verify the ABI contract). Highlighted are the genuinely substantial phases: <code>COMPILE</code> (the <i>build</i> of the tier binaries) and <code>EXECUTE/MEASURE</code> (the <i>measurement</i>); <code>Codegen/Load/Persist</code> are pure plumbing. The out-of-process path (bottom, the recommended route) separates the two tasks out-of-process: the <code>BuildOrchestrator</code> builds <i>all tier binaries first</i> (resumable, RAM-aware), after which <code>perm_runner</code> loads and measures each binary individually (one DLL per process) through the observer interface <code>IObservableTier</code>	58
4.7	The downstream evaluation toolchain	58
A.1	Cycles per organism permutation (<code>v5_pipeline_demo</code>)	103
A.2	Cycles per organism permutation (<code>cartesian_smoke43</code>)	104
A.3	Heatmap: overall latency (ns/op) (search algorithm x workload)	104
A.4	Heatmap: insert latency p50 (ns/op) (search algorithm x workload)	104
A.5	Heatmap: lookup latency p50 (ns/op) (search algorithm x workload)	105
A.6	Heatmap: erase latency p50 (ns/op) (search algorithm x workload)	105
A.7	Heatmap: scan latency p50 (ns/op) (search algorithm x workload)	105
A.8	Heatmap: rmw latency p50 (ns/op) (search algorithm x workload)	105
A.9	3D raw-data view: overall latency (ns/op) (search algorithm x workload)	105
A.10	3D raw-data view: insert latency p50 (ns/op) (search algorithm x workload)	106
A.11	3D raw-data view: lookup latency p50 (ns/op) (search algorithm x workload)	106
A.12	3D raw-data view: erase latency p50 (ns/op) (search algorithm x workload)	106
A.13	3D raw-data view: scan latency p50 (ns/op) (search algorithm x workload)	106
A.14	3D raw-data view: rmw latency p50 (ns/op) (search algorithm x workload)	106
A.15	working-set sweep: overall latency (ns/op)	107
A.16	working-set sweep: insert latency p50 (ns/op)	107
A.17	working-set sweep: lookup latency p50 (ns/op)	108
A.18	stacked per-axis latency attribution	111
A.19	latency spread p50-p99 per search algorithm x operation; point=p50, whisker=p99. . .	112
A.20	ECDF of overall latency – distribution over configurations	113
A.21	Pareto scatter: median vs tail latency (one point per configuration and operation kind; diagonal = no tail surcharge)	114
A.22	Ratio to reference <code>linear_scan</code> : overall latency (ns/op) (1 = same as reference)	114
A.23	Ratio to reference <code>linear_scan</code> : insert latency p50 (ns/op) (1 = same as reference) . . .	114
A.24	Ratio to reference <code>linear_scan</code> : lookup latency p50 (ns/op) (1 = same as reference) . . .	114
A.25	Ratio to reference <code>linear_scan</code> : erase latency p50 (ns/op) (1 = same as reference)	114
A.26	Ratio to reference <code>linear_scan</code> : scan latency p50 (ns/op) (1 = same as reference)	115
A.27	Ratio to reference <code>linear_scan</code> : rmw latency p50 (ns/op) (1 = same as reference)	115

List of Tables

2.1	Cache-near structure designs and the hardware parameters they explicitly bind into the design.	17
3.1	Dialectical adoption of the state-of-the-art instances per composition axis (in canonical slot order T0–T17; the earlier numbering T0–T18 still counted telemetry and ISA as composition axes — both have moved into the system sphere, Section 3.4.1. Cluster codes: A trie/node, B hybrid/layout, C allocation, D prefetching, E synchronization, F hardware/measurement/engineering)	23
3.2	Axis dissection of the search methods: eighteen organ axes in canonical slot order (T0–T17), the quantities moved into the system sphere, and the probe build axis; per axis the sources contributing an organ.	25
3.3	Allocator profiles with license and workload affinity	26
3.4	Target hardware and scheduling profiles assigned in this thesis per reconstructed SOTA method (selection; the profiles are reconstruction targets of this thesis, not necessarily original-paper properties; the full catalogue follows in the appendix).	26
3.5	Benchmark frameworks and using publications	27
3.6	Load-profile catalogue (14 profiles, derived from the 33-paper analysis)	28
3.7	SOTA algorithm profiles with workload affinity	30
3.8	Mapping of the three-stage evaluation onto the mandatory measurement series	44
5.1	Profile-aware workload routing (fallback heuristic)	65
5.2	Real datasets and their characteristics	66
A.1	Measurement series v5_pipeline_demo: pre-configured organism permutations (in-process path, run_workload)	101
A.2	Measurement series cartesian_smoke43: pre-configured organism permutations (in-process path, run_workload)	102
A.3	Bias-break matrix (median ns/op, search method x load profile)	103
A.4	Per-axis observer detail values per tier binary (measured counters only)	108
A.5	Axis exchangeability: search_algo (sibling-pair diffs, median rel. Δ ns/op w.r.t. v ; 0 sibling pairs; confounding: least confounded)	112
A.6	Axis exchangeability: node_type (sibling-pair diffs, median rel. Δ ns/op w.r.t. v ; 0 sibling pairs; confounding: caveat)	113
A.7	Axis exchangeability: memory_layout (sibling-pair diffs, median rel. Δ ns/op w.r.t. v ; 0 sibling pairs; confounding: caveat)	113
A.8	Axis exchangeability: prefetch (sibling-pair diffs, median rel. Δ ns/op w.r.t. v ; 0 sibling pairs; confounding: least confounded)	113

A.9	Honest limitations: unresolved caveats (no finding silently lapses)	115
F.1	Complete <code>std::map</code> interface (C++23)	171
F.2	Complete <code>std::vector</code> interface (C++23)	174
F.3	Composite functions (“under the hood”)	176

A Complete Measurement Series

This appendix contains the complete measurement series of the pre-configured organism permutations. Table A.1 lists the metrics tabulated per permutation (operations; clock cycles approximated; cache misses L1–L3 are 0 in this PMC-less measurement series – the measurement chain collects L1D, last-level, dTLB and, since M-3a, branch misses for real when built with `COMDARE_ENABLE_PMC`, cf. limitation 1 in Section A.10); Figure A.1 compares the clock cycles graphically. The series documented in this appendix are smoke proofs of the complete chain run with axis variation; they do not replace the mandatory measurement series A–C (Section 5.1), whose full-build results are still pending.

Permutation	ops	cycles	L1	L2	L3
ArtComposition_0	2000	825053	0	0	0
HotComposition_1	2000	740176	0	0	0
StartComposition_2	2000	805750	0	0	0
SurfComposition_3	2000	873976	0	0	0
WormholeComposition_4	2000	696350	0	0	0
MasstreeComposition_5	2000	770871	0	0	0

Table A.1: Measurement series v5_pipeline_demo: pre-configured organism permutations (in-process path, run_workload)

A.1 Cartesian smoke series (43 permutations, real DLL measurement)

The following series results from the full end-to-end chain: the built permutation DLLs (43 combinations in two sub-blocks: 27 cartesian $\text{SIMD} \times \text{layout} \times \text{allocator}$ combinations plus 16 more over node width, path compression, internal search, and telemetry (2^4) — in total seven varied axes of the then nineteen-axis numbering, interim state of July 2026) were measured at the test dock, persisted as binary measurement records, converted via pipeline stage 03 into the 16-column schema, and embedded here automatically. Table A.2 and Figure A.2 show the results.

A.2 Bias-break matrix

Table A.3 summarises the median latency (ns/op, nearest-rank) measured per search method and load profile across all dynamic settings and repetitions; only the runs marked `two_phase_valid` are taken into account. The matrix reveals under which load profiles a given method breaks its systematic bias.

Permutation	ops	cycles	L1	L2	L3
avx2_aos_jemalloc	1000	8025	0	0	0
avx2_aos_mimalloc	1000	7995	0	0	0
avx2_aos_std	1000	7995	0	0	0
avx2_hybrid_jemalloc	1000	315164	0	0	0
avx2_hybrid_mimalloc	1000	243930	0	0	0
avx2_hybrid_std	1000	230759	0	0	0
avx2_soa_jemalloc	1000	423930	0	0	0
avx2_soa_mimalloc	1000	400634	0	0	0
avx2_soa_std	1000	398265	0	0	0
scalar_aos_jemalloc	1000	6975	0	0	0
scalar_aos_mimalloc	1000	7005	0	0	0
scalar_aos_std	1000	6975	0	0	0
scalar_hybrid_jemalloc	1000	1366319	0	0	0
scalar_hybrid_mimalloc	1000	1279365	0	0	0
scalar_hybrid_std	1000	1356329	0	0	0
scalar_soa_jemalloc	1000	440835	0	0	0
scalar_soa_mimalloc	1000	394200	0	0	0
scalar_soa_std	1000	438960	0	0	0
sse4_aos_jemalloc	1000	16845	0	0	0
sse4_aos_mimalloc	1000	14160	0	0	0
sse4_aos_std	1000	14324	0	0	0
sse4_hybrid_jemalloc	1000	1281180	0	0	0
sse4_hybrid_mimalloc	1000	1275404	0	0	0
sse4_hybrid_std	1000	1279350	0	0	0
sse4_soa_jemalloc	1000	397095	0	0	0
sse4_soa_mimalloc	1000	392294	0	0	0
sse4_soa_std	1000	396780	0	0	0
pa_compact_lazy_binary_leaf_only	1000	348105	0	0	0
pa_compact_lazy_binary_sampling	1000	375600	0	0	0
pa_compact_lazy_linear_leaf_only	1000	445815	0	0	0
pa_compact_lazy_linear_sampling	1000	423090	0	0	0
pa_compact_none_binary_leaf_only	1000	360765	0	0	0
pa_compact_none_binary_sampling	1000	346980	0	0	0
pa_compact_none_linear_leaf_only	1000	398939	0	0	0
pa_compact_none_linear_sampling	1000	391875	0	0	0
pa_wide_lazy_binary_leaf_only	1000	309300	0	0	0
pa_wide_lazy_binary_sampling	1000	311805	0	0	0
pa_wide_lazy_linear_leaf_only	1000	445605	0	0	0
pa_wide_lazy_linear_sampling	1000	492495	0	0	0
pa_wide_none_binary_leaf_only	1000	307110	0	0	0
pa_wide_none_binary_sampling	1000	311925	0	0	0
pa_wide_none_linear_leaf_only	1000	445140	0	0	0
pa_wide_none_linear_sampling	1000	478635	0	0	0

Table A.2: Measurement series cartesian_smoke43: pre-configured organism permutations (in-process path, run_workload)

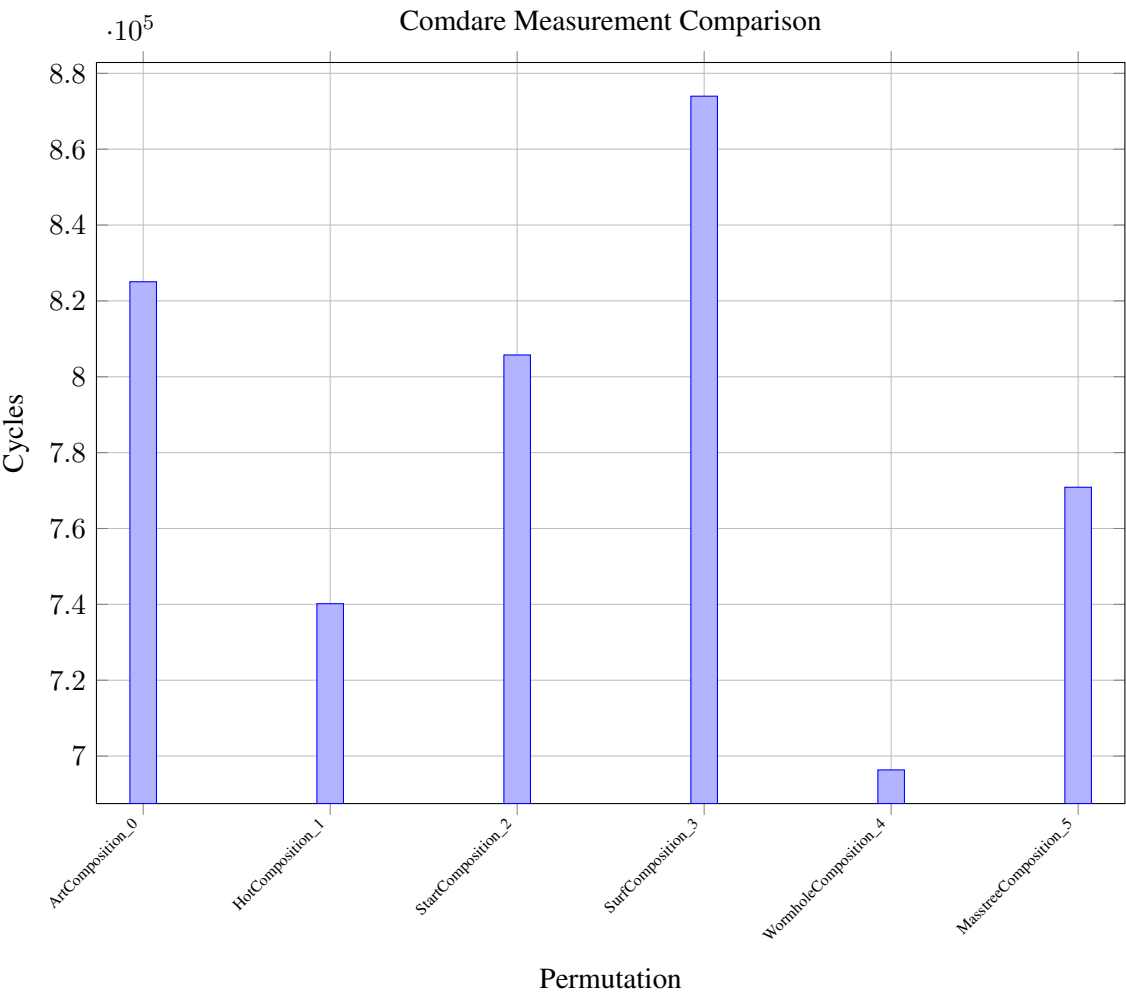


Figure A.1: Cycles per organism permutation (v5_pipeline_demo)

Search method	y _{csb} -c
k_ary	672

Table A.3: Bias-break matrix (median ns/op, search method x load profile)

(Measured, but not drawable as a surface: lookup latency p50 (ns/op) spans only a 1×1 matrix (search algorithms $\times$ workloads) in the present corpus; the pgfplots surface form requires at least 2×2 . The measured value itself is present in the corpus; the surface is omitted honestly instead of failing the build.)

Figure A.5: Heatmap: lookup latency p50 (ns/op) (search algorithm x workload)

(No measured values: erase latency p50 (ns/op) was never executed in the present corpus. This surface is honestly omitted instead of claiming a 0 ns measurement.)

Figure A.6: Heatmap: erase latency p50 (ns/op) (search algorithm x workload)

(No measured values: scan latency p50 (ns/op) was never executed in the present corpus. This surface is honestly omitted instead of claiming a 0 ns measurement.)

Figure A.7: Heatmap: scan latency p50 (ns/op) (search algorithm x workload)

(No measured values: rmw latency p50 (ns/op) was never executed in the present corpus. This surface is honestly omitted instead of claiming a 0 ns measurement.)

Figure A.8: Heatmap: rmw latency p50 (ns/op) (search algorithm x workload)

A.4 Three-dimensional raw-data view

The same six surfaces are additionally plotted as a true 3D surface over the plane (search algorithm $\times$ load profile). Both surface axes are *nominal*: the spanned surface deliberately carries *no* interpolation or trend claim and is to be read solely as a raw-data and inspection view alongside the heatmap.

3D raw-data view: overall latency (ns/op) (search algorithm x workload)

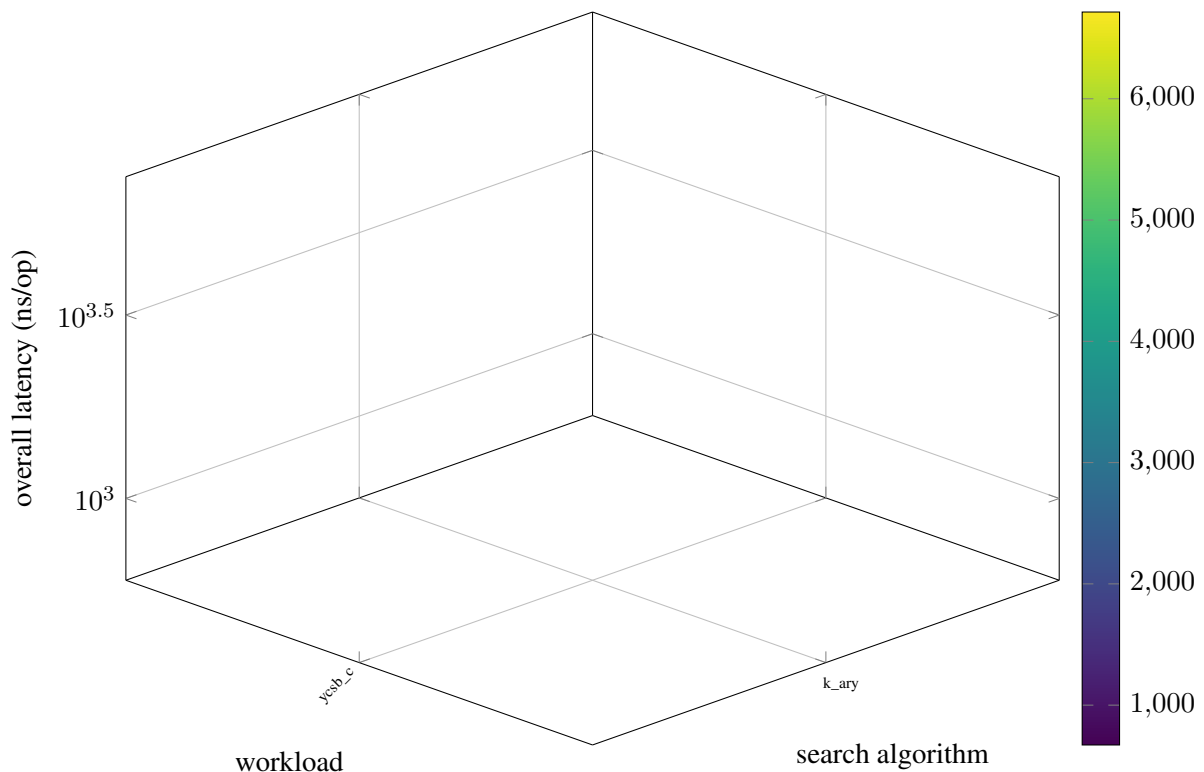


Figure A.9: 3D raw-data view: overall latency (ns/op) (search algorithm x workload)

(No measured values: insert latency p50 (ns/op) was never executed in the present corpus. This surface is honestly omitted.)

Figure A.10: 3D raw-data view: insert latency p50 (ns/op) (search algorithm x workload)

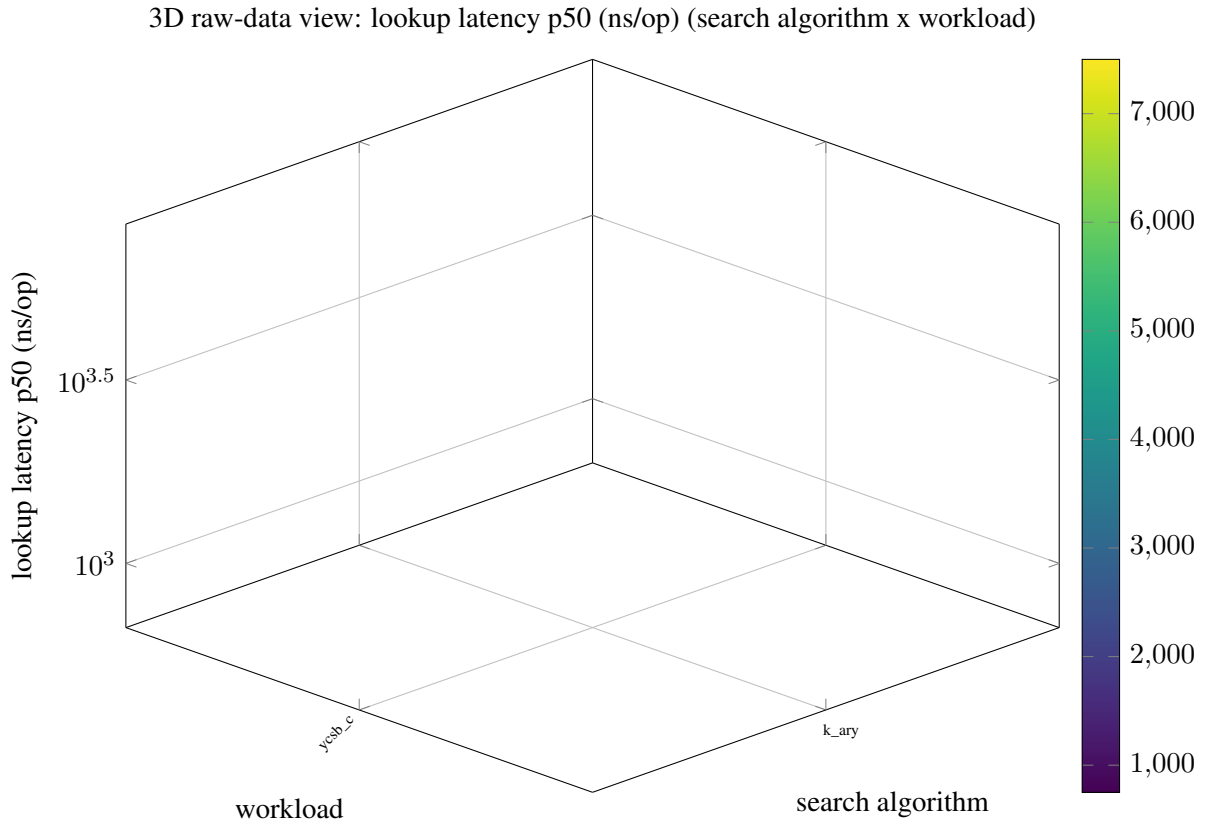


Figure A.11: 3D raw-data view: lookup latency p50 (ns/op) (search algorithm x workload)

(No measured values: erase latency p50 (ns/op) was never executed in the present corpus. This surface is honestly omitted.)

Figure A.12: 3D raw-data view: erase latency p50 (ns/op) (search algorithm x workload)

(No measured values: scan latency p50 (ns/op) was never executed in the present corpus. This surface is honestly omitted.)

Figure A.13: 3D raw-data view: scan latency p50 (ns/op) (search algorithm x workload)

(No measured values: rmw latency p50 (ns/op) was never executed in the present corpus. This surface is honestly omitted.)

Figure A.14: 3D raw-data view: rmw latency p50 (ns/op) (search algorithm x workload)

A.5 Working-set sweep

The following curves plot the latency over the working-set size n (keys) — one curve per swept axis value, for the overall latency (`ns_per_op`) as well as the p50 latencies of `insert` and `lookup`. Measurement series without a `working_set_n` column deliberately produce *no* curve; no support point is fabricated.

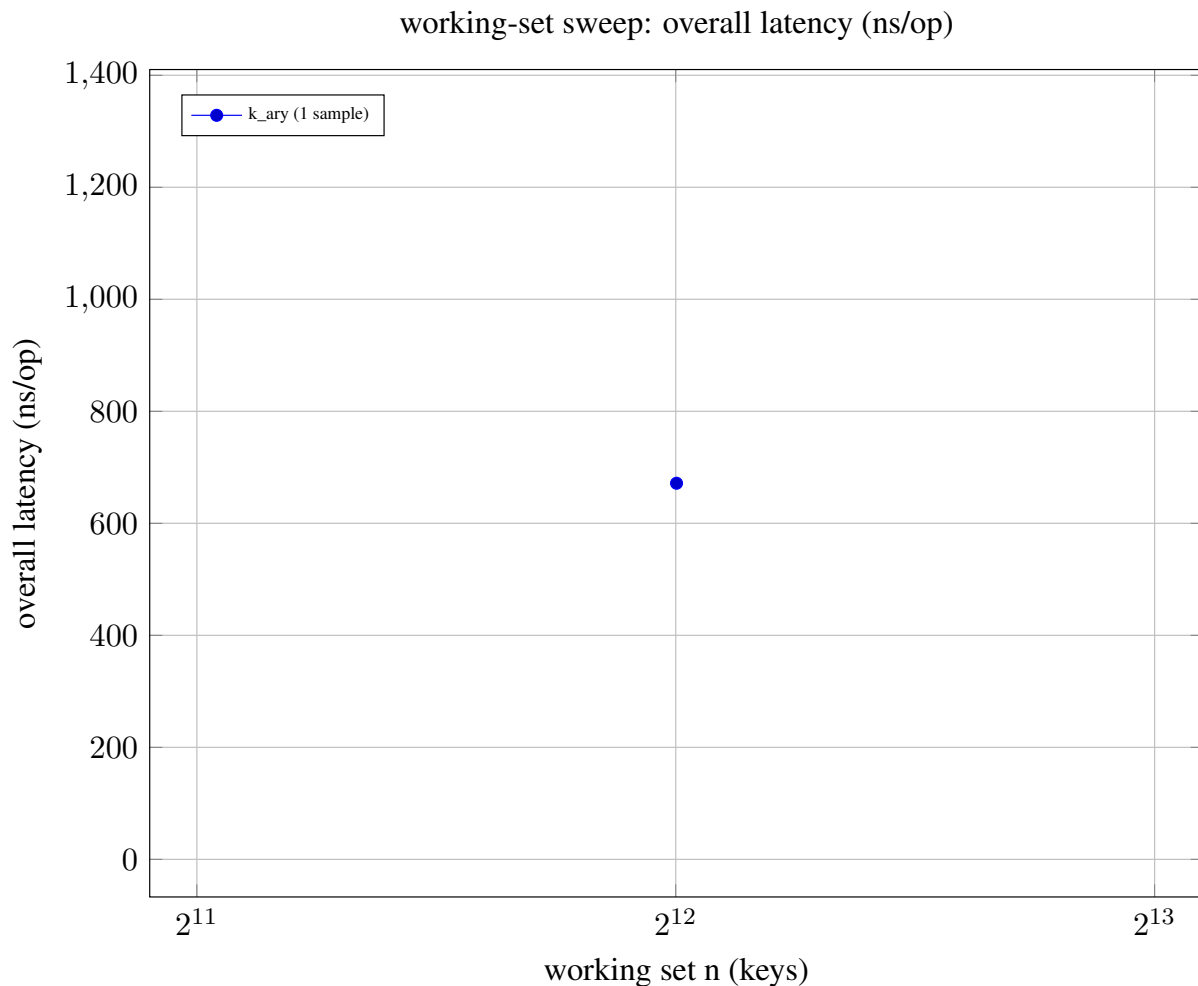


Figure A.15: working-set sweep: overall latency (ns/op)

(No measured values: the operation of this metric was never executed in the present corpus. The curve is honestly omitted instead of showing an invented zero.)

Figure A.16: working-set sweep: insert latency p50 (ns/op)

A.6 Per-axis latency attribution

The following figure decomposes the segment runtime per search method into the contributions of the individual organ axes (stacked bars). The 100 % whole per bar is the wall-clock of the separate segment run (`seg_run_total_ns`), *not* the real-workload latency (`total_ns`); the segments — twenty in the data state of the then nineteen-axis numbering (interim state of July 2026): nineteen organ axes plus the framework remainder — sum exactly to this base (coverage ≈ 1.0).

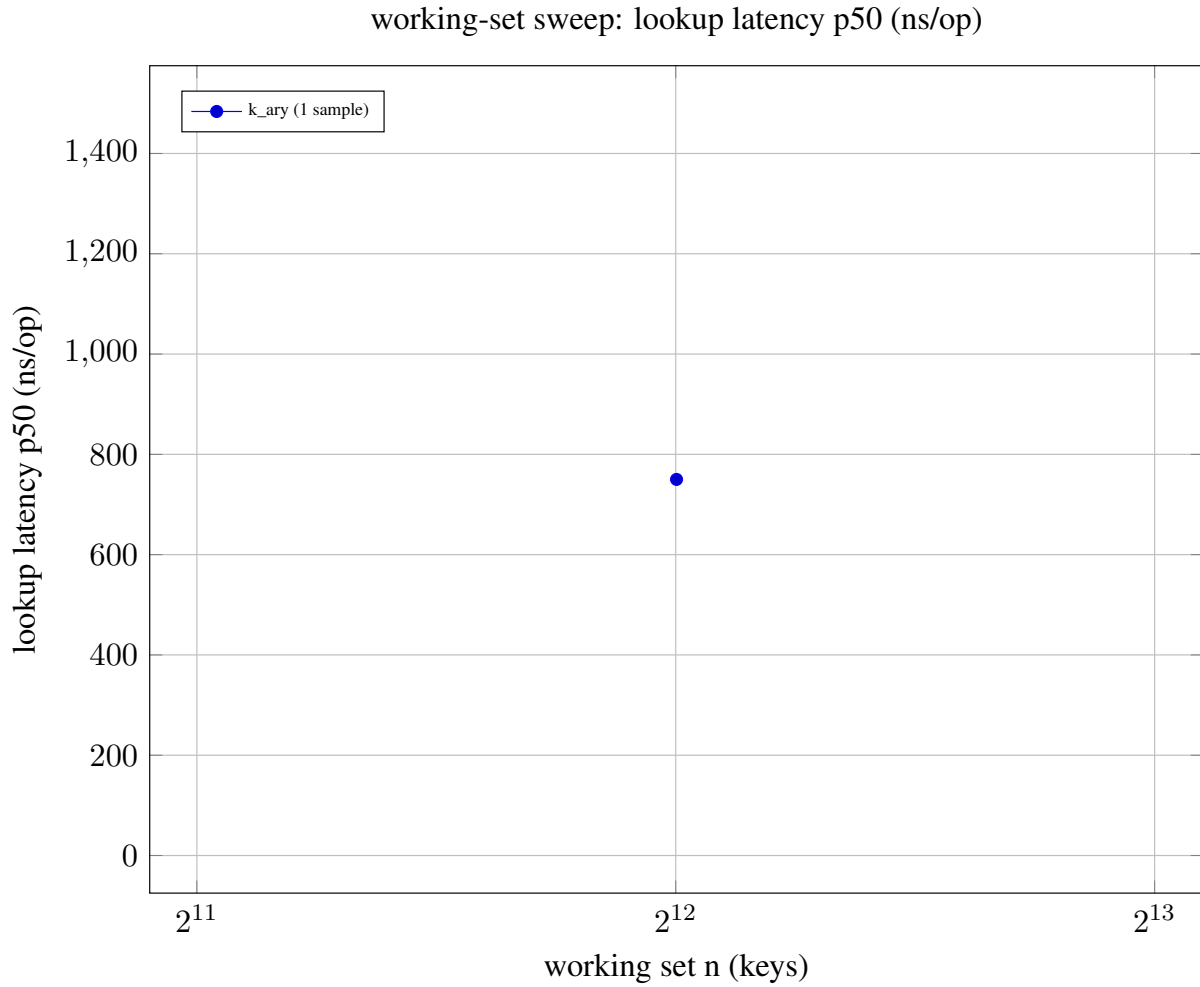


Figure A.17: working-set sweep: lookup latency p50 (ns/op)

A.6.1 Per-axis observer detail

The following longtable lists, per tier binary, the actually measured observer counters (`stat_*` columns from the DLL-side axis schema) — per axis and observer field the measured uint64 counter value; fields that were not collected (n/a) are omitted (no fabricated 0 value).

Table A.4: Per-axis observer detail values per tier binary (measured counters only)

axis	observer field	value
search_algo=k_ary/cache_traversal=linear_fanout/mapping=direct_placement/path_compression=path_compression_none/ node_type=node4/memory_layout=memory_layout_cache_line_aligned/allocator=std_malloc/prefetch=prefetch_none/ concurrency=concurrency_none/serialization=serialization_raw_binary/value_handle=value_handle_inline/ index_organization=index_org_heap/io_dispatch=io_in_memory_only/migration_policy=migration_none/filter=filter_bloom/ queuing_q1=no_buffer/queuing_q2=eager_flush/persistence_target=persistence_memory_only [ycsb_c]		
allocator	alloc_cnt	2099211
	budget_reject	0
	bytes_alloc	537444328
	bytes_in_use	286720
	dealloc_cnt	2098186
	fail	0
cache_traversal	batch_size	4096

continued on next page

Table A.4 – continued

axis	observer field	value
concurrency	batch_visited	32768
	peak_tracked	0
	register	0
	resolve	20000
	resolve_hit	20000
	resolve_miss	0
	unregister	0
	acquire	20000
	contention	0
	pattern_id	0
filter	release	20000
	validation_fail	0
	checksum	18432
	hash_probes	16384
	neg	0
index_organization	pos	4096
	probe	1
	checksum	16378
	indirect_lookups	0
	predicate_evals	4096
io_dispatch	records	4096
	scan	1024
	align_adjusts	0
	bytes	262144
	checksum	16378
mapping	dispatch_cnt	4096
	rounds	1024
	indirect_steps	20000
	peak_mapped	0
	register	0
memory_layout	resolve	20000
	resolve_hit	20000
	resolve_miss	0
	reverse_lookup	0
	cache_lines	4096
migration_policy	checksum	16378
	field_bytes	32768
	line_bytes	64
	records	4096
	scan	1024
node_type	cold_votes	0
	decisions	4096
	hot_votes	0
	migrations	0
	tier_moves	0
path_compression	checksum	40496
	find	1
	keys_stored	4
	queries	4096
	bytes_saved	70000
persistence_target	checksum	7
	compress	10000
	cuts	10000
	prefix_len	70000
	bytes_staged	0
	checksum	0

continued on next page

Table A.4 – continued

axis	observer field	value
prefetch	device_flushes	0
	records_staged	36864
	rounds	9
	addrs_enqueued	0
	hot_path_hints	0
	max_queue_depth	0
queuing_q1	suggestions	0
	trigger	0
	get	0
	overflow	0
queuing_q2	peak_size	0
	put	0
	underflow	20000
	decisions	0
	flush_complete	0
	full_flush	0
search_algo	no_flush	0
	partial_flush	0
	erase	0
	hit	10000
	insert	0
	lookup	10000
serialization	miss	0
	peak	0
	bytes	262144
	checksum	16378
	records	4096
value_handle	serialize	1024
	access	4096
	indirect_deref	0
	peak_chain_depth	1
	version_strips	0

A.7 Latency distribution

The range bars show the p50–p99 span per search method and operation (dot at p50, whisker to p99); since only these two percentiles are measured, no box plot with interpolated quartiles is drawn on purpose. The ECDF shows the distribution of the overall latency *across configurations* (share of permutations with $ns_per_op \leq x$) — not a per-operation CDF, as raw per-operation samples are not available.

Finally, the Pareto scatter plots the median against the tail latency (one point per configuration and operation kind). This makes the lower left front visible: those configurations that are favourable in both metrics at once and are therefore not surpassed by any other configuration in both metrics simultaneously.

A.8 Reference-relative evaluation

The figures below relate all measured values to *one* reference. That reference is a measured axis value of the corpus itself (default `search_algo/linear_scan`) and expressly *not* an external library baseline: a measured `std::map` performance series does not exist, because every `std::map` in the corpus is the conformance oracle of the test dock and thus a correctness, not a performance data point. If the reference

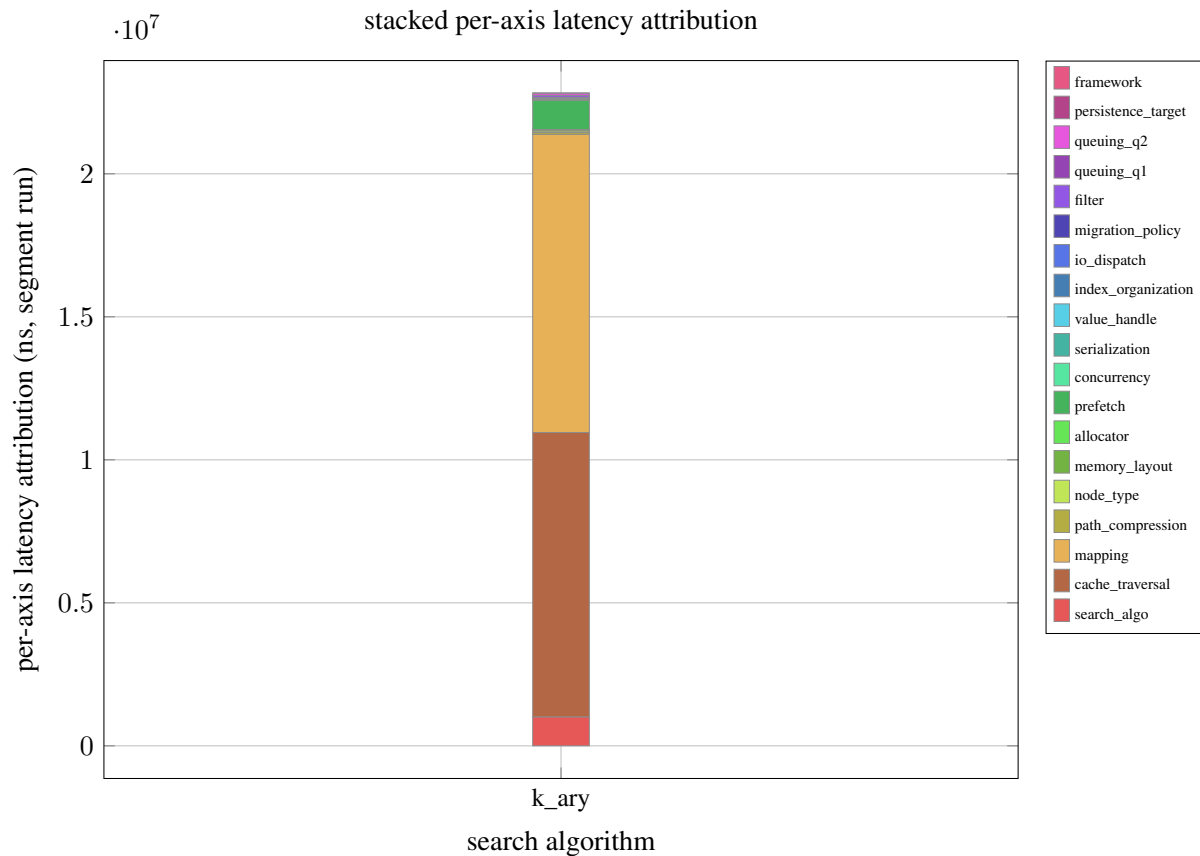


Figure A.18: stacked per-axis latency attribution

does not occur in a measurement series, the figures of this section are omitted without substitute.

A.8.1 Ratio matrices

Per interface function, one matrix shows the quotient of the measured median and the reference median. The diverging colour scale is centred on equality (ratio 1), so that advantage and disadvantage relative to the reference are distinguishable without comparing numbers. In the analysis role this matrix supersedes the heatmap of Section A.3; the raw heatmap remains alongside it as the raw-value view.

A.8.2 Normalised bars

The same statement condensed across the load profiles: one bar per search algorithm, with the reference line at 1. Where the ratio matrix shows the load-profile pattern, the bar gives the overall statement per search algorithm.

*(This is where the normalised bars of the **overall latency** (`ns_per_op`) would stand — one bar per search algorithm, reference line at 1. They are not present: they share the denominator of the ratio matrices, and the reference axis value was not measured alongside in the series available so far.) (This is where the same bars for the **p50 latency of insert** would stand; they too lack the reference axis value measured alongside.) (This is where the same bars for the **p50 latency of lookup** would stand; they too lack the reference axis value measured alongside.) (This is where the same bars for the **p50 latency of erase** would stand; they too lack the reference axis value measured alongside.) (This is where the same*

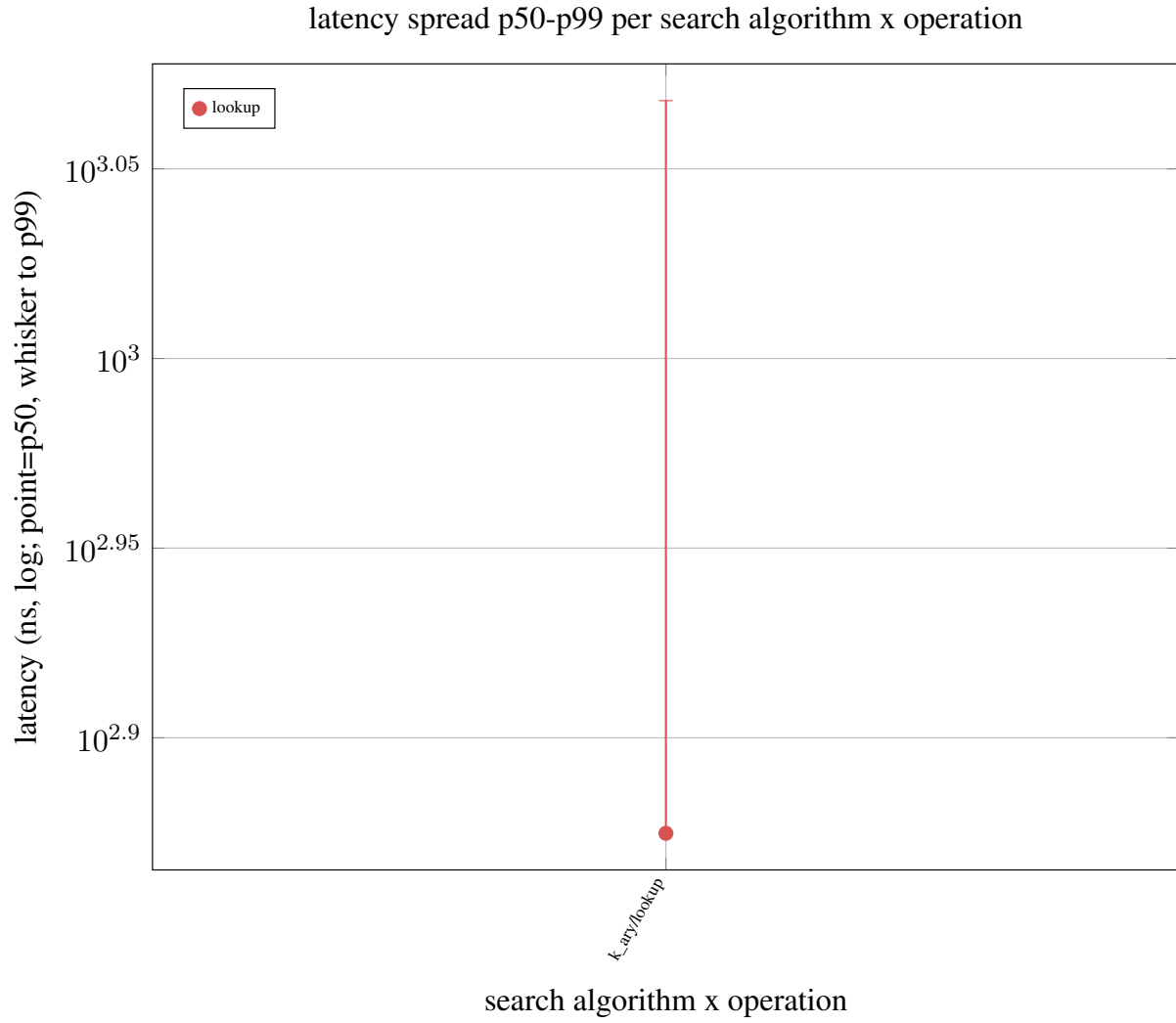


Figure A.19: latency spread p50-p99 per search algorithm x operation; point=p50, whisker=p99.

bars for the **p50 latency of scan** would stand; they too lack the reference axis value measured alongside.)
 (This is where the same bars for the **p50 latency of rmw** would stand; they too lack the reference axis value measured alongside.)

A.9 Axis exchangeability

The four following longtables provide the exchangeability evidence per variable axis from the sibling-pair differences (median rel. Δ ns/op w.r.t. v): `search_algo` (Table A.5), `node_type` (Table A.6), `memory_layout` (Table A.7) and `prefetch` (Table A.8). The least confounded are `search_algo` and `prefetch`.

Table A.5: Axis exchangeability: `search_algo` (sibling-pair diffs, median rel. Δ ns/op w.r.t. v ; 0 sibling pairs; confounding: least confounded)

v	v'	function	median abs. Δ (ns)	median rel. Δ	IQR rel. Δ	n	diagnostic

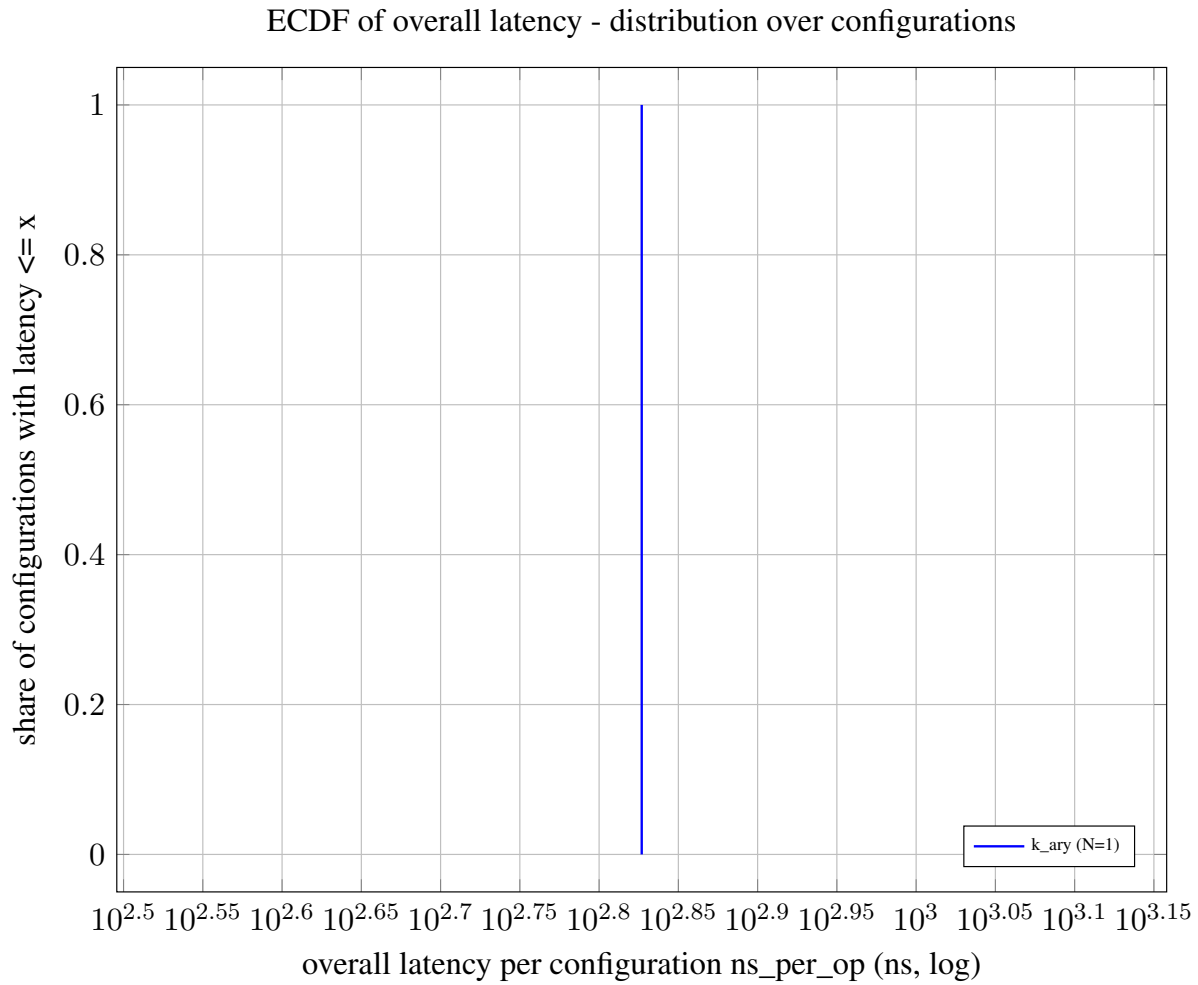


Figure A.20: ECDF of overall latency – distribution over configurations

Table A.6: Axis exchangeability: node_type (sibling-pair diffs, median rel. Δ ns/op w.r.t. v ; 0 sibling pairs; confounding: caveat)

v	v'	function	median abs. Δ (ns)	median rel. Δ	IQR rel.	n	diagnostic

Table A.7: Axis exchangeability: memory_layout (sibling-pair diffs, median rel. Δ ns/op w.r.t. v ; 0 sibling pairs; confounding: caveat)

v	v'	function	median abs. Δ (ns)	median rel. Δ	IQR rel.	n	diagnostic

Table A.8: Axis exchangeability: prefetch (sibling-pair diffs, median rel. Δ ns/op w.r.t. v ; 0 sibling pairs; confounding: least confounded)

v	v'	function	median abs. Δ (ns)	median rel. Δ	IQR rel.	n	diagnostic

The following forest plot summarises the exchangeability of the ns_per_op metric (dot = median rel. Δ , whisker = IQR, zero reference line; small- n rows greyed out). The whisker is symmetric around the median (via the IQR width), as the aggregate stores only the IQR width, not the quartile bounds.

(The forest plot appears after the next measurement run.)

The second forest plot shows the same aggregates, but turns all pairs onto *one* fixed reference axis value in the sense of Section A.8, so that the yardstick of the comparison is named unambiguously instead of

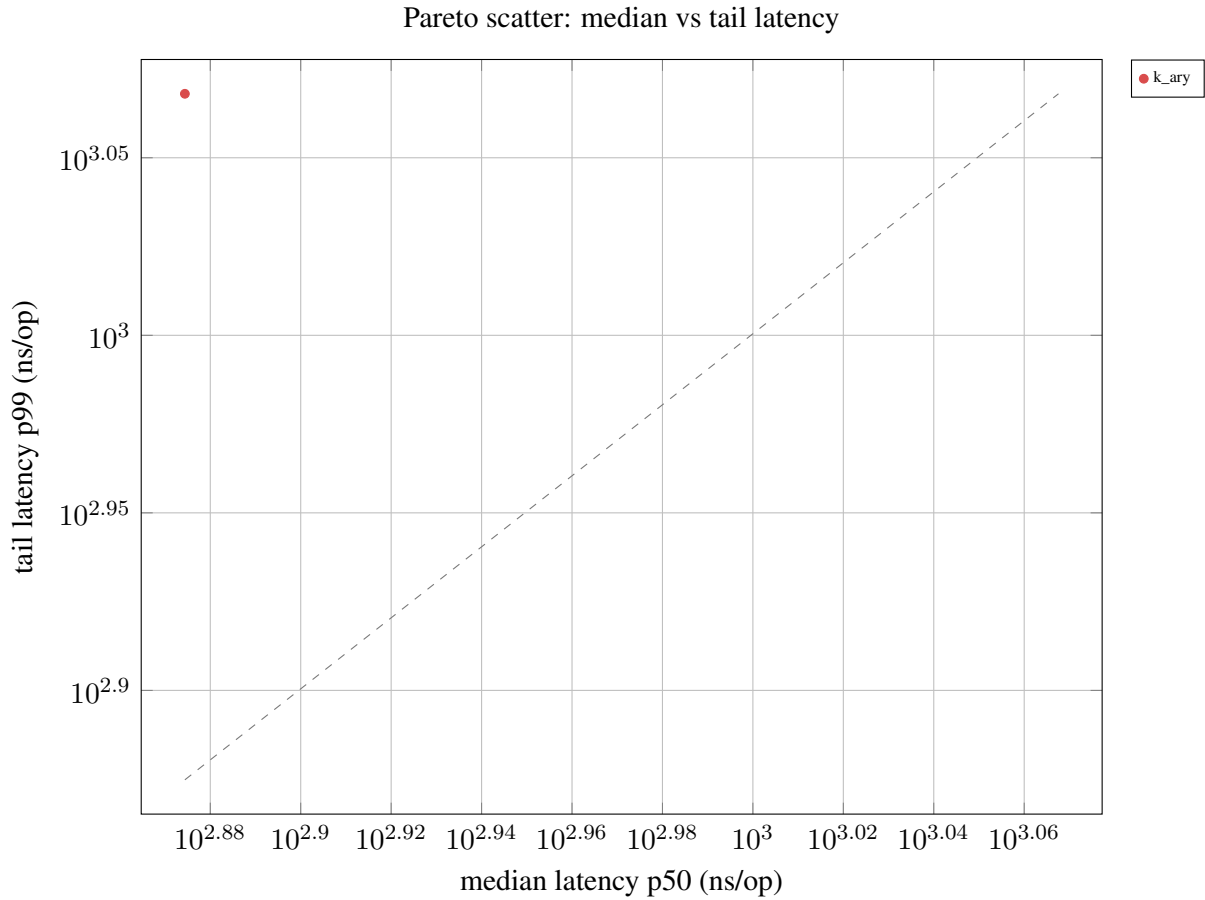


Figure A.21: Pareto scatter: median vs tail latency (one point per configuration and operation kind; diagonal = no tail surcharge)

(No comparable measurements: overall latency (ns/op) has no reference series for linear_scan in the present corpus (or was never executed). This surface is honestly omitted instead of claiming a ratio against a missing reference.)

Figure A.22: Ratio to reference linear_scan: overall latency (ns/op) (1 = same as reference)

(No comparable measurements: insert latency p50 (ns/op) has no reference series for linear_scan in the present corpus (or was never executed). This surface is honestly omitted instead of claiming a ratio against a missing reference.)

Figure A.23: Ratio to reference linear_scan: insert latency p50 (ns/op) (1 = same as reference)

(No comparable measurements: lookup latency p50 (ns/op) has no reference series for linear_scan in the present corpus (or was never executed). This surface is honestly omitted instead of claiming a ratio against a missing reference.)

Figure A.24: Ratio to reference linear_scan: lookup latency p50 (ns/op) (1 = same as reference)

(No comparable measurements: erase latency p50 (ns/op) has no reference series for linear_scan in the present corpus (or was never executed). This surface is honestly omitted instead of claiming a ratio against a missing reference.)

Figure A.25: Ratio to reference linear_scan: erase latency p50 (ns/op) (1 = same as reference)

(No comparable measurements: scan latency p50 (ns/op) has no reference series for linear_scan in the present corpus (or was never executed). This surface is honestly omitted instead of claiming a ratio against a missing reference.)

Figure A.26: Ratio to reference linear_scan: scan latency p50 (ns/op) (1 = same as reference)

(No comparable measurements: rmw latency p50 (ns/op) has no reference series for linear_scan in the present corpus (or was never executed). This surface is honestly omitted instead of claiming a ratio against a missing reference.)

Figure A.27: Ratio to reference linear_scan: rmw latency p50 (ns/op) (1 = same as reference)

remaining open. The turn converts the relative delta exactly — d becomes $d' = -d/(1 + d)$, because the denominator changes together with the reference value; a mere sign inversion would be an arithmetic error here. Points that cannot be mapped this way are omitted, not substituted.

(The reference-relative forest plot appears after the next measurement run, provided the reference axis value was measured.)

A.10 Limitations

Table A.9 lists all unresolved caveats of the measurement series, so that no finding silently lapses; row 1 names the remaining PMC gaps of the core metric (L2 and coherence without a portable counter; energy best-effort).

Table A.9: Honest limitations: unresolved caveats (no finding silently lapses)

#	Caveat	Status / consequence
1	Cache misses (core metric): L2 + coherence structurally 0; L1/L3/dTLB only collected with the PMC switch set; energy best-effort	Without <code>COMDARE_ENABLE_PMC</code> (build default OFF, so that <code>test:unit</code> builds without HW access) <code>NullPmcSource</code> reports <code>available=false</code> and all HW columns stay 0 — that is how the tabulated zeros arose. With the switch, <code>LinuxPerfPmcSource</code> collects L1, L3 (last-level) and dTLB misses for real via <code>perf_event_open(2)</code> (kernel syscall, no vendor library, no PAPI required — PAPI remains an optional secondary path). L2 and coherence invalidations stay structurally 0 even then, for lack of a portable generic counter; L3 can additionally fail to open on individual AMD platforms; energy runs best-effort via RAPL sysfs and stays empty without read access to the zone. The Intel PCM Windows driver is still pending for the Windows lane (#26/P4). The remaining L2/coherence gap is observable only indirectly via the wall-clock proxy (<code>seg_memory_layout_ns/ns_per_op</code>).
2	14 pinned axes = 0 exchangeability evidence	Only 4 of the 18 composition axes vary (<code>search_algo</code> , <code>node_type</code> , <code>memory_layout</code> , <code>prefetch</code>). Pinned (1 value each): <code>cache_traversal</code> , <code>mapping</code> , <code>path_compression</code> , <code>allocator</code> , <code>concurrency</code> , <code>serialization</code> , <code>value_handle</code> , <code>index_organization</code> , <code>io_dispatch</code> , <code>migration_policy</code> , <code>filter</code> , <code>queuing_q1</code> , <code>queuing_q2</code> , <code>persistence_target</code> . (Since INC-2d, <code>isa</code> is the system axis <code>target_isa</code> , and since INC-2c telemetry is no longer a binary_id composition axis — neither therefore counts as a pinned organ axis any more.)
3	Observer probe counters (*_find/*_probe/*_get) by design ≈ 1	Expected behaviour, no phantom: one probe per lookup. Says nothing about internal iterations.
4	Honest-0 of inactive sub-features of pinned strategies	<code>concurrency/migration/io/value_handle</code> report 0 (feature inactive), not a measurement error.

continued on next page

Table A.9 – continued

#	Caveat	Status / consequence
5	Wall-clock not bit-reproducible	Seed drives keys (deterministic), NOT CPU timing; wall-clock varies run-to-run.
6	RC dimension only nominal	K1: $\times 18 \rightarrow \times 3$ degenerated; RC axis does not provide full value breadth.
7	2/21 scan profiles = no-op (excluded)	ycsb_e and lp_range_scan perform no real scan; excluded for scan diffs (19 valid workloads).
8	Insert profiles measure upserts; stat_* columns include load phase	The insert path is effectively an upsert; the stat_* counters also accumulate the load phase.
9	prefetch measures key values as pseudo-addresses (K9)	The prefetch axis interprets key values as addresses; no real memory prefetch.
10	node_type + memory_layout: Q2 step-4 shadowing	Apparatus artefact possible (search_organ_shadowing); search_algo + prefetch are the least confounded.
11	container_mirror (path B): O(n) flatten+rebuild as extra apparatus	Path-B creatures (SOTA trees/tries/hash) keep a sorted container_mirror alongside the real search organ; its O(n) flatten+rebuild (on new inserts + successful erases; #211, unresolved as of M3) is EXTRA apparatus work the real tree does not incur $\rightarrow$ the measured ns_per_op of path-B tiers is inflated by this mirror share. (For the path-A array family the same O(n) rebuild is the GENUINE structural cost, not an extra artefact.) Resolved with #188/#211 + successor rebuild (#215).
12	memory_layout effect partly sub-noise	Part of the layout differences lies below the measurement noise $\rightarrow$ PMC externally gated (#26).
13	Secondary indirection organs (cache_traversal/mapping): uint16 key saturation	The uint64 PRIMARY key is narrowed (static_cast) into the smaller slot_index/key_type fields of the PINNED cache_traversal/mapping organs when auto-coupling (low bits); for records > type width their observer entries collide. Does NOT affect the primary lookup (uint64); observer-only limitation of pinned axes (cf. rows 2/4).
14	Capsule-internal (swallowed) OOM in a measured op	An OOM propagated outward now hard-invalidates the measurement (two_phase_valid=false). An OOM caught INTERNALLY by tier code via catch(...) (e.g. defensive memento skip) is not observable from outside $\rightarrow$ documented as a conservative residual caveat, not forced.
15	Timer overhead (clock::now()) not calibrated	The wall-clock segment timings include the non-subtracted intrinsic overhead of the std::chrono::steady_clock::now() calls (an additive, run-stable ns component per measured segment). NO empty-loop calibration is performed $\rightarrow$ documented caveat; affects the absolute ns origin, not the relative per-axis comparisons (G5 audit, MESS-MINOR-7).
16	Design labels = naming convention, no measurement effect	A few pattern labels in code/docs are naming/design caveats with no measurement effect: “hybrid visitor” (in reality a memento, no accept()/host visit), plus interpreter/template/decorator labels and reserve/if constexpr hygiene. Pure naming convention $\rightarrow$ documented as a caveat, no influence on measured values (G5 audit, PATTERN-MINOR-1..7).

B Code Structure

This appendix documents the real source-code structure of the three repositories from Section 4.1 in a verifiable way at directory level. The code state was surveyed on 2 August 2026 via directory listings (two to three levels deep); the authoritative commits are 2b0a9bd8 (comdare-cache-engine), 32e8ffa (comdare-prt-art), and daee6c85 (probst-Diplomarbeit-cache-engine, super-repository). Build artefacts (build*/), IDE directories, and the vendored third-party code under ext/ are hidden or collapsed into a single collective entry; all other paths listed here exist in the surveyed state. Paths are given relative to the respective repository root.

comdare-cache-engine — building-block library and builder

The repository follows the Pitchfork layout: apps/ contains the executables, libs/ the libraries, and libs/common the cross-cutting code. The vendored third-party code of the A/P series is collected under ext/ and is not expanded here.

```
comdare-cache-engine/                                state: commit 2b0a9bd8
|-- adapters/                                         ABI adapters of the A/P series
|                                                     (A01-hoard ... A20-dlmalloc, P01-ART ... P30-HazardPointers)
|-- apps/                                             executables (Pitchfork)
|  |-- cache_engine_builder/    builder main program (main.cpp)
|  |-- perm_runner/            runner for generated permutation modules
|  |-- anatomy_codegen_tool/, catalog_codegen_tool/, adhoc_emitter/,
|      f15_compare/, h2_score_akte_tool/, is_original_validator/
|      codegen and checking tools
|-- libs/                                             libraries (Pitchfork)
|  |-- cache_engine/          CacheEngine: axis building blocks + builder library
|  |  |-- axes/              one subdirectory per axis (alloc, layout,
|  |  |                      prefetch_axis, telemetry_axis, concurrency_axis,
|  |  |                      persistence_target, ...)
|  |  |-- topics/, anatomy/, compositions/, profile_facade/
|  |  |                      axis family roof (AxisKind), anatomy core,
|  |  |                      composition and catalogue layer
|  |  |-- builder/          codegen/, permutation_loop/, module_loader/,
|  |  |                      experiment_driver/ (ExperimentDriver),
|  |  |                      experiment_tree/ (ExperimentPlanDirector),
|  |  |                      build_orchestrator/, pruef_dock/, bestandslog/,
|  |  |                      curve_fit/, ...
|  |  |-- include/cache_engine/abi/  ABI headers (baustein_variants.hpp,
|  |  |                      resolve_baustein.hpp, module_abi_v1.hpp,
|  |  |                      anatomy_version_stamp.hpp, ...)
|  |  |-- include/cache_engine/measurement/  realm roots + registries
|  |  |                      (system_axis.hpp, measurement_*_registry.hpp,
|  |  |                      ram_frequency_reading.hpp, spd_ddr5_parser.hpp,
```

```

| | | ram_probe_chain.hpp, hardware_probe_factory.hpp)
| | '-- algorithm_profiles/ sota/ (33 profile files *.profile.xml +
| | sota_h2_scores.xml), allocators/, load_profiles/,
| | thesis_profiles/, permutation_axes.xml
| |-- common/ cross-cutting code: config/, measurement/, platform/,
| | serialization/ (xml_config_parser/), succinct/
| |-- execution_engine/ ExecutionEngine library (result_aggregator)
| |-- search_engine/ SearchEngine building blocks (node/, page/, traversal/,
| | operations/, memory_layout/, value_handle/, ...)
| '-- test_infra/ benchmark_suite/, test_data/, workload_generator/
|-- ext/ vendored third-party repositories of the A/P series
|-- tests/ unit/, integration/, generic_module_tests/,
| module_specific_tests/, fixtures/
|-- tools/ auxiliary tools (abi_stability_test/, gen_golden_fullpilot/,
| best_binary_selector/, p27_bundle_finder/, ycsb_cli/,
| axis_registry_gen/ et al.; the former permutation_codegen
| second channel has been removed, cf. Chapter 4)
'-- docs/ architecture, status, and session documentation

```

Component	Path	Purpose
CacheEngineBuilder	apps/cache_engine_builder/	builder executable (configuration to permutation modules)
ExperimentDriver	libs/cache_engine/builder/ experiment_driver/	seven-phase experiment driver (host side)
Code generation	libs/cache_engine/builder/codegen/	generate_module_ from_profile incl. templates
Permutation loop	libs/cache_engine/builder/ permutation_loop/	enumeration of the axis permutations
Module loader	libs/cache_engine/builder/ module_loader/	loading of the generated modules
ABI headers	libs/cache_engine/include/ cache_engine/abi/	incl. baustein_ variants.hpp, resolve_baustein.hpp
XML config parser	libs/common/serialization/ xml_config_parser/	parser for the profile and experiment XMLs
Axis building blocks	libs/cache_engine/axes/	one subdirectory per axis
SOTA profiles	libs/cache_engine/ algorithm_profiles/sota/	33 profile files (*.profile.xml): the 30 standalone SOTA algorithm profiles (Section 3.2) plus three files for the concurrency disciplines olc, rcu, and hazard_pointers
A/P-series adapters	adapters/	ABI adapters over the ext/ third-party code (10 A, 12 P entries)

[illegible]

Component	Path	Purpose
PrtArtSearchEngine	prt_art/include/prt_art/identity/ prt_art_search_engine.hpp	hybrid engine of the probe (vector/map/tuple API)
ExecutionEngine adapter	prt_art/include/prt_art/identity/ prt_art_execution_engine_ adapter.hpp	ABI coupling to the measurement infrastructure
Pool allocator	prt_art/include/prt_art/allocator/	descriptors of the 4+1+2 pools
OLC	prt_art/include/prt_art/ concurrency/	OLC with reserved blocks
Prefetch	prt_art/include/prt_art/prefetch/	path-oriented prefetch, redirect prefetch
Multi-level layout	prt_art/include/prt_art/ memory_layout/	multi_level_ layout.hpp et al.
Density tracker, H1–H3	prt_art/include/prt_art/ measurement/	density_tracker.hpp, hypothesis_ metrics.hpp
legacy_reimpl	prt_art/legacy_reimpl/	14 comparison re-implementations (P11–P27)
Probe profile	prt_art/algorithm_profiles/	profile and axis-extension XML

probst-Diplomarbeit-cache-engine — super-repository

The super-repository bundles the measurement driver, the experiment configurations, the evaluation pipeline, and the manuscript; the two code repositories are embedded as submodules under `Code/external/`.

```

probst-Diplomarbeit-cache-engine/          state: commit daee6c85
|-- Code/
|   |-- 01_sample_data_generator/    generation of sample data
|   |-- 02_messung_driver/           measurement driver (main.cpp, v32_orchestrator.hpp,
|   |                               plugin_loader.hpp, measurement_writer.hpp, ...)
|   |-- 03_binary_to_csv/            evaluation stage 1: binary measurement data -> CSV
|   |-- 04_csv_to_latex/             evaluation stage 2: CSV -> LaTeX tables
|   |-- 05_diagram_generator/        evaluation stage 3: diagram generation
|   |-- 06_latex_to_pdf/            manuscript build (build_thesis.sh/.bat)
|   |-- 07_tier_binary_report/       report on the tier measurement binaries
|   |-- 08_appendix_generator/       in-process generator of the measurement
|   |                               appendix (replaces the .ps1 helper)
|   |-- 09_tex_formatter/            typesetting-neutral LaTeX source formatter
|   |-- experiment_config/           config_a_prt_art_vs_sota.xml,
|   |                               config_b_cache_engine_perms.xml,
|   |                               config_c_merge_alt_neu.xml, messreihen.xml,
|   |                               sample_data/
|   |-- external/                   submodules: comdare-cache-engine, comdare-prt-art
|   |-- tests/, cmake/, tools/      repo-wide tests, build and helper scripts
|-- thesis/diplomarbeit/            manuscript: diplomarbeit.tex, Makefile,
|                                   kapitel/{de,en}/, anhang/{de,en}/,
|                                   tabellen/, tikz/

```

```

|-- docs/                ledger, architecture, and session documentation
|-- Forschungsarbeiten/  literature collection per axis (collective entry)
|-- measurement/         measurement write-back (persisted CI runs)
`-- Messdaten-Backup/    additive backup of the raw measurement data

```

Component	Path	Purpose
messung_driver	Code/02_messung_driver/	measurement driver: loads permutation modules, orchestrates the measurement series
Experiment configurations	Code/experiment_config/	XML configurations A/B/C and messreihen.xml
binary_to_csv	Code/03_binary_to_csv/	evaluation pipeline, stage 1
csv_to_latex	Code/04_csv_to_latex/	evaluation pipeline, stage 2
diagram_generator	Code/05_diagram_generator/	diagram generation from CSV data
latex_to_pdf	Code/06_latex_to_pdf/	manuscript build scripts
Submodules	Code/external/	embedding of the two code repositories
Manuscript	thesis/diplomarbeit/	LaTeX sources of the thesis (DE/EN)
Messdaten-Backup	Messdaten-Backup/	additive backup of the raw measurement data

C Glossary

This glossary summarizes the abbreviations and central terms used in the thesis.

ABI Application Binary Interface — the binary-stable interface through which the builder loads the permutation modules (living beings).

Adaptive vs. dynamic vs. measurement-driven Distinction of terms for adapting the algorithm behaviour. *Adaptive*: adaptation based on run-time-observed data characteristics (e.g. adaptive node sizes in ART). *Dynamic*: the run-time selection of the fully compiled permutation *module* via the permutation identifier (module loading) — all eighteen organ axes are baked in per binary at compile time; in addition there are run-time-*variable* parameters of dynamic sub-axes (prefetch distance, batch size, inline threshold, pool budget; cf. Section 3.4), no run-time choice of individual axis strategies. *Measurement-driven*: selection of the configuration based on empirically collected measurement series — the leitmotif of this thesis; the corresponding selection mechanism is implemented in a first increment (`best_binary_selector`: ranking and delivery of the best permutation per metric from the measurement CSVs), while the fully automatic, load-profile-spanning selection remains outlook.

Anatomy The *body* of a living being: the fixed composition of its eighteen axis organs and, above all, their *wiring* — the generalized interfaces through which one organ draws on the services of the others (e.g. the shared allocation interface). Not a second model beside the search algorithm but its internal structure (cf. Section 3.3.1).

ART Adaptive Radix Tree [LKN13]; reference design with adaptive node sizes (Node4/16/48/256).

Autonomous profile-driven heuristic evaluation The intended mechanism that automatically derives, per load profile, the best axis configuration from the measurement series and stores it as a reusable heuristic policy (target mechanism: ML classification of load-profile/axis features onto configurations). First building blocks are implemented — the automatic best-binary selection from measurement series (`best_binary_selector`) and the extraction/storage as an XML load-profile result —; the ML classification itself remains outlook (cf. Section 6.3).

AVX, AVX-512 x86 SIMD extensions (256- and 512-bit, respectively).

Axis An orthogonal partial decision (“organ”) of a search algorithm; the framework maintains eighteen organ composition axes (T0–T17) alongside the three system main axes and the measurement realm (cf. Section 3.4.1).

built_stem Field of the textual plan (revision v1.1) that states, per build step, the actually built file stem — produced by the very function the build orchestrator calls during the real build; evidence of the *single* main channel (cf. Section 4.3.2).

CacheEngineBuilder The autonomous platform-measurement system of the chain: root of the system realm; builds and stamps the tier binaries per platform and executes the seven-phase pipeline via the `ExperimentDriver` (cf. Sections 3.4.5 and 4.3.2).

CLU Cache-line utilization — the fraction of used bytes per loaded cache line.

Complex axis Bracket over a fixed recombination of several main axes that jointly act as *one* axis: the outer bracket over target ISA, operating system, and extension hardware, and the inner complex axis `target_isa` (RAM frequency, CAS latency, CPU fabrication); in the identity stamp a complex axis occupies exactly one field (cf. Section 3.4.2).

CPI, IPC Cycles per instruction and instructions per cycle, respectively.

CRTP Curiously Recurring Template Pattern — C++ idiom for static polymorphism.

dTLB Data translation lookaside buffer; caches data address translations.

Fit stamp (Passungs-Stempel) *Design* of a check that validates the compiled-in platform *class* of a tier binary against the run-time verdict of the hardware probing on every measurement run; not built in the state of this thesis (cf. Section 6.1).

Genus An externally visible interface (Map, Container, Graph) — the topmost level (level 1) of the framework; see *Three-level model*.

Genus subclass The second level beneath a genus, with a fixed axis set (code term: `AnatomyGenus`, “animal subclass”). In the biological reading the “mammal” = `SearchAlgorithm` (the full eighteen-axis anatomy); under `Container` the subclasses `Set`, `Sequence`, `Adapter`, and `View` (“bird”, “reptile”, “invertebrate”, “plant”).

GoF “Gang of Four” — the canonical design patterns after Gamma et al. [GHJV94].

HBM High-Bandwidth Memory.

HOT Height Optimized Trie [BZP⁺18].

Living being A complete search algorithm as a composition of all axes (metaphorical code term; technical identifier: `SearchAlgorithm` instance).

LOUDS Level-Ordered Unary Degree Sequence — succinct tree encoding (Jacobson 1989).

Measurement realm The third axis realm beside organ and system axes: the concerns of measuring itself, rooted at the experiment *planner*; its first main axis is the load framework, and all measurement-realm axes are strictly binary-id-neutral (cf. Section 3.4.1).

MESI, MOESI, MESIF Cache coherence protocols (base protocol and AMD/Intel variants, respectively).

NINE Non-Inclusive-Non-Exclusive — inclusion policy of the L3 in Intel server processors.

NUMA Non-Uniform Memory Access.

OLC Optimistic Lock Coupling — optimistic concurrency technique (ARTSync, P08) [LABN16].

Organ Metaphorical term for an *axis* (level 3 of the three-level model); a search algorithm has eighteen organ axes.

Planner The experiment planner (`ExperimentPlanDirector`): resolves the user XML against the three offer catalogues, selects the measurement axes, and steers the builder orders of the chain; root of the measurement realm (cf. Section 3.4.5).

PMC (Hardware) Performance Monitoring Counter.

Probe The new search algorithm to be integrated for testing (here PRT-ART); it is compiled by the builder against the core constituents of the cache engine.

Probe dock The externally visible measurement interface of a genus, through which the builder loads a living being, drives it through the interface operations, measures it, and persists the results (cf. Section 3.3.2).

Provenance level Trust level of a collected hardware value — configured-measured, SPD-JEDEC base value, or declared-not-measured — carried by every result together with its collection timestamp; a lower level may displace a higher one only with the label travelling along (cf. Section 3.4.3).

PRT-ART Probst Redirect Tree / ART hybrid — the probe of this thesis.

Rank Centrality classification of the state-of-the-art publications (Rank-1/2/3); explained in Section 1.3.

RCU Read-Copy-Update — reclamation scheme (McKenney, P29) [McK01].

RRIP, DRRIP (Dynamic) Re-Reference Interval Prediction — cache replacement heuristic [JTJE10].

SIMD Single Instruction, Multiple Data.

SOSD Search On Sorted Data — index benchmark [KMvR⁺19].

SOTA State of the Art.

SuRF Succinct Range Filter [ZLL⁺18].

SVE, NEON, RVV ARM (SVE, NEON) and RISC-V (RVV) SIMD extensions, respectively.

Three-level model The layering of the one architecture: *genus* (level 1, the outer algorithm interface — Map, Container, Graph), *genus subclass* (level 2, fixed axis set), and *organs/axes* (level 3). See Section 3.3.2.

TLB Translation Lookaside Buffer.

Two-gate protocol The acceptance protocol per implementation package: gate 1 is the strict double-run gate of the engine repository (exclusively “0 failed”, no exception list), gate 2 the integration gate of the super-repository with a nominally frozen exception set (cf. Section 4.4).

XML load-profile result The persistence format demanded by the thesis assignment (sub-task 7) in which the heuristic policy extracted for an overall algorithm — reusable per architecture focus — is stored under reference to the applied load profiles; it serves as input to the autonomous profile-driven heuristic evaluation. Import parser and export writer including the extractor are implemented (round-trip-capable schema `comdare_load_profile`); what remains as outlook is the autonomous heuristic evaluation built on top (ML classification).

YCSB Yahoo! Cloud Serving Benchmark [CST⁺10].

D Building-Block Matrix

This appendix contains the complete building-block matrix referenced by the state of the art (Section 3.2). Each axis is an *organ* of the living being: a concrete search algorithm (technically `SearchAlgorithm`, metaphorically a living being) is the fixed composition of its eighteen organ axes (T0–T17), each of which is an organ; the building-block variants listed per axis are the interchangeable forms of that organ, and their permutation amounts to a genetic experiment on the living being. The *anatomy* of this living being is the wiring *between* the organs: axis algorithms use the interfaces of other axes (the allocation axis, for instance, provides a shared allocation interface for the remaining organs). For each axis it lists the purpose, the *static* sub-axes (compile-time-selectable variant families) and the *dynamic* sub-axes (run-time parameters), as well as the concrete building-block variants with their source (paper identifier or code/baseline). The entries are verified against the `comdare-cache-engine` code and the paper-provenance map (Doc 18).

Beyond the eighteen organ slots this appendix documents the two remaining axis categories of the triad (cf. Section 3.4.1). The *system realm* comprises exactly three main axes in canonical, code-guarded order — target ISA (`target_isa`), operating system (`operating_system`) and extension hardware (`external_utils`) — plus the outer complex bracket `build_target_complex`, which is explicitly not a fourth main axis; the order is not an editorial decision but follows the single ordering source of the code. The *measurement realm* comprises the measurement categories, the measurement tooling and the load framework as meta-meta axis at the end of the realm. In the anatomy metaphor, system and measurement axes are not a further organ but the *blood*: not permutable, not part of the permutation tree and strictly neutral towards the binary identity. The entries of both spheres are additionally verified against the three axis registries of the code (`cache_engine_axis_registry.xml` for the organ realm, `system_axis_registry.xml` and `measurement_axis_registry.xml`). Historically conserved legacy sections — the SIMD offering catalogue under the extension hardware as well as the page-type catalogue Build-PG and the platform hardware strategy Build-HW — are marked as such and stand at the end of the respective block.

D.1 T0 Search Algorithm Axis T0 (Search Structures and Methods)

This axis is the *search organ* (T0) of the living being `SearchAlgorithm` (cf. Section 3.2).

Systematic permutation of 21 concrete search algorithms and structures (S01–S21: 17 base building blocks — array-based, sorted vectors, original paper wrappers, search methods like k-ary/interpolation, and ordered structures like skip-list/B-tree — plus 4 opt-in per-K k-ary wrappers K2/K4/K8/K16, disabled by default) to measurably quantify the impact of search paradigm choice on performance characteristics within a uniform `std::map` interface. Sub-axes classify by access density (SA1 dense, SA2 sparse, SA3/SA4 multilevel), enable compliance proofs through paper mixins (Habich requirement: `is_original` marking per function/wrapper), and identify SIMD-capable and store-traversable variants.

Static sub-axes

ID	Name	Description
SA1	Direct-Addressed High Density (Dense Access)	Array family: direct-addressed ($O(1)$ lookup), high fill density $\geq 50\%$. Includes <code>Array256SearchAlgo</code> (256-slot compact array, ART Node256 re-impl) and <code>Array65535SearchAlgo</code> (uint16-based, mid-density 25-50% direct-addressed).
SA2	Sparse/Ordered (Sparse Access)	Sorted vectors, ordered search methods, and tree/hash structures: low fill density, range-scan support, variable $O(\log n)$ – $O(1)$ lookup. Includes <code>VectorU8U8SearchAlgo</code> , original paper wrappers (ART/HOT/START/Wormhole/SuRF), search methods (k-ary, interpolation, Eytzinger, hash), and structures (skip-list, BST, B-tree).
SA3	Multi-Byte Discriminators (Multilevel Access)	Multi-byte search structures with indexed sub-discriminators (e.g., START with cost-DP multi-level). <code>VectorU16U16SearchAlgo</code> (u16-keys, sorted lower_bound) and <code>OriginalStartSearchAlgo</code> (paper P05) belong here.
SA4	Direct-Addressed Medium Density (Direct Multibyte)	Shares direct-addressing access pattern (like SA1) with multi-byte granularity (uint16 size) and medium fill density 25–50% (vs. SA1 $\geq 50\%$). Example: prt-art <code>Array65535</code> (REV 6 §5.17). Sub-axis added to not break SA1 abstraction but capture density variant.

Building blocks

Variant	Description	Source
<code>Array256SearchAlgo</code> (S01)	ART Node256 direct-addressed re-implementation: <code>std::array<std::optional<u64>, 256></code> with $O(1)$ lookup/insert/erase. Cache-line-aligned, SIMD-capable, no heap allocation. Baseline for dense accesses.	Code/Baseline (Leis ICDE 2013 ART Paper P01)
<code>VectorU8U8SearchAlgo</code> (S02)	Sparse sorted key-value vector pair: <code>std::vector<u8> keys + std::vector<u64> values</code> with <code>std::lower_bound</code> . $O(\log n)$ lookup, $O(n)$ insert/erase. Concept-equivalent to HOT-k-constrained. SIMD-capable (bitmask scan).	Code/Baseline (Binna SIGMOD 2018 HOT Paper P02)
<code>VectorU16U16SearchAlgo</code> (S03)	Sparse sorted u16 vector pair: <code>std::vector<u16> keys + std::vector<u64> values</code> . Simplified START concept (multi-byte discriminators without cost-DP). $O(\log n)$ lookup, $O(n)$ insert/erase.	Code/Baseline (Fent ICDEW 2020 START Paper P05)
<code>Array65535SearchAlgo</code> (S09)	Mid-density direct-addressed uint16-based (25–50% fill density). Migrated from prt-art legacy code (REV 6 §5.17). $O(1)$ lookup but with memory overhead vs. SA1 denser arrays.	Code/Baseline (prt-art F.6-Migration)
<code>OriginalArtSearchAlgo</code> (S04)	ART paper binding (<code>unodb::db::is_original_module()=true</code> (4/4 functions original). Paper mixin via SHA256-validated tool code generation. Habich compliance: <code>get_compiler()='gcc-9.5'</code> , all insert/lookup/erase/clear marked original.	P01 (Leis/Kemper/Neumann ICDE 2013, Apache-2.0 unodb)
<code>OriginalHotSearchAlgo</code> (S05)	HOT paper binding (<code>HOTRowex::is_original_module()=false</code> (2/4 original). insert/lookup original, but erase/clear gaps (HotRowex is append-only; re-impl fallback). Paper P02.	P02 (Binna et al. SIGMOD 2018, ISC ISC)
<code>OriginalStartSearchAlgo</code> (S06)	START paper binding (<code>sosd-adapter::is_original_module()=false</code> (2/4 original). insert/lookup original, erase/clear gaps (no remove in sosd-adapter). Paper P05, multi-byte discriminators.	P05 (Fent/Jungmair/Kipf/Neumann ICDEW 2020, MIT)

Variant	Description	Source
OriginalWormholeSearchAlgo (S07)	Wormhole paper binding: <code>is_original_module()=false</code> (3/4 original). <code>put/get/del</code> original, clear gap. Hybrid hash+trie+B+ index. GPL-3.0 licensed (blocks original code linking for thesis).	P07 (Wu/Ni/Jiang EuroSys 2019, GPL-3.0)
OriginalSurfSearchAlgo (S08)	SuRF paper binding (FST with LOUDS bitmap): <code>is_original_module()=false</code> (1/4 original, lookup only). Succinct range filter, multi-header library. Paper P10.	P10 (Zhang/Lim/Andersen SIGMOD 2018, Apache-2.0)
KArySearchAlgo (S10)	k-ary search method (Schlegel/Gemulla/Lehner DaMoN 2009): k-way partition vs. binary halving. K is iterable_ aspect ($K \in \{2,4,8,16\}$). $O(\lceil \log_{K+1} n \rceil)$ iterations, parallel ILP comparisons. Array family but no dedicated traversal organ (SA2 classification). C++23 re-impl, <code>is_original=false</code> .	Schlegel et al. DaMoN 2009 [SGL09]
InterpolationSearchAlgo (S11)	Interpolation search method (Perl/Itai/Avni CACM 1978): distribution-aware position estimation vs. blind halving. $O(\log \log N)$ avg with uniform distribution, $O(N)$ worst-case. Not SIMD-vectorizable (data-dependent index). C++23 re-impl.	Perl/Itai/Avni CACM 1978 [PIA78]
EytzingerSearchAlgo (S12)	Eytzinger/BFS-layout branch-free search (Khuong/Morin JEA 2017): cache-layout search method with branch-free kernel. Keys in BFS order of implicit search tree. For large arrays per paper fastest general-purpose layout. C++23 re-impl (experiment harness without standard license).	Khuong/Morin JEA 2017 [KM17b]
SkipListSearchAlgo (S13)	Skip-list probabilistic ordered STRUCTURE (Pugh CACM 1990): $O(\log n)$ expected search/insert/erase without explicit rebalancing. Each node has 1..kMaxLevel forward pointers (coin-flip probability 0.5). Index-based (<code>std::vector<Node></code> with uint32 indices, value-semantic). Textbook algorithm, C++23 re-impl.	Pugh CACM 1990 [Pug90]
HashSearchAlgo (S14)	Open-addressing hash table UNORDERED (Knuth TAOCP 3 §6.4): Fibonacci hash, linear probing, tombstone-erase on resize. $O(1)$ expected lookup/insert at <code>load_factor < 0.7</code> . No range-scan support (distinctive classification). Correct tombstone marking for probe-chain integrity. C++23 re-impl.	Knuth TAOCP 3 §6.4 [Knu98]
LinearScanSearchAlgo (S15)	Unsorted linear scan baseline (Leis ICDE 2013 ART Node4): $O(n)$ lookup on unsorted KV-array. Simplest comparison null-point structure: insert $O(1)$ -amortized, erase $O(1)$ via swap-and-pop. Store-traversable (array family). No range-scan support.	Code/Baseline (Leis ICDE 2013 ART Paper P01 Node4)
BinarySearchTreeSearchAlgo (S16)	Unbalanced binary search tree (Knuth TAOCP 3 §6.2.2 Hibbard-deletion): deterministic comparison-tree baseline (vs. skip-list probabilistic). Index-based (<code>std::vector<Node></code> with uint32 indices, free-list). Hibbard-erase (3 cases: leaf/1 child/2 children via in-order successor). $O(n)$ worst-case with sorted input.	Knuth TAOCP 3 §6.2.2 [Knu98]
BTreeSearchAlgo (S17)	Balanced block-oriented multi-way B-tree (Bayer/McCreight Acta Inf. 1972 / CLRS Ch. 18): $t=4$, so $[3..7]$ keys per node, up to 8 children. ALWAYS balanced ($O(\log n)$ guaranteed). Index-based, <code>alignas(64)</code> nodes (cache-line alignment). Top-down splitting on insert, borrow/merge on erase. DB index workhorse.	Bayer/McCreight Acta Inf. 1972 / CLRS [BM72]

Contributing sources: P01, P02, P05, P07, P10, k-ary (Schlegel et al. 2009), interpolation (Perl et al. 1978), Eytzinger (Khuong/Morin 2017), skip list (Pugh 1990), hashing/BST (Knuth TAOCP 3), B-tree (Bayer/McCreight 1972) [LKN13, BZP⁺18, FJKN20, WNJ19, ZLL⁺18, SGL09, PIA78, KM17b, Pug90, Knu98,

BM72].

D.2 T1 Cache Traversal (Fanout Lookup)

This axis is the *cache-traversal organ* (T1) of the living being `SearchAlgorithm` (cf. Section 3.2). Axis `axis_03b` defines three distinct strategies for key resolution in small fanout arrays (typically B+ tree or trie inner nodes). It spans a spectrum from linear scan ($O(N)$, cache-friendly, small), via multiplicative hashing with open addressing ($O(1)$ amortized, Knuth-Fibonacci), to sorted binary search ($O(\log N)$, ordered traversal, B+ tree standard). All three are textbook baselines without original code linking, implemented as pure C++23 re-implementations.

Static sub-axes

ID	Name	Description
CT1	Linear Access	Linear-scan-based lookup methods on small, unsorted fanout arrays. Characterized by $O(N)$ resolution with simple comparisons, typical for nodes with fanout < 32 in B+ trees. Sub-axis for <code>LinearFanout</code> and <code>BinarySearchFanout</code> (both: <code>axis_tag = linear_access_tag</code>).
CT2	Hash-Based Access	Hash lookup with open addressing and linear probing. $O(1)$ amortized resolution, characterized by Fibonacci multiplier (11400714819323198485 $\approx 2^{64}/\varphi$), power-of-2 bucket count, dynamic rehashing when load factor > 0.7 . Only <code>HashLookup</code> wrapper. Sub-axis: <code>axis_tag = hash_access_tag</code> .

Building blocks

Variant	Description	Source
<code>LinearFanout</code>	Unsorted fanout traversal via <code>std::find_if</code> and linear scan. $O(N)$ lookup, no hash overhead, cache-friendly for small N (< 32). Classic B-tree baseline (Bayer/McCreight 1972). Belongs to sub-axis CT1.	Code/Baseline — Bayer/McCreight 1972 (Acta Informatica)
<code>HashLookup</code>	Fibonacci-hash open addressing with linear probing (Knuth TAOCP Vol. 3, §6.4). $O(1)$ amortized, dynamic rehashing (doubles capacity when load ≥ 0.7), iterable runtime capacity (<code>kIterableInitialCapacities: {8, 16, 64, 256, 1024}</code>). Standalone C++23 re-implementation, no original code binding. Belongs to sub-axis CT2.	Code/Baseline — Knuth 1998 (The Art of Computer Programming Vol.3)
<code>BinarySearchFanout</code>	Sorted fanout traversal via <code>std::lower_bound</code> . $O(\log N)$ lookup with sorted ordering, preserves key sequencing for range-capable routing layers. Classic B+ tree inner-node search (Bayer/McCreight 1972), third distinct strategy of the axis. C++23 re-implementation (R7.2, 2026-05-29), <code>is_original=false</code> . Belongs to sub-axis CT1.	Code/Baseline — Bayer/McCreight 1972 (Acta Informatica)

Contributing sources: Bayer/McCreight 1972, Knuth TAOCP Vol. 3 [BM72, Knu98].

Related literature: P17, P21, P32 [BDFC05, CGM01, SKK⁺25].

D.3 T2 Slot-to-Offset Mapping (Traversal Sublayer)

This axis is the *mapping organ* (T2) of the living being `SearchAlgorithm` (cf. Section 3.2).

Mediation between logical slot indices and physical memory offsets in trie inner nodes and memory pools. The axis provides two fundamentally different mapping strategies: direct absolute offsets (linear packing) or pool-relative offsets with automatic rebase mechanism supporting memory reallocation and persistent data structures.

Static sub-axes

ID	Name	Description
MP1	Direct-Access Mapping (MP1)	Direct slot-to-absolute-offset mapping without indirection. Wrapper family for classic array-of-pointers layout as in B-tree inner nodes (Bayer/McCreight 1972). Simple cache locality, $O(N)$ lookup via <code>std::find_if</code> .
MP2	Pool-Relative Mapping (MP2)	Slot-to-pool-relative-offset mapping using based-pointer arena pattern. All offsets stored relative to dynamic <code>pool_base_address</code> . Enables $O(1)$ rebase on memory reallocation — core property for persistent data structures (Driscoll et al. 1989) and <code>CustomAlignedStructure</code> pattern in prt-art.

Building blocks

Variant	Description	Source
DirectPlacement	Wrapper for direct slot-to-absolute-offset mapping (linear packing). <code>std::vector<pair<slot_index_type, offset_type>;</code> <code>resolve_offset/register_slot</code> via linear <code>std::find_if</code> . Usage: prt-art <code>INode::placement_page_</code> + mapping table. Storage: 16-bit slot index + <code>size_t</code> offset per entry. No pool base dependency.	CE-Baseline
PoolRelative	Wrapper for pool-relative-offset mapping with $O(1)$ rebase capability. <code>std::vector<pair<slot_index_type, relative_offset>;</code> + mutable <code>pool_base_address</code> . On pool reallocation: <code>rebase(new_pool_base)</code> updates only <code>pool_base_</code> — all relative offsets remain valid. Used with prt-art <code>CustomAlignedStructure</code> + versioning scenarios (persistent data structures per Driscoll/Sarnak/Sleator/Tarjan 1989).	CE-Baseline

Contributing sources: — (CE baseline, own wrappers).

Related literature: Bayer/McCreight 1972, Driscoll et al. 1989, P03, P07 [BM72, DSST89, MKM12, WNJ19].

D.4 T3 Path Compression

This axis is the *path-compression organ* (T3) of the living being `SearchAlgorithm` (cf. Section 3.2).

Defines the compression granularity and strategy for key paths in trie structures. Controls how inner nodes store and navigate redundant byte sequences or significant bits of common prefixes to optimize memory and cache efficiency.

Static sub-axes

ID	Name	Description
pc1	Compression Granularity (PC1)	Defines the minimal unit of path compression: bit-wise (Patricia), byte-wise (ART), or no compression (raw path). Determines trie height and optimal code blocks for traversal.
pc2	Skip Strategy (PC2)	Defines the navigation pattern at inner nodes: linear over 8 bytes (ByteWise), single-bit comparison (Patricia), or direct jump without intermediate steps. Influences branch density and prefetch behavior.
pc3	Decode Complexity (PC3)	Classifies the runtime complexity when extracting prefix information: $O(1)$ for direct extraction, $O(\log n)$ for structure-based search, $O(n)$ for sequential check. Important for operations like <code>common_prefix_len()</code> and key comparisons.

Building blocks

Variant	Description	Source
PathCompression None	No path compression; raw key paths are stored completely and uncompressed in nodes. Serves as measurement baseline with <code>compression_ratio=1.0</code> . Used as control case to quantify memory and height savings of compressing strategies.	Code/Baseline
ByteWisePath Compression	Byte-by-byte compression (ART-style); stores up to 7 prefix bytes per node in a compact u64 word with length tag in high byte. Typical 2x reduction in trie height (<code>compression_ratio=0.5</code>). Origin: Leis et al., ART ICDE 2013 (P01). Employs goldstandard operations <code>common_prefix_len()</code> and <code>cut()</code> for navigation optimization.	P01
PatriciaPath Compression	Single-bit-split path compression (Patricia trie); navigates bit-wise instead of byte-wise, reduces trie height by typically factor 3 (<code>compression_ratio=0.3</code>). Origin: Morrison PATRICIA JACM 1968; modernly implemented in HOT (Binna SIGMOD 2018, P02) and Wormhole (Wu Euro Sys 2019, P07). F15-operative primitives: <code>key_split_bit()</code> for MSB-first bit extraction, <code>path_descend_scan()</code> for goldstandard-compliant <code>_scan</code> signature with key-dependent variable depth.	P02

Contributing sources: P01, P02, P07, Morrison 1968 [LKN13, BZP⁺18, WNJ19, Mor68].

Related literature: P04, P05, P09 [BFTV24, FJKN20, Jac89].

D.5 T4 Node Types (Node-Type Axis)

This axis is the *node-type organ* (T4) of the living being `SearchAlgorithm` (cf. Section 3.2).

The axis `axis_04_node_type` defines the four node formats of the Adaptive Radix Tree (ART) as exchangeable variants: Node4, Node16, Node48, and Node256. Each node type offers a different balance between memory, cache efficiency, and lookup speed. This axis forms the foundation for adaptive tree reorganization and is central to the design of search algorithms.

Static sub-axes

ID	Name	Description
NT1	Capacity Class (NT1)	Classification by fanout size and slot count: small (Node4 with 4 slots, linear search), medium (Node16 with 16 slots, SIMD binary search) to maximal (Node256 with 256 slots, direct-addressed).
NT2	Access Pattern (NT2)	Method of key lookup within a node: linear scan (Node4), SIMD binary search (Node16), indirect indexing (Node48), or direct addressing (Node256).
NT3	Compactness (NT3)	Memory layout and density: sparse and cache-aligned (Node4), dense with indirection (Node48), or maximally direct (Node256), with impacts on memory usage and cache-line misses.

Building blocks

Variant	Description	Source
Node4NodeType	Small ART node with 4 slots and linear search. Ideal for sparsely populated upper tree levels, fits completely in a cache line (64 bytes). Implemented as CE re-implementation of the ART specification with <code>node_find_scan()</code> as linear scan over sorted keys.	P01
Node16NodeType	Medium ART node with 16 slots, optimized for SIMD-based binary search (SSE2 PCMPSTRM / AVX2 VPCMPSTRM). Balance between memory and search speed; largest capacity without indirection. CE re-implementation of the Leis ICDE 2013 specification.	P01
Node48NodeType	Large ART node with 48 slots, using indirection: 256-byte ChildIndex (byte→slot mapping, 0=empty) plus 48-slot ChildArray. Trade-off between memory (vs Node256 direct) and cache misses (vs Node4 linear). Double-indirection access pattern (index + child).	P01
Node256NodeType	Maximal ART node with 256 slots, directly addressed without index. O(1) lookup (no scan, no index touch). High memory overhead (256 pointers), but optimal search speed for dense nodes. Default for lower tree levels.	P01
Internal/Border	Node-type pair of Masstree (trie of B ⁺ trees): internal nodes navigate the 8-byte key slices of the B ⁺ level, border nodes form the leaf level with the values or the transition to the next trie layer. Paper node format (reconstruction target of this thesis, no CE baseline).	P03 (Mao et al. 2012) [MKM12]
Decision/Span	Node-type pair of the B ² -tree (B ⁺ with embedded binary inner nodes, cache-aware): decision nodes carry the binary branching decision within the page, span nodes group contiguous key ranges. Paper node format (reconstruction target of this thesis, no CE baseline).	P06 (Schmeisser et al. 2022) [SSL ⁺ 21]
LOUDS-Dense/Sparse	Node-type pair of the succinct range filter SuRF: LOUDS-Dense encodes the upper, densely populated trie levels bitmap-based with direct addressing, LOUDS-Sparse the lower, sparsely populated levels byte-sequentially succinct. Paper node format (reconstruction target of this thesis, no CE baseline).	P10 (Zhang et al. 2018) [ZLL ⁺ 18]
NodeTypeSlotStore	Storage organ: node-type-driven bounded slot storage. Spans <code>std::array</code> with capacity <code>N::max_capacity()</code> over a shared <code>uint64</code> key. Fulfills the StorageOrgan concept and is drop-in substrate for ComposedSearch. Baseline variant without layout/allocator integration.	Code/Baseline
ComposedStore	3-axis storage organ: <code>node_type(N) ⊕ layout(L) ⊕ allocator(A)</code> . Unbounded vector over allocator A; layout flows in as static <code>cache_line_size</code> and via optional <code>organ_scan_field_sum()</code> . Really measurable (F15 bridge).	Code/Baseline
NodeChunkedStore	3-axis storage organ with runtime impact of node_type axis: Slots lie in chunks of capacity <code>N::max_capacity()</code> . Chunk count = number of node-sized allocations; thus the node_type axis is REALLY measurable (Node4 many chunks, Node256 few). Proof of delegation.	Code/Baseline
LayoutAware ChunkedStore	Layout-honoring variant of NodeChunkedStore: stores records at layout-driven stride (<code>eff_stride</code>). Each record occupies <code>eff_stride</code> bytes (key 0..8, value 8..16, rest padding). Makes memory_layout axis really measurable, OOB-safe for scan methods.	Code/Baseline

Variant	Description	Source
ObservableNode Type	ObservableAxis wrapper around a node_type strategy. Passes through static API (max_capacity/name/topic_tag) + offers observe_node_find() as instance driver. Tracks find_count, keys_stored, queries_run, last_checksum. Format-divergent lookup latency measurable (out-of-process path).	Code/Baseline

Contributing sources: P01 [LKN13], P03 [MKM12], P06 [SSL⁺21], P10 [ZLL⁺18].

D.6 T5 Memory Layout

This axis is the *memory-layout organ* (T5) of the living being SearchAlgorithm (cf. Section 3.2). This axis classifies the spatial organization of records in memory, directly impacting cache efficiency, SIMD vectorization potential, and false-sharing characteristics. It captures four orthogonal design dimensions: (1) alignment strategies (cache-line padding vs. strict packing), (2) data organization (row-oriented AoS vs. column-oriented SoA vs. hybrid AoSoA), (3) packing density (dense vs. succinct bit-packed), (4) stride patterns (sequential vs. interleaved). Layout choice is a primary driver of cache locality and SIMD capability during index traversal.

Static sub-axes

ID	Name	Description
HM1	Alignment Strategy	Choice between cache-line-padded layout (false-sharing avoidance) and strict packing (memory density). Determines stride overhead during sequential traversal.
HM2	Data Organization	Primary memory ordering: Array-of-Structs (AoS, row-oriented), Struct-of-Arrays (SoA, column-oriented), or hybrid AoSoA (block-mixed). Critical for cache-line utilization during field-selective scans.
HM3	Packing Density	Storage density level: from standard records (8–64 bytes) to compact bit-packing (e.g., LOUDS bitmaps with ~1 bit per slot). Affects decode latency and L1/L2 utilization.
HM4	Stride Pattern	Addressing pattern during traversal: sequential strides (consecutive records), interleaved (e.g., to separate hot/cold data), or cache-aware layouts (e.g., Eytzinger BFS). Optimizes prefetch efficiency.

Dynamic sub-axes

ID	Parameter	Description
CacheLineSize	Cache-Line Size (Subaxis KF-5)	Runtime parameter for target platform cache-line size: 64 bytes (standard x86-64/ARM64), 128 bytes (Azure, some Power ISAs), or 32 bytes (embedded). Selected at compile-time via CacheLineConfig NTTP and affects alignment/padding overhead for all aligned layouts.

Building blocks

Variant	Description	Source
AoSStrictMemoryLayout	Array-of-Structs, tightly packed without cache-line alignment. Advantage: minimal memory footprint (record_size strides, dense). Disadvantage: false-sharing under concurrent writes, poor cache-hit ratio for narrow field scans. Typical for Wormhole index.	Code/Baseline
CacheLineAlignedMemoryLayout	AoS with 64-byte per-record alignment. Avoids false-sharing via padding (stride = round_up(record_size, 64)). Standard for ART/HOT/Masstree/START. Operationally: at record_size=48 (bench build), creates 16-byte overhead per record → measurably higher latency than aos_strict, genuine cache effect.	Code/Baseline
SoAMemoryLayout	Struct-of-Arrays: all fields stored column-wise sequentially. Advantage: contiguous field scans (e.g., keys-only without values) maximize cache-line utilization (~16x fewer cache-lines than AoS for single-field scan). Ideal for SIMD vectorization and OLAP indexes. Benefit described in Abadi et al. SIGMOD 2008.	Abadi et al. 2008 [AMH08]
PackedBitmapMemoryLayout	Bit-packed succinct layout (LOUDS / SuRF FST). Stores n entries in ~n*lg(sigma) bits instead of bytes. Extreme compactness (32x fewer cache-lines than AoS), but higher decode cost per lookup. Foundation: Jacobson LOUDS FOCS 1989 (P09). Modern impl: SuRF Apache-2.0.	P09 (Jacobson LOUDS FOCS 1989)
AoSoAMemoryLayout	Array-of-Structures-of-Arrays: hybrid layout with block width W=8 (AVX2 lanes). Within block: SoA (fields contiguous). Across blocks: AoS (blocks strided). Unites SIMD vectorization (SoA within) with spatial locality (AoS across). Standard in HPC/SIMD engines. Convention; no single originating paper, but ISPC idiom.	Code/Baseline (HPC/SIMD convention)

Contributing sources: P09, Abadi et al. 2008 [Jac89, AMH08].

Related literature: P06, P11, P12, P32 [SSL⁺21, RR99, RR00, SKK⁺25].

D.7 T6 Allocator Axis (T6)

This axis is the *allocation organ* (T6) of the living being `SearchAlgorithm` (cf. Section 3.2).

This axis provides memory management strategy for the cache engine, defining 25 allocator wrappers covering a spectrum from classical baselines (Buddy, Slab) through modern research allocators (mimalloc, jemalloc, snmalloc) to specialized variants for NUMA, PIM, and formally verified systems. All wrappers satisfy standardized concepts and are orthogonally structured by 7 compile-time variant families (freelist topology, size-class schema, thread locality, synchronization, allocation policy, reclamation, fragmentation strategy).

Static sub-axes

ID	Name	Description
AA1	Free-List Topology	How are free memory blocks organized? Examples: per-page sharding (mimalloc A04), per-thread caching (snmalloc A07), buddy trees (A19), single-global free-list (dlmalloc A20), spans (scallop A08), hybrid strategies (hmalloc A15).

ID	Name	Description
AA2	Size-Class Schema	How are block sizes classified? Examples: slab per object size (A02), dlmalloc bins (A20), jemalloc size-classes (A05), scallo spans (A08), buddy power-of-2 (A19), formally verified bins (StarMalloc A13), std::pmr pools (A22).
AA3	Thread Locality	How thread/core-local is allocation? Examples: tcmalloc thread-local caches (A06), rpmalloc thread-heaps (A10), snmalloc message-passing (A07), tcmalloc-warehouse hyperscale (A14), Hoard per-heap locks (A01).
AA4	Synchronization	What concurrency mechanism? Examples: lock-free CAS (Michael A03, LRMalloc A11), per-heap locks (Hoard A01), single-threaded (Exgen A18), wait-free (Crystalline A17).
AA5	Allocation Policy	How are allocation requests routed? Examples: NUMA-origin awareness (NUMAlloc A09), cache-set awareness (CAMA A12), NUCA awareness (TCMalloc-Warehouse A14), PIM awareness (PIM-malloc A16), polymorphic memory resource (std::pmr A22).
AA6	Reclamation	How are deleted blocks reclaimed? Examples: magazine cache (Slab A02), page-heap release (TCMalloc A06), lock-free reclamation (Crystalline A17), deferred-free (mimalloc A04), Vmem magazine extension (A23).
AA7	Fragmentation Strategy	How is fragmentation prevented? Examples: dlmalloc coalescing (A20), object coloring (Slab A02), buddy splitting (A19), page-local sharding (mimalloc A04), span pooling (scallop A08).

Building blocks

Variant	Description	Source
HoardAllocator (A01)	Per-heap free-lists with SMP scalability. Berger et al. (PPoP 2000). Synchronization via per-heap locks and thread-local heap caches.	P— (OSS, Apache-2.0)
SlabAllocator (A02)	Object-caching kernel allocator with magazine-cache strategy. Bonwick (USENIX 1994, 2001). Classic textbook paradigm without standalone OSS implementation.	P— (Baseline/Textbook)
MichaelLockFree Allocator (A03)	Lock-free CAS allocator with hazard pointers. Maged Michael (PLDI 2004). Re-implementation with IBM patent background.	P— (LGPL Re-Impl)
MimallocAllocator (A04)	Free-list sharding with deferred-free and page-local management. Leijen/Zorn/de Moura (MSR-TR-2019-18, APLAS 2019). Original code binding (is_original=2/2).	P— (OSS, MIT, is_original)
JemallocAllocator (A05)	Arena-based size-classes with per-CPU arenas and tcache layer. Evans (BSDCan 2006). Standard allocator of FreeBSD and Facebook. Original code binding (is_original=2/2).	P— (OSS, BSD-2-Clause, is_original)
TCMallocAllocator (A06)	Thread-caching malloc with central page-heap. Ghemawat/Menage (Google ca. 2005) + TEMERAIRE (OSDI 2021). Google production standard.	P— (OSS, Apache-2.0)
SnmallocAllocator (A07)	Message-passing allocator with asymmetric heap operations. Paul Liétar et al. (ISMM 2019). Original code binding (is_original=2/2).	P— (OSS, MIT, is_original)
ScallopAllocator (A08)	Span-based free-list with virtual spans for multi-core scalability. Aigner/Kirsch/Lippautz/Sokolova (OOPSLA 2015). Low fragmentation via span pooling.	P— (OSS, BSD-3-Clause)
NUMAllocAllocator (A09)	NUMA-origin-aware allocator for faster allocation in multi-socket systems. UTSASRG (ISMM 2023). Optimized for multi-socket architectures.	P— (OSS, vermutlich Apache-2.0)
RPMallocAllocator (A10)	Per-thread span caching without dependencies on other allocator libraries. Mattias Jansson (2017). Original code binding (is_original=2/2).	P— (OSS, Public-Domain/MIT, is_original)

Variant	Description	Source
LRMallocAllocator (A11)	Lock-free allocator with hazard pointers for multi-core scalability. Leite/Rocha (VECPAR 2018, LNCS 11333). Original code binding (is_original=2/2).	P— (OSS, MIT, is_original)
CAMAAAllocator (A12)	Cache-Aware Memory Allocator with predictable behavior. Herter/Backes/Haupenthal/Reineke (ECRTS 2011, Saarland). No standalone OSS implementation available.	P— (Baseline/Research)
StarMallocAllocator (A13)	Formally verified hardened memory allocator with F*/Steel. Inria-Prosecco: Reitz/Fromherz/Protzenko et al. (OOPSLA 2024). Mathematically verified allocator.	P— (OSS, Apache-2.0)
TCMallocWarehouse Allocator (A14)	TCMalloc hyperscale extension with hugepage support for warehouse-scale computing. TEMERAIRE (Hunter OSDI 2021) + warehouse characterization (Zhou ASPLOS 2024). NUCA-aware TCMalloc variant.	P— (OSS, Apache-2.0)
HMallocAllocator (A15)	Hybrid lock-free allocator with scalable memory management. Li/Yao/Tang/Lin (IEEE ICPADS 2019). No standalone OSS implementation available.	P— (Research Baseline)
PIMMallocAllocator (A16)	Processing-in-memory allocator for heterogeneous hardware with fast local memory. VIA-Research (HPCA 2026, arXiv:2505.13002). Specialized for PIM hardware.	P— (OSS, MIT)
CrystallineReclamation (A17)	Wait-free safe-memory-reclamation family (Nikolaev/Ravindran DISC 2021 / PLDI 2024). Reclamation policy, not a standalone allocator.	P— (Research)
ExgenAllocator (A18)	Single-threaded specialized allocator with optimizations for single-threaded workloads. UT Austin (IEEE CAL 2025, arXiv:2510.10219). “Old is Gold” principle.	P— (Research, Konfidenz medium)
BuddyAllocator (A19)	Buddy system with power-of-2 splitting for simple fragmentation control. Knuth (TAoCP Vol.1, 1968). Classic textbook paradigm.	P— (Baseline/Textbook)
DlmallocAllocator (A20)	Doug Lea malloc with bins-based classification. Lea (1987–2012). Original code binding (is_original=2/2).	P— (OSS, Public-Domain/CC0, is_original)
PtMalloc2Allocator (A21)	ptmalloc2 (glibc malloc) with explicit wrapper control. Wolfram Gloger / Ulrich Drepper (1990s–2000s). License-blocked (LGPL-2.1+).	P— (OSS, LGPL-2.1+, blockiert Linking)
StdMalloc (A22a)	Standard libc malloc (proof-of-concept) — portable fallback allocator. Baseline for all standard library implementations.	P— (Standard-Library)
PmrResourceAllocator (A22b)	std::pmr::memory_resource (polymorphic memory resource). Halpern (N3916, C++17). Abstract interface without behaviorally distinct default implementation.	P— (Standard-Library, C++17)
PoolResourceAllocator (A22c)	std::pmr::unsynchronized_pool_resource with own size-class pool. Halpern (N3916, C++17). Behaviorally distinct without vendor linking (F15-operational).	P— (Standard-Library, C++17, F15-operativ)
VmemMagazinesAllocator (A23)	Vmem plus magazines as extension of slab allocator for multi-core scaling. Bonwick (USENIX ATC 2001). No standalone OSS implementation available.	P— (Baseline/Textbook)

Contributing reclamation sources: P29, P30 [McK01, Mic04a]; full allocator corpus A01–A23 see section “A-Corpus”.

D.8 T7 Prefetch Strategy

This axis is the *prefetch organ* (T7) of the living being `SearchAlgorithm` (cf. Section 3.2).

The Prefetch axis determines the strategy for anticipatory loading of nodes and addresses along the expected search path in a trie data structure. It supports four concrete strategies with different trigger

mechanisms and heuristics: no prefetch (measurement baseline), distance-estimated software prefetch based on node density and cache latency (ART standard), explicit hardware prefetch instructions (Wormhole standard), and path-oriented prefetch with bundle granularity (PRT-ART thesis).

Static sub-axes

ID	Name	Description
PF1	Trigger Mechanism	Type of control over the prefetch timing: whether no mechanism (baseline), through compiler hints via distance estimator, explicit hardware prefetch instructions, or path-oriented prediction
PF2	Distance Heuristic	Calculation method for prefetch distance (how far ahead to load): linear, adaptive based on node density and tier latency, or no heuristic
PF3	Granularity	Prefetch size per trigger: cache-line (64 bytes, individual), page (4K, OS window), or bundle (multiple cache-lines, trie-path-oriented)

Building blocks

Variant	Description	Source
NonePrefetch	No prefetch strategy (baseline). Serves as comparison reference for measurement series. Trigger: none; heuristic: none; granularity: N/A. <code>is_active()</code> returns false.	Code/Baseline
DistanceEstimator Prefetch	Distance-estimated software prefetch (ART standard per Leis ICDE 2013). Estimates cache distance to next node based on node density [%] and tier latency [cycles] with heuristic: $\text{estimate} = \text{clamp}(1 + \text{sparseness} * 8 + \text{latency_factor}, [1, 16])$. Trigger: compiler hints via <code>__builtin_prefetch</code> ; heuristic: adaptive (density + latency); granularity: cache-line. Applies ART pattern (unodb/libart).	P01 (ART/unodb, Leis ICDE 2013)
HardwarePrefetch	Explicit hardware prefetch instructions (Wormhole standard per Wu EuroSys 2019). Uses <code>PREFETCHT0/T1/T2/NTA</code> on hash-anchor-chain without software heuristic; CPU estimates distance autonomously. Trigger: explicit hardware instruction; heuristic: hardware-managed; granularity: hardware-determined. License GPL-3.0 blocks <code>is_original-linking</code> .	P07 (Wormhole, Wu EuroSys 2019)
PathOriented Prefetch	Path-oriented prefetch heuristic (PRT-ART thesis 2026). Pre-loads all nodes along predicted trie path before search starts. Tracker follows search path and recommends next address via linear step extrapolation. V11.1 Hot-Path-Hints: interprets first 8 bytes of key as uint64 address. Trigger: path-oriented prediction; heuristic: linear (step extrapolation); granularity: bundle (multiple cache-lines). Related literature: Chen/Gibbons/Mowry SIGMOD 2001 (pB+-Trees Prefetching) + Mowry 1994.	P21 (related: Chen/Gibbons/Mowry SIGMOD 2001); thesis 2026

Contributing sources: P01, P07, P21 [LKN13, WNJ19, CGM01].

Related literature: P23, P25, P26, P27 [Kha11, MWR25, ZSZ⁺24, ZGH⁺25].

D.9 T8 Concurrency Synchronization Axis (Concurrency Axis T8)

This axis is the *concurrency organ* (T8) of the living being `SearchAlgorithm` (cf. Section 3.2).

This axis controls synchronization patterns and safe memory reclamation for concurrent access to tree/index structures. The axis provides compile-time selectable synchronization patterns (None/Blocking/Reader-Writer/Optimistic/Lock-Free/Wait-Free) as well as orthogonal reclamation schemes (RCU/Hazard-Pointer) for safe memory deallocation in lock-free structures.

Static sub-axes

ID	Name	Description
CC1	Synchronization Pattern (CC1)	Compile-time variant family of synchronization patterns: None (single-threaded baseline), Blocking (pessimistic mutex), ReaderWriter (shared_mutex), Optimistic (lock-free version validation), LockFree (CAS-loops), WaitFree (bounded steps). Each family has distinct runtime characteristics (0-overhead to mutex-overhead).
CC2	Reclamation Scheme (CC2)	Orthogonal compile-time variant family for safe memory deallocation in lock-free structures: RCU (deferred grace-period via userspace-rcu), Hazard-Pointer (per-thread in-use marks). Composable with lock-free patterns, optional for Blocking/Optimistic.

Building blocks

Variant	Description	Source
NoneConcurrency	No synchronization, single-threaded baseline. Both acquire() and release() are no-ops (volatile dummy). Reference null-point for concurrency-overhead measurement (F15).	Code/Baseline
Blocking Concurrency	Pessimistic mutex locking with std::mutex. Global coarse-grained lock. Serializes all accesses. Upper-bound for locking overhead. Paper: Dijkstra CACM 1965.	Code/Baseline
ReaderWriter Concurrency	Single-writer/multi-reader via std::shared_mutex. Lock-sharing for read-heavy workloads. Middleweight between pessimistic and optimistic patterns. Paper: Courtois et al. CACM 1971.	Code/Baseline
OlcOptimistic Concurrency	Optimistic Lock-Coupling (ART-Sync): version-counter per node, lock-free readers with retry-on-conflict. Cheap under low contention. Paper: Leis DaMoN 2016 / DEBULL 2019 (P08, is_original=true, Apache-2.0, unodb).	P08
LockFree Concurrency	Lock-free via Compare-And-Swap (std::atomic CAS-loops). System-wide progress guarantee. Generic CAS-pattern without blocking. Paper: Michael/Scott PODC 1996 (anchor paper).	Code/Baseline
WaitFree Concurrency	Wait-free: each thread completes in bounded steps (strongest guarantee). Without retry-loop like LockFree. Realized via sequence-lock / fetch-add. Paper: Herlihy TOPLAS 1991 (theory anchor).	Code/Baseline
RcuConcurrency	Read-Copy-Update (McKenney OLS 2001): lock-free readers via per-thread nesting counter (relaxed-order, no memory-barrier on read-side). Reclamation-scheme orthogonal to pattern. Local re-impl (P29 userspace-rcu LGPL-2.1, is_original=false).	P29
HazardPointer Concurrency	Hazard Pointers (Michael TPDS 2004, P30): per-thread marked in-use pointers prevent premature deallocation (ABA-safe). Reclamation-scheme. Local re-impl (P30 NO LICENSE, is_original=false). SEQ_CST-store on acquire().	P30
OlcReservedBlocks Concurrency	OLC + cache-line-reserved value blocks (multi-writer cache-coherence-storm avoidance). Specialization of Optimistic pattern with additional atomic block-ID reservation (fetch_add). F.6 Doku 19 (prt-art optional, not is_original).	Code/Baseline

Contributing sources: Dijkstra 1965, Courtois et al. 1971, P08, Michael/Scott 1996, Herlihy 1991, P29, P30 [Dij65, CHP71, LABN16, MS96, Her91, McK01, Mic04a].

D.10 T9 Serialization (Encoding and Compression)

This axis is the *serialization organ* (T9) of the living being `SearchAlgorithm` (cf. Section 3.2).

Axis 10 determines how data is encoded and compressed within the Cache-Engine system: from raw-binary storage layouts to variable-length encoding and succinct bit-packing. The serialization strategy choice directly impacts CPU cost (delta-encoding, bit-manipulation, LEB128 varint), memory overhead, and IO-bandwidth utilization — critical for LSM-trees and read-heavy workloads with approximate filters (SuRF/LOUDS).

Static sub-axes

ID	Name	Description
SR1	Byte-Order and Word Format	How are bytes arranged within a record? Raw (memcpy raw-layout), Varlen (LEB128-varint with signaling bits), or Packed (bit-by-bit packing)?
SR2	Density (Memory Utilization)	Memory overhead per record: Sparse (complete records), Dense (with padding), or Succinct (information-theoretically dense, $n \cdot H + o(n)$ bits)?
SR3	Compression and Preprocessing	Block compression (LZ4/Snappy, LZ77-based) or delta-encoding (zigzag-transform for negative deltas)? Or no compression?

Building blocks

Variant	Description	Source
RawBinary Serialization	Raw byte storage layout (memcpy). Baseline without transformation. Minimal CPU cost. Typical for in-memory workloads without compression. Variant: <code>serialize_scan()</code> reads 32-bit fields + order-sensitive checksum.	Code/Baseline
VarLen Serialization	Variable-length encoding with signaling bits (LEB128-VarInt). Standard for ART (Leis ICDE 2013 P01). Small values = few bytes. Signaling bits mark byte-length. Data-dependent loop (1-5 bytes). FNV-1a-mix of varint bytes.	P01
Succinct Serialization	Bit-by-bit packing with minimal bit-width (LOUDS/SuRF). Achieves information-theoretic minimum $n \cdot H + o(n)$ bits. Read-only after build. Optimal for approx-filters + read-heavy. Highest per-element CPU cost (real bit-manipulation). Variant: <code>serialize_scan()</code> with 64-bit accumulator.	P10,P09
Compressed Serialization	Block compression (LZ4/Snappy) layered on raw-binary. Trade-off: CPU cost vs IO/memory bandwidth. Variant: delta-encoding against predecessor + zigzag-transform (negative delta $\rightarrow$ small unsigned). Typical for LSM-trees. Medium CPU cost.	Code/LZ77 (Ziv/Lempel 1977) [ZL77]

Contributing sources: P01, P09, P10, Ziv/Lempel 1977 [LKN13, Jac89, ZLL⁺18, ZL77].

D.11 T10 Value Handle Storage Strategy

This axis is the *value-handle organ* (T10) of the living being `SearchAlgorithm` (cf. Section 3.2).

The Value-Handle axis defines how data values (Values) are stored and accessed in the cache index: inline within node slots, externally in a pool, via immutable-shared references (RCU-style), versioned pointers

(MVCC), or chained external references (multi-value). This axis characterizes the storage-layout strategy for values and determines access latency (direct vs. indirect) and concurrency semantics (snapshot vs. versioning).

Static sub-axes

ID	Name	Description
VH1	Storage Location	Compile-time variants for physical value placement: inline in node slot (combines key/value without indirection) or external in separate pool (node holds offset/pointer only).
VH2	Ownership Semantics	Compile-time variants for ownership and sharing strategy: owned (exclusive per key), shared (immutable reference-counted via RCU-style), weak (borrowing without increment).
VH3	Versioning Strategy	Compile-time variants for concurrency control: no versioning (simple ownership), version-tags in pointer (MVCC for reader snapshot isolation).

Building blocks

Variant	Description	Source
<code>InlineValueHandle</code>	Value embedded directly in node slot (no indirection). Standard for small values like <code>uint64_t</code> . Minimal access latency (1 direct read), but node memory consumed by larger values. Source: ART (Leis ICDE 2013, P01).	P01
<code>ExternalPoolValueHandle</code>	Value stored externally in pool; node holds offset only. Enables variable value sizes and compact nodes, costs 1 additional cache miss per lookup (pointer chasing). Source: Wormhole (Wu EuroSys 2019, P07).	P07
<code>ImmutableSharedRefValueHandle</code>	Value via reference-counted pointer (RCU-style snapshot sharing). Optimal for reader concurrency: readers see stable snapshots, writers create new immutable snapshots. Requires <code>RefCount</code> acquire/release per access. Source: RCU (McKenney OLS 2001, P29).	P29
<code>VersionedPointerValueHandle</code>	Pointer with version tag in upper byte (MVCC). Enables multi-version concurrency control: readers get snapshot version for visibility control, writers create new version. Tombstone detection via version bit. Source: Masstree (Mao EuroSys 2012, P03) and SMART (OSDI 2023).	P03
<code>ChainRefValueHandle</code>	Multi-value chained external reference (linked-list). Node holds offset to chain head in pool; each chain node holds (value_offset, next_offset). Most expensive variant with 2 dependent dereferences per value access. Source: prt-art (local reimplementaion, F.6 migration).	Code/Baseline

Contributing sources: P01, P03, P07, P29, SMART (OSDI 2023) [LKN13, MKM12, WNJ19, McK01, LZS⁺23].

D.12 T11 Index Organization

This axis is the *index-organization organ* (T11) of the living being `SearchAlgorithm` (cf. Section 3.2). Classification of data storage organization and index structuring across three orthogonal dimensions: storage-order (heap/clustered/unordered), index-count (none/single/multiple), and data-embedding (separate vs. leaf-embedded). This axis models fundamental DBMS architectural decisions that decisively influence cache behavior, access patterns, and I/O characteristics based on alignment of index-order with storage-order.

Static sub-axes

ID	Name	Description
io1_ storage_ order	Storage Order	Arrangement of tuples in storage: unordered (insertion-order in heap), sorted by primary key (clustered), or via pointer indirection (non-clustered). Determines sequential vs. random access patterns during range scans.
io2_index_ count	Index Multiplicity	Number of supported indexes per table: none (heap baseline), single primary index (clustered), or multiple secondary indexes (non-clustered). Influences OLTP workload support.
io3_data_ embedding	Data Embedding	Storage placement of data relative to index: separate row storage (Clustered/NonClustered with pointer-hop), or directly embedded in index leaf-pages (IOT/trie-style like ART/HOT).

Building blocks

Variant	Description	Source
ClusteredIndex Organization	Storage-order aligns with index-order (1 primary key). Optimized for range-scans along primary key with sequential disk reads (cache-friendly). Standard: MySQL InnoDB, PostgreSQL CLUSTER, SQL Server CLUSTERED INDEX. Access pattern: sequential forward iteration without predicate evaluation, $O(n)$ aggregation scan.	Bayer & McCreight, Acta Informatica 1972 (10.1007/BF00288683)
HeapIndex Organization	Storage-order = insertion-order, NO index (baseline). Lookup via $O(n)$ full-scan with per-record predicate evaluation (data-dependent branch, no early exit). Standard: PostgreSQL heap tables, HBase. Access pattern: unordered sequential scan with comparison overhead and branch misprediction.	Garcia-Molina, Ullman & Widom, Database Systems (Textbook Baseline)
IotIndexOrganization	Data directly embedded in index leaf-pages (Oracle IOT, SQL Server CLUSTERED INDEX style). Saves pointer indirection vs. non-clustered. Equivalent to B+tree leaves with embedded data or trie nodes (ART/HOT/Wormhole). Access pattern: sequential like clustered, but with overhead from fat leaf layout (two fields per record).	Srinivasan et al., VLDB 2000 (Oracle8i IOT) [SDF ⁺ 00]
NonClusteredIndex Organization	Index-order $\neq$ storage-order, multiple secondary indexes per table. Lookup via index + pointer-hop to row data (random-stride, cache-misses). Standard: SQL Server NONCLUSTERED INDEX, PostgreSQL CREATE INDEX. Access pattern: index navigation followed by random storage hops (LCG-deterministic, REAL cache-miss latency from TLB/L2 invalidation), then a residual-predicate comparison per fetched record (data-dependent branch on the fetched row data after the pointer-hop).	Comer, ACM Computing Surveys 1979 (The Ubiquitous B-Tree, 10.1145/356770.356776)

Contributing sources: Bayer & McCreight 1972, Comer 1979, Srinivasan et al. 2000, Garcia-Molina, Ullman & Widom (Textbook) [BM72, Com79, SDF⁺00, GMUW09].

D.13 T12 I/O Dispatch Axis

This axis is the *I/O-dispatch organ* (T12) of the living being *SearchAlgorithm* (cf. Section 3.2).

Modeling and selection of I/O strategies for persistence and memory access in index operations: in-memory storage only (baseline), buffered via OS page cache (standard), direct I/O with `O_DIRECT` flag (NVMe-optimized), or memory-mapped file I/O (persistent memory). This axis decouples search and tree structures from persistence implementation and enables A/B measurements of dispatch overhead profiles under identical workloads.

Static sub-axes

ID	Name	Description
IO1	Persistence Model	Classifies storage type: RAM-only (no persistence, pure measurement baseline), or disk/persistent-memory (file-backed indices with various caching strategies).
IO2	Caching Strategy	Characterizes OS buffer usage: Buffered (OS page cache, read-ahead + write-back, standard), Direct I/O (<code>O_DIRECT</code> , page cache bypass, 512B/4 KB alignment), or Memory-Mapped (file-backed <code>mmap</code> with automatic OS paging and page-fault-driven access).
IO3	Write Durability	Defines consistency guarantees: no <code>fsync</code> (highest performance, no durability), <code>fsync</code> after batch (compromise, batch commits), or atomic writes (highest safety, lowest performance). In this implementation primarily present as a conceptual sub-axis; operational realization occurs through the caching strategy.

Building blocks

Variant	Description	Source
InMemoryOnly	Pure RAM-based index without persistence. Direct sequential record access without dispatch overhead. Fastest path and measurement baseline of the axis. Used for performance reference and measurement under zero-I/O-latency conditions.	Code/Baseline
BufferedIo	Standard <code>read()/write()</code> via OS page cache with automatic read-ahead and write-back. Default for general-purpose DBMS. Trade-off: convenient but double-buffering (app cache + OS cache results in higher memory footprint). Paper anchor: Stonebraker CACM 1981 (position paper Operating System Support for Database Management, DOI 10.1145/358699.358703). Confidence: medium (engineering pattern, no algorithm code).	P-Stonebraker-1981
DirectIo	Bypasses OS page cache via <code>O_DIRECT</code> flag (<code>open(2)</code>), NVMe-SSD-optimal. Requires 512B/4KB alignment. Enables DBMS-owned cache strategies (RocksDB, MySQL). Higher CPU cost but no cache pollution from background activity. No dedicated paper source (Linux API since 2.4.10); treated as intentional engineering baseline.	Code/Baseline
MmapIo	Memory-mapped file I/O (<code>mmap</code>) with persistent memory or read-heavy workloads. OS paging automatic, no copy between user and kernel space. Trade-off: page-fault latency spikes vs. reuse-friendliness. Paper anchor: Crotty et al. CIDR 2022 (anti-pattern baseline Are You Sure You Want to Use MMAP in Your DBMS?, db.cs.cmu.edu/.../cidr2022-p13-crotty.pdf). Deliberately cited as anti-pattern reference to document limits and overhead. OSS reference MIT-licensed.	P-Crotty-2022

Contributing sources: Stonebraker-1981, Crotty-2022 [Sto81, CLP22]

D.14 T13 Migration Strategy Axis

This axis is the *migration organ* (T13) of the living being `SearchAlgorithm` (cf. Section 3.2).

The Migration Strategy axis (T13, `axis_migration`) encapsulates decision logic for data placement across multi-tier storage hierarchies (RAM/SSD/HDD, Hot/Cold tiers). It models when and how frequently accessed data should be promoted to faster storage or demoted when cold, measuring decision logic only without actual tier migration. Supports four concrete strategies: static (baseline), Hot/Cold separation, multi-tier hierarchy, ML-driven adaptive.

Static sub-axes

ID	Name	Description
MG1	Trigger Mechanism (MG1)	Determines when migration decision is triggered: none (static), threshold-based, or observation-driven. Compile-time variant per strategy.
MG2	Migration Direction (MG2)	Determines direction of data movement: none (static), up-migration (RAM→SSD), down-migration (SSD→RAM), or bidirectional. Compile-time variant.
MG3	Migration Granularity (MG3)	Specifies the unit granularity for migration decisions: none (static), page (4 KB), block (64KB), object (single item). Compile-time category.

Dynamic sub-axes

ID	Parameter	Description
D1	Decision Threshold	Runtime parameter: Hot/Cold classification boundary (e.g., recency counter limit for LRU-based promotions/demotions). User-configurable.
D2	Feature Scoring Weights	Runtime parameters for AdaptiveMigration: weights (<code>w_freq</code> , <code>w_rec</code>) for frequency and recency features in ML linear combination. Self-tuning possible.
D3	Tier Configuration	Runtime: number of storage tiers (e.g., <code>kTiers=3</code> for RAM/SSD/HDD in Tier BasedMigration), capacities per tier, promotion/demotion thresholds.

Building blocks

Variant	Description	Source
<code>NoMigration</code>	Static placement without migration (baseline). Data positioned once, no movement between tiers. Benchmark zero-point for comparison with active strategies.	Code/Baseline
<code>HotColdMigration</code>	Hot/Cold separation via LRU + probabilistic recency counting. Frequently accessed data (hot) promoted to faster cache, rarely accessed (cold) demoted to slow storage. Standard for LRU-K and LFU cache systems.	Anti-Caching (PVLDB 2013) [DPT ⁺ 13]
<code>TierBased Migration</code>	Multi-tier migration RAM→NVM→SSD→HDD with modulo-based tier mapping. Each data item mapped to target tier via modulo field value. Standard for RocksDB, LSM-trees, and tiered cache systems with configurable thresholds.	Managing NVM in DBMSs (SIGMOD 2018) [vRLK ⁺ 18]

Variant	Description	Source
AdaptiveMigration	ML-driven adaptive migration with online learning (Cachelib/LeCaR-style). Weighted linear combination of frequency and recency features computes adaptive score; running learning accumulator models online learning. Higher overhead versus optimal hit-rate.	ML-based LeCaR (USENIX HotStorage 2018) [VRM ⁺ 18]

Contributing sources: Anti-Caching (PVLDB 2013), LeCaR (HotStorage 2018), Managing NVM (SIGMOD 2018) [DPT⁺13, VRM⁺18, vRLK⁺18].

D.15 T14 Filter Axis

This axis is the *filter organ* (T14) of the living being `SearchAlgorithm` (cf. Section 3.2).

The Filter Axis T14 provides selectable probabilistic and deterministic membership-query filters for point and range queries. It covers classic Bloom filters with k hash functions, the deletable Cuckoo Filter with bucket hashing and fingerprints, the succinct Range Filter SuRF using LOUDS-Trie navigation, and the space-efficient Xor Filter with constant 3-way-XOR latency. Purpose: Make space, query latency, mutability, and false-positive-rate trade-offs directly measurable in cache-engine experiments.

Static sub-axes

ID	Name	Description
FT1	Query Type	Distinction between point queries (membership: key present?) and range queries (range: keys in $[a,b]$?). Point filters are Bloom/Cuckoo/Xor; only RangeSurfFilter supports range semantics.
FT2	Mutability	Division into immutable filters (Bloom, RangeSurf, Xor immutable after build) vs. mutable (Cuckoo with delete operation). Determines wrapper design and caching possibilities during runtime.
FT3	Error Profile	Characterization of error-tolerant semantics: false-positive-only (Bloom, Cuckoo, Xor — filter answers “yes” incorrectly) vs. lossless (deterministic answer). Bloom: approx. $k \cdot n$ hash tests; Xor: exactly 3 XOR ops (branch-free).

Building blocks

Variant	Description	Source
BloomFilter	Classical Bloom Filter per Bloom 1970: m -bit bitmap, k independent hash functions (double-hashing per Kirsch/Mitzenmacher 2006). Point queries only; false positives possible, no deletes. Implementation with a real persistent m -bit bitmap (8 KiB, $m=65536$, $k=4$ double-hashing), probed in the out-of-process measurement path over the actually inserted keys; the honestly declared pseudo-bitmap probe ($k=4$ hash tests per query, characteristic early-exit latency on mismatch) remains only in the in-process segment-timing scan.	Bloom 1970 (CACM 13:422-426)

Variant	Description	Source
CuckooFilter	Cuckoo Filter per Fan et al. CoNEXT 2014: Bucketed cuckoo hashing with 1-byte fingerprints. Supports deletes and lookup counting (advantage over Bloom). Probe characteristic: 2 bucket lookups + fingerprint comparison (partial-key cuckoo per CoNEXT 2014 §3: $i2 = i1 \text{ XOR hash}(fp)$). Higher insert costs, but flexible mutability.	Fan et al. CoNEXT 2014 (DOI:10.1145/2674005.2674994)
RangeSurfFilter	SuRF (Succinct Range Filter) per Zhang et al. SIGMOD 2018: LOUDS-bitmap trie for range queries. Only filter with range semantics (lower_bound/upper_bound at byte-prefix level). Probe characteristic: multi-level trie descent with succinct-LOUDS navigation (up to kMaxDepth=6), not exactly 1 hash test, hence latency variable per prefix length. Practical in RocksDB SST optimizations.	P10 (Zhang et al. SIGMOD 2018)
XorFilter	Xor Filter per Graf+Lemire 2020 (JEA 25): XOR-based filter with approx. 9 bits/key (vs. 10 bits/key Bloom at same FP rate). Immutable after build. Probe characteristic: exactly 3 independent table accesses (3 disjoint table thirds h0, h1, h2) + 3-way XOR, branch-free, constant latency without early-exit. Safe against path optimization, smallest memory footprint of all filters.	Graf+Lemire 2020 (JEA 25:Article 1.5, DOI:10.1145/3376122)

Contributing sources: Bloom 1970, Fan et al. CoNEXT 2014, Zhang et al. SIGMOD 2018, Graf+Lemire JEA 2020 [Blo70, FAKM14, ZLL⁺18, GL20].

D.16 T15 Buffering Q1 (Operations Buffer)

This axis is the *buffering organ* (*Q1*) (T15) of the living being `SearchAlgorithm` (cf. Section 3.2). The `axis_q1_queueing` realizes various buffering strategies for operations in the cache-engine context. It encompasses textbook ADTs (FIFO, LIFO, Append-Only, Priority-Heap), specialized versioning techniques (Delta-Chain, Copy-on-Write, Epoch, Tombstone), and modern lock-free queues (SPSC/MPMC following Lamport and Vyukov). Strategies are selected at compile time via the axis registry (the `mp_list` of research wave 2, filtered by enable flags) per permutation binary (no runtime switch) and are classifiable by access pattern (sequential, ordered, cyclic, versioned, batched, lock-free).

Static sub-axes

ID	Name	Description
QS1	Sequential Access	FIFO/LIFO/Append-Only: Access at head or tail, typical for write-coalescing and memtable buffering
QS2	Ordered Access	Skip-List/Priority-Heap: Sorted drain, typical for LSM compaction and hot-key eviction
QS3	Cyclic Access	Bounded Ring-Buffer: Fixed capacity, Disruptor pattern, wrap-around with overflow handling
QS4	Versioned Access	Delta-Chain/Tombstone/CoW/Epoch: MVCC, snapshot isolation, reclamation windows
QS5	Batched Operations	Batched-Insert-Buffer: Accumulation into sub-batches, SIMD vectorization, cache-locality optimization
QS6	Lock-Free Concurrent	SPSC/MPMC: CAS-loop or wait-free Lamport model, true lock-freedom without mutex

Building blocks

Variant	Description	Source
NoBuffer	Passthrough/Identity (Baseline): <code>put()</code> is no-op, <code>get()</code> always returns <code>std::nullopt</code> . Useful as control point and in classic B+/ART scenarios with direct leaf access. Family Q01, QS1 <code>sequential_access</code> .	Code/Baseline
AppendOnlyBuffer	Append-Only Linear Buffer (LSM memtable, Bw-Tree delta): monotonically growing <code>std::vector</code> without middle deletion. Optimized for pure append + bulk drain. Family Q02, QS1 <code>sequential_access</code> . References: Levandoski/Lomet/Sengupta Bw-Tree ICDE 2013, O’Neil LSM 1996.	Code/Baseline
FIFOQueueBuffer	FIFO-Queue (<code>std::deque</code>): First-In-First-Out via <code>deque::push_back / deque::pop_front</code> . Unbounded, efficient middle growth. Textbook ADT, typical for write-coalescing. Family Q03, QS1 <code>sequential_access</code> .	Code/Baseline
LIFOStackBuffer	LIFO-Stack (<code>std::vector</code>): Last-In-First-Out via <code>vector::push_back / vector::pop_back</code> . Unbounded, optimized for hot-path reuse and version tombstones. Family Q04, QS1 <code>sequential_access</code> .	Code/Baseline
BoundedRingBuffer	Bounded Ring-Buffer (Disruptor Pattern): Fixed capacity, cyclic access with wrap-around. Overflow: <code>drop_oldest</code> (default). <code>iterable_aspect_t</code> : Capacity as runtime loop. Family Q05, QS3 <code>cyclic_access</code> . Reference: LMAX Disruptor 2011.	LMAX 2011
PriorityHeapBuffer	Priority-Heap (<code>std::priority_queue</code>): Max-heap, <code>get()</code> returns highest priority first. First strategy with <code>supports_priority_ordering()=true</code> . Unbounded, textbook ADT (Williams 1964). Application: LRU approximation, hot-key promotion, eviction queues. Family Q06, QS2 <code>ordered_access</code> .	Code/Baseline
DeltaChainBuffer	Delta-Chain Buffer (Bw-Tree, Levandoski 2013): First versioned strategy (<code>is_versioned()=true</code>). Each <code>put()</code> appends delta-record with ascending version-ID to top of chain. <code>get()</code> LIFO-drain with version-ID. Append-versioning paradigm. Family Q07, QS4 <code>versioned_access</code> .	Bw-Tree (Levandowski et al. ICDE 2013) [LLS13]
SkiplistBuffer	Skip-List Buffer (sorted buffer, RocksDB-style): <code>put()</code> inserts in sorted order via <code>std::set</code> . <code>get()</code> ASCENDING-drain (minimum first), typical for LSM-compact output. Distinction from <code>PriorityHeapBuffer</code> : <code>skiplist=ascending</code> , <code>heap=descending</code> . Reference: Pugh CACM 1990. Family Q08, QS2 <code>ordered_access</code> .	Pugh 1990, RocksDB
TombstoneBuffer	Tombstone-Marker Buffer (LSM + MVCC): <code>put()</code> creates live entry, deleted entries remain as tombstones (<code>std::nullopt</code>) until compaction. <code>get()</code> skips tombstones FIFO-like, <code>is_versioned()=true</code> (erase-versioning, unlike delta-chain append-versioning). <code>size()</code> counts live-slots only. Family Q09, QS4 <code>versioned_access</code> . Reference: O’Neil LSM-tree 1996.	O’Neil 1996
CopyOnWriteBuffer	Copy-on-Write Snapshot-Buffer (Persistent Data Structures): <code>put() / get()</code> increments snapshot version, earlier snapshots remain logically via <code>shared_ptr</code> . Snapshot-versioning paradigm (unlike delta-chain append, tombstone erase). <code>is_versioned()=true</code> . Family Q10, QS4 <code>versioned_access</code> . Reference: Driscoll/Sarnak/Sleator/Tarjan JCSS 1989.	Driscoll et al. 1989
EpochBuffer	Epoch-based Buffer / QSB (Quiescent-State-Based-Reclamation): Entries accumulated in current epoch. After <code>epoch_threshold</code> transition => prior epoch can be reclaimed. <code>iterable_aspect_t</code> : <code>epoch_threshold</code> as runtime parameter. <code>is_versioned()=true</code> (reclamation-window-versioning). Family Q11, QS4 <code>versioned_access</code> . References: McKenney RCU OLS 2001, Masstree EuroSys 2012, SMART OSDI 2023.	P29 (RCU OLS 2001), Masstree 2012

Variant	Description	Source
BatchedInsert Buffer	Batched-Insert Buffer (OLAP indexing, ART bulk-insert): <code>put()</code> buffers into sub-batches of <code>batch_size_</code> , complete batches passed on (bulk-drain). Goal: amortized insert cost via bulk operations, SIMD vectorization, cache-locality. First QS5 strategy. Family Q12, QS5 <code>batched_access</code> . Reference: Leis ART ICDE 2013 §6.	P01 (Leis ART ICDE 2013)
LockFree SPSCBuffer	Lock-Free SPSC Ring-Queue (Lamport 1977): Single-Producer/Single-Consumer. <code>put()</code> wait-free (1 atomic.store + 1 atomic.load), <code>get()</code> wait-free. No CAS needed due to disjoint modified-sets. <code>ProgressGuarantee::LockFree</code> (even wait-free). <code>iterable_aspect_t</code> : Capacity. Overflow: dropping. Contract: exactly 1 producer + 1 consumer. Family Q13a, QS6 <code>lock_free_access</code> .	Lamport 1977
LockFree MPMCBuffer	Lock-Free Bounded MPMC Queue (Vyukov 2010–11): Multi-Producer/Multi-Consumer via per-cell atomic sequence-number and CAS-loop. <code>Cell.sequence == pos</code> (producer writes), then <code>pos+1</code> (consumer reads). Retries on contention (wait-free per op NOT guaranteed). Capacity MUST be power-of-2. <code>iterable_aspect_t</code> . Family Q13b, QS6 <code>lock_free_access.is_original_eligible=true</code> (Vyukov BSD-2).	Vyukov 1024cores 2010–11, Michael/Scott PODC 1996
OriginalLockFree MpmcConcurrent Queue	<code>moodycamel::ConcurrentQueue</code> (Cameron Desrochers, block-based MPMC): Habich-compliance wrapper with original-code linking (2/6 methods original: <code>put</code> , <code>get</code> ; 4/6 gaps: <code>emplace</code> , <code>peek_front</code> , <code>peek_back</code> , <code>clear</code> are re-impl). <code>is_original_module()=false</code> due to gap-filling. Header-only, BSD-2/BSL-1.0 dual. Analog <code>LockFree MPMCBuffer</code> but with true submodule binding. Family Q15, QS6 <code>lock_free_access.is_original_eligible=true</code> (moodycamel BSD-2).	moodycamel 2014, BSD-2/BSL-1.0

Contributing sources: LMAX 2011, P01 (Leis ART ICDE 2013), Bw-Tree (Levandowski et al. ICDE 2013), P29 (RCU OLS 2001), Lamport 1977, Vyukov 1024cores 2010–11, Michael/Scott PODC 1996, Pugh CACM 1990, O’Neil LSM-tree 1996, Driscoll et al. 1989, Masstree EuroSys 2012, SMART OSDI 2023, Williams 1964, moodycamel 2014 [TFB⁺11, LKN13, LLS13, McK01, Lam77, Vyu11, MS96, Pug90, OCGO96, DSST89, MKM12, LZS⁺23, Wil64, Des14].

D.17 T16 Buffering Q2 (Flush Policy)

This axis is the *buffering organ* (Q2) (T16) of the living being `SearchAlgorithm` (cf. Section 3.2). Axis Q2 defines automatic flush strategies for write buffers. It determines WHEN a filled buffer propagates its output to the next stage — after every operation (Eager), never automatically (Lazy), when threshold is exceeded (Watermark), after time window (Timed), or adaptively by workload frequency via EWMA-controlled LSM-style logic. This is central to memory/throughput tradeoff and batching efficiency in cache engines.

Static sub-axes

ID	Name	Description
FS1	Event-Triggered	Flush is triggered by explicit events (<code>put/get</code> operation or eviction). Encompasses <code>EagerFlush</code> (after every operation) and <code>LazyFlush</code> (only on eviction).

ID	Name	Description
FS2	Threshold-Triggered	Flush is triggered when buffer fill exceeds a threshold (default 75%). Reduces unnecessary flushes in sparse traffic.
FS3	Time-Triggered	Flush is triggered periodically after a time window (micro-batching), independent of buffer fill. Classic Hadoop/Spark/Kafka pattern for bounded-latency guarantees.
FS4	Adaptive-Triggered	Flush threshold is adaptively learned from workload frequency via EWMA. High write-rate → flush early (60% threshold); low rate → flush late (95% threshold). First Q2 strategy with self-tuning.

Building blocks

Variant	Description	Source
EagerFlush	Pessimistic Write-Through policy: flushes completely (FullFlush) after EVERY <code>put()</code> operation. Lowest latency, but maximum flush frequency and highest throughput overhead — classic synchronous-write pattern.	Textbook Write-Through Policy / Code Baseline
LazyFlush	Optimistic Write-Back policy: never flushes automatically, only on explicit eviction from buffer. Maximum batching, minimum flush frequency — classic deferred-write pattern. <code>should_flush()</code> always returns <code>NoFlush</code> .	Textbook Write-Back Policy / Code Baseline
WatermarkFlush	Threshold-driven flush policy (default 75% capacity). Iterable aspect with 5 variants (50%, 65%, 75%, 85%, 95%) for runtime permutation. Related to LSM-Tree Memtable-Trigger (O’Neil 1996).	LSM-Tree O’Neil 1996 (related) / Code Implementation
TimedFlush	Time-window-based micro-batching policy (iterable aspect: 10/100/1000/10000 ms). Inspired by Kafka (NetDB 2011) and Spark D-Streams (SOSP 2013) for bounded-latency guarantees. Independent of buffer fill.	Kafka NetDB 2011 / Spark SOSP 2013 / Code Implementation
AdaptiveLsmFlush	Self-tuning adaptive watermark policy with EWMA over burst-rate. Threshold interpolates between 60% (high write-rate) and 95% (low rate). Conceptually RocksDB Dynamic Leveled Compaction + O’Neil LSM 1996; EWMA logic is CE-specific. First Q2 strategy with <code>is_adaptive=true</code> .	RocksDB Dynamic Leveled Compaction / O’Neil LSM-Tree 1996 / Code Implementation

Contributing sources: O’Neil et al. 1996 (LSM-Tree), Kafka (NetDB 2011), Spark D-Streams (SOSP 2013) [OCGO96, KNR11, ZDL⁺13].

D.18 T17 Persistence Target

This axis is the *persistence-target organ* (T17) of the living being `SearchAlgorithm` (cf. Section 3.2); it joined as the eighteenth organ slot with the axis reordering.

The axis makes the storage target of the structure explicit as an organ slot of its own instead of carrying an implicit in-memory assumption. In the canonical measurement catalogue it is pinned to its in-memory representative so that the permutation space stays at 2^{17} binary identities per system permutation (Section 3.4.5); every `binary_id` nevertheless carries the eighteenth segment `persistence_target=persistence_memory_only`.

Building blocks

Variant	Description	Source
MemoryOnlyTarget	In-memory storage as the default target; the only enabled representative of the canonical measurement catalogue. No device write-back path (<code>has_device_writeback_path()</code> =false).	code/baseline
DiskWritebackTarget	Disk write-back target; fully present as a building block (type, registry, sub-axes PT1–PT2) but switched off per option in the canonical catalogue, so the axis remains single-element there.	code/baseline

Contributing sources: — (CE baseline, own building blocks).

D.19 System sphere: target ISA (`target_isa`)

In the nineteen-axis numbering this quantity was the *ISA organ* (then T12); since the axis reordering it is the system main axis `target_isa` — itself a complex axis (cf. Sections 3.4.1 and 3.4.2).

The axis specifies the target CPU instruction set architecture (ISA) as a compile-time constant, enabling cross-platform optimizations across four dominant server architectures: x86-64 (Intel/AMD), AArch64 (ARM), Power-ISA (IBM), and RISC-V (open-source). Each variant defines a hardware-specific SIMD baseline (SSE2, NEON, VSX, scalar) and thus characterizes the vector acceleration potential for cache-engine operations such as `simd_field_sum` (accumulation of 32-bit word fields).

Static sub-axes

ID	Name	Description
IS1	Word Size	CPU data-path width: 32-bit, 64-bit, or 128-bit. In the cache engine standardized to 64-bit for all four ISA profiles.
IS2	CPU Vendor/Family	Instruction set family: <code>intel_amd</code> (x86-64), <code>arm</code> (AArch64), <code>ibm</code> (Power-ISA), <code>open</code> (RISC-V). Determines SIMD extension compatibility: SSE2/AVX2/AVX-512 (x86-64), NEON/SVE/SVE2 (AArch64), VSX (Power), RVV (RISC-V).
IS3	ZIH Cluster Affinity	Production machines at TU Dresden: Barnard (Intel Xeon Platinum 8470, Sapphire Rapids, x86-64), Capella (AMD EPYC 9334, x86-64), Grace Hopper (NVIDIA Grace Hopper ARMv9 Neoverse V2, AArch64), N/A (research boards, RISC-V/Power). Enables hardware-specific experiments with real production cluster profiles.

Beyond these three design dimensions, the build-driving system registry carries four system sub-axes on the complex wrapper: two compile-time ones (`target_isa_complex` and `scheduling`, each `stage=ct` with `kind=fixed_enum_tuple`) and two dynamic ones (`numa_node` and `page`). The first bundles the named recombinations of the target machines (`complex_count=2`) with three fixed members each (owner OD-2).

Recombination	Member	Declaration
<code>prod1_zen5</code> (<code>isa=x86_64</code>)	<code>ram_frequency_mhz</code>	<code>declared=false</code> — no declared figure.
<code>prod1_zen5</code>	<code>cas_latency_cl</code>	<code>declared=false</code> — no declared figure.
<code>prod1_zen5</code>	<code>cpu_fabrication</code>	<code>declared=true</code> , <code>value=AuthenticAMD/26/68/0</code> (vendor/family/model/stepping).
<code>prod2_raptor_lake</code> (<code>isa=x86_64</code>)	<code>ram_frequency_mhz</code>	<code>declared=true</code> , <code>value=4800</code> , <code>declaration_source=dmidcode 2026-07</code> .

Recombination	Member	Declaration
prod2_raptor_lake	cas_latency_cl	declared=false — no declared figure.
prod2_raptor_lake	cpu_fabrication	declared=true, value=GenuineIntel/6/151/2.

Note: the values originate from `kDeclaredMachines`, which in turn mirrors the `<machines>` block of the user XML (O-4b). `declaration_source` preserves the frozen origin of a declared figure (A2 note, not a provenance level); runtime levels never appear in this XML (P3).

The second compile-time sub-axis `scheduling` carries five fixed members. It used to be a system main axis and was re-attached as a `sub_axis` of the `target_isa` complex wrapper with package A3 — a relocation, not a deletion.

Member	Registry value	Description
<code>worker_pool_layout</code>	<code>value_ordinal=0</code>	Distribution of the worker pool across the cores.
<code>simd_worker_count_limit</code>	<code>value=2</code>	Upper bound of concurrent SIMD workers.
<code>hetero_core_dispatch</code>	<code>value_ordinal=0</code>	Dispatch across heterogeneous cores (P and E cores).
<code>co_routine_strategy</code>	<code>value_ordinal=0</code>	Coroutine strategy of job processing.
<code>batch_granularity</code>	<code>value_ordinal=0</code>	Granularity of batch processing.

Note: the ordinals are raw enum values without a compile-time label template; the identifier strings are compile-coupled to the accessor calls of `concepts::scheduling_strategy.hpp`.

Dynamic sub-axes

ID	Parameter	Description
<code>numa_node</code>	NUMA node	<code>stage=runtime, value_type=token, option_source=machine_resolved</code> . Mirror of the organ member <code>alloc_hw.numa_node</code> ; the admissible values are the node identifiers of the concrete machine and are not knowable at build time — hence a declared option source instead of an option catalogue.
<code>page</code>	Page-size release	<code>stage=runtime, value_type=token, option_source=machine_resolved</code> . The release of page sizes (4k/2m/1g) as a system capability; enforcement lives on the organ side in the <code>AllocPageHint</code> NTTP, this axis is the system side of the release.

Name trap (guarded at compile time): the sub-axis is called `page` and is not `page_type`. `axis_01_page_type` is the tree-node `PageKind` of the organ world and an entirely different axis (blueprint D2.6/IV-BLOCKER-5); the `static_assert` in `target_isa_sub_axes.hpp` records this. The building-block catalogue of the page type remains unchanged in the conserved legacy section “Build-PG”.

Building blocks

Variant	Description	Source
Amd64Isa	x86-64 / Intel 64 ISA (AMD/Intel Server+Desktop). Dominant server platform with SSE2 SIMD baseline (4 uint32 lanes per 128-bit register). Implements genuine SSE2 vectorization via <code>_mm_loadu_si128 + _mm_add_epi32</code> on native x86-64 hosts; scalar fallback on non-x86 build hosts. ZIH presence: Barnard (Intel Xeon Platinum 8470, Sapphire Rapids), Capella (AMD EPYC 9334).	Vendor spec (AMD 2000 / Intel ongoing)
Aarch64Isa	ARMv8-A 64-bit ISA (ARM Holdings). Growing server relevance (Apple M-Series, AWS Graviton, Ampere Altra). SIMD baseline: NEON (128-bit, 4 uint32 lanes). Cache engine implementation: 4-way unrolled scalar fallback (models NEON width without ARM intrinsics on x86 build hosts); genuine NEON vectorization on native ARMv8 hosts. ZIH presence: GraceHopper (NVIDIA Grace Hopper, Neoverse V2 + Hopper GPU).	Vendor spec (ARM Ltd., since 2011/2013)
PowerPcIsa	Power ISA (POWER9/10, ppc64le little-endian). IBM Power Systems (AC922, IC922, S1022). SIMD baseline: VSX (Vector-Scalar Extension, 128-bit, 4 uint32 lanes). Cache engine implementation: 4-way unrolled scalar fallback (models VSX width without Power intrinsics on x86 build hosts). Relevant for HPC workloads with NVLink GPU connectivity (AC922). ZIH presence: not primary, but available for research comparisons.	Vendor/standards spec (IBM/ OpenPOWER, v3.0 2017, v3.1 2020)
RiscVIsa	RISC-V RV64GC open-source ISA (RV64 General + Compressed). Growing relevance (SiFive, Alibaba Xuantie, Andes, ETH Zurich cores). SIMD baseline: NONE (RVV is optional, not part of RV64 GC baseline). Cache engine implementation: deliberately plain scalar path (unrolled) – models lack of vector unit in base ISA and thus creates real, slower runtime versus x86-64 and ARM. ZIH presence: embedded + research boards.	Standards spec (RISC-V Intl.; UCB TR since 2011)

Registry state: the build-driving system registry currently carries two enabled building blocks — `x86_64` (native, without triple and march flags) and `aarch64` (cross compilation with `triple=--target aarch64-linux-gnu` and `march=--march=armv8-a`). The four-ISA catalogue listed above is retained as an *offering* catalogue: Power-ISA and RISC-V are described as building blocks but are not enabled in today's build offering.

Contributing sources: —

D.20 System sphere: operating system (`operating_system`)

This quantity is the second system main axis and thus part of the *blood* and of no organ (cf. Section 3.4.1); in the canonical, code-guarded order it occupies position two of three.

The main axis carries the *class* identity only: exactly three operating-system families (instance catalogue OP-10, `baustein_count=3`), together with `stage=ct` and `binary_id=never`. The *instance* — distribution, release, kernel and patch level — resides entirely in the three sub-axes; entering a distribution as a fourth expression of the main axis would miss that cut. The real fleet (Windows 11 and Windows Server 2022, the Linux Docker matrix, two macOS machines) falls entirely into the three families.

Dynamic sub-axes

ID	Name	Description
os_version	Distribution/release identity	The release identity of the operating-system instance within its family: on Linux ID and VERSION_ID from <code>/etc/os-release</code> , on Windows the product line, on macOS the product version.
kernel	Kernel release	The kernel release of the instance: on Linux and macOS the <code>uname</code> release, on Windows the NT kernel triple.
build	Build and patch level	The build and patch level of the instance <i>including</i> the update state (owner decision E-12, verbatim: “Wir mergen den Update Zustand in Build”). A separate <code>update_zustand</code> is therefore explicitly <i>not</i> a fourth sub-axis but a constituent of this one.

Note: all three are `stage=runtime` with `value_type=token` and `option_source=machine_resolved`: they have no compile-time option catalogue because their admissible values depend on the concrete machine and are not knowable at build time. Runtime acquisition is a build step of its own (package OS-U3, `operating_system_probe_{linux,macos,windows}.hpp`); the attachment itself is only the offering. The names have been fixed since owner decision E3 of 2 August 2026 (package A14/OS-U1).

Building blocks

Variant	Description	Source
linux	Linux family (<code>LinuxOperatingSystem, posix=true</code>). Covers the entire Docker distribution matrix; the individual distribution is a matter for the sub-axis <code>os_version</code> , not for this expression.	Code/Baseline
windows	Windows family (<code>WindowsOperatingSystem, posix=false</code>). Covers desktop and server product lines (Windows 11, Windows Server 2022).	Code/Baseline
macos	macOS family (<code>MacosOperatingSystem, posix=true</code>). Covers the Apple Silicon and Intel machines of the fleet.	Code/Baseline

Contributing sources: — (CE baseline, own building blocks).

Stamp neutrality (A-15): the three sub-axes are runtime sub-axes and therefore never appear in the binary stamp. Their attachment changes neither the stamp lines nor the build-version suffix, and it must not raise the axis code level of `operating_system` (bump prohibition guard): such a bump would shift the SHA512 of all new builds against the existing stock and would force exactly the full rebuild of all legacy binaries that the owner decision rules out.

D.21 System sphere: extension hardware (`external_utils`)

This quantity is the third system main axis; it is exclusively the collecting hub of the hardware meta axes — SIMD and vector extensions, GPU, FPGA and NPU as well as external accelerators (cf. Section 3.4.1). The hub carries *only* system meta-metas; the load framework explicitly does not belong to it. Because meta-meta axes receive no stamp line of their own but appear as a bracketed suffix at the end of the respective realm line, this hub closes the system realm: the suffix is folded type-driven from the hub’s member type list (`ExternalUtilsHub` → `SimdExternalUtilsFamily`), and the level resides in the bracket depth rather than in the name (owner decision E2 and owner answer Q1 of 2 August 2026).

In today’s build-driving offering the axis carries `baustein_count=1`, `stage=ct` and `binary_id=never`.

Dynamic sub-axes

ID	Name	Description
<code>simd</code>	Vector extension level	<code>stage=runtime</code> , <code>value_type=token</code> , <code>option_count=3</code> . The family identifier is at the same time the sub-axis label (notation <code>external_utils.simd=...</code>); the per-dialect reflected compiler flags come from the option, not from the family.

The three options of the sub-axis with their flags per compiler dialect:

Option	Flag (gpp and clang)	Flag (msvc)
<code>no_extension</code>	— (empty)	— (empty)
<code>avx2</code>	<code>-mavx2</code>	<code>/arch:AVX2</code>
<code>avx512</code>	<code>-mavx512f</code>	<code>/arch:AVX512</code>

Note (Q2/K2): `simd` materializes as a flag of the compilation seam and therefore appears in the build-version suffix (cf. Section 3.4.4) — never as a runtime type switch and never in the `binary_id`.

The sub-axis is accompanied by a fine-grained feature catalogue `simd_feature_catalog` with `count=23` features, tiered by `tier`: fourteen features of tier `avx512`, four of tier `avx256`, three companion features (`companion`) and two scalar ones (`scalar`). The flags are identical throughout for `gpp` and `clang`.

Feature (cpuinfo)	Flag (gpp and clang)	Tier (tier)
<code>avx512f</code>	<code>-mavx512f</code>	<code>avx512</code>
<code>avx512cd</code>	<code>-mavx512cd</code>	<code>avx512</code>
<code>avx512vl</code>	<code>-mavx512vl</code>	<code>avx512</code>
<code>avx512dq</code>	<code>-mavx512dq</code>	<code>avx512</code>
<code>avx512bw</code>	<code>-mavx512bw</code>	<code>avx512</code>
<code>avx512ifma</code>	<code>-mavx512ifma</code>	<code>avx512</code>
<code>avx512vbmi</code>	<code>-mavx512vbmi</code>	<code>avx512</code>
<code>avx512_vbmi2</code>	<code>-mavx512vbmi2</code>	<code>avx512</code>
<code>avx512_vnni</code>	<code>-mavx512vnni</code>	<code>avx512</code>
<code>avx512_bitalg</code>	<code>-mavx512bitalg</code>	<code>avx512</code>
<code>avx512_vpopcntdq</code>	<code>-mavx512vpopcntdq</code>	<code>avx512</code>
<code>avx512_vp2intersect</code>	<code>-mavx512vp2intersect</code>	<code>avx512</code>
<code>avx512_bf16</code>	<code>-mavx512bf16</code>	<code>avx512</code>
<code>avx512_fp16</code>	<code>-mavx512fp16</code>	<code>avx512</code>
<code>avx2</code>	<code>-mavx2</code>	<code>avx256</code>
<code>fma</code>	<code>-mfma</code>	<code>avx256</code>
<code>f16c</code>	<code>-mf16c</code>	<code>avx256</code>
<code>avx_vnni</code>	<code>-mavxvnni</code>	<code>avx256</code>
<code>gfni</code>	<code>-mgfni</code>	<code>companion</code>
<code>vaes</code>	<code>-mvaes</code>	<code>companion</code>
<code>vpclmulqdq</code>	<code>-mvpclmulqdq</code>	<code>companion</code>
<code>bmi2</code>	<code>-mbmi2</code>	<code>scalar</code>
<code>popcnt</code>	<code>-mpopcnt</code>	<code>scalar</code>

Machine signatures (count=3): the registry records for each target machine the subset of features actually present — `prod1_zen5` (`host_isa=x86_64`) with 22 features, `prod2_raptor_lake`

with 9 and `odroid_gracemont` with 9. Only the Zen 5 machine carries the `avx512` tier; the other two end at `avx256` together with the companion and scalar features.

Building blocks

Variant	Description	Source
<code>simd</code>	Vector extension family (<code>SimdExternalUtilsFamily</code> , <code>family_id=simd</code> , enabled). The only member of the hub enabled today; the families for GPU, FPGA and NPU are foreseen in the design but are not part of the build-driving offering.	Code/Baseline

Contributing sources: — (CE baseline, own building blocks).

D.22 Build-SIMD: Vector/Accelerator Extension Axis (Hardware Accelerators)

Conserved offering and history catalogue: the legacy axis identifier `axis_09b` has been superseded by the main axis `external_utils` with its sub-axis `simd`. The eight-building-block catalogue below is retained in full as an offering and history catalogue under that main axis, as is the sub-axis table SE1–SE3 as a historical block.

The `axis_09b_simd_extension` models available SIMD/accelerator extensions of the CPU instruction set architecture as a sub-axis to `axis_09` (main ISA). It enables compile-time selection among various vector extensions (SSE2/AVX2/AVX-512 for x86; NEON/SVE2 for ARM; RVV for RISC-V) plus baseline (scalar) and GPU acceleration (CUDA), with backward-cumulative layer hierarchy and hardware-specific topology (SIMD units per socket, P/E core access).

Static sub-axes

ID	Name	Description
SE1	Vector Width	Compile-time-known maximum vector width of the SIMD extension in bits: 0 (none), 128, 256, 512, or -1 (scalable). Determines lane width and parallelization degree.
SE2	Accelerator Type	Categorizes accelerator class: none (baseline), simd (vector SIMD), matrix (matrix engine), gpu (GPU), npu (neural processing unit). Influences composition with hardware topology (<code>axis_12</code>).
SE3	Compatibility Family	Constraint to main ISA (<code>axis_09</code>): none (universal), x86, arm, riscv, cuda. Ensures compile-time validation: e.g., <code>Sse2SimdExtension</code> compatible only with <code>Amd64Isa</code> .

Building blocks

Variant	Description	Source
NoSimdExtension	Baseline variant without SIMD acceleration. Scalar fallback strategy for all permutations without accelerator. Universally compatible with all main ISAs (x86/ARM/RISC-V/PowerPC). Mandatory comparison baseline.	Code/Baseline
Sse2SimdExtension	Intel SSE2 128-bit vector extension (x86_64 ABI baseline since 2003). Compatible only with Amd64Isa. Baseline on x86_64; provides 4x uint32 lanes/iteration. Standard on cluster nodes Barnard/Capella.	Intel ISA Ext Ref 319433-060
Avx2SimdExtension	Intel AVX2 256-bit vector extension (Haswell+ 2013). X86_64 only. Backward-cumulative: includes all SSE+AVX. Default optimization level for server DBMS. 2 AVX2 units/socket. ZIH standard on Barnard (Intel Sapphire Rapids) and Capella (AMD EPYC 9334).	Intel ISA Ext Ref 319433
Avx512SimdExtension	Intel AVX-512 512-bit vector extension (Skylake-X+, Ice Lake Server, Zen 4+). Backward-cumulative: all SSE/AVX/AVX2 + 15 separate flags (VNNI for vector indexes, VPOPCNTDQ for bitmap indexes). Not on all x86_64 CPUs (e.g., Haswell/Raptor Lake disabled). 1 AVX-512 unit/socket; E-cores no access (Intel Hybrid).	Intel ISA Ext Ref 319433-060
NeonSimdExtension	ARM NEON / Advanced SIMD 128-bit (AArch64 ABI baseline ARMv8+). Compatible only with AArch64Isa. Baseline on all AArch64 CPUs (Apple M-Series, AWS Graviton 2/3/4, NVIDIA Grace). 1 NEON unit/socket; all cores accessible (big.LITTLE architecture).	ARM ARM DDI 0487 / Cortex-A8 TRM
Sve2SimdExtension	ARM SVE2 (Scalable Vector Extension 2) ARMv9 mandatory (scalable 128–2048 bit). Compatible only with AArch64Isa. ZIH cluster Grace Hopper (NVIDIA Grace = ARM Neoverse V2 with 128-bit SVE2). Scalable lane width; 1 unit/socket; all cores.	SVE [SBB ⁺ 17] (IEEE Micro 2017); SVE2/Armv9 per ARM
RvvSimdExtension	RISC-V Vector Extension (RVV v1.0, scalable VLEN 128–65536 bit). Compatible only with RiscVIsa. Optional (not baseline like SSE2/NEON). Growing relevance: SiFive Performance P870 (128-bit), T-Head Xuantie C908 (128-bit). Research boards at TU Dresden/ZIH.	RVV-v1.0-Spec [RIS21]; Vitruvius+ (TACO 2023) [MPU ⁺ 23]
CudaGh200SimdExtension	NVIDIA CUDA on Hopper GH200 (Grace Hopper Superchip GPU accelerator). ZIH cluster Alpha Centauri with NVIDIA HGX. Compatible with AArch64Isa (Grace CPU) AND Amd64Isa (Hopper host). NVLink-C2C 900 GB/s between Grace CPU and Hopper GPU. Massive parallel (96 GB HBM3 GPU; ~16,896 CUDA cores). <code>units_per_socket () = -1</code> (GPU is separate device).	NVIDIA Tesla [LNOM08] (IEEE Micro 2008) + NVIDIA GH200 Whitepaper

Sources: ISA specifications and whitepapers (Intel, ARM, IEEE Micro, ACM TACO, NVIDIA); see the source column of the building blocks.

D.23 System sphere: outer complex bracket (`build_target_complex`) and build toolchain (`build_toolchain`)

This quantity is the command-pattern bracket wrapper over the fixed recombination of the three system main axes (`member_count=3`: `target_isa`, `operating_system`, `external_utils`) and thus closes the system realm block (cf. Section 3.4.2).

As a bracket it is explicitly *not* a fourth main axis: the canonical system axis order stays at three. Three compile-time departure guards (`static_assert`) record this — `compiler`, `scheduling` and

`load_framework` must not return as main axes; their departures were relocations, not deletions. The load framework is not a *member* of this bracket either: since package A3 it lives in the measurement realm, whereas the system meta-metas travel under the hub `external_utils`. The axis carries `stage=ct` and `binary_id=never`.

Static sub-axes

The sub-axis group `build_toolchain` (`sub_axis_count=3`) hangs on the bracket. The registry names the three members flat; the nesting underneath shows the actual parent chain.

Member	Parent reference	Description
<code>compiler</code>	<code>build_target_complex</code>	<code>stage=ct</code> , <code>baustein_count=2</code> . The compiler choice itself; it used to be a system main axis and became the sub-axis group of the outer bracket with package A3.
<code>opt_level</code>	<code>compiler</code>	<code>stage=runtime</code> , <code>value_type=token</code> , <code>option_count=5</code> . Optimization level per compiler dialect.
<code>atomic128</code>	<code>compiler</code>	<code>stage=runtime</code> , <code>value_type=token</code> , <code>option_count=2</code> . Release of the 128-bit compare-and-swap instruction.

Note: the group hangs on `build_target_complex` and not on `target_isa` because it describes the *build* arising from the recombination of all three main axes and not a property of the target ISA alone (O-1r).

Building blocks

Variant	Description	Source
<code>gcc</code>	GNU compiler (<code>GccCompilerAxis</code> , driver <code>g++-16</code> , <code>supports_fno_gnu_unique=true</code>).	Code/Baseline
<code>clang</code>	LLVM compiler (<code>ClangCompilerAxis</code> , driver <code>clang++-22</code> , <code>supports_fno_gnu_unique=false</code>).	Code/Baseline

The five options of the sub-axis `opt_level` with their flags per dialect; the column `ieee754_deterministic` states whether the level leaves floating-point semantics untouched:

Option	Flag (gpp and clang)	Flag (msvc)	<code>ieee754_deterministic</code>
<code>O0</code>	<code>-O0</code>	<code>/Od</code>	<code>true</code>
<code>O1</code>	<code>-O1</code>	<code>/O1</code>	<code>true</code>
<code>O2</code>	<code>-O2</code>	<code>/O2</code>	<code>true</code>
<code>O3</code>	<code>-O3</code>	<code>/O2</code>	<code>true</code>
<code>Ofast</code>	<code>-Ofast</code>	<code>/O2</code>	<code>false</code>

The two options of the sub-axis `atomic128`:

Option	Flag (gpp and clang)	Flag (msvc)
<code>no_cx16</code>	— (empty)	— (empty)
<code>cx16</code>	<code>-mcx16</code>	— (empty)

Build version identifier: `opt_level`, `simd` and `atomic128` materialize as flags of the compilation seam and therefore appear in the build-version suffix of the binary — an ordered eight-segment sequence

in which an empty member simply produces no segment (cf. Section 3.4.4). They never appear in the `binary_id`.

Version flag grammar v2 (owner decision core of 7 August 2026, fully supersedes the Q3 grammar): the version of every axis algorithm is uniformly three-part; a `v` prefix no longer exists since flag grammar v2, and short forms remain forbidden. Compile-time guards enforce the form:

```
version := UINT '.' UINT '.' UINT [ '.' flag ]*
flag    := basis [ '{' sub [ '.' sub ]* '}' ] | companion
basis   := 'c' | 'g' | 'f' | 'n' | 'x128' | 'x256' | 'x512'
sub     := token [ '{' sub [ '.' sub ]* '}' ] (recursive)
```

A dot precedes every flag, including the first (`1.0.0.c`, never `1.0.0c`); the basis attaches directly to its opening brace, with no dot of its own in between (`c{p.e}`, never `c.{p.e}`). The suffix `e` now denotes an *efficiency core* and is a sub-flag like any other under the basis `c`; the earlier concept of an experimentally introduced specimen algorithm has been dropped without replacement. Example form: `memory_layout=SoaMemoryLayout@1.0.0.c{p.e}.x512{f.v1.bw.dq}.gfni`.

D.24 Measurement sphere: measurement categories (`measurement_category`)

This quantity belongs to the third realm — the measurement axes, the *blood of measuring* (cf. Section 3.4.1).

It carries `stage=ct`, `binary_id=never` and `baustein_count=16`. Its values manifest as columns of the result tables and as the setting label of a run, never in the binary identity. In the chapter model the sixteen categories are the *sub*-axis of the measurement tooling; the measurement registry additionally carries them as an offering block of their own, because the tooling does not invent its categories but selects them from this catalogue. The column `regime_ordinal` separates the time- and observer-based regime (0) from the counter-based PMC regime (1).

Building blocks

Category	category_ordinal	Regime (regime_ordinal)
CLU	0	0 — time/observer
CACHE_MISS_L1	1	1 — PMC
CACHE_MISS_L2	2	1 — PMC
CACHE_MISS_L3	3	1 — PMC
DTLB_MISS	4	1 — PMC
MEMORY_FOOTPRINT	5	0 — time/observer
BRANCH_MISS	6	1 — PMC
IPC_CPI	7	1 — PMC
LATENCY_MEAN	8	0 — time/observer
LATENCY_P50	9	0 — time/observer
LATENCY_P95	10	0 — time/observer
LATENCY_P99	11	0 — time/observer
LATENCY_P999	12	0 — time/observer

Category	category_ ordinal	Regime (regime_ordinal)
THROUGHPUT	13	0 — time/observer
ENERGY_J	14	1 — PMC
FILL_BUFFER_OCCUPANCY	15	0 — time/observer

Regime split: nine categories lie in the time and observer regime (CLU, MEMORY_FOOTPRINT, the five latency categories, THROUGHPUT, FILL_BUFFER_OCCUPANCY), seven in the PMC regime (the three cache-miss levels, DTLB_MISS, BRANCH_MISS, IPC_CPI, ENERGY_J). The split coincides with the “nine/seven” division of Section 3.4.1.

Contributing sources: — (CE baseline, own building blocks).

D.25 Measurement sphere: measurement tooling/collector (collector)

This quantity is the instrument axis of the measurement realm: it records with which tool a category is acquired (cf. Section 3.4.1).

It carries `stage=ct`, `binary_id=never` and `baustein_count=3`. Every building block carries its regime and the set of categories it serves; the union of the three sets covers ten of the sixteen categories. Of the remaining six, the four latency-percentile categories (LATENCY_P50 through LATENCY_P999) are served not by a collection tool but by the downstream evaluation (nearest-rank over the raw values, complemented by the HDR histogram, Section 4.5); MEMORY_FOOTPRINT and FILL_BUFFER_OCCUPANCY are described in the offering catalogue but not yet assigned to a tool.

Building blocks

Variant	Regime	Categories served
WallClockSystemAxis	0	LATENCY_MEAN, THROUGHPUT
ObserverSnapshot	0	CLU
SystemAxis		
PmcSystemAxis	1	CACHE_MISS_L1, CACHE_MISS_L2, CACHE_MISS_L3, DTLB_MISS, BRANCH_MISS, IPC_CPI, ENERGY_J

Contributing sources: — (CE baseline, own building blocks).

D.26 Measurement sphere: telemetry building blocks (conserved catalogue)

In the nineteen-axis numbering this quantity was the *telemetry organ* (then T10); since the axis reordering it is a *two-way split* main quantity — sweep sub-axis of the measurement tooling in the planner *and* compile-time main quantity in the builder (cf. Section 3.4.1). Its building-block catalogue is retained here as a conserved catalogue.

Defines strategies for measuring index operations (insert, lookup, erase) with different overhead profiles. Supports density tracking for adaptive node-type selection, global insert counters for throughput measurements, HDR histograms for latency quantiles, and leaf-only counters to avoid cache-line false sharing in measurement cycles.

Static sub-axes

ID	Name	Description
TM1	Scope	Validity scope of counter management: leaf-only (leaf nodes only), per-node (all nodes), or global (single accumulator for entire structure). Compile-time choice, impacts overhead through contention on shared cache lines.
TM2	Metric Type	Type of measured metric: counter (event counter), histogram (distribution with quantiles), or density (per-node slot occupancy). Impacts memory footprint and measurement granularity.
TM3	Overhead Level	Measurement overhead magnitude: low (atomic operations), medium (per-node data structures), high (histogram logging). Determines suitability for different workload scenarios (benchmarking vs. production).

Building blocks

Variant	Description	Source
LeafOnly Counter	Counter in leaf nodes only (avoids cache-line ping-pong by skipping inner-node updates). Low overhead with leaf-only scope. Implements coherence-friendly counter concept from Kuehn DaMoN 2023 motivation, although original paper describes cache-friendly B+-layout, not leaf-counter algorithm — CE baseline for false-sharing avoidance.	P28 (Kuehn DaMoN 2023, context; Code/Baseline)
DensityTracker	Per-node density/slot-occupancy tracking for adaptive node-type selection (4→16→48→256). Contextual telemetry tied to ART structure (Leis ICDE 2013), measures slot occupancy across all nodes. Higher overhead than LeafOnly, but provides adaptation signal for dynamic rebalancing decisions.	P01 (ART/Leis ICDE 2013, context; Code/Baseline)
InsertCounter	Global atomic insert counter with minimal overhead (1 atomic increment per insert). Contextual telemetry tied to HOT structure (Binna SIGMOD 2018), global scope. Sufficient for throughput measurement series, no latency granularity.	P02 (HOT/Binna SIGMOD 2018, context; Code/Baseline)
Latency Histogram	HDR histogram for lookup latency distribution (p50/p95/p99 quantiles). Based on HdrHistogram (Gil Tene), CC0-1.0 license. Highest overhead (per lookup), but provides complete latency statistics for SLA-oriented and research-grade measurement series. Only is_original candidate of the axis.	HdrHistogram (github.com/HdrHistogram/HdrHistogram_c, CC0-1.0, is_original)
Probability-Hints	Probability hints of the cache-oblivious Bender line (P16/P17): distribution- and probability-related occupancy hints that serve as input to the layout decisions. Paper building block of the axis dissection (reconstruction target of this thesis, no CE baseline).	P16/P17 (Bender 2000/2005) [BDFC00, BDFC05]

Contributing sources: HdrHistogram (Gil Tene) [Ten12], P16/P17 (Bender) [BDFC00, BDFC05].

Related literature: P01, P02, P28 [LKN13, BZP⁺18, Kue23].

D.27 Measurement sphere: load framework (`load_framework`) — meta-meta axis

This quantity is the meta-meta main axis of the measurement realm and therefore stands at its end (cf. Section 3.4.1).

It carries `category=measurement_meta_meta`, `stage=ct`, `binary_id=never` and `baustein_count=1`. Its relocation history is part of its identity: with package A3 it left the system world *without replacement* towards the measurement realm, where it is the planner's meta-meta main axis; it explicitly does not belong under the hub of the system meta-metas. In the stamp it receives *no* meta-meta line of its own: its identifier appears as a dynamic bracketed suffix at the *end* of the existing measurement realm line (owner decision E2 and owner answer Q1 of 2 August 2026). The feeding path here is entry-driven — the chosen identifier together with its version comes from the framework registry of the measurement realm and therefore does not reside in the type, unlike the type-driven system hub.

Dynamic sub-axes

The registry carries six sweep dimensions (`resource_control_version=1`); all are `stage=runtime`.

Dimension	Origin (source)	Value type and meaning
<code>workload</code>	<code>measurement:load_framework</code>	token — the load profile; the single source is the sub-axis label of the load framework itself.
<code>thread_count</code>	<code>resource_control_pod</code>	uint — number of worker threads.
<code>prefetch_distance</code>	<code>resource_control_pod</code>	uint — prefetch distance.
<code>pool_budget_bytes</code>	<code>resource_control_pod</code>	uint — memory budget of the pool.
<code>batch_size</code>	<code>resource_control_pod</code>	uint — batch size per operation.
<code>inline_threshold_bytes</code>	<code>resource_control_pod</code>	uint — threshold of inline storage.

Honesty note: the repetition count of a run and the measurement modes are deliberately *not* in this list — for two *different* reasons. The repetition count remains a pure label dimension without a compile-time anchor and becomes reflectable only once it is typed. The modes, by contrast, are typed by now: there are *four* of them (debug, measurement, release and comparison execution), carried by a compile-time registry without runtime branching, where comparison execution today carries selectability and not yet its own procedure. They are nevertheless absent here because the fan-out of this sub-axis into the reflection matrix is still pending. A hand-written entry would be a phantom in either case — the honest-0 doctrine forbids it (open item W2-B).

Building blocks

Variant	Description	Source
<code>ycsb</code>	The YCSB load-profile framework; enabled, with the sub-axis label <code>workload</code> . The only building block of the axis enabled today.	Code/Baseline

Contributing sources: — (CE baseline, own building blocks).

D.28 Build-PG Page Type Axis (PAGE-TYPE)

Delimitation and conservation: this section describes the `PageKind` of the organ world (`axis_01_page_type`) and is distinct from the system sub-axis `page` of the target ISA (page-size release 4k/2m/1g) — both quantities carry similar names but share neither building blocks nor semantics. The catalogue is retained in full as a conserved offering and history catalogue.

The page type axis defines the structural and indexing strategies for branch pages in the trie index: from dense direct-addressed byte indexing (`DenseByte`), through multi-level density variants (`ExtendedDense`, `SparsePatricia`), to path-collapse optimizations (`Redirect`, `CustomCache`) and B+-style sorted pages (`BPlus`). All page types are cache-line aware and implement the `PageTypeStrategy` concept interface with guaranteed interchangeability (F15 permutation).

Static sub-axes

ID	Name	Description
PG1	Structure Role (Branching)	Classification of branch page architecture: full-branching branch page, collapsed single-path node (<code>Redirect</code>), or specialized cache-optimized variant.
PG2	Density Class (Density)	Degree of slot utilization and resulting indexing strategy: densely populated (0-256 slots), extended (>256 slots), or sparse with Patricia compression.
PG3	Path Collapse Strategy	Handling of unique remainder paths: no compression (standard), or collapse via <code>Redirect/CoCo</code> technique for space minimization.

Building blocks

Variant	Description	Source
<code>DenseBytePageType</code>	Dense byte-indexed branch page with direct 256-slot addressing (direct-addressed). Page type 1: direct mapping from key byte to slot index without indirection. Source: Adaptive Radix Tree (ART), Leis et al. 2013.	P01
<code>ExtendedDensePageType</code>	Extended dense page for >256 entries with multi-level indexing scheme (e.g., 2-level lookup or variable k-byte span). Page type 2: density class for complex slot arrangements. Source: Adaptive Radix Tree extensions, PRT-ART.	P01 (extended)
<code>SparsePatriciaPageType</code>	Sparsely populated page with Patricia path compression (single-bit split). Page type 3: density class with bit-vector traversal for low branching factor. Source: HOT (Height Optimized Trie, Binna et al. 2018, based on Patricia Trie, Morrison 1968).	P02
<code>RedirectPageType</code>	Collapsed unique remainder path (single-path node). Page type 4: path-collapse strategy for space optimization under monopoly unique successor sequences. Neither classical branch nor leaf, stores only suffix until next real branching or value node. Source: CoCo-Trie (Boffa et al. 2024).	P04
<code>CustomCachePageType</code>	Cache-line-oriented custom page with explicit layout design for 64-byte alignment. Page type 5: structure role branch with hardware-aware geometry. Source: PRT-ART proprietary implementation with TLB-offset + cache-line-aligned memory layout (Axis 5).	PRT-ART
<code>BPlusPageType</code>	B+-style sorted branch page with range-walk traversal. Page type 6: structure role branch, but with sorted key-array organization instead of byte indexing. Source: Masstree (Mao et al. 2012), B+-tree family.	P03

Contributing sources: P01, P02, P03, P04, Morrison 1968, PRT-ART [LKN13, BZP⁺18, MKM12, BFTV24, Mor68].

Related literature: P05, P06, P10, P11, P12, P13, P20 [FJKN20, SSL⁺21, ZLL⁺18, RR99, RR00, HP03, MBL25].

D.29 Build-HW — Platform Hardware Strategy

Current status (conserved catalogue): this axis is excluded from the build-driving offering of the system registry. The registry records the exclusion verbatim: `extension_hardware_system_axis.hpp` is a deprecated island, `hardware_isa_system_axis.hpp` a pure host descriptor that does not drive the build — neither belongs to the build-driving offering. The static platform profiles with their fixed constants for cache-line size, page size and SIMD bit width are conceptually superseded by the runtime hardware detection of the `CacheEngineBuilder` — a factory over ISA and operating system that acquires hardware values instead of setting them statically (cf. Section 3.4.3). The five sub-axis entries (`hw1` to `hw5`) and the three hardware profiles are retained in full as a historical catalogue.

Defines the platform hardware family (CPU architecture, SIMD capabilities, cache topology, memory paging) as compile-time constants for all cache-engine algorithms. The axis captures which platforms are configurable at build time and provides static properties (cache-line size, page size, SIMD bit-width, NUMA capability, hugepage capability) for algorithm-appropriate data structures.

Static sub-axes

ID	Name	Description
<code>hw1_simd_family</code>	12.1 SIMD-Family	Which vector instruction set is actively used? (AVX2, AVX-512, NEON, SVE2, scalar/none). Determines the maximum vector width for parallel operations within an algorithm.
<code>hw2_cache_level_targeting</code>	12.2 Cache-Level-Targeting	Which cache level is optimized (L1-aware, L2-aware, L3-aware, HBM-aware)? Critical for prefetch strategies and memory layout decisions.
<code>hw3_numa_strategy</code>	12.3 NUMA-Strategy	How is NUMA topology considered (single-node, multi-node, NUMA-aware-pinning, NUMA-balanced)? Affects allocator locality and thread-pinning strategy.
<code>hw4_prefetch_hardware</code>	12.4 Prefetch-Hardware-Instructions	Which explicit hardware prefetch instructions are available (PREFETCH, PREFETCHNTA, PREFETCHW, none)? Core part of distance-estimator and path-oriented prefetch.
<code>hw5_atomic_instruction_family</code>	12.5 Atomic-Instruction-Family	Which atomic read-modify-write instructions are available (CAS, LL-SC, RMW-extended via HTM/XBEGIN/XEND, none)? Fundamental basis for lock-free concurrency and optimistic lock coupling.

Building blocks

Variant	Description	Source
X86_64Hardware Profile	Modern x86-64 desktop/server platform (Intel/AMD). Build-time constants: <code>cache_line_size=64</code> , <code>memory_page_size=4096</code> , <code>simd_width_bits=256</code> (AVX2 conservative), <code>numa_capable=true</code> , <code>huge_page_capable=true</code> . Baseline for desktop and server-side experiments.	Code/Baseline
Aarch64Hardware Profile	ARM64/AArch64 server and mobile platforms (Ampere Altra, Apple Silicon M-series, Linux ARM). Build-time constants: <code>cache_line_size=64</code> , <code>memory_page_size=4096</code> , <code>simd_width_bits=128</code> (NEON standard), <code>numa_capable=true</code> , <code>huge_page_capable=true</code> . Enables cross-platform comparisons.	Code/Baseline
GenericHardware Profile	Conservative fallback profile for platform-agnostic configurations. Build-time constants: <code>cache_line_size=64</code> , <code>memory_page_size=4096</code> , <code>simd_width_bits=0</code> (scalar-only), <code>numa_capable=false</code> , <code>huge_page_capable=false</code> . Lowest common denominator when no specialized adapter matches.	Code/Baseline

Sources: platform profiles as code/baseline (no external papers).

D.30 A-Korpus Allocator Corpus (A01-A23)

The allocator axis documents 25 memory management strategies (identifiers A01–A21, the three standard-library baselines A22a–A22c, and A23) affecting dynamic allocation in search workloads. It covers textbook allocators (Buddy, Slab), industrial standards (dlmalloc, jemalloc, TCMalloc, mimalloc, snmalloc), specialized variants (NUMA-aware, PIM-Malloc, cache-aware), and modern hardened implementations (StarMalloc). All 25 wrappers satisfy the `AllocatorStrategy` concept and enable systematic measurement of memory management’s impact on search tree performance (F15 operativity).

Static sub-axes

ID	Name	Description
AA1	Free-List Topology	Organization of free memory blocks: per-page sharding (mimalloc), per-thread caching (snmalloc, rpmalloc, TCMalloc), buddy tree (Buddy), single global free-list (dlmalloc), per-heap locks (Hoard).
AA2	Size-Class Schema	Classification of block sizes: slab-per-object-size (Slab), dlmalloc bins, jemalloc size-classes, scallo spans, Buddy power-of-2.
AA3	Thread Locality	Degree of thread isolation: tcmalloc thread-local caches, rpmalloc per-thread spans, snmalloc message-passing, Hoard per-heap locks.
AA4	Synchronization	Concurrency mechanism: lock-free CAS (Michael, LRMalloc), per-heap locks (Hoard), wait-free (Crystalline), mutex-based.
AA5	Allocation Policy	Routing of allocation requests: NUMA-origin-aware (NUMAAlloc), cache-set-aware (CAMA), NUCA-aware (TCMalloc-Warehouse), PIM-aware (PIM-Malloc).
AA6	Reclamation	Reclamation of deleted blocks: magazine cache (Slab), page-heap release (TCMalloc), lock-free reclamation (LRMalloc), deferred free (mimalloc).
AA7	Fragmentation Strategy	Fragmentation avoidance: dlmalloc coalescing, object coloring (Slab), buddy splitting, page-local sharding (mimalloc), span pooling (scallop).

Building blocks

Variant	Description	Source
HoardAllocator (A01)	Per-heap free-lists with SMP scalability. Three-tier structure: thread-local heaps, global heap for cross-thread migration, false-sharing reduction via segregated allocation.	A01 [BMBW00] (Berger/McKinley/Blumofe/Wilson, ASPLOS 2000, 10.1145/378993.379232) + Code/Baseline
SlabAllocator (A02)	Object-caching kernel allocator. Slab-per-object-size with per-CPU magazine cache (Bonwick 1994/2001). Classic Solaris kernel implementation for pre-allocated object pools.	A02 [Bon94] (Bonwick, USENIX 1994) + Code/Baseline
MichaelLockFree Allocator (A03)	Lock-free CAS allocator per Michael 2004. Hazard-pointer-protected lock-free data structures, LGPL re-implementation with IBM patent background.	A03 [Mic04b] (Michael, PLDI 2004, 10.1145/996841.996848) + Code/Baseline
MimallocAllocator (A04)	Free-list sharding (3 free-lists per page). Microsoft Research 2019: deferred_free hooks, temporal cadence, page-local sharding. is_original: MIT direct binding.	A04 [LZdM19] (Leijen/Zorn/de Moura, APLAS 2019, 10.1007/978-3-030-34175-6_13) + is_original (github.com/microsoft/mimalloc, MIT)
JemallocAllocator (A05)	Arena-based size-class allocator. Evans 2006 / Free BSD. Per-CPU arenas with tcache layer for thread-local caches. Facebook standard. is_original: BSD-2-Clause direct binding.	A05 [Eva06] (Evans, BSDCan 2006, papers.freebsd.org/.../jemalloc) + is_original (github.com/jemalloc/jemalloc, BSD-2-Clause)
TCMallocAllocator (A06)	Thread-cache + central-cache + page-heap. Google gperftools 2005. Later TEMERAIRE (OSDI 2021) with warehouse-scale optimizations.	A06 [HKT ⁺ 21] (TCMalloc 2005; TEMERAIRE, OSDI 2021) + Code/Baseline
SnmallocAllocator (A07)	Message-passing allocator. Paul Liétar et al. ISMM 2019: free operations via lock-free queue to allocating thread, cross-thread lock contention avoidance. is_original: MIT direct binding.	A07 [LDPB19] (Liétar, ISMM 2019, 10.1145/3315573.3329980) + is_original (github.com/microsoft/snmalloc, MIT)
ScallocAllocator (A08)	Span-based, virtual-spans technique. Aigner/Kirsch/Lippautz/Sokolova OOPSLA 2015. Multicore-scalable with low fragmentation.	A08 [AKLS15] (Aigner et al., OOPSLA 2015) + Code/Baseline
NUMAAllocator (A09)	NUMA-origin-aware allocator. UTSASRG ISMM 2023. Node hint for local NUMA allocation, multi-node systems optimized.	A09 [YZZ ⁺ 23] (Yang et al., ISMM 2023) + Code/Baseline
RPMallocAllocator (A10)	Per-thread span caching. Mattias Jansson 2017, OSS without paper. Special case: mandatory init (rpmalloc_initialize, rpmalloc_thread_initialize). is_original: Public-Domain/MIT direct binding.	Code/Baseline (github.com/mjansson/rpmalloc, Public-Domain/MIT) + is_original
LRMallocAllocator (A11)	Lock-free + hazard pointers. Leite/Rocha VECPAR 2018 (LNCS 11333). Modern lock-free allocator with ABA protection. is_original: MIT direct binding.	A11 [LR18] (Leite/Rocha, VECPAR 2018, 10.1007/978-3-030-15996-2_17) + is_original (github.com/ricleite/lrmalloc, MIT)
CAMAAAllocator (A12)	Cache-aware memory allocator. Herter/Backes/Haupenthal/Reineke ECRTS 2011 (Saarland). Predictable cache behavior, no public code reference.	A12 [HBHR11] (Herter et al., ECRTS 2011) + Code/Baseline
StarMalloc Allocator (A13)	Formally verified hardened allocator. Inria-Prosecco (Reitz/Fromherz/Protzenko et al.) OOPSLA 2024. F*/Steel formal verification, Apache-2.0 license.	A13 [RFP24b] (Reitz et al., OOPSLA 2024) + Code/Baseline
TCMallocWarehouse Allocator (A14)	TCMalloc-Warehouse / Hyperscale. Google Cloud OSDI 2021 / ASPLOS 2024. Hugepage-aware variant, marketing variant without separate artifact.	A14 [HKT ⁺ 21, ZGV ⁺ 24] (Hunter OSDI 2021 / Zhou ASPLOS 2024) + Code/Baseline
HMallocAllocator (A15)	Hybrid, scalable, lock-free memory allocator. Li/Yao/Tang/Lin ICPADS 2019. Hybrid approach for scalability, no public code reference.	A15 [LYTL19] (Li/Yao/Tang/Lin, ICPADS 2019, 10.1109/ICPADS47876.2019.00064) + Code/Baseline
PIMMalloc Allocator (A16)	Processing-in-memory allocator. VIA-Research HPCA 2026 (arXiv:2505.13002). Memory distribution across compute nodes, PIM hardware optimized.	A16 [VIA26] (VIA-Research, HPCA 2026, arxiv.org/abs/2505.13002) + Code/Baseline

Variant	Description	Source
Crystalline Reclamation (A17)	Wait-free / lock-free safe-memory-reclamation family for non-blocking data structures. Not a standalone malloc/free allocator but a reclamation policy.	A17 [NR21, NR24] (Nikolaev/Ravindran, DISC 2021 / PLDI 2024) + Code/Baseline
ExgenAllocator (A18)	Single-threaded specialized allocator. UT Austin IEEE CAL 2025 (arXiv:2510.10219). “Old is Gold” optimizations for single-threaded workloads, no public code reference.	A18 [LJY25] (UT Austin, IEEE CAL 2025, arxiv.org/abs/2510.10219) + Code/Baseline
BuddyAllocator (A19)	Buddy system (power-of-2 splitting). Knuth TAoCP Vol.1 (1968), CACM 1965. Classic textbook allocator, binary tree splitting without OSS code reference.	A19 [Kno65] (Knuth/Carlson, CACM 1965, 10.1145/365628.365655) + Code/Baseline
DlmallocAllocator (A20)	Doug Lea malloc (bins-based). Lea 1987–2012, public-domain (CC0). Classic since 1987, bins size-class schema, oldest production implementation. is_original: public-domain direct binding.	A20 [Lea96] (Lea, gee.cs.oswego.edu/.../malloc.html, 1996) + is_original (github.com/ennorehling/dlmalloc, Public-Domain/CC0)
PtMalloc2 Allocator (A21)	ptmalloc2 (glibc malloc). Wolfram Gloger 1990s-2000s. LGPL-2.1+ copyleft, linking blocked (no is_original). Standard basis for glibc-based systems.	Code/Baseline (malloc.de/en/) + License restriction
StdMalloc (A22a)	Standard libc malloc (proof of concept), portable fallback allocator. Baseline for all standard-library implementations without vendor linking.	Code/Baseline (C++ Standard Library) + Baseline
PmrResource Allocator (A22b)	std::pmr::memory_resource. WG21 N3916 (2014 → C++17). Polymorphic Memory Resources standard library, fallback to new/delete allocator.	Code/Baseline (C++17 Standard Library, open-std.org/.../n3916.pdf) + Baseline
PoolResource Allocator (A22c)	std::pmr::unsynchronized_pool_resource. WG21 N3916 (C++17). Own size-class pool, F15-operative (first wrapper with own memory retention behavior without vendor linking).	Code/Baseline (C++17 Standard Library, open-std.org/.../n3916.pdf) + is_operative
VmemMagazines Allocator (A23)	Vmem + magazines (slab extension). Bonwick 2001 USENIX ATC. Per-CPU magazines for scalability, kernel classic without public code reference.	A23 [BA01] (Bonwick, USENIX ATC 2001, unix.org/.../bonwick.html) + Code/Baseline

Contributing sources: cited per allocator A01–A23 in the source column of the building-block table.

D.31 Axis inventory from the generated registries

The following table is not maintained by hand: the appendix generator (`Code/08_appendix_generator`) derives it from the three generated axis registries of the code — organ, system and measurement realm — and writes it to `anhang/en/tabellen/` on each run. It is therefore a projection of the compile-time reflection rather than a second, hand-kept state beside the registries. If the registries contain no axis, no file is produced on purpose; no empty table is invented.

(The axis inventory appears after the next run of the appendix generator.)

E Architecture Decisions

This appendix collects the load-bearing design decisions of this thesis as compact *Architecture Decision Records* (ADRs): for each decision the context in which it was made, the decision itself, and its most important consequence. The goal is quick traceability — *why* the apparatus is built the way the chapters describe it; the detailed derivation and justification resides in the section referenced under *Evidence* and is not repeated here.

ADR-1 — C-stable module boundary instead of a C++ module ABI

Context: Each permutation is compiled as a standalone tier binary and loaded at run time; C++23 modules structure the translation units but, by themselves, guarantee no portable ABI stability [Xu24]. **Decision:** A uniform C module boundary (`comdare_create_anatomy() → IAnatomyBase`) exports exactly one creature per permutation; behind it lies the three-layer abstraction chain `CacheEngine → ExecutionEngine → SearchEngine`. **Consequence:** ABI stability is established by the C boundary, not by the module system; the ABI contract becomes verifiable per module (opt-in ABI verify). **Evidence:** Sections 1.4, 3.3.3, and 4.3.1.

ADR-2 — Compile-time permutation via type lists instead of runtime adaptation

Context: Eighteen organ composition axes already count $2^{17} = 131072$ binary identities per system permutation in the canonical measurement catalogue — one of them, the persistence target, pinned to its in-memory representative —, and the raw variant product space over all axis catalogues including their sub-axes lies orders of magnitude above that; the hot path must carry neither `virtual` calls nor runtime switches. **Decision:** Permutation happens at compile time via Boost.MP11 type lists (`mp_list`) and concept-checked `if constexpr(resolve_baustein)` — no runtime tag, no `std::variant`; the literal `std::variant` catalogue exists only as the historical eleven-axis precursor (the `baustein_variants` fallback). **Consequence:** The axis abstraction remains a zero-cost abstraction. **Evidence:** Sections 1.4, 3.3.3, 4.3, and 3.4.5.

ADR-3 — The three-level model as the *one* architecture

Context: The anatomy metaphor and the technical engine view threaten to drift apart into parallel model trees. **Decision:** There is exactly one hierarchy without parallel trees — genus (outer interface, the test dock) → genus subclass (fixed axis set) → axis organs — under the common measurable root `IExecutionEngine`; a separate genus would arise only if the base axis set differed fundamentally. **Consequence:** “SearchEngine” is merely the ABI runtime view of the same creature, not a second model. **Evidence:** Sections 3.3, 3.3.2, and 4.3.1.

ADR-4 — Pass-through doctrine and enforced full assignment

Context: Non-tree-shaped methods (such as the hash cross-check) cannot meaningfully carry tree- and trie-specific axes like path compression or node type. **Decision:** Such axes are not dropped but assigned as pass-through variants (`None`); the `IsComposition` concept enforces, at

compile time, the assignment of all eighteen organs, so no composition with an open axis compiles.

Consequence: Non-tree-shaped methods require no genus of their own but a restricted axis subset of the same genus. **Evidence:** Sections 3.2, 3.3.2, and 4.3.

ADR-5 — Layer model E4–E0 with frozen contracts and contract tests

Context: Two implementation increments spanning several layers at once had to be rolled back entirely; with a product space that rules out any complete materialisation, multi-layer changes are practically impossible to accept. **Decision:** The experiment pipeline is organised into layers E4–E0 (plus the cross-cutting concern M); each layer is completed individually, with an explicitly frozen interface contract towards its neighbouring layer, and contract-tested against a compile-time substitute (fake) of that neighbour. **Consequence:** Every layer can be accepted individually before the next one begins; the E3→E2 boundary is already golden-tested. **Evidence:** Sections 3.3.4 and 4.4.

ADR-6 — honest-0 doctrine: honest zero and honest emptiness

Context: Values that cannot truly be collected (counters without hardware access, axes without a real runtime parameter) invite plausible phantom values that would systematically poison the heuristic derivation. **Decision:** The system reports honestly invalid or empty instead of a phantom value — per-event validity flags per measurement; the wall-clock source, for instance, refuses to label its mean as a percentile. **Consequence:** Gaps remain visible and can be closed deliberately; the doctrine is programmed into the building blocks rather than merely documented. **Evidence:** Sections 3.4 and 4.5.

ADR-7 — Three-repository split with orthogonal subsystems

Context: Measurement apparatus, test subject, and evaluation have separate responsibilities and life cycles. **Decision:** Three repositories — `comdare-cache-engine` (*how* measuring is done), `comdare-prt-art` (the test subject), and `probst-Diplomarbeit-cache-engine` (*what* is tested) —, with `comdare-cache-engine` following a Pitchfork source-tree layout; the four subsystems (measurement driver, CacheEngineBuilder, CacheEngine, test subject) lie orthogonal to this split. **Consequence:** Builder and CacheEngine share one repository as an executable and a library respectively; the roles remain cleanly separated. **Evidence:** Section 4.1.

ADR-8 — Family-wise adapter fallbacks for self-contained buildable trees

Context: Third-party code (allocator vendors, SOTA building blocks) is not available or activated on every platform and in every build. **Decision:** Compile-time flag families enable or disable adapters, each with a safe, family-wise chosen fallback: allocator adapters replace the missing vendor with `std::malloc/std::free`, data-structure adapters with a stub that, when used, throws a controlled `std::runtime_error` exception. **Consequence:** The build remains self-containedly runnable without any third-party activation. **Evidence:** Section 4.3.

ADR-9 — Measurement gating and four delivery variants

Context: Measurement and observer couplings cost time and must not burden the production path. **Decision:** The measurement couplings of the runtime adapter are gated behind `COMDARE_MEASUREMENT_ON`; this yields four delivery variants of the same permutation (debug with observers, measurement with aggregator, release without measurement couplings, and compare

as an already selectable label kept build-identical to release for the future replay comparison), and the measurement aggregator is run host-side by the `ExperimentDriver`. **Consequence:** The production path stays measurement-free and carries no measurement overhead. **Evidence:** Sections 4.1 and 4.3.2.

ADR-10 — The `std::map` contract as common denominator with a conformance gate

Context: Structurally different search algorithms can only be compared fairly through a common, well-understood outer interface. **Decision:** All methods are mapped onto the C++ contract of the ordered map `std::map<Key, Value>` (with `std::vector` in parallel as the container comparison case); a conformance gate verifies this contract before every measurement. **Consequence:** Only comparables are compared; the axes describe exclusively the inner behaviour behind the fixed interface. **Evidence:** Section 3.4.

ADR-11 — Design-pattern selection with negative list; Command only in hybrid mode

Context: The apparatus is deliberately a composition of named design patterns; unclear pattern roles would dilute the architecture. **Decision:** Used are, among others, *Strategy*, *Abstract Factory*, *Composite*, *Iterator*, *Adapter*, *Observer*, *Decorator*, *Memento*, and *CRTP*; *not* used are *Singleton*, *Flyweight*, and *Bridge*, and the *Command* pattern appears exclusively as the command channel of the hybrid mode. **Consequence:** Every pattern has exactly one named role in the wiring of the apparatus. **Evidence:** Sections 3.3.3 and 3.6.2.

ADR-12 — Dual measurement regime: immutable Talos and root Linux

Context: Production fidelity and measurement depth are in conflict: a production-faithful system denies precisely the hardware-counter access that deep measurements require. **Decision:** On the production target machines, every measurement is collected under two operating-system regimes: an immutable operating system (Talos) as the production mirror and a root Linux with full hardware-counter access (`perf/MSR`). **Consequence:** The time- and observer-based categories run identically under both regimes and secure comparability; the PMC categories are collected only under the privileged regime. **State of execution:** The decision stands but is only half redeemed — all reported measurements originate from the root-Linux regime; the Talos mirror remains a requirement and will be collected after submission. **Evidence:** Sections 5.1 and 1.4.

F Comparison Interfaces: `std::map` and `std::vector`

This appendix presents — complementing Section 3.4 — the complete listing of all standard interface functions of the two comparison shells `std::map` (ordered associative) and `std::vector` (sequence) as of C++23, with their officially expected semantics and complexity guarantee per the ISO C++ standard [ISO24]. It also serves as a central reference for the further development of the cache engine: because the search-algorithm shell replaces its internals with dynamically interchangeable axis algorithms (Section 3.4), it becomes explanation-worthy, per standard function, what behaviour the external contract guarantees.

Variadic mapping. One template parameter provides the `std::vector` API, two parameters the `std::map` API, and $N > 2$ a `std::map<K, std::tuple<...>>` (Section 4.3.1); the shared functions (iteration, `size/empty/clear`, `swap`, `get_allocator`) carry the same semantics in both shells.

Composite functions. Some functions arise “under the hood” as a composition of more primitive operations — e.g. `operator[]` as `lower_bound` followed by default insertion on a missing key, or `at` as `find` with a thrown `std::out_of_range` exception on absence —; such cases are marked in the “Note” column. The “Since” column names the introducing standard version. The exact cache-engine-internal function-handle hops are documented in a dedicated cache-engine developer document.

Conformance subset. The conformance gate (Section 3.4) checks a core subset of the `std::map` operations before each measurement, so that only comparable things are compared; it is marked as such in the `std::map` table.

F.1 `std::map` (ordered associative)

Table F.1 lists the complete interface of `std::map<Key, T, Compare, Allocator>` as of C++23. A † marks the core operations that the conformance gate (Section 3.4) checks before each measurement, so that only comparable things are compared. Complexities refer to $n = \text{size}()$; for range operations N is the number of input elements.

Table F.1: Complete `std::map` interface (C++23)

Function	Official semantics	Complexity	Since
Construction, assignment, allocator			
(constructor)	<code>empty</code> ; <code>from</code> <code>Compare/Allocator</code> ; <code>range</code> ; <code>copy/move</code> (incl. <code>allocator-extended</code>); <code>initializer_list</code> ; C++23 <code>from_range</code>	$O(1)$ to $O(N \log N)$	C++98

Function	Official semantics	Complexity	Since
(destructor)	destroys all elements and nodes	$O(n)$	C++98
operator=	copy/move/initializer_list assignment with allocator propagation	$O(n)$	C++98
get_allocator	copy of the associated allocator	$O(1)$	C++98
Element access			
at†	reference to the mapped value; throws <code>std::out_of_range</code> on absence	$O(\log n)$	C++11
operator[]†	reference; inserts a value-initialized element on absence (never throws)	$O(\log n)$	C++98
Iterators			
begin/end†	iterator to first element / end, in key order (cbegin/cend since C++11)	$O(1)$	C++98
rbegin/rend	reverse iterators (crbegin/crend since C++11)	$O(1)$	C++98
Capacity			
empty†	empty test (<code>begin() == end()</code>); [[nodiscard]] since C++20	$O(1)$	C++98
size	number of elements	$O(1)$	C++98
max_size	theoretical maximum size	$O(1)$	C++98
Modifiers			
clear†	remove all elements	$O(n)$	C++98
insert†	element(s) without overwriting (value, P&&, hint, range, initializer_list, node handle since C++17)	$O(\log n)$; hint amort. $O(1)$	C++98
insert_range	insert an input range, no overwrite	$O(N \log(n+N))$	C++23
insert_or_ assign†	insert, or assign to an existing value; bool reports insertion	$O(\log n)$	C++17
emplace†	construct in place, only if the key is absent (otherwise discarded)	$O(\log n)$	C++11
emplace_hint	like <code>emplace</code> , near the hint	$O(\log n)$; amort. $O(1)$	C++11
try_emplace†	like <code>emplace</code> , leaves the arguments untouched on an existing key	$O(\log n)$	C++17
erase†	by iterator/range/key (key: 0 or 1 removed)	iter. amort. $O(1)$; key $O(\log n)$	C++98

Function	Official semantics	Complexity	Since
<code>swap</code>	exchange contents and comparator; iterators stay valid	$O(1)$	C++98
<code>extract</code>	unlink an element as a node handle (no copy/move)	iter. amort. $O(1)$; key $O(\log n)$	C++17
<code>merge</code>	splice nodes from a source map; existing keys remain there	$O(N \log(n+N))$	C++17
Lookup			
<code>find†</code>	iterator to the key, or <code>end()</code>	$O(\log n)$	C++98
<code>count†</code>	number (0 or 1)	$O(\log n)$	C++98
<code>contains†</code>	membership (bool)	$O(\log n)$	C++20
<code>lower_bound†</code>	first key $\geq$ key	$O(\log n)$	C++98
<code>upper_bound†</code>	first key $>$ key	$O(\log n)$	C++98
<code>equal_range†</code>	pair [<code>lower_bound</code> , <code>upper_bound</code>)	$O(\log n)$	C++98
Observers			
<code>key_comp†</code>	copy of the key comparator	$O(1)$	C++98
<code>value_comp</code>	<code>value_compare</code> (compares pairs by key)	$O(1)$	C++98
Non-member functions			
<code>operator==</code>	equal size and pairwise-equal elements	$O(n)$	C++98
<code>operator<=></code>	lexicographical three-way comparison; synthesizes <code><=>=>=</code>	$O(n)$	C++20
<code>std::swap</code>	specialization; calls <code>lhs.swap(rhs)</code>	$O(1)$	C++98
<code>std::erase_if</code>	removes all elements with <code>pred==true</code> ; returns the count	$O(n)$	C++20
Deduction guides	CTAD from <code>range</code> / <code>initializer_list</code> / <code>from_range</code>	— (compile time)	C++17

The pre-C++20 standalone relational operators `<=>=>=` are synthesized from `operator<=>` since C++20, and `!=` is rewritten from `operator==`. The heterogeneous (transparent-key) overloads of `operator[]`, `at`, `try_emplace`, and `insert_or_assign` belong to C++26 and are excluded here; the C++23 additions are exactly the `from_range` constructor, `insert_range`, and the transparent-key overloads of `erase/extract`.

F.2 `std::vector` (sequence)

Table F.2 lists the complete interface of `std::vector<T, Allocator>` as of C++23 ($n = \text{size}()$). All members are `constexpr` since C++20.

Table F.2: Complete `std::vector` interface (C++23)

Function	Official semantics	Complexity	Since
Construction, assignment, allocator			
(constructor)	<code>empty/Allocator; n×</code> <code>default/value; range; copy/move (incl. allocator-extended);</code> <code>initializer_list; C++23</code> <code>from_range</code>	$O(1)$ or $O(n)$	C++98
(destructor)	destroys all elements, frees storage	$O(n)$	C++98
<code>operator=</code>	<code>copy/move/initializer_list</code>	$O(n)$	C++98
<code>assign</code>	<code>replace contents with n×</code> <code>value/range/initializer_list</code>	$O(n)$	C++98
<code>assign_range</code>	replace contents with an input range	$O(n)$	C++23
<code>get_allocator</code>	copy of the associated allocator	$O(1)$	C++98
Element access			
<code>at</code>	bounds-checked access; throws <code>std::out_of_range</code>	$O(1)$	C++98
<code>operator[]</code>	unchecked access (UB out of range)	$O(1)$	C++98
<code>front/back</code>	reference to first/last element (UB if empty)	$O(1)$	C++98
<code>data</code>	pointer to the contiguous storage (not in <code>vector<bool></code>)	$O(1)$	C++11
Iterators			
<code>begin/end,</code> <code>rbegin/rend</code>	forward/reverse iterators in index order (c . . . variants since C++11)	$O(1)$	C++98
Capacity			
<code>empty/size/</code> <code>max_size</code>	empty test, element count, maximum size	$O(1)$	C++98
<code>reserve</code>	<code>capacity ≥ n</code> ; reallocates and invalidates; throws <code>length_error</code> if <code>n > max_size()</code>	$O(n)$	C++98
<code>capacity</code>	currently allocated capacity	$O(1)$	C++98
<code>shrink_to_fit</code>	non-binding reduction of capacity toward <code>size()</code>	$O(n)$	C++11
Modifiers			

Function	Official semantics	Complexity	Since
<code>clear</code>	remove all elements (capacity unchanged)	$O(n)$	C++98
<code>insert</code>	before a position: <code>value/n×/range/initializer_list</code> ; shifts the tail	$O(n)$	C++98
<code>insert_range</code>	insert an input range before a position	$O(n)$	C++23
<code>emplace</code>	construct an element in place before a position	$O(n)$	C++11
<code>erase</code>	remove a position/range, shift the tail down	$O(n)$	C++98
<code>push_back</code>	append an element (copy/move)	amort. $O(1)$	C++98
<code>emplace_back</code>	append in place; returns a reference since C++17	amort. $O(1)$	C++11
<code>append_range</code>	append an input range	amort. $O(n)$	C++23
<code>pop_back</code>	remove the last element (UB if empty)	$O(1)$	C++98
<code>resize</code>	truncate to n , or fill with default/value	$O(n)$	C++98
<code>swap</code>	exchange contents and capacity with another vector	$O(1)$	C++98
Non-member functions			
<code>operator==</code>	equal size and element-wise equality	$O(n)$	C++98
<code>operator<=></code>	lexicographical three-way comparison	$O(n)$	C++20
<code>std::swap</code>	specialization; calls <code>lhs.swap(rhs)</code>	$O(1)$	C++98
<code>std::erase</code>	removes all elements equal to a value; returns the count	$O(n)$	C++20
<code>std::erase_if</code>	removes all elements with <code>pred==true</code> ; returns the count	$O(n)$	C++20
Deduction guides	CTAD from <code>range/from_range</code>	— (compile time)	C++17

The specialization `std::vector<bool>` packs boolean values bitwise: it has no `data()` and no contiguity guarantee, returns a proxy type reference instead of `bool&` via `operator[]/at/front/back`, and additionally offers `flip()` (at the vector and the reference level), a static `swap(reference, reference)`, and a `std::hash` specialization. The comparison operators behave as for `std::map`.

F.3 Composite Functions and the Conformance Subset

Several interface functions are “under the hood” compositions of more primitive operations (Table F.3); the table describes the ISO standard contract. The measurement wrapper of this work deliberately deviates in two places, because the ABI conformance contract demands `noexcept` and a throwing wrapper counts as non-conformant: `there`, `operator[]` is a non-mutating `std::optional` read (the write path goes through `set()/insert_or_assign`), and `at` does not throw (`optional/status` return). For this thesis that is the central observation: all map accessors and modifiers rest on a single ordered-search primitive (`lower_bound/find`); an axis implementation therefore only needs to provide a correct ordered search plus node linking, and the rest follows as a fixed composition — exactly the substitutability whose comparison is the central measurement act (Section 3.4). The conformance gate checks the core subset marked with † in Table F.1.

Table F.3: Composite functions (“under the hood”)

Function	Composition	Note
<code>map::</code>	<code>lower_bound</code> + default insertion	mutates on absence;
<code>operator[]</code>	on absence $\rightarrow$ reference	never throws
<code>map::at</code>	<code>find</code> + throw	the only throwing
	<code>std::out_of_range</code> on absence	access; never mutates
<code>map::insert /</code> <code>emplace</code>	ordered search + link a node only if the key is absent	no overwrite; <code>emplace</code> may still consume arguments on a hit
<code>map::try_</code> <code>emplace</code>	like <code>emplace</code> , but arguments untouched on a hit	preserves move-only arguments
<code>map::insert_</code> <code>or_assign</code>	ordered search + insert or assign	overwrites (unlike <code>insert</code>)
<code>map::count /</code> <code>contains /</code> <code>equal_range</code>	reduce to <code>find</code> resp. <code>lower_bound+upper_bound</code>	always 0 or 1 for a unique-key map
<code>vector::</code> <code>push_back /</code> <code>emplace_back</code>	possible growth (realloc + relocation) + construction at the end	hidden cost + invalidation on realloc
<code>vector::insert</code> <code>/emplace</code>	possible growth + tail shift + insertion	extra $O(n)$ shift
<code>vector::resize</code>	append (default/value) or truncate the tail	bidirectional; does not shrink capacity
<code>vector::assign</code>	clear + insert $\text{range}/n \times$	capacity reused where possible

Functions that both shells *share* by name — `size`, `empty`, `clear`, the iterator families, `swap`, `get_allocator`, and `insert/emplace/erase` — carry key semantics in the map and position/index semantics in the vector; `operator[]`, for instance, inserts in the map but is an unchecked index access in the vector. The exact cache-engine-internal function-handle hops behind these contracts are documented in a dedicated cache-engine developer document.

Acknowledgments

My thanks go first to apl. Prof. Dr.-Ing. Dirk Habich — a modest, disciplined person — for the dedicated supervision of this thesis: despite persistently scarce time, he always made time for precise questions, clear priorities, and the freedom to develop this work largely independently, with targeted support where it was needed. I thank Dr.-Ing. Alexander Krause for accompanying this work as second advisor. Finally, I thank the team at UltiHash: the instructive years at this startup shaped many of the experiences and questions that this work now implements technically.

Copyright Information

Confidentiality notice (restricted)

This thesis itself is not subject to a confidentiality notice. A restricted confidentiality notice concerns the software core of the `comdare-cache-engine` and the probe PRT-ART: both may be inspected and tested for assessment; any business use, private individual use or distribution beyond the assessment requires a separate licence from the rights holder, BEP Venture UG (haftungsbeschränkt), owner of the registered trademark Comdare. Research is exempt: for research purposes at an institution both may be used freely. The experiments described in this thesis use the cache engine as a client via freely selectable test parameters and are not affected by the restriction.

Licences and copyright

The code of the diploma-thesis repository (measurement driver and evaluation pipeline) is licensed under Apache-2.0; bundled submodules and third-party components keep their own licences. The software core of the `comdare-cache-engine` and the probe PRT-ART are licensed under the Comdare Research License 1.0: the source code is readable by everyone; research at an institution may use it freely; business use and private individual use each require a separate written licence from the rights holder. For the `comdare-cache-engine` the change date 2031-08-10 applies: from that day it becomes available to everyone under the Apache License 2.0. Previously published revisions keep their original licence irrevocably (`comdare-cache-engine`: Apache-2.0 before 2026-08-02, a dual research/commercial model from 2026-08-02 to 2026-08-09; PRT-ART: Apache-2.0 from 2026-08-02 to 2026-08-09). The rights to both software cores rest with BEP Venture UG (haftungsbeschränkt), Comdare brand. The text of this thesis is copyright Benjamin-Elias Probst and is licensed under CC BY 4.0 (Creative Commons Attribution 4.0 International).

Third-party material

Verbatim and paraphrased passages taken from other works are marked as quotations and referenced in the bibliography; they remain the property of their respective authors and are not covered by the licence of this thesis. All figures and tables in this thesis were produced by the author from the project's own measurement data and design documents (TikZ/pgfplots or generator output of the evaluation pipeline); no third-party graphics are included. The logo on the title page is a mark of Technische Universität Dresden (ZIH), is used according to its rules for theses and is excluded from the CC BY licence. Third-party benchmark datasets (such as the datasets of the SOSD benchmark) are not part of this thesis; they are only cited or used as input through the dataset loader, and their use is governed by the terms of their publishers. Third-party code vendored in the `comdare-cache-engine` (directory `ext/`) is not part of

the software within the meaning of the Comdare Research License and keeps its respective original licence; the complete attribution is given in the files `NOTICE`, `REUSE.toml` and `LICENSE_AUDIT_EXT.md` of the repository. *Comdare* is a registered trademark of the rights holder; its mention in this thesis does not grant any trademark rights to third parties.